

Introducción a las redes neuronales

Índice general

1. Teoría de aprendizaje estadístico	9
1.1. Definición de aprendizaje	9
1.2. Modelos de aprendizaje estadístico	11
1.3. Tipos de aprendizaje	12
1.3.1. Modelos discriminativos y generativos	14
1.4. Entrenamiento y evaluación	15
1.4.1. Entrenamiento	18
1.4.2. Evaluación	23
1.4.3. Subajuste y sobreajuste	30
1.5. Representación de datos	31
1.5.1. Datos no estructurados	34
1.5.2. Normalización de datos	37
1.5.3. Datos sintéticos	39
1.5.4. Conjetura de la variedad	40
1.6. Regresión lineal	42
1.7. Regresión logística	46
2. El Perceptrón	53
2.1. Aprendizaje en el perceptrón	56
2.2. Separabilidad lineal	60
2.2.1. Perceptrón y problemas lógicos	63
2.3. Modelos de una neurona	66
3. Redes feedforward	71
3.1. Capas ocultas y redes feedforward	71
3.1.1. Funciones de activación	74
3.2. Teoremas de aproximador universal	81
3.2.1. Otros teoremas de aproximador universal	86
3.3. Redes neuronales profundas	87
4. Aprendizaje en redes multicapa	91
4.1. Descenso por gradiente	93
4.1.1. Lotes y mini lotes	99
4.2. Optimizadores basados en gradiente	102

4.3. Retro-propagación	112
4.4. Implementación del aprendizaje	121
4.4.1. Gráficas computacionales	124
5. Regularización	129
5.1. Regularización de Tychonoff	129
5.2. Aumento de datos	133
5.3. Robustecimiento por ruido	135
5.4. Aprendizaje multitarea	138
5.5. Early stopping	139
5.6. Métodos por conjuntos	141
5.7. Dropout	144
6. Redes neuronales recurrentes	149
6.1. Definición de red recurrente	149
6.2. Tipos de redes recurrentes	153
6.2.1. Redes encoder-decoder	155
6.2.2. Redes recurrentes bidireccionales	156
6.3. Aprendizaje en capas recurrentes	158
6.4. Capas de memoria	165
6.4.1. Gated Recurrent Unit (GRU)	166
6.4.2. Long-Short Term Memory (LSTM)	168
7. Redes convolucionales	175
7.1. Convolución y correlación cruzada	175
7.2. Capas convolucionales	178
7.3. Pooling	184
7.4. Redes convolucionales	186
7.5. Entrenamiento en capas convolucionales	189
8. Redes atencionales	193
8.1. Atención en redes recurrentes	193
8.2. Auto-atención	198
8.2.1. Derivación de una capa de auto-atención	203
8.3. Auto-atención enmascarada	207
8.4. Cabezas de atención y atención multi-cabeza	210
8.5. Transformadores	212
8.5.1. Encoder	214
8.5.2. Decoder	218
8.5.3. Salida e inferencia	220
8.5.4. Optimizador Noam	223

Introducción

Las redes neuronales artificiales son modelos de aprendizaje estadístico que buscan emular la forma en que una neurona biológica procesa información. En la actualidad, las redes neuronales han tenido un gran impacto, puesto que han permitido el desarrollo de herramientas de inteligencia artificial como no lo habíamos visto antes. Las redes neuronales han hecho posible un desarrollo de la llamada inteligencia artificial generativa que no había sido posible con otras perspectivas. Esto debido a que las redes neuronales tienen una gran flexibilidad para adaptarse a distintos problemas y resolverlos de manera eficiente, siempre que se cuente con la cantidad suficiente de datos.

El presente trabajo busca dar una primera introducción a las redes neuronales desde una perspectiva que busca fundamentar los conceptos matemáticos que están detrás de éstas. Estas notas han sido elaboradas pensando en el curso de redes neuronales que se imparte en la Facultad de Ciencias de la UNAM. Sin embargo, se busca adoptar un enfoque actualizado que tome en consideración las aplicaciones más recientes en el área. En tanto introducción, el enfoque del libro son únicamente los modelos supervisados más usados en la actualidad. Otros modelos, que suelen revisarse en el curso, como las arquitecturas no supervisadas (máquinas de Boltzman, SOM, Autocodificadores) o generativas (GANs, modelos de difusión), se dejan para otro trabajo.

La inteligencia artificial es un área que cambia rápida y constantemente. En los últimos años, el área de las redes neuronales ha presentado grandes avances. Se habla de aprendizaje profundo, puesto que las redes utilizadas incorporan mayor profundidad, un mayor número de neuronas y son más complejas. Han surgido un conjunto de nuevas arquitecturas enfocadas a procesar datos con estructuras complejas. Las redes convolucionales que se enfocan en el procesamiento de imágenes, las redes recurrentes y los transformadores que han tenido amplias aplicaciones en texto. Asimismo, se han presentado numerosas propuestas para mejorar el aprendizaje dentro de las redes neuronales: optimizadores basados en gradiente o estrategias de regularización. Constantemente se publican nuevas investigaciones que se enfocan en proponer nuevas arquitecturas, o estrategias de aprendizaje. Seguir el ritmo de este desarrollo se vuelve una tarea titánica. El objetivo de este texto no es presentar las innovaciones más recientes, sino ser una introducción de los que creemos son los temas más relevantes para adentrarse al mundo de las redes neuronales y el aprendizaje profundo.

Con este objetivo en mente, gran parte del texto desarrolla las nociones básicas del aprendizaje profundo, la definición de neurona artificial, de red neuronal profunda y las estrategias de aprendizaje con optimizadores basados en descenso por gradiente y el algoritmo de retro-propagación. Asimismo se dedica un capítulo a los métodos de

regularización, que se enfocan en mejorar la generalización del aprendizaje en contextos donde no se cuenta con un gran número de datos. El primer capítulo presenta una introducción muy general al aprendizaje estadístico, área donde se inserta el aprendizaje profundo. El objetivo es distinguir las nociones básicas de aprendizaje, así como las estrategias de evaluación que estarán presentes en todo desarrollo de redes neuronales. En este mismo capítulo se presentan los modelos de regresión lineal y regresión logística, los cuáles, como se muestra más adelante, también pueden pensarse como modelos de una neurona.

El siguiente capítulo presenta el perceptrón, el cual es el modelo estándar de una neurona. Además de presentar el perceptrón se realiza una discusión teórica sobre los límites de este modelo para resolver problemas. Se muestra que el perceptrón sólo es capaz de solucionar problemas que sean linealmente separables y que es incapaz de lidiar con problemas complejos como el problema lógico de XOR. Finalmente, se ven las relaciones que guarda con la regresión lineal y logística. Es en el tercer capítulo donde se introducen las redes profundas, las llamadas redes feedforward. En este capítulo se abordan los conceptos de capas ocultas y se profundiza en las funciones de activación que pueden aplicarse en las redes neuronales. Finalmente, se discute la propiedad de este tipo de redes de ser aproximadores universales.

Los siguientes dos capítulos abordan las estrategias básicas del aprendizaje sobre las redes profundas. En primer lugar, se discute el método del descenso por gradiente, así como el concepto de lote, esencial en todo entrenamiento en redes neuronales. Posteriormente se amplían los métodos de optimización por gradiente revisando métodos basados en momento y adaptativos. Finalmente, se revisa el algoritmo central de aprendizaje en redes profundas, la retro-propagación. En el capítulo sobre regularización se revisan diferentes estrategias que permiten una mejor generalización durante el entrenamiento, como la regularización L_1 y L_2 , el aumento de datos, aprendizaje multi-tarea, *early stopping*, y dropout, el cual es el método de regularización más común actualmente.

En estos primeros capítulos se busca abordar los temas recurriendo a sus fundamentos matemáticos. Algunos de los conceptos explicados profundizan en temas que quizá no sean ampliamente conocidos por todos los lectores. Un ejemplo claro es la discusión que se hace sobre los teoremas de aproximador universal, los cuáles requieren de la aplicación de resultados de análisis funcional (como el teorema de Hahn-Banach o el de representación de Riesz). Estos resultados pueden omitirse sin afectar la lectura del texto. Particularmente, las demostraciones de estos teoremas pueden omitirse en una primera lectura, enfocada más a los conceptos de aplicación técnica.

La idea del presente texto es profundizar en los fundamentos que están detrás del aprendizaje profundo, pues consideramos que esto ayuda al mejor entendimiento de las redes neuronales. El libro está dirigido a estudiantes de las carreras de ciencias de la computación, matemáticas, física, actuaría, de donde provienen la mayoría de los alumnos que toman los cursos del área. Por tanto, hemos asumido un nivel de conocimientos matemáticos correspondiente a estas carreras. Para fortalecer el conocimiento de los conceptos, en cada capítulo se presentan un conjunto de ejercicios tanto teóricos como prácticos. En la parte práctica, se cuenta con una serie de ejemplos escritos en el lenguaje de programación Python que pueden encontrarse en el siguiente enlace:

<https://github.com/VictorMijangosDeLaCruz/Redes-Neuronales>.

Este libro se escribió gracias al apoyo del proyecto PAPIIT TA100924 “Investigación de sesgos inductivos en aprendizaje profundo y sus aplicaciones”, otorgado por la DGAPA de la UNAM.

Capítulo 1

Teoría de aprendizaje estadístico

El aprendizaje profundo es parte de una teoría de mayor alcance conocida como aprendizaje estadístico o aprendizaje de máquina. Hacemos una distinción entre ambos términos, llamando aprendizaje estadístico a la parte teórica del área (Vapnik, 1998), mientras que el término de aprendizaje de máquina o automático referirá a la aplicación de los métodos y algoritmos orientados a aplicaciones específicas. De esta forma, el aprendizaje estadístico nos dará herramientas teóricas para abordar a las redes neuronales. Asimismo, podremos considerar la implementación de modelos de redes neuronales como parte del aprendizaje automático.

Por tanto, antes de partir a estudiar las redes neuronales, es importante conocer las nociones básicas de la teoría de aprendizaje. En este capítulo desarrollamos estas nociones; introducimos el concepto de aprendizaje, los tipos más comunes de aprendizaje, así como la metodología general que se sigue cuando se usa un modelo de aprendizaje para resolver un problema.

1.1. Definición de aprendizaje

El concepto de aprendizaje es central en nuestro estudio. Buscamos que las máquinas “aprendan” a partir de experiencia para que sean capaces de resolver problemas específicos. Aquí el término de aprendizaje difiere a lo que se podría entender en ámbitos como la psicología o las ciencias cognitivas, pues nos orientamos a definir agentes computacionales que resuelvan tareas más que a entender los procesos cognitivos humanos. Con base en esto, damos una definición de aprendizaje basada en Mitchell (1997).

Definición 1 (Aprendizaje). *Un programa de computadora se dice que aprende de la experiencia E con respecto a una tarea T y una medida de desempeño P , si su desempeño con respecto a T , medido con P , mejora con la experiencia E .*

Como se desprende de esta definición, decimos que un programa aprende cuando mejora su desempeño en una tarea con base en experiencia proporcionada. Esta definición se liga al modelo estándar de la inteligencia artificial (Russell and Norvig, 2021),

en donde se busca que los programas solucionen las tareas de manera óptima; esto es, buscando los valores óptimos para alguna medida de desempeño. Lo que caracteriza a los modelos de aprendizaje y los diferencia de otros métodos en inteligencia artificial, es que su desempeño varía con base en experiencia que los modelos incorporan. Esta experiencia no son otra cosa que datos que el programa observa para inferir soluciones al problema con base en la información que la máquina puede encontrar en ellos. De aquí proviene la analogía con el aprendizaje humano, pues el programa será mejor en resolver un problema entre más datos (instancias del problema) vea. Sin embargo, no tendría sentido que el programa observará todas las instancias del problema, pues esto implicaría que conocemos las soluciones al problema.

El aprendizaje estadístico observa un conjunto limitado de instancias del problema que permitan al programa generalizar las soluciones a otras instancias que no hayan sido vistas previamente. Por ejemplo, si el objetivo del programa es solucionar ecuaciones diferenciales ordinarias, podemos darle a éste un conjunto finito de ecuaciones (y sus soluciones) para que infiera estrategias para solucionarlas, de tal forma que cuando se le presente una nueva ecuación pueda encontrar su solución sin conocerla previamente. Presentamos otro ejemplo a continuación.

Ejemplo 1. *Para complementar la definición de aprendizaje que hemos dado, pensemos en el siguiente ejemplo: supongamos que queremos crear un programa que se dedique a clasificar spam de nuestro correo electrónico. Entonces, para poder aplicar un algoritmo de aprendizaje requerimos definir los siguientes elementos:*

Tarea: *La tarea T es determinar si un correo electrónico, una cadena de texto, es correo deseado (ham) o correo no deseado (spam).*

Experiencia: *La experiencia E consistirá en el conjunto de correos electrónicos en los que previamente el usuario ha determinado si se trata de correo deseado o no deseado.*

Desempeño: *Podemos definir diferentes medidas de desempeño P : una medida simple sería determinar el porcentaje de correo electrónico correctamente clasificado ya sea en ham o spam.*

De esta forma, un algoritmo de aprendizaje automático toma los correos electrónicos previamente clasificados para poder inferir qué es lo que determina si se trata de correo deseado o no deseado. Asimismo, seríamos capaces de evaluar qué tan bien ha aprendido a clasificar los correos.

Podemos observar que en la mayoría de los casos, el que un correo sea spam depende de los gustos de los usuarios. Por tanto, no podemos definir un algoritmo preciso que determine cuando se trata de spam o no. Un algoritmo de aprendizaje tiene la ventaja de que puede aprender lo que un usuario en particular considera spam. Si bien, en principio podría tener un desempeño bajo, la experiencia que le proporciona el usuario (que en este caso podría consistir en corregir los errores del programa señalando cuando ha enviado un correo a spam sin serlo o cuando ha dejado pasar un correo que para el usuario es spam) el algoritmo de aprendizaje puede mejorar su desempeño.

1.2. Modelos de aprendizaje estadístico

Nuestro problema consiste en entender y construir máquinas que aprendan. Como en otros problemas con que los humanos nos enfrentamos, resulta aquí de suma importancia el determinar un modelo matemático que describa las capacidades que tiene una máquina para aprender sobre problemas. Se han hecho varias propuestas en la teoría de aprendizaje estadístico: una de ellas, propuesta por Valiant (1984), es el aprendizaje PAC (por sus siglas en inglés *Probably Approximate Correct Learning*); otra propuesta es el aprendizaje VC (siglas de los apellidos de sus creadores, los investigadores rusos Vladimir Vapnik y Alexey Chervonenkis). Se ha probado que ambas teorías son equivalentes. Aquí nos enfocamos en la última: la teoría VC.

En este marco, Vapnik (1998) propone el **modelo general de aprendizaje a partir de ejemplos**, el cual busca sintetizar el proceso de aprendizaje por medio de tres elementos esenciales:

- **Generador:** Se encarga de generar la experiencia (ejemplos), que se describen a partir de una distribución de probabilidad $p(x)$. Generalmente estos ejemplos x son vectores y la función de probabilidad define la forma en que se comportan.
- **Supervisor:** Determina una solución al problema, asociando a cada ejemplo x su solución y . La supervisión puede o no estar presente durante el aprendizaje, en cada caso hablaremos de un tipo distinto de aprendizaje.
- **Máquina de aprendizaje:** Es una función f que toma un ejemplo de entrada x y le asigna un valor de salida, de tal forma que $f(x) = \hat{y}$, donde $\hat{y}$ es una predicción hecha por la máquina de aprendizaje que busca ser lo más cercana a la supervisión y .

De esta forma, el objetivo del aprendizaje automático es encontrar una máquina de aprendizaje que genere salidas $\hat{y}$ que sean lo más cercanas posibles a los elementos del supervisor y . En este sentido, la máquina de aprendizaje es una función:

$$f : X \rightarrow Y$$

donde X es el espacio de los datos y Y el de las soluciones. El generador proporciona un conjunto de datos $\{x : x \in X\}$, mientras que el supervisor asocia a cada dato x una respuesta $y \in Y$. La Figura 1.1 presenta de forma esquemática este modelo de aprendizaje.

Generalmente, la función f depende de un conjunto de parámetros o pesos que se modifican cada vez que la máquina ve un nuevo ejemplo;¹ de esta forma, estos pesos se ajustan hasta obtener las salidas correctas en la máquina de aprendizaje. Por lo que podemos pensar a la máquina de aprendizaje como la función:

$$f : X \times \Theta \rightarrow Y$$

¹Existen métodos que no están determinados por un conjunto de parámetros. Estos suelen llamarse modelos no paramétricos.

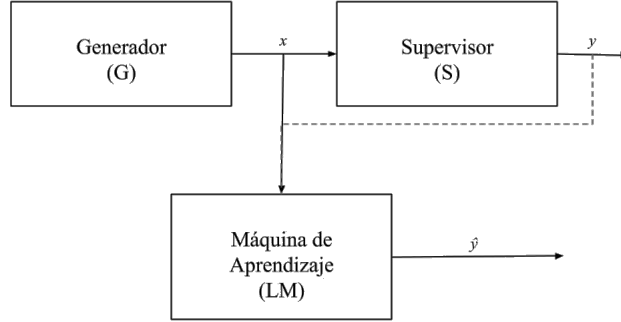


Figura 1.1: Esquema del modelo general de aprendizaje a partir de ejemplos

Donde Θ puede llamarse el **espacio de hipótesis**; es decir, las soluciones al problema deben buscarse en Θ , de tal forma que el proceso de aprendizaje consista en buscar los elementos del espacio Θ que resuelvan el problema de forma óptima. Entonces, si $\theta \in \Theta$ es una hipótesis, podemos determinar la máquina de aprendizaje como la función:

$$f(x; \theta) = \hat{y}$$

El objetivo del proceso aprendizaje es encontrar el conjunto de pesos θ^* que solucione el problema de manera adecuada. Idealmente, buscamos que $f(x; \theta) = y$, es decir, que la máquina de aprendizaje de exactamente las mismas respuestas que el supervisor. Esto no es factible en general. Pero podemos encontrar aproximaciones que sean lo más cercanas posibles a las respuestas del supervisor.

Un paso importante de la teoría de aprendizaje estadístico es convertir el problema de aprendizaje en un problema de optimización. Para esto, debemos definir una función de riesgo (también llamada función objetivo) $R : \Theta \rightarrow \mathbb{R}^+$, que sea capaz de decirnos que tan bien una máquina de aprendizaje soluciona un problema. De tal forma que nuestro objetivo se definirá como:

$$\theta^* = \arg \min_{\theta \in \Theta} R(\theta; f)$$

La función R dependerá del problema de aprendizaje. De forma general usaremos sólo $R(\theta)$ para denotar esta función. De esta forma, nuestro objetivo será encontrar los parámetros que mejor resuelvan el problema a partir de minimizar la función de riesgo.

Antes de pasar a explicar con más a detalle la forma en que podemos solucionar los problemas por medio del aprendizaje, pasamos a exponer de manera breve los diferentes tipos de aprendizaje, que también se desprenden del modelo de aprendizaje a partir de ejemplos. De esta forma, nos adentramos al entrenamiento y evaluación donde profundizamos en la función de riesgo así como en las capacidades de generalización de una máquina de aprendizaje.

1.3. Tipos de aprendizaje

En la Figura 1.1 se puede ver una línea punteada entre la salida y del supervisor y la máquina de aprendizaje. Lo que esto quiere indicar es que la máquina de aprendizaje puede trabajar tanto con información supervisada (el supervisor le proporciona algún conocimiento de las salidas esperadas) o bien sin tal información. Lo que queda claro es que la máquina de aprendizaje necesita de un conjunto de ejemplos para aprender; ejemplos que son proporcionados por el generador. A este conjunto de ejemplos se le suele llamar **conjunto de datos** (en inglés *dataset*). Los datos en este conjunto de datos son esenciales en el aprendizaje, pues dotan a la máquina de aprendizaje de ejemplos para que ésta pueda aprender.

Los ejemplos que se le proporcionan a la máquina de aprendizaje pueden contener o no información del supervisor. Cuando contienen esta información, los ejemplos dotan a la máquina de información sobre las salidas esperadas. Por ejemplo, si la tarea es la clasificación de spam, un ejemplo (x, y) contará con un correo electrónico x (que puede ser la cadena textual o alguna otra codificación del texto) así como una indicación $y \in \{0, 1\}$, que le indicará a la máquina si x es spam (1) o si no lo es (0). Por tanto, durante su proceso de aprendizaje, la máquina podrá ver en algunos ejemplos cuáles son las salidas esperadas. A partir del conjunto de ejemplos que se proporcionan en la tarea, podemos definir los tipos de aprendizaje.

Definición 2 (Aprendizaje supervisado). *El aprendizaje supervisado es aquel que se realiza con un conjunto de datos que cuenta con una supervisión (solución esperada por cada ejemplo). El conjunto de datos supervisado está determinado como:*

$$\mathcal{S} = \{(x, y) : x \in X, y \in Y\}$$

donde Y es el conjunto de soluciones o supervisión del problema.

Dentro del aprendizaje supervisado la máquina de aprendizaje aprende en base a ejemplos que cuentan con la solución esperada de éstos, de tal forma que la máquina pueda ver lo que se espera obtener y generalizar este conocimiento a datos que no han sido vistos. Dentro del aprendizaje supervisado también tenemos dos grandes problemas que resolver. El primero de estos problemas refiere a la clasificación.

Definición 3 (Problema de clasificación). *Un problema de clasificación es un problema supervisado, donde se busca estimar una función $f : X \rightarrow \{0, 1, 2, \dots, M\} \subseteq \mathbb{N}$; es decir, clasifica los datos en clases discretas.*

Como se puede observar, la clasificación se basa en asignar a cada dato un valor de clase. Un tipo de clasificación común es la clasificación binaria, en donde el conjunto de clases es $Y = \{0, 1\}$. Estas clases pueden representar diferentes cosas; por ejemplo, una enfermedad (1) y la ausencia de ésta (0), si el dato representa a un animal (1) o no (0), etc.

El segundo problema dentro del aprendizaje supervisado es el de regresión.

Definición 4 (Problema de regresión). *Un problema de regresión es un problema supervisado en donde se busca estimar una función $f : X \rightarrow Y \subseteq \mathbb{R}^k$, $k \in \{1, 2, \dots, n\}$; en este caso, Y toma valores continuos.*

En la regresión los valores que se asignan a los datos de entrada son continuos. Por tanto, no es necesario (de hecho, no es factible) que la máquina de aprendizaje observe todos los valores posibles en el entrenamiento. Lo que se busca es estimar una función continua que, cuando llegué un nuevo dato, pueda asignarle un valor real. Un caso particular es cuando se estima un escalar; es decir, se busca una función $f : X \rightarrow \mathbb{R}$. Algunos ejemplos de este tipo de problemas es la estimación de precios: dada la descripción de un producto por medio de un vector en $\mathbb{R}^d$, se busca determinar un precio para éste.

En contraste con el aprendizaje supervisado, en el aprendizaje no-supervisado la máquina de aprendizaje no cuenta con información del supervisor para poder aprender. La máquina de aprendizaje sólo podrá observar los ejemplos x .

Definición 5 (Aprendizaje no-supervisado). *El aprendizaje no-supervisado es aquel que no cuenta con una supervisión, y se determina por un conjunto de datos dado como:*

$$\mathcal{U} = \{x : x \in X\}$$

Este tipo de aprendizaje resulta, por lo general, más complejo, pues se busca que la máquina pueda encontrar patrones dentro de los datos que la lleven a tomar ciertas decisiones sobre estos. Dentro del aprendizaje no-supervisado, podemos hablar de los siguientes problemas típicos:

1. *Agrupamiento* (clustering): En el agrupamiento, como su nombre lo dice, la máquina de aprendizaje busca agrupar los datos dentro de grupos, conocidos como clústers, a partir de sus similitudes. Los elementos dentro de un mismo clúster tendrán características similares, y se diferenciarán de los elementos de otros clústers.
2. *Reducción de dimensionalidad*: La reducción de dimensionalidad busca comprimir la información de los datos de tal forma que se puedan conservar las características más relevantes de éstos. Es de especial utilidad para visualizar datos que tengan alta dimensionalidad.

Los problemas en que se enfoca el aprendizaje no-supervisado son principalmente exploratorios. Debido a la falta de una orientación de los resultados esperados, la máquina de aprendizaje debe inferir características relevantes de los datos de manera autónoma.

Otro tipo de aprendizaje importante es el **aprendizaje por refuerzo** (*reinforcement learning*), en donde se busca minimizar el costo para que un agente complete una meta. En este caso, hay un agente que realizará ciertas acciones que dependen del estado del entorno en que el agente se encuentre; el objetivo es realizar las acciones que minimicen una función de refuerzo. Otro tipo de aprendizaje es el aprendizaje auto-supervisado, que se utiliza en los modelos del lenguaje basados en redes neuronales. Estos tipos de aprendizaje no serán profundizado aquí.

1.3.1. Modelos discriminativos y generativos

Una distinción común dentro del aprendizaje, que es relevante para las redes neuronales, consiste en la diferencia entre los modelos discriminativos y modelos generativos.

En el caso de las redes neuronales, éstas primero surgen como modelos discriminativos (el perceptrón, las redes feedforward). Posteriormente se desarrollan arquitecturas de redes neuronales generativas, tales como las Redes Generativas Adversariales o GANs (*Generative Adversarial Networks*) (Goodfellow et al., 2014), los Autoencoders variacionales (Kingma and Welling, 2013), los Transformers (Vaswani et al., 2017) o los modelos de difusión (Ho et al., 2020).

Tanto los modelos discriminativos como los modelos generativos pueden considerarse como modelos gráficos. Un **modelo gráfico** representa las variables del problema por medio de nodos, y las relaciones entre estas variables con aristas. Esencialmente, los modelos discriminativos se representan con aristas no dirigidas, mientras que los modelos generativos tienen aristas dirigidas (véase Figura 1.2). Esta caracterización determina el tipo de distribuciones que ambos tipos de modelos pueden estimar.

Definición 6 (Modelo discriminativo). *Un modelo discriminativo es un modelo gráfico no dirigido que estima distribuciones condicionadas a un vector de observación de la siguiente forma:*

$$p(Y = y|x) = \prod_i \phi(x_i, y)$$

Donde Y es la variable de clases y cada x_i es una entrada del vector x . Cada valor $\phi(x_i, y)$ se conoce como un factor.

Los modelos discriminativos suelen aplicarse en la clasificación de elementos de entrada (vector de rasgos x) dentro de una clase y a partir de discriminar las clases probables. Un ejemplo de modelo discriminativo es la regresión logística o el perceptrón. Este tipo de modelos contrasta con los modelos generativos.

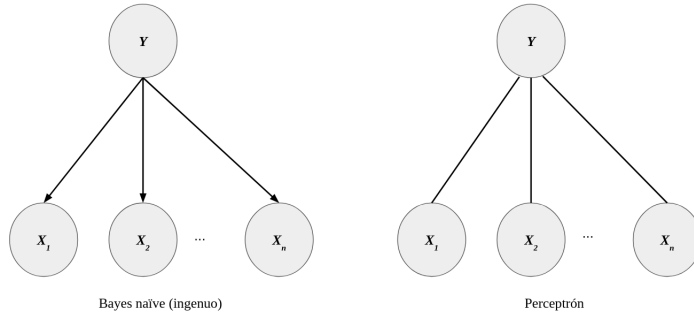


Figura 1.2: Los modelos generativos (como Bayes ingenuo) pueden modelarse como una gráfica que se dirige de las categorías o clases a los ejemplos; los modelos discriminativos (como el perceptrón) están determinados por una gráfica no dirigida.

Definición 7 (Modelo generativo). *Un modelo generativo es un modelo gráfico dirigido que estima distribuciones de datos x condicionadas a una clase y de la siguiente forma:*

$$p(y, x) = p(x|y)p(y) = \prod_i p(x_i|y)p(y)$$

Donde x_i son las entradas del vector x .

Tanto en la Definición 6 y 7, los casos de factorización son excepcionales. En los modelos generativos se asume que los datos observados x fueron generados por una clase y . De tal forma que se puede hacer una clasificación basada en la clase que con mayor probabilidad genera x . Un modelo generativo típico es el de Bayes naïve o ingenuo (Figura 1.2).

Dentro del aprendizaje profundo se tienen modelos tanto de índole discriminativo como generativo. En este texto abordamos en mayor amplitud los modelos discriminativos. Los modelos generativos tienen particularidades que no profundizaremos. Algunas redes generativas son los autoencoders variacionales o las máquinas de Boltzman. También existen modelos híbridos, que combinan módulos generativos con discriminativos; tal es el caso de las Redes Generativas Adversariales o GANs.

1.4. Entrenamiento y evaluación

Dos partes esenciales en el procesamiento de aprendizaje es el entrenamiento y la evaluación. Entrenar un modelo es el proceso central del aprendizaje. Durante el entrenamiento, se obtiene los pesos óptimos de una red neuronal, los cuales les permitirán resolver los problemas específicos. La evaluación, por su parte, nos dice la capacidad de generalizar que tiene una máquina de aprendizaje.

Generalmente el proceso de aprendizaje consta de los siguientes pasos: 1) generar un modelo de aprendizaje a partir de entrenar la máquina utilizando datos de entrenamiento (ejemplos); y 2) determinar la capacidad de generalizar del modelo a partir de aplicarlo en datos nuevos para determinar qué tan bien los resuelve (véase Figura 1.3). El proceso de entrenamiento determina la forma en que la máquina resolverá el problema, es propiamente el aprendizaje. La evaluación es esencial para conocer la capacidad de nuestro modelo para resolver el problema.

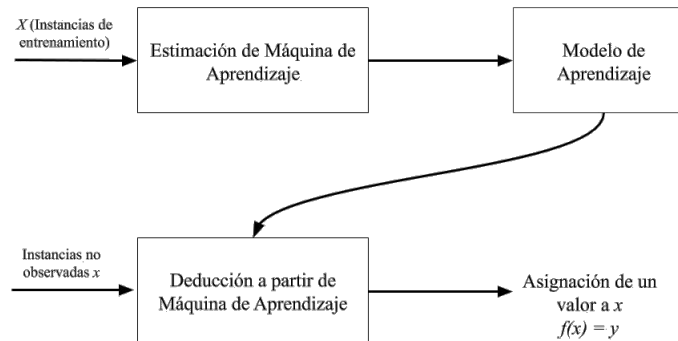


Figura 1.3: Proceso de aprendizaje

En el entrenamiento y la evaluación buscaremos minimizar una función de riesgo (también llamada **función objetivo**) que nos indica qué tan bien está resolviendo la

tarea la máquina de aprendizaje. Para definir la función de riesgo, hablaremos de una **función de pérdida**, que es una función $L: \mathcal{F} \times Y \rightarrow \mathbb{R}^+$, donde $\mathcal{F} = \{f(\cdot; \theta) : \theta \in \Theta\}$ es el espacio de las máquinas de aprendizaje que dependen de los parámetros θ . Esta función puede entenderse como un indicador de qué tanto la máquina f se equivoca en el problema.

Distinguimos dos funciones de riesgo, la primera de ellas responde al riesgo de generalización:

Definición 8 (Función de riesgo de generalización). *Dada una función de pérdida L y una máquina de aprendizaje $f: X \rightarrow Y$, una función de riesgo de generalización es una función $R_G: \Theta \rightarrow \mathbb{R}^+$ que se define como:*

$$\begin{aligned} R_G(\theta) &= \mathbb{E}_\Omega [L(f(X; \theta), Y)] \\ &= \int_X L(f(x; \theta), y) d\mu(x) \end{aligned}$$

donde μ la función de distribución sobre el espacio de datos.

La función de riesgo de generalización no se puede calcular empíricamente; es decir, a partir de un conjunto de datos, pues busca representar el espacio del problema, que generalmente es continuo y demasiado grande. Esta función busca determinar la capacidad de la máquina de aprendizaje sobre todo el espacio del problema. Para tratar de estimar la capacidad de generalización de un modelo se usan técnicas de evaluación que nos permitirán determinar, de manera aproximada, qué tan bien lo hace el modelo de aprendizaje en el problema (véase la Sección 1.4.2).

En el entrenamiento, la estimación del modelo se realiza mediante un conjunto de datos finito. Por tanto, determinamos una función de riesgo empírico.

Definición 9 (Función de riesgo empírico). *Dada una función de pérdida L y una máquina de aprendizaje $f: X \rightarrow Y$, se le llama función de riesgo empírico $R_E: \Theta \rightarrow \mathbb{R}^+$ a la función definida como:*

$$R_E(\theta) = \frac{1}{N} \sum_{i=1}^N L(f(x_i; \theta), y_i) \quad (1.1)$$

Decimos N es el tamaño del dataset o el número de ejemplos.

La función de riesgo empírico, que corresponde a la etapa de entrenamiento, busca estimar los parámetros $\hat{\theta}$ que minimicen la función. De esta forma, el problema de aprendizaje corresponde a un problema de optimización. Por tanto, se habla de la minimización del riesgo empírico.

Definición 10 (Minimización del Riesgo Empírico). *El principio de la minimización del riesgo empírico (Empirical Risk Minimization o ERM) señala que en el aprendizaje el mejor conjunto de parámetros o pesos $\hat{\theta}$ para una máquina de aprendizaje f es aquel que minimice la función de riesgo empírico. Formalmente:*

$$\hat{\theta} = \arg \min_{\theta \in \Theta} R_E(\theta) \quad (1.2)$$

Este principio es de gran relevancia para la teoría de aprendizaje, puesto que, bajo las condiciones necesarias, se puede comprobar que con el ERM podemos obtener la máquina de aprendizaje f que solucione adecuadamente el problema que nos interesa. Esto queda manifestado en el siguiente teorema:

Teorema 1. *La minimización del riesgo empírico converge en probabilidad a la minimización del riesgo de generalización. Esto es:*

$$\frac{1}{N} \sum_{i=1}^N L(f(x_i; \theta), y_i) \xrightarrow{P} \int_X L(f(x; \theta), y) d\mu(x) \quad (1.3)$$

cando $N \rightarrow \infty$.

El Teorema 1, que es elaborado en textos sobre teoría de aprendizaje como el de Vapnik (1998), apunta que con un número suficientemente grandes de datos podemos estimar, durante el entrenamiento, una función que resuelva eficientemente el problema. En este contexto, la capacidad de generalizar del problema depende del proceso de optimización en el entrenamiento. Si consideramos los teoremas de aproximador universal (Sección 3.2), podemos concluir teóricamente que podemos estimar funciones que resuelvan prácticamente cualquier problema. En la teoría de aprendizaje, sin embargo, es necesario estudiar bajo qué condiciones se puede asegurar la convergencia, así como la relación del tamaño del conjunto de datos y la cercanía de la función del riesgo empírico y el riesgo general.

Para realizar esto de manera empírica, al menos en el caso supervisado, se toma un conjunto de datos $\mathcal{S} = \{(x, y) : x \in \mathbb{R}^d, y \in Y\}$. El conjunto de datos se divide en dos partes: entrenamiento y evaluación, aunque algunas otras veces se considera también la validación. Se suele usar un esquema 70-30, que consiste en:

1. Conjunto de entrenamiento (70 %): con el que se genera el modelo de aprendizaje durante la fase de entrenamiento.
2. Conjunto de evaluación (30 %): Con el que se evalúa el modelo aprendido.
3. Conjunto de validación: Representa un 10 % de los datos; se utiliza para ajustar hiperparámetros de la máquina de aprendizaje y se suele integrar a los datos de entrenamiento, una vez validado el modelo. Se suele usar un 10 % que se toma del subconjunto de evaluación.

Este esquema será general cuando se trate de problemas supervisados. Los problemas no supervisados se tratan de forma distinta, pues suelen ser más complejos de evaluar. En la Sección 1.4.2 abordamos con más detalle las cuestiones de evaluación.

1.4.1. Entrenamiento

El proceso de entrenamiento consiste en estimar un modelo de aprendizaje; el modelo de aprendizaje es el conjunto de parámetros óptimo, que resuelva de la manera más eficiente el problema al que nos enfrentamos. Según el tipo de problema, tanto la máquina de aprendizaje como el proceso de entrenamiento pueden variar.

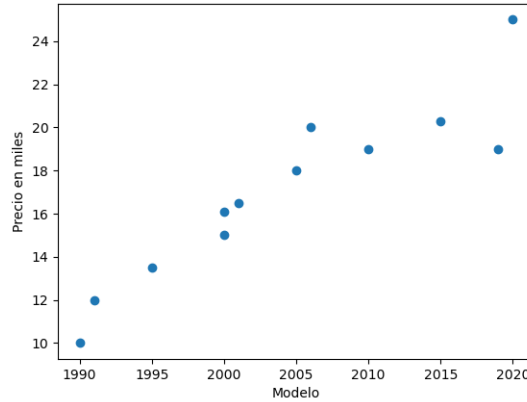


Figura 1.4: Ejemplo de regresión: estimar precios de autos por su modelo

Una distinción esencial en los problemas de aprendizaje es la de los problemas supervisados contra los no supervisados (véase Sección 1.3). Para los fines que perseguimos, será de principal importancia los problemas supervisados. Estos problemas son aquellos donde los datos son de la forma (x, y) , tal que $x \in \mathbb{R}^d$ es un ejemplo y y es su supervisión, la respuesta esperada para este caso. Distinguimos, además, dos problemas básicos dentro de los casos supervisados: la regresión y la clasificación.

Entrenamiento en regresión

La regresión busca estimar una máquina de aprendizaje $f : X \rightarrow Y$ donde $X \subseteq \mathbb{R}^d$ es el espacio de los ejemplos y $Y \subseteq \mathbb{R}^m$. Esto último nos señala que la máquina de aprendizaje busca obtener valores en un espacio continuo. Por tanto, la función de pérdida indicará qué tan lejos está la predicción de f con respecto al valor esperado y . En la regresión se predice un valor real continuo (que puede ser un intervalo o todo el conjunto de los reales), por lo que la máquina de aprendizaje no da una respuesta correcta o incorrecta en términos discretos; podemos hablar, más bien, de qué tan cercano o lejano está el valor de la máquina de aprendizaje del valor real.

Un ejemplo de regresión es el estimar el valor del precio de autos según el modelo de éstos. El entrenamiento constaría de una serie de datos (x, y) donde x es un valor que representa el modelo del auto (por ejemplo, 2000, 2005), mientras que y es el valor del precio de ese auto. En la Figura 1.4 se muestran algunos datos con los que el modelo se podría entrenar. En este ejemplo, podría pasar que un mismo modelo de automóvil tenga dos valores de precios distinto (véanse los datos con modelo 2000); podemos pensar que hay variables que no estamos tomando en cuenta (la marca, el uso). Por tanto, más que estimar el valor exacto, lo que el modelo buscará predecir es el valor esperado. En este caso, siempre habrá un margen de error entre el valor de los datos y el que la máquina de aprendizaje predice, pero buscamos que este margen de error sea el más pequeño posible.

En la regresión, usaremos una función de pérdida que nos permita determinar qué tan cercano del valor real está el valor predicho. Definimos entonces la función $L(f(x; \theta), y) = \|y - f(x; \theta)\|_2^2$. Con esta función de pérdida, nuestra función de riesgo estará definida por el **error cuadrático**:

$$R(\theta) = \frac{1}{2} \sum_{(x,y) \in \mathcal{S}} \|y - f(x; \theta)\|_2^2 \quad (1.4)$$

El objetivo del entrenamiento será minimizar esta función dada los ejemplos del conjunto de entrenamiento $\mathcal{S}$. Minimizar esta función implica minimizar el valor de la diferencia que la máquina de aprendizaje f predice con respecto a los valores reales y . Usamos la norma euclidiana ya que la regresión puede predecir vectores continuos. En el ejemplo anterior, la predicción es un escalar, por lo que esta norma sólo representa el cuadrado de la resta. En general, los problemas de regresión toman como función de riesgo el error cuadrático.

Problema de clasificación

En el problema de clasificación, la predicción es discreta. La máquina de aprendizaje debe asignar a cada dato x una clase o categoría en un conjunto finito de éstas. Es decir, la máquina de aprendizaje es una función $f : X \rightarrow Y$, donde $X \subseteq \mathbb{R}^d$ es el espacio de datos y $Y = \{0, 1, 2, \dots, M\}$ son las clases. De esta forma, la función de pérdida nos indicará en que casos la máquina de aprendizaje ha asignado la clase correcta y en qué casos ha errado en la asignación.

La forma más simple de clasificación es la clasificación binaria. En ésta, el conjunto de clases es $Y = \{0, 1\}$ y el objetivo de la máquina es asignar 1 cuando el vector de entrada pertenezca a la clase y 0 cuando no. Por ejemplo, si nuestro problema es clasificar un conjunto de síntomas dentro de una enfermedad, la máquina de aprendizaje f asignará 1 cuando los síntomas representados en el dato x correspondan a la enfermedad y 0 cuando no lo hagan. En la Figura 1.5 se puede observar la representación de un problema de clasificación: los puntos representados por vectores en $\mathbb{R}^2$ se separan en dos clases disjuntas.

Una primera intuición al respecto de la función de riesgo es que podríamos aplicar el error cuadrático medio también para los problemas de clasificación. De hecho, notamos que si la predicción de la máquina es errada, entonces $y - f(x; \theta) = 1$, mientras que si la predicción acierta, entonces $y - f(x; \theta) = 0$. Al aplicar el error cuadrático a un problema de clasificación se obtiene el número de errores que realiza f . Sin embargo, en la práctica usar el error cuadrático no suele ser eficiente. En los problemas de clasificación, los modelos de aprendizaje, tales como las redes neuronales, más que predecir si un dato de entrada pertenece o no a una clase, el modelo estima una probabilidad de que el dato pertenezca a tal clase. De tal forma que $f(x; \theta)$ es una función de probabilidad.

Abordemos primero el caso binario. En este caso, buscamos que cuando el dato x pertenece a la clase 1, la función $f(x; \theta)$ sea cercana a 1. Mientras que si el dato no pertenece, entonces $f(x; \theta)$ debe acercarse a 0, o lo que es lo mismo, $1 - f(x; \theta)$ debe ser cercano a 1. Por tanto, podemos definir la siguiente función de riesgo para

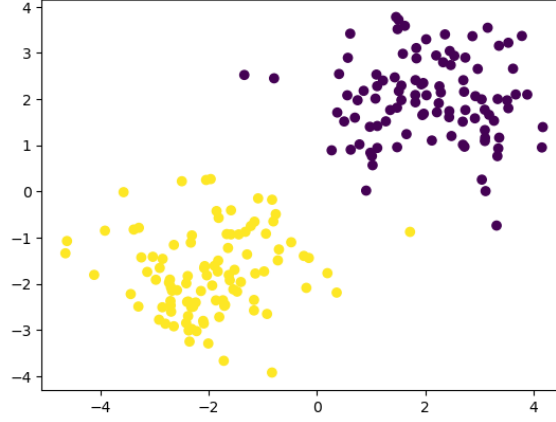


Figura 1.5: Ejemplo de un problema de clasificación binaria

clasificación binaria:

$$R(\theta) = -\left(y \ln(f(x; \theta)) + (1 - y) \ln(1 - f(x; \theta))\right)$$

Cuando tengamos la clase $y = 1$, el minimizar esta función corresponde a maximizar la función $f(x; \theta)$ gracias al símbolo negativo. Cuando $y = 0$, se maximizará $1 - f(x; \theta)$ que corresponde a minimizar $f(x; \theta)$.

Ahora, en el caso general, cuando contamos con M clases, podemos usar un principio similar. Buscaremos que cuando un elemento pertenezca a una clase, la probabilidad $f(x; \theta)$ sobre esta clase se maximice. Generalmente, en la clasificación multi-clase $f(x; \theta)$ corresponde a un vector de probabilidades, donde cada entrada $f_i(x; \theta)$ es la probabilidad de la clase i . Por tanto, la función de riesgo que definiremos estará dada como:

$$R(\theta) = -\sum_x \sum_i \delta_{x,i} \ln f_i(x; \theta) \quad (1.5)$$

donde $\delta_{x,i}$ es una función indicadora definida como:

$$\delta_{x,i} = \begin{cases} 1 & \text{si } x \text{ pertenece a la clase } i \\ 0 & \text{en otro caso} \end{cases}$$

A esta función se le llama la **entropía cruzada**. En general, esta función será central en las redes neuronales, y la revisaremos con más detalle cuando abordemos el entrenamiento en éstas.

Generalización de la función de riesgo

Watanabe (2009) propone una función de riesgo general en la teoría de aprendizaje estadístico. En la teoría de aprendizaje estadístico, se asume que los datos son generados por una distribución de probabilidad $p(y, x)$. El objetivo del aprendizaje es encontrar

una función $f(x; \theta)$ que sea lo más cercana posible a la función de probabilidad real $p(y, x)$. Una forma de medir la cercanía entre distribuciones de probabilidad es usando la **divergencia de Kullback-Liebler** o divergencia KL. Ésta se define como:

$$D[p||f] = \mathbb{E}_p \left[\ln \frac{p(y, x)}{f(x; \theta)} \right] \quad (1.6)$$

La divergencia KL cumple que siempre es mayor a 0, y es 0 cuando $p \equiv f$. De tal forma que la divergencia KL nos dice qué tanto se aleja la estimación f del valor real de la distribución de los datos p . La divergencia KL se usa como función de riesgos en algunos modelos neuronales, como las *Generative Adversarial Networks* (GANs) en su versión más simple. Sin embargo, en otros casos es difícil de implementar debido a que la distribución real p , no es conocida.

Para lidiar con esto, se aprovechan resultados teóricos que nos llevan a proponer las funciones de riesgo de entropía cruzada y error cuadrático medio para los problemas de clasificación y regresión, respectivamente. La entropía cruzada, en su forma general, se define como:

$$H(f) = -\mathbb{E}_p \left[\ln f(x; \theta) \right] \quad (1.7)$$

Podemos comprobar que esta definición de entropía cruzada puede darse en términos de la divergencia de KL y de la **entropía de Shannon**, la cuál es una medida de información definida como:

$$H(Y, X) = -\mathbb{E} \left[\ln p(y, x) \right]$$

Lema 1. La entropía cruzada $H(f)$ se define como:

$$H(f) = H(Y, X) + D[p||f]$$

Demostración. Sea p una distribución sobre X , y sea f una distribución empírica también sobre X . Entonces:

$$\begin{aligned} H(f) &= -\mathbb{E}_p \left[\ln f(x; \theta) \right] \\ &= \mathbb{E}_p \left[\ln \frac{1}{f(x; \theta)} \right] \\ &= \mathbb{E}_p \left[\ln \frac{1}{f(x; \theta)} \right] + \left(\mathbb{E}_p \left[\ln p(y, x) \right] - \mathbb{E}_p \left[\ln p(y, x) \right] \right) \\ &= \left(\mathbb{E}_p \left[\ln \frac{1}{f(x; \theta)} \right] + \mathbb{E}_p \left[\ln p(y, x) \right] \right) - \mathbb{E}_p \left[\ln p(y, x) \right] \\ &= \mathbb{E}_p \left[\ln \frac{1}{f(x; \theta)} + \ln p(y, x) \right] - \mathbb{E}_p \left[\ln p(y, x) \right] \\ &= \mathbb{E}_p \left[\ln \frac{p(y, x)}{f(x; \theta)} \right] - \mathbb{E}_p \left[\ln p(y, x) \right] \\ &= D[p||f] + H(Y, X) \end{aligned}$$

□

Con base en este resultado, podemos obtener la equivalencia de minimizar la entropía cruzada y la divergencia KL.

Teorema 2. *Minimizar la entropía cruzada minimiza la divergencia KL. Esto es:*

$$\arg \min_{\theta} \mathbb{E}_p \left[\ln \frac{p(y, x)}{f(x; \theta)} \right] = \arg \min_{\theta} \mathbb{E}_p [- \ln f(x; \theta)]$$

Demostración. La demostración usa el Lema 1 y parte del hecho de que la entropía de Shannon no depende de los parámetros θ , sólo la entropía cruzada depende de éstos; por tanto, el argumento que minimiza sólo se busca en la entropía cruzada. Desarrollando:

$$\begin{aligned} \arg \min_{\theta} \mathbb{E}_p \left[\ln \frac{p(y, x)}{f(x; \theta)} \right] &= \arg \min_{\theta} \left(\mathbb{E}_p [\ln p(y, x)] - \mathbb{E}_p [\ln f(x; \theta)] \right) \\ &= \mathbb{E}_p [\ln p(y, x)] - \arg \min_{\theta} \mathbb{E}_p [\ln f(x; \theta)] \\ &= \arg \min_{\theta} \mathbb{E}_p [- \ln f(x; \theta)] \end{aligned}$$

□

Podemos, entonces, enfocarnos únicamente en la entropía cruzada. De hecho, dentro de las redes neuronales, la entropía cruzada será la función de riesgo de la que generalmente partiremos. Sin embargo, al no conocer la distribución real de las clases $p(y_i, x)$, podemos utilizar la función indicadora $\delta_{i,x}$ (notando que $\mathbb{E}[\delta_{i,x}] = p(y_i, x)$). En los problemas de clasificación, siempre buscaremos minimizar la entropía cruzada. En los problemas de regresión, minimizamos el error cuadrático. En el siguiente resultado podemos ver que esto equivale a minimizar la entropía cruzada (bajo ciertas condiciones).

Proposición 1. *Asúmese que la distribución de $p(y, x)$ es normal con media $f(x; \theta)$ y varianza 1, entonces:*

$$\arg \min_{\theta} \mathbb{E} [- \ln p(y, x)] = \arg \min_{\theta} \frac{1}{2} \mathbb{E} [\|y - f(x; \theta)\|_2^2]$$

Demostración. Si $p(y, x)$ tiene distribución normal con media $f(x; \theta)$ y varianza 1, entonces:

$$p(y, x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2} \|y - f(x; \theta)\|_2^2}$$

Si minimizamos la entropía cruzada con respecto a esta distribución, obtenemos:

$$\begin{aligned} \arg \min_{\theta} \mathbb{E} [- \ln p(y, x)] &= \arg \min_{\theta} - \mathbb{E} \left[\ln \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2} \|y - f(x; \theta)\|_2^2} \right] \\ &= \arg \min_{\theta} - \left(\mathbb{E} \left[\ln \frac{1}{\sqrt{2\pi}} \right] + \mathbb{E} \left[\ln e^{-\frac{1}{2} \|y - f(x; \theta)\|_2^2} \right] \right) \\ &= \ln \sqrt{2\pi} - \arg \min_{\theta} \mathbb{E} \left[-\frac{1}{2} \|y - f(x; \theta)\|_2^2 \right] \\ &= \arg \min_{\theta} \frac{1}{2} \mathbb{E} [\|y - f(x; \theta)\|_2^2] \end{aligned}$$

□

En la regresión, los valores que se pueden estimar por una máquina de aprendizaje, como una red neuronal, no son valores precisos, sino los valores esperados o medias. Del Teoremas 2 y la Proposición 1 podemos ver que la divergencia KL es un caso general de la entropía cruzada, y a su vez la entropía cruzada generaliza el error cuadrático medio. En lo que sigue usaremos la entropía cruzada extensivamente como función de riesgo en las redes neuronales para clasificación; por su parte, el error cuadrático medio se usará en problemas de regresión.

1.4.2. Evaluación

La evaluación es un proceso fundamental en toda metodología de aprendizaje automático. El proceso de evaluación consiste en determinar la capacidad de generalización de la máquina de aprendizaje. Como no es factible evaluar las predicciones de la máquina de aprendizaje sobre todos los datos, se utilizan estrategias estadísticas para evaluar la calidad de las predicciones. La evaluación también depende del tipo de problema. Pasamos a revisar los tipos de evaluación más comunes para regresión y clasificación. Finalmente, revisamos de forma breve la evaluación en problemas no supervisados.

Evaluación de regresión

En los problemas de regresión, se busca estimar un valor esperado dado un dato de entrada x . El valor $f(x)$ es la media de los valores y . Como tenemos un valor continuo, la evaluación debe determinar el error en términos de qué tan alejado está el valor predicho del valor real. Una forma simple de hacer esto es por medio del *Mean Absolute Error*.

Definición 11 (Mean absolute error). *El error absoluto medio (o Mean Absolute Error) de las predicciones de una función $f(x)$ sobre un conjunto de datos de evaluación $\{(x, y) : x \in \mathbb{R}^d, y \in \mathbb{R}\}$ está dado como:*

$$MAE = \frac{1}{N} \sum_{(x,y)} |y - f(x)| \quad (1.8)$$

Esta función de evaluación determina el promedio del error que comete la máquina de aprendizaje f . Otra forma de determinar el error, es a partir del error cuadrático medio.

Definición 12 (Mean Squared Error). *El error cuadrático medio (o Mean Squared Error) de las predicciones de una función $f(x)$ sobre un conjunto de datos de evaluación $\{(x, y) : x \in \mathbb{R}^d, y \in \mathbb{R}\}$ está dado como:*

$$MSE = \frac{1}{N} \sum_{(x,y)} (y - f(x))^2 \quad (1.9)$$

Tanto el error absoluto medio como el error cuadrático medio son métricas que toman valores desde 0 a infinito. Claramente, si la máquina no comete errores, cualquiera

de las métricas dadas será igual a 0. Sin embargo, cuando la máquina de aprendizaje comete errores, en ambos casos es difícil comparar la capacidad de una misma máquina de aprendizaje sobre diferentes problemas, pues los valores no están acotados superiormente. Para lidiar con esto, se propone una métrica de evaluación que toma valores entre -1 y 1.

Definición 13 (Coeficiente de determinación). *El coeficiente de determinación o puntaje R^2 sobre las predicciones de una función $f(x)$ sobre un conjunto de datos de evaluación $\{(x, y) : x \in \mathbb{R}^d, y \in \mathbb{R}\}$ está dado como:*

$$R^2 = 1 - \frac{\sum (y - f(x))^2}{\sum (y - \hat{\mu}_y)^2} \quad (1.10)$$

donde $\hat{\mu}_y = \frac{1}{N} \sum y$ es la media empírica de la variable Y .

El coeficiente de determinación se interpreta de forma distinta a las métricas de errores. Si la máquina de aprendizaje no comete errores, entonces $R^2 = 1$. Entre más errores cometa, entonces el coeficiente será más cercano a 0 o será negativo. Aunque debe observarse que el hecho de que sea 0 no implica que el error sea máximo, el error será 0 siempre que el valor de $f(x)$ sea similar al valor medio de y .

Ejemplo 2. *Para ejemplificar el uso de estas métricas consideremos que se tiene una máquina de aprendizaje que predice como se muestra en el Cuadro 1.1. Sabiendo que*

y	$f(x)$
1	1
2	3
2.5	2
3	4
1.5	1

Cuadro 1.1: Ejemplo de los valores reales y de regresión y los valores predichos $f(x)$

la media empírica sobre Y es $\hat{\mu}_y = \frac{1}{5}(1 + 2 + 2.5 + 3 + 1.5) = \frac{10}{5} = 2$, obtenemos los resultados del Cuadro 1.2.

MAE	MSE	R^2
0.6	0.5	0

Cuadro 1.2: Métricas de evaluación de regresión

Se puede observar que en este ejemplo el valor de R^2 es 0, aunque los errores MAE y MSE no son necesariamente altos.

Evaluación de clasificación

En la clasificación, se estiman valores de clases. Estos valores son discretos y generalmente representan valores $0, 1, 2, \dots, M - 1$. El objetivo de la evaluación es determinar

qué tan bien la máquina de aprendizaje puede aprender a clasificar los datos en sus clases correctas. La forma más simple de hacer esto es obteniendo el valor promedio de los aciertos. A este tipo de métrica se le llama exactitud o *Accuracy*. Sin embargo, la información que aporta la métrica de exactitud no es suficiente. Muchas veces queremos saber qué tan preciso resulta el modelo para una clase, o qué tantos elementos de una clase puede clasificar adecuadamente.

Para lidiar con esto, en la clasificación se suelen utilizar otro tipo de métricas. En particular, se suele usar una matriz de confusión.

Definición 14 (Matriz de confusión binaria). *Una matriz de confusión para un problema de clasificación binaria es una matriz de 2×2 que se representa como:*

<i>Real / Predicha</i>	<i>Positiva</i>	<i>Negativa</i>
<i>Positiva</i>	Verdaderos positivos (TP)	Falsos negativos (FN)
<i>Negativa</i>	Falsos positivos (FP)	Verdaderos negativos (TN)

Las columnas corresponden a la clase predicha por la máquina de aprendizaje, y los renglones corresponden a las clases reales.

En la matriz de confusión, se pone el número de valores que coinciden entre las predicciones de la máquina de aprendizaje y los valores reales. Estos se basan en una clase (generalmente la clase 1): los valores positivos son las predicciones de esta clase, y los negativos son las predicciones sobre la clase contraria (la clase 0). Con respecto a esto, se tienen 4 casos:

1. **Verdaderos positivos:** Los verdaderos positivos son aquellos valores que realmente pertenecen a la clase positiva y que la máquina dijo que pertenecían a esta clase.
2. **Verdaderos negativos:** Son aquellos valores que pertenecían a la clase negativa y que la máquina clasificó en esa clase.
3. **Falsos positivos:** Son los errores en que la clase negativa real es clasificada por la máquina como positivos. También se llama **error de tipo II**.
4. **Falsos negativos:** Son los errores en que la clase positiva real es clasificada por la máquina como negativos. También se llama **error de tipo I**.

Como se puede observar, los aciertos están en la diagonal de la matriz. Con base en la matriz de confusión podemos calcular diferentes tipos de métricas de evaluación.

Definición 15 (Precisión). *La métrica de precisión se define sobre la clase positiva (1) como:*

$$Prec(y = 1) = \frac{TP}{TP + FP}$$

Corresponde a la columna positiva de la matriz de confusión.

En contraste a la precisión, se tiene la medida de exhaustividad:

Definición 16 (Exhaustividad). *La métrica de exhaustividad o recall se define sobre la clase positiva (1) como:*

$$Recc(y = 1) = \frac{TP}{TP + FN}$$

Corresponde al renglón positivo de la matriz de confusión.

La precisión y la exhaustividad suelen ser complementarias. Un modelo con alta precisión puede llegar a tener baja exhaustividad y viceversa. Si tanto la exhaustividad como la precisión son altas, el modelo ha aprendido adecuadamente. Para evaluar la precisión y la exhaustividad de manera simultánea, se usa la media geométrica entre ambas métricas.

Definición 17 (Medida F_1). *La medida F_1 es la media geométrica de la precisión y la exhaustividad, determinada de la siguiente forma:*

$$F_1(y = 1) = 2 \cdot \frac{Prec(y = 1) \cdot Recc(y = 1)}{Prec(y = 1) + Recc(y = 1)}$$

Es común reportar estas medidas cuando se tiene una clasificación binaria, además de la exactitud. En términos de la matriz de confusión, podemos entender la **exactitud** de la siguiente forma:

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN}$$

Ejemplo 3. *Supongamos que, después de entrenar, obtenemos un modelo que hace las predicciones $\hat{y}$, mientras que las predicciones reales son y . Entonces, para cada ejemplo se tienen los resultados mostrados en el Cuadro 1.3.*

Ejemplo	0	1	2	3	4	5	6	7	8	9
y	0	0	0	0	0	1	1	1	1	1
$\hat{y}$	0	1	0	1	0	1	1	1	1	0

Cuadro 1.3: Ejemplo de resultados de predicciones $\hat{y}$, comparados con valores reales y

Para el ejemplo 1, la clase real es 0 y la predicción es acertada, pues predice la clase 0. En el ejemplo 2, la clase real es 0, pero la máquina de forma errónea le asigna un 1. Con base en estos resultados contruimos la matriz de confusión.

<i>Real / Predicha</i>	<i>Positiva</i>	<i>Negativa</i>
<i>Positiva</i>	4	1
<i>Negativa</i>	2	3

A partir de esta matriz de confusión podemos calcular las diferentes métricas de evaluación. En el caso de la precisión tenemos:

$$Prec(y = 1) = \frac{4}{6} = \frac{2}{3} \approx 0,66$$

Por su parte, la exhaustividad es:

$$Recc(y = 1) = \frac{4}{5} = 0,8$$

Por tanto, podemos calcular la métrica F_1 de la siguiente forma:

$$F_1(y = 1) = 2 \cdot \frac{0,8 \cdot 0,66}{0,8 + 0,66} \approx \frac{1,066}{1,46} = 0,73$$

Esta métrica nos dice de forma más global qué tan bien lo está haciendo el modelo. Podemos contrastarla con la métrica de exactitud que también es global.

$$Accuracy = \frac{7}{10} = 0,7$$

La medida de exactitud se puede interpretar que el modelo acierta el 70 % de las veces. Como vemos, además el valor es cercano al de F_1 .

Para generalizar la evaluación a diversas clases, primero que nada debemos mencionar que la evaluación se hace sobre cada una de las clases. En el caso binario, las métricas se realizan sobre la clase 1, que se llama clase positiva. En este caso, la clase de interés variará y las métricas se aplicarán en cada clase. En primer lugar, generalizamos el concepto de matriz de confusión.

Definición 18 (Matriz de confusión). Una matriz de confusión M para clasificación n -aria es una matriz de $n \times n$ que se define como:

$$M_{i,j} = |Y_i \cap \hat{Y}_j|$$

donde $Y_i = \{y : y = i \text{ es clase real}\}$ y $\hat{Y}_j = \{\hat{y} : \hat{y} = f(x) = j\}$.

Es decir, la matriz de confusión contiene en cada entrada el número de valores que se intersectan entre las diferentes clases reales y las clases predichas por la máquina de aprendizaje. Puede verse que en el caso en que las clases sean $n = 2$ tenemos la matriz binaria. De igual forma, podemos generalizar las métricas de evaluación.

Definición 19 (Precisión). La precisión de un modelo de clasificación n -aria para la clase k se define como:

$$Prec(y = k) = \frac{M_{k,k}}{\sum_j M_{k,j}}$$

Definición 20 (Exhaustividad). La exhaustividad de un modelo de clasificación n -aria para la clase k se define como:

$$Recc(y = k) = \frac{M_{k,k}}{\sum_i M_{i,k}}$$

Finalmente, tenemos también la medida F_1 para cada una de las clases definida de forma similar al caso binario.

Definición 21 (Medida F_1). La medida F_1 para un modelo de clasificación n -aria para la clase k se define como:

$$F_1(y = k) = 2 \frac{Prec(y = k)Recc(y = k)}{Prec(y = k) + Recc(y = k)}$$

Ejemplo 4. Supóngase que se tiene un modelo $f(x)$ que realiza las predicciones $\hat{y}$ para un conjunto de datos con clases reales y . En el Cuadro 1.4 se muestran los resultados obtenidos con este modelo de aprendizaje.

Ejemplo	0	1	2	3	4	5	6	7	8	9
y	0	0	0	1	1	1	1	2	2	2
$\hat{y}$	2	0	0	0	1	2	0	1	2	2

Cuadro 1.4: Ejemplo de resultados de predicciones $\hat{y}$, comparados con valores reales y para una predicción multiclase

De estas predicciones, obtenemos la matriz de confusión de 3×3 que se muestra en el Cuadro 1.5. Por ejemplo, para la clase real 0, podemos observar que tuvo 2 aciertos, pero que uno de los datos que pertenecían a esta clase, el modelo lo asignó como parte de la clase 2, por lo que este 1 se coloca en el último renglón de la columna correspondiente a esta clase. Puede verse que aquí los aciertos se encuentran en la diagonal, mientras que los errores son los números fuera de ésta.

Real / Predicha	Clase 0	Clase 1	Clase 2
Clase 0	2	2	0
Clase 1	0	1	1
Clase 2	1	1	2

Cuadro 1.5: Matriz de confusión para un ejemplo multiclase

A partir de esta matriz de confusión podemos obtener las métricas de evaluación para cada clase. Por ejemplo, para la clase 0 tenemos:

$$Prec(y=0) = \frac{2}{3}$$

Y la exhaustividad está dada como:

$$Recc(y=0) = \frac{2}{4} = \frac{1}{2}$$

Podemos observar que la precisión toma el valor de la diagonal y divide entre los valores de la columna, mientras que la exhaustividad divide entre la suma de los valores del renglón.

Evaluación no supervisada

La evaluación de un conjunto no supervisado presenta la dificultad de no contar con un indicador de cuáles son los valores óptimos que debe obtener la máquina de aprendizaje. Para realizar una evaluación de un modelo no supervisado, se suele definir un *gold-standard*.

Definición 22 (Gold-standard). Un *gold-standard* es un conjunto $\hat{C} = \{\hat{c}_1, \dots, \hat{c}_N\}$ donde cada $\hat{c}_i$ es una posible solución al problema; por ejemplo, un grupo óptimo para una agrupación de los datos.

El *gold-standard* funciona como una especie de supervisión, y muchas veces no es posible obtenerlo. Se utiliza principalmente en problemas de agrupamiento. A partir de éste se pueden definir métricas para la evaluación de modelos no supervisados.

Definición 23 (Pureza). Sea $C = \{c_1, c_2, \dots, c_N\}$ el conjunto de grupos (clústers) realizado por un algoritmo no supervisado de agrupamiento; y sea $\hat{C} = \{\hat{c}_1, \dots, \hat{c}_N\}$ el *gold-standard*. Entonces, definimos la pureza como:

$$\text{Purity}(C, \hat{C}) = \frac{1}{N} \sum_{i=1}^N \max_j |\hat{c}_i \cap c_j|$$

La pureza determina qué tan similares son los grupos del modelo con respecto a los del *gold-standard*. También podemos definir la métrica de información mutua.

Definición 24 (Información mutua). Sea $C = \{c_1, c_2, \dots, c_N\}$ el conjunto de grupos (clústers) realizado por un algoritmo no supervisado de agrupamiento; y sea $\hat{C} = \{\hat{c}_1, \dots, \hat{c}_N\}$ el *gold-standard*. Asúmase que se tiene la distribución conjunta $p(\hat{c}_i \cap c_j)$, definimos la información mutua entre ambos conjuntos como:

$$MI(C, \hat{C}) = \sum_i \sum_j p(\hat{c}_i \cap c_j) \ln \frac{p(\hat{c}_i \cap c_j)}{p(\hat{c}_i)p(c_j)}$$

Ejemplo 5. Supóngase que se tiene un modelo que realiza el agrupamiento de un conjunto de datos en tres grupos como se muestra en la Figura 1.6. El *gold-standard*

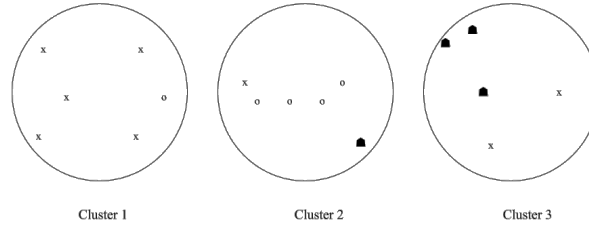


Figura 1.6: Ejemplo de agrupamiento en 3 grupos. El *gold-standard* está definido por las formas de los puntos

está determinado por las formas de los datos, que también define tres grupos. De esta forma, podemos calcular la pureza: ésta nos dice que para el primer clúster del modelo c_1 tenemos que determinar las intersecciones con el *gold-standard*. En la figura, el clúster 1 interseca con el grupo formado por x y con los círculos, mientras que la intersección con el grupo de las figuras en negro es nula. Por tanto, la cardinalidad de las intersecciones es 4, 1 y 0, respectivamente. El máximo es, por tanto, 4. Procediendo de esta forma obtenemos la pureza:

$$\begin{aligned} \text{Purity}(C, \hat{C}) &= \frac{1}{17} (4 + 4 + 3) \\ &= \frac{10}{17} = 0,588 \end{aligned}$$

Para calcular la información mutua, tenemos que estimar las probabilidades conjuntas entre los grupos del modelo y del gold-standard. Si tomamos como $\hat{c}_1$ al grupo x , tenemos que la probabilidad frecuentista es $p(\hat{c}_1 \cap c_1) = \frac{5}{16}$. Asimismo, notamos que si $\hat{c}_3$ es el grupo conformado por los puntos en negro, notamos que $p(\hat{c}_3 \cap c_1) = 0$. Procediendo de esta forma, obtenemos que $MI(C, \hat{C}) \approx 0,41$.

1.4.3. Subajuste y sobreajuste

Dentro de la teoría de aprendizaje tiene especial relevancia estudiar la relación entre el error de entrenamiento y el error de generalización. Garantizar que la diferencia entre estos errores sea pequeña determina la calidad del aprendizaje, pues esto determina que un buen proceso de entrenamiento conlleva una buena generalización del modelo.

Definición 25 (Capacidad de generalización). *Dado un riesgo de entrenamiento $R(\theta)$ y un riesgo de generalización $R_G(\theta)$, la capacidad de generalización del modelo $f(x; \theta)$ que depende del conjunto de parámetros θ está determinado por:*

$$\sup_{\theta \in \Theta} |R_G(\theta) - R(\theta)|$$

Bajo este criterio, podemos hablar de tres posibles casos que determinan dos conceptos importantes en la teoría de aprendizaje: el subajuste y sobreajuste.

Definición 26 (Subajuste). *El subajuste (o underfitting) se da cuando el error de entrenamiento es alto y, por tanto, también es alto el error de generalización bajo el modelo obtenido.*

Bajo el subajuste, la capacidad de generalización no es buena, pero esto se debe a que el modelo no puede aprender adecuadamente y esto se refleja también en el entrenamiento. El subajuste se da, por lo general, cuando se tiene un modelo de poca capacidad, es decir, que no es capaz de ajustarse a los datos. Por ejemplo, en la Figura 1.7 se muestra el caso del subajuste, donde por medio de un modelo lineal (estimación por medio de una recta) se busca aproximar en un problema de regresión de datos que no tienen una dependencia lineal. Por tanto, para corregir el subajuste se suele buscar un modelo de mayor capacidad (en el caso del ejemplo, un modelo polinomial).

Otro caso se da cuando la capacidad de generalización es mala, pero no se puede ver durante el entrenamiento.

Definición 27 (Sobreajuste). *El sobreajuste (u overfitting) se da cuando el error de entrenamiento es bajo, pero el error de generalización es alto.*

El sobreajuste, de forma metafórica, se da cuando el modelo de aprendizaje memoriza sobre los datos de entrenamiento, pero no es capaz de generar un conocimiento que le permita generalizar. En la Figura 1.7 se puede ver que un modelo de gran capacidad se ajusta de manera muy precisa a los datos que observa durante en el entrenamiento, pero precisamente por esto, su capacidad de generalizar se ve limitada. Para corregir el sobreajuste se proponen varias perspectivas:

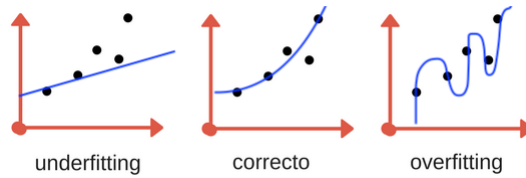


Figura 1.7: Ejemplos de overfitting y underfitting en un problema de regresión

- Utilizar un modelo de aprendizaje de menor capacidad, de tal forma que su capacidad de ajustarse a los datos de entrenamiento se reduzca. Esto implica la suposición de que el problema tiene una solución más sencilla; por ejemplo, si los datos tienen una dependencia lineal, utilizar un modelo de regresión polinomial podría llevar al sobreajuste, por lo que es preferible usar un modelo lineal.
- Aumentar el número de datos de tal forma que el modelo pueda observar mejor el comportamiento del problema (véase la Sección 1.5.3). Aumentar el número de datos permite que un modelo pueda observar mejor la distribución de los datos. Hay una relación fuerte entre el número de datos y la capacidad del modelo. Generalmente, cuando se cuenta con pocos datos se prefiere un modelo de baja capacidad para evitar el sobreajuste, mientras que un modelo de alta capacidad requiere de una mayor cantidad de datos.
- Para evitar el sobreajuste en las redes neuronales es común utilizar diferentes estrategias que caen bajo el concepto de regularización. La regularización será revisada en el Capítulo 5.

Finalmente, hablamos de una capacidad de generalización correcta cuando tanto el error de entrenamiento como el de generalización son ambos buenos (cerca de 0). Poder garantizar que un modelo de aprendizaje tenga una buena capacidad de generalización al tener un error de entrenamiento bajo es una parte importante de la teoría de aprendizaje (véase Vapnik (1998)).

1.5. Representación de datos

Una máquina de aprendizaje es una función que toma de entrada una representación de los datos que, generalmente, se da de forma tensorial; es decir, como un arreglo numérico. El tipo de representación que obtendremos de los datos depende en gran medida de las características de estos. En particular, se tienen dos tipos generales de datos: datos estructurados y datos no estructurados.

Definición 28 (Datos estructurados). *Los datos estructurados son aquellos datos que se encuentran bien organizados bajo criterios específicos, a los cuales es fácil de acceder, así como son sencillos de procesar por una computadora.*

Los datos estructurados pueden encontrarse en base de datos, hojas de cálculo o formularios. Cuando contamos con datos estructurados, podemos representarlos en vectores en $\mathbb{R}^d$. Cada una de las dimensiones del vector representa una de las celdas de la tabla de datos. Por ejemplo, supóngase que se tiene la siguiente tabla de información:

Dato	Edad	Peso	Altura
1	35	74.3	175
2	27	63	172

Estos datos se encuentran organizados y son fáciles de interpretar por una computadora. La información cuenta con tres campos ‘edad’, ‘peso’ y ‘altura’, por lo que se representarán en vectores de 3 dimensiones. Por ejemplo, el dato 1 se representa como el vector:

$$x = \begin{pmatrix} 35 \\ 74.3 \\ 175 \end{pmatrix}$$

En una representación vectorial se asume que las entradas de los vectores son variables aleatorias. Cada vector $x \in \mathbb{R}^d$ es un vector aleatorio cuyas entradas son valores de las variables $X_1, \dots, X_d$. Se asume que las variables son independientes. De esta forma:

$$x^T = (X_1 = x_1 \quad \dots \quad X_d = x_d)$$

Cada valor que toma una variable $X_i = x_i$ se conoce como **rasgo** (o *feature*). Al espacio de los vectores de los datos $X \subseteq \mathbb{R}^d$ se le conoce como **espacio de rasgos** (*feature space*). De tal forma, un conjunto de datos es el muestreo de un fenómeno que presenta valores específicos para las variables aleatorias definidas. En general, el conjunto de datos se representa como una matriz $X \in \mathbb{R}^{n \times d}$, donde cada uno de los n renglones es un dato (vector) con d dimensiones.

Hemos tomado un ejemplo en que los datos están en principio en un formato numérico. Sin embargo, se pueden presentar otros casos en que se tengan datos sin un formato numérico, datos categóricos.

Definición 29 (Datos categóricos). *Los datos categóricos son variables que se identifican por medio de cadenas de texto: nombres o categorías.*

Los datos categóricos pueden ser clases cerradas, como un conjunto de categorías de nivel educativo, o clases abiertas, como nombres propios. En general, no existe un orden para este tipo de datos, mas que el orden lexicográfico que, en la mayoría de los casos, no es informativo para los modelos de aprendizaje. Podemos considerar el siguiente ejemplo:

Dato	¿Alergias?	Nivel educativo	Nombre
1	Sí	Primaria	Pedro
2	No	Licenciatura	María

En este caso, los datos estructurados no presentan características numéricas. La primera variable ‘¿alergias?’ es, de hecho, una variable booleana pues puede tomar

únicamente dos valores: ‘sí’ y ‘no’. En este caso, su representación numérica se basa en los valores booleanos: 1 para ‘sí’ y 0 para ‘no’.

La variable ‘nivel educativo’ posee una estructura jerárquica, que va de un nivel más bajo hacia un nivel más alto. Podemos considerar varios niveles y a cada nivel asignarle un nivel ascendente:

Pre-escolar	Primaria	Secundaria	Preparatoria	Licenciatura	Posgrado
1	2	3	4	5	6

De esta forma, la representación numérica corresponde al valor numérico asignado previamente.

Finalmente, la variable de ‘nombre’ contiene un número abierto de valores que representan el conjunto de posibles nombres. En este caso, su representación es problemática, pero dentro del aprendizaje profundo pueden representarse de manera sencilla los datos de entrada para después aprender una representación más efectiva. En particular, dado un conjunto de valores $\Omega = \{\omega_1, \dots, \omega_N\}$, se asocia a cada objeto con un índice $i \in I \subseteq \mathbb{N}$ y se genera un vector **one-hot** que representa al objeto ω_i con índice i ; las entradas del vector one-hot se definen como:

$$x_j = \begin{cases} 1 & \text{si } i = j \\ 0 & \text{en otro caso} \end{cases}$$

Las redes neuronales pueden aprender una función que tome este vector indexal y lo represente en un vector de rasgos distribuido (con entradas continuas); a este tipo de representaciones se les conoce como **embeddings**. La asignación de los índices es arbitraria, y se asocia a cada uno de los valores categóricos.

En el ejemplo anterior, tendríamos un índice para ‘Pedro’ y otro para ‘María’. La dificultad es que esta representación está limitada a los valores observados en el conjunto de datos. Se puede utilizar una etiqueta especial para aquellos casos que no son observados en el conjunto de datos de entrenamiento. De esta forma, la representación de los datos del ejemplo anterior sería la siguiente:

Dato	¿Alergias?	Nivel educativo	Pedro	María
1	1	2	1	0
2	0	5	0	1

Cada dato está representado por un vector de 4 dimensiones. Un conjunto de datos puede contener variables de diferentes tipos, numéricas o categóricas. La representación vectorial de estos datos requieren de un análisis de las variables que los representan. El aprendizaje profundo permite tener representaciones más sencillas, ya que éste es parte del aprendizaje representacional; es decir, aprende representaciones (en las capas ocultas) que permiten solucionar el problema. Por esto mismo, el aprendizaje profundo presenta grandes ventajas para trabajar con datos no estructurados.

1.5.1. Datos no estructurados

Las redes neuronales surgieron como soluciones para problemas sobre datos estructurados (perceptrón y redes feedforward). Sin embargo, su flexibilidad ha permitida

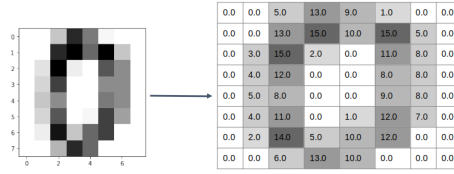


Figura 1.8: Codificación de una imagen en escala de grises

adaptarlas a problemas para datos con estructuras más complejas, los datos no estructurados.

Definición 30 (Datos no estructurados). *Los datos no estructurados son aquellos datos que carecen de organización y estructura aparente.*

El nombre de ‘no estructurado’ podría hacernos pensar que estos datos no tienen estructura. Más bien, este tipo de datos cuenta con una estructura subyacente que no es comprensible (sin procesamiento) por una computadora. Ejemplos típicos de datos no estructurados son el texto y las imágenes. También pueden presentarse otros tipos de datos en que un conjunto de elementos se relacionan entre sí para conformar una elemento mayor. Estos datos pueden representarse por gráficas representacionales.

Cada uno de estos tipos de datos tiene un manejo particular: se representan mediante diferentes relaciones. Asimismo, se suelen procesar por medio de diferentes redes neuronales. Por ejemplo, las imágenes se procesan por medio de redes convolucionales (Capítulo 7), mientras que el texto se puede procesar por medio de redes recurrentes o transformers (Capítulos 6 y 8).

Imágenes

Las imágenes se representan por medio de píxeles que son vectores que codifican los colores. Una imagen en escala de grises puede codificarse por medio de una valor escalar. Una imagen con colores se representa por medio de los valores de rojo, verde y azul o RGB. Llamaremos a cada uno de estos valores de los tres colores un canal. Un píxel puede tener cuatro canales en RGB- α , donde además del rojo, verde y azul se considera un valor α que codifica la transparencia del píxel. Por tanto, cada píxel será un vector en $\mathbb{R}^C$ donde C es el número de canales (1, 3 ó 4). Una imagen será, entonces, una arreglo rectangular de píxeles. Cada imagen tendrá una altura H y una anchura W . Por tanto una imagen será un dato $x \in \mathbb{R}^{H \times W \times C}$.

En la Figura 1.8 se muestra un ejemplo con una imagen en escala de grises. Esta imagen se codifica por medio de un canal; es decir, $C = 1$ y tiene una altura de 8 y anchura también de 8 píxeles. Por tanto, es un dato en $8 \times 8 \times 1$. Una imagen con un mayor número de canales implicaría que cada píxel fuera un vector de mayor dimensionalidad. Las imágenes deben ser procesadas conservando la información de la estructura de los píxeles, pues estos no son independientes, si no que dependen de los píxeles que están cercanos a ellos. Una primera aproximación a su procesamiento es el aplanamiento (*flattening*).

Definición 31 (Flattening). *El flattening o aplanamiento es el mapeo de un tensor $x \in \mathbb{R}^{H \times W \times C}$ hacia un vector $\text{flat}(x) \in \mathbb{R}^{HWC}$ de dimensión igual al producto $d = HWC$.*

Ejemplo 6. *Supóngase que se tiene una imagen de un sólo canal (en escala de grises) de tamaño 3×3 (altura y anchura). El aplanamiento de esta imagen será un vector de dimensión $d = 9$.*

$$\text{flat} \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} = \begin{pmatrix} x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ x_{2,1} \\ x_{2,2} \\ x_{2,3} \\ x_{3,1} \\ x_{3,2} \\ x_{3,3} \end{pmatrix}$$

A partir del procesamiento de aplanamiento se puede aplicar un método de aprendizaje automático tradicional. Sin embargo, este tipo de métodos no son capaces de determinar la estructura subyacente de los datos, pues se asume que las entradas de este vector aplanado son independientes. Dentro del aprendizaje profundo se proponen las redes convolucionales para procesar de manera eficiente las imágenes. Estas se tratan en el Capítulo 7.

Texto

El texto presenta otra complejidad ya que no hay información numérica que permita representar estos datos. La primera dificultad que presenta el texto es que la computadora no es capaz de determinar cuáles son las unidades que componen una cadena textual. En principio, podríamos asumir que estas unidades son las palabras, pero este concepto suele ser ambiguo, por lo que se prefiere usar el término de tóken. Un tóken es una cadena que compone una unidad textual más grande. Pueden ser palabras, pero en la práctica suelen ser elementos más pequeños que las palabras (a veces llamados *subwords*). Por ejemplo, del siguiente texto:

En algún lugar, de cuyo nombre no quiero acordarme.

Se pueden obtener los siguientes tókens (separados por comas):

en, algún, lugar, de, cuy##, ##o, nombre, no, quier##,
##o, acord##, ##ar##, ##me.

La mayoría de los tókens corresponden a palabras, pero otros tókens corresponden a unidades significativas más pequeñas. En estos casos, se usa una marca ## para indicar que conforman cadenas más complejas. Asimismo, se suele eliminar los signos de puntuación y pasar el texto a minúsculas, aunque no en todos los casos, pues esto depende de la tarea que se busca resolver.

Los tókens de una cadena textual pueden ser tratados como variables categóricas, de tal forma que se pueden representar por medio de vectores one-hot. Sin embargo, la representación one-hot asume que cada tóken es ortogonal a los otros, por lo que

no existe ninguna relación entre los diferentes tokens. Esto contradice la noción que tenemos del lenguaje, donde hay conjuntos de palabras que tienen significados similares. Para lidiar con esto, en el aprendizaje profundo se propone el uso de encajes de palabras o *word embeddings*.

Definición 32 (Encajes de palabras). *Dado un token w_i con una representación one-hot $x \in \mathbb{R}^n$, un encaje de palabra es un vector que resulta del mapeo:*

$$\text{emb}(w) = Wx$$

donde $W \in \mathbb{R}^{d \times n}$ es una matriz de pesos y d es la dimensión que se desea.

Es decir, el encaje de palabra es una proyección lineal (multiplicación por una matriz) del vector one-hot de la palabra. La matriz de pesos $W \in \mathbb{R}^{d \times n}$ es una matriz que se aprende dentro de la red neuronal. De nuevo, las redes neuronales sirven para aprender representaciones de los datos. Los encajes de palabras entran dentro de estos procedimientos. Los tokens dentro de un texto tienen una relación secuencial que no puede ser capturada por un método tradicional de aprendizaje automático. Dentro del aprendizaje profundo, este tipo de datos suele trabajarse por medios de las redes neuronales recurrentes (Capítulo 6) y actualmente por medio de los transformers (Capítulo 8).

Otros tipos de datos

El aprendizaje profundo permite trabajar con diferentes estructuras relacionales. Podemos hablar de datos no estructurados que se componen por medio de relaciones de sus componentes.

Definición 33 (Datos con estructuras gráficas). *Un dato con estructura gráfica es un dato que cuenta con: 1) un conjunto de elementos representados por vectores $x \in \mathbb{R}^d$; y 2) relaciones entre estos elementos. Por tanto, pueden representarse por medio de una gráfica $G = (V, E)$ donde los vértices V se asocian a los vectores x y las aristas E representan las relaciones.*

El aprendizaje profundo permite la construcción de redes neuronales que permiten interpretar estas relaciones dentro de los datos para capturar sus estructuras subyacentes. Tanto las imágenes como el texto pueden verse como este tipo de datos. En el caso de las imágenes, los píxeles representan las aristas y las relaciones de vecindad de estos píxeles las relaciones, lo que define una gráfica de cuadrícula (véase la Figura 1.9). En el caso del texto, las aristas son los encajes de palabra de los tokens y se tiene una relación secuencial, lo que define una gráfica de cadena (Figura 1.9). Otro tipo de estructuras más complejas pueden definirse. Por ejemplo, una figura en 3D se puede representar por un conjunto de vectores en 3 dimensiones y relaciones de triangulación que definen la superficie.

1.5.2. Normalización de datos

Muchos algoritmos de aprendizaje trabajan mejor cuando los datos tengan una escala normalizada. En particular, cuando la distribución a la que pertenecen está estandarizada. Es común que los datos se normalicen llevándolos a una media 0 y a una varianza 1.

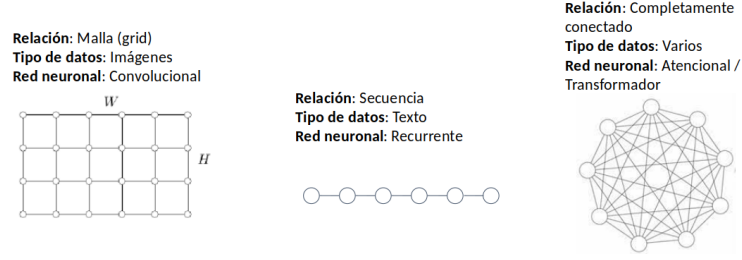


Figura 1.9: Diferentes estructuras de datos representados como gráficas

Definición 34 (Normalización). Sea $X \in \mathbb{R}^{n \times d}$ un conjunto de n datos. La normalización de un dato $x \in \mathbb{R}^d$ se define como:

$$\text{norm}(x) = \frac{x - \mu}{\sigma} \quad (1.11)$$

Donde μ es la media empírica y σ la desviación estándar.

La media empírica se estima a partir del conjunto de datos como:

$$\mu = \frac{1}{n} \sum_x x$$

Por su parte, la desviación estándar se calcula a partir de la media como:

$$\sigma = \sqrt{\frac{1}{n-1} \sum_x (x - \mu)^2}$$

Como lo señalamos, la normalización de los datos lleva éstos hacia una distribución normal estándar. Es decir, con media 0 y varianza 1.

Proposición 2. Sea $X \in \mathbb{R}^{n \times d}$ un conjunto de n datos. La normalización de los datos tiene una distribución normal estándar $\mathcal{N}(0, 1)$.

Demostración. Demostramos que la media de los datos normalizados es 0. Sabemos que μ es la media empírica de los datos, por lo que tenemos que:

$$\begin{aligned} \mathbb{E}[\text{norm}(X)] &= \mathbb{E}\left[\frac{X - \mu}{\sigma}\right] \\ &= \frac{1}{\sigma} \mathbb{E}[X - \mu] \\ &= \frac{1}{\sigma} (\mathbb{E}[X] - \mathbb{E}[\mu]) \\ &= \frac{1}{\sigma} (\mu - \mu) = 0 \end{aligned}$$

Recuérdese que la esperanza de una constante, como lo es μ , es esa constante. □

Existen otras formas de escalar los datos que permiten acotar los valores que toman los valores de las variables. Por ejemplo, si buscamos que los datos se encuentren acotados superiormente por 1 e inferiormente por 0, podemos tomar la siguiente normalización:

$$norm_1(x) = \frac{x - \min\{x\}}{\max\{x\} - \min\{x\}}$$

Puede comprobarse que el valor máximo de los datos en esta normalización es 1. Asimismo que el valor mínimo es 0. La normalización de los datos ayuda al procesamiento de los datos. En el caso de las redes neuronales la normalización será de gran importancia; más adelante revisaremos la normalización por lotes (Sección 4.1.1).

1.5.3. Datos sintéticos

Una forma de mejorar la calidad del modelo de aprendizaje es aumentando el número de datos de entrenamiento. Sin embargo, muchas veces aumentar los datos resulta complicado por la misma dificultad que presenta obtener estos datos de forma natural. En algunos casos será posible generar datos de forma artificial a partir del conocimiento de un conjunto de datos dado. A estos datos generados de forma artificial los llamaremos **datos sintéticos**.

Los datos sintéticos consisten en datos generados de manera automática por algún método, como algún algoritmo de aprendizaje generativo o alguna estrategia que aproveche los datos conocidos para modificarlos y así obtener nuevos datos. Aunque crear nuevos datos puede resultar complicado para tareas como la estimación de probabilidades o aprendizaje no supervisado. Es común que el aumento de datos se haga dentro de los problemas de clasificación, donde se crea un nuevo dato que pertenezca a algunas de las clases y.

Una forma común de aumentar los datos es aplicar una transformación a los datos ya existentes. Sea $\mathcal{S} \subseteq X$ un conjunto de datos y $T : X \rightarrow X$ una transformación sobre este mismo tipo de datos, entonces podemos crear nuevos datos $T(x)$ aplicando la transformación a los datos $x \in \mathcal{S}$. Debemos tomar en cuenta que las transformaciones aplicadas nos den como resultado el mismo tipo de datos que estamos considerando, así como que generen datos que pertenezcan a la clase que nos interesa en el caso de la clasificación.

Por ejemplo, si nuestros datos son imágenes, podemos aplicar transformaciones como rotaciones, traslaciones o escalamientos. Este tipo de transformaciones generan nuevas imágenes que pueden mejorar el rendimiento de nuestro algoritmo de aprendizaje. Debemos tomar en cuenta el objetivo de la tarea para limitar el tipo de transformaciones que podemos aplicar. En una tarea de reconocimiento óptico de caracteres (llevar la imagen de un texto a texto legible por computadora) las imágenes no pueden tener rotaciones de 180 grados, puesto que esto podría llevar a confundir una 'b' por una 'd'. Pero se podrían aplicar ligeras rotaciones o traslaciones de la imagen.

Otra forma de aumentar datos es introducir ruido. Por ejemplo, en un conjunto de datos de audio (para tareas de voz a texto) se puede introducir ruido gaussiano a los datos para generar nuevos datos $x + N$, donde $N \sim \mathcal{N}(\mu, \sigma)$ es ruido de una muestra gaussiana. El ruido también se puede introducir cuando tenemos tareas de regresión logística asumiendo que éste no debería afectar la decisión de la máquina de

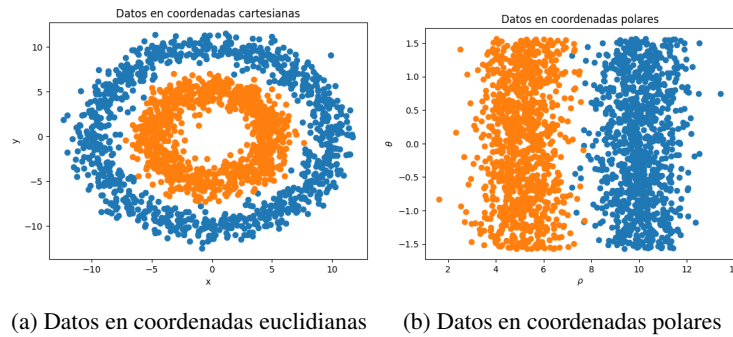


Figura 1.10: Visualización de un mismo conjunto de datos con dos tipos de representaciones

aprendizaje. En la Sección 5.3 veremos una técnica que introduce ruido en diferentes niveles de una red neuronal profunda.

En general, los datos sintéticos se crearán dependiendo del tipo de datos que estemos trabajando y la tarea que estemos resolviendo. Otras estrategias que hagan uso del conocimiento de los datos (como generar texto por medio de gramáticas formales o modificando ciertos tokens) se pueden utilizar mejorando el rendimiento del aprendizaje. Estas estrategias caen dentro de lo que en el área se ha dado en llamar *data augmentation* o aumento de datos.

1.5.4. Conjetura de la variedad

El aprendizaje profundo es parte del aprendizaje representacional, el cual se basa en aprender representaciones de los datos que permitan encontrar una solución al problema planteado sobre estos datos. El ejemplo más sencillo es el de la clasificación, donde un algoritmo de aprendizaje buscará separar dos o más conjuntos de datos pertenecientes a diferentes clases; si tomamos dos clases como en la Figura 1.10 (a) tenemos dos clases que un algoritmo de clasificación lineal no podría separar. Este tipo de algoritmos buscaría trazar una recta para separar los datos de ambas clases. Claramente aquí no es posible. En 1.10 (b), por el contrario, tal separación es posible, pues podemos trazar una recta que pase por en medio de ambos conjuntos de datos, separando casi completamente ambas clases.

En la Figura 1.10 trabajamos con un mismo conjunto de datos pero con diferentes representaciones. La representación en (a) se basa en coordenadas euclidianas, mientras que en (b) las coordenadas son polares. Esta simple modificación de las coordenadas de los datos permite que en el primer caso no sea posible resolver el problema con un algoritmo lineal, mientras que en el segundo caso sí es posible.

Como veremos más adelante cuando tratemos de las redes neuronales profundas, este tipo de redes aprenden representaciones que permitan separar los datos de forma lineal. Es decir, dado un conjunto de datos como los de la Figura 1.10 (a), estas redes aprenden una transformación de los datos que los permite separarlos. Muchas veces, esta representación debe realizarse sobre un espacio de mayor dimensión para poder

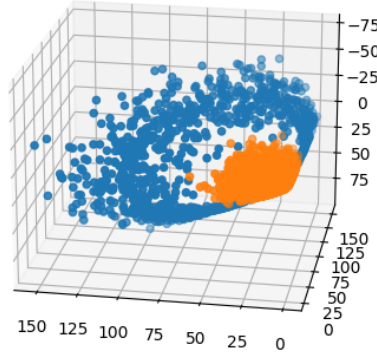


Figura 1.11: Proyección en 3 dimensiones de los datos de la Figura 1.10

encontrar una solución al problema. Por ejemplo, en la Figura 1.11 se puede ver una transformación de los datos de la Figura 1.10 (a) en un espacio de 3 dimensiones. En este espacio también se puede realizar una clasificación lineal, pues se puede encontrar un plano que separa a las dos clases.

El poder de las redes neuronales profundas radica principalmente en su capacidad de aprender este tipo de representaciones a partir de diferentes estructuras de datos de entrada. Bajo este supuesto, algo que se ha propuesto dentro de la teoría del aprendizaje automático es lo que se conoce como la conjetura de la variedad. Aquí explicamos de manera superficial esta conjetura, pues no profundizamos en el concepto de variedad, cuya definición formal omitimos.

Definición 35 (Variedad). *Una variedad d -dimensional X es un conjunto localmente homeomorfo a $\mathbb{R}^d$; es decir, para cada elemento $x \in X$ existe una función continua (y con inversa continua) $f : \mathcal{N}(x) \rightarrow \mathbb{R}^d$ que mapea los puntos en una vecindad de x , $\mathcal{N}(x)$, a $\mathbb{R}^d$.*

Un ejemplo de variedad es una esfera, pues para cada punto en la esfera podemos encontrar una vecindad de ese punto y una función que nos permite mapear esa vecindad en $\mathbb{R}^2$. Si pensamos que esta esfera es el planeta Tierra, entonces el mapeo en $\mathbb{R}^2$ se puede ver como un mapa de una región de la tierra. En general, cuando hablamos de variedades en $\mathbb{R}^d$, se trata de conjuntos acotados y cerrados. A partir de esto, en la teoría de aprendizaje se asume la siguiente conjetura.

Definición 36 (Conjetura de la variedad). *El espacio de los datos X corresponde a una variedad.*

La relevancia de esta conjetura es que nos dice que los datos están conformados por subconjuntos acotados. Por ejemplo, esto se puede ver en las imágenes. Las imágenes se encuentran acotadas, es decir, si nos movemos demasiado del conjunto de imágenes encontraremos sólo ruido. La conjetura de la variedad nos dice que las imágenes se

encuentran en una región del espacio en que se representan. Lo mismo se puede pensar con los datos textuales, pues podríamos pensar que buscar de forma aleatoria cadenas de caracteres, nos daría cadenas sin sentido en un lenguaje dado.

La conjetura de la variedad también tiene consecuencia en la aplicación de las redes neuronales a la resolución de tareas específicas. Como veremos más adelante en la prueba de las redes neuronales como aproximadores universales (Sección 3.2), la demostración de que una red neuronal profunda puede aproximar cualquier función asume que el dominio de éstas es compacto. Es decir, la conjetura de la variedad nos da condiciones para decir que una red neuronal puede resolver los problemas sobre un conjunto de datos.

1.6. Regresión lineal

Tomaremos, como primer ejemplo concreto de un modelo de aprendizaje automático, a la regresión lineal. La regresión lineal es un modelo, como su nombre lo dice, lineal; es decir, busca aproximar un conjunto de datos por medio de una función lineal: un hiperplano parametrizado por un conjunto de pesos $w \in \mathbb{R}^d$ y $b \in \mathbb{R}$.

Sea $\mathcal{S} = \{(x, y) : x \in \mathbb{R}^d, y \in Y \subseteq \mathbb{R}\}$ un conjunto de datos supervisado. Como mencionamos en la Sección 1.3, el problema de regresión asume que los valores a predecir pertenecen a un conjunto continuo. Comenzamos por asumir que se trata de un intervalo de los números reales. En los problemas de regresión, estimamos un valor esperado μ condicionado a un valor de entrada x . En particular, en la regresión lineal se asume que $Y \sim N(\mu, \sigma^2)$ (los valores a predecir tienen una distribución normal) con una media lineal; esto es:

$$\mu = \mathbb{E}[Y|x] = wx + b$$

Es decir, la esperanza condicional de Y dado el dato x es una función lineal dependiente de $w \in \mathbb{R}^d$ y $b \in \mathbb{R}$. Con base en esto, podemos definir la función objetivo, $R(w, b)$, que nos permita encontrar los parámetros w y b que mejor aproximen la media. Para poder determinar una función objetiva adecuada, demostramos el siguiente resultado.

Teorema 3. Sea $\mathcal{S} = \{(x, y) : x \in \mathbb{R}^d, y \in Y \subseteq \mathbb{R}\}$ un conjunto de datos en un problema de regresión lineal. Asíumase que Y tiene distribución normal con media $f(x) = wx + b$. Entonces:

$$\arg \min_{w, b} R(w, b) = \arg \min_{w, b} - \sum_{(x, y)} \ln p(y|x) = \arg \min_{w, b} \frac{1}{2} \sum_{(x, y)} (y - f(x))^2 \quad (1.12)$$

Demostración. Dado que se asume una media lineal $\mu = f(x) = wx + b$, tenemos que:

$$\begin{aligned} - \sum_{(x, y)} \ln p(y|x) &= - \sum_{(x, y)} \ln \left(\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(y-f(x))^2}{2\sigma^2}} \right) \\ &= \sum_{(x, y)} \ln \sqrt{2\pi\sigma^2} + \frac{1}{2\sigma^2} \sum_{(x, y)} (y - f(x))^2 \end{aligned}$$

Ya que el término $\sum_x \ln \sqrt{2\pi\sigma^2}$ es constante, e ignorando σ^2 , tenemos que:

$$\arg \min_{w,b} - \sum_{(x,y)} \ln p(y|x) = \arg \min_{w,b} \frac{1}{2} \sum_{(x,y)} (y - f(x))^2$$

□

Este resultado señala que para encontrar la media que aproxime mejor la distribución de los datos basta minimizar la función de error cuadrático. Más aún, podemos notar que al encontrar la media $f(x)$ que minimiza la función objetivo, también minimizamos la varianza, la cual está implícita en la función objetivo del error cuadrático. Ahora podemos definir el método de regresión lineal.

Definición 37 (Regresión lineal). *El método de regresión lineal sobre un conjunto de datos (x, y) está dado por una función $f : X \rightarrow Y$ definida como:*

$$f(x) = wx + b \quad (1.13)$$

Y cuya función objetivo está dada por el error cuadrático medio:

$$R(w, b) = \frac{1}{2} \sum_{(x,y)} (y - f(x))^2$$

Existen otras funciones objetivo que pueden llevar hacia la solución del problema de regresión. En particular, también se puede definir la función objetivo basada en el valor absoluto:

$$R(w, b) = \frac{1}{n} \sum_{(x,y)} |y - f(x)|$$

A esta función objetivo se le conoce como error absoluto medio (MAE del inglés *Mean Absolute Error*). Sin embargo, es común utilizar el error cuadrático en tanto esta función objetivo tiene derivadas en todos los puntos. Con base en esta función objetivo podemos observar que una solución analítica está dada en la siguiente proposición.

Proposición 3. *Dado un problema de regresión con matriz de datos X , donde a cada dato se le ha concatenado un 1 ($x = (x_1 \ x_2 \ \dots \ x_d \ 1) \in \mathbb{R}^{d+1}$), y Y es el vector de valores a predecir, los valores que minimizan la función objetivo de error cuadrático, $R(w, b) = \frac{1}{2} \sum_{(x,y)} (y - f(x))^2$, en la regresión lineal están dados como:*

$$(w, b) = (X^T X)^{-1} X^T Y$$

donde X es la matriz de datos y Y un vector con las clases de cada ejemplo como entradas.

Demostración. Ejercicio. □

El concatenar un 1 a cada dato se hace para que la resolución del problema se de en un solo paso. Pero pueden estimarse w y b de forma separada. Cabe mencionar que la regresión lineal también puede resolverse por medio del método de descenso por gradiente que se revisará en la Sección 4.1.

Ejemplo 7. Supongamos que tenemos el problema de predecir el precio de una casa dependiendo de los metros cuadrados de ésta. Formalmente, tenemos datos (x, y) donde $x \in \mathbb{R}$ son los metros cuadrados e $y \in \mathbb{R}$ son los precios. Los datos de entrenamiento se muestran en el Cuadro 1.6.

Núm. casa	m ²	Precio (millones)
0	10	3
1	12	4
2	12	5
3	15	7
4	18	10
5	18	9
6	20	15
7	25	20
8	30	20
9	45	50

Cuadro 1.6: Datos de casas y sus precios

De igual forma, los datos se pueden visualizar en la Figura 1.12 (a), vemos que existe cierta dependencia lineal. Para encontrar la recta que mejor aproxima los datos debemos solucionar el problema de regresión lineal. Para esto, primero generaremos una matriz con los datos de metros cuadrados, agregando un 1 a cada entrada. Si llamamos a esta matriz X , podemos observar que:

$$X^T X = \begin{pmatrix} 5211 & 205 \\ 205 & 10 \end{pmatrix}$$

Más aún, podemos obtener la inversa de este producto obteniendo la matriz:

$$(X^T X)^{-1} = \begin{pmatrix} 0,0009 & -0,0203 \\ -0,0203 & 0,516 \end{pmatrix}$$

Recordemos que la solución al problema de regresión está dado como $(X^T X)^{-1} X^T Y$. Sabiendo que $X^T Y$ es el vector $\begin{pmatrix} 3985 & 143 \end{pmatrix}$ observamos que:

$$(X^T X)^{-1} X^T Y = \begin{pmatrix} 0,0009 & -0,0203 \\ -0,0203 & 0,516 \end{pmatrix} \begin{pmatrix} 4235 \\ 143 \end{pmatrix} = \begin{pmatrix} 1,29 \\ -12,19 \end{pmatrix}$$

En este caso, la última entrada del vector resultante corresponde a b . Por lo que tenemos que $w = 1,29$ y $b = -12,19$. La función de regresión lineal obtenida está dada como:

$$f(x) = 1,29x - 12,19$$

La recta definida por esta función se puede observar en la Figura 1.12 (b). Esta recta es la recta que minimiza la distancia promedio entre los puntos del conjunto de datos. Si probamos el modelo obtenido a los datos obtenemos los resultados obtenidos en el Cuadro 1.7.

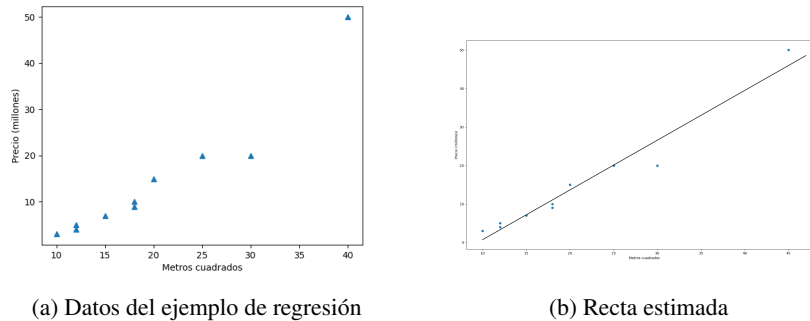


Figura 1.12: Ejemplo de aplicación de regresión a un conjunto de datos y estimación del modelo lineal

Núm. casa	m ²	Precio (millones)	Predicción
0	10	3	-0.01
1	12	4	2.84
2	12	5	2.84
3	15	7	7.14
4	18	10	11.4
5	18	9	11.4
6	20	15	14.3
7	25	20	21.4
8	30	20	28.6
9	45	50	42.9

Cuadro 1.7: Resultados de aplicar la predicción del modelo de regresión lineal

Los datos del cuadro anterior no representan una evaluación, pues la evaluación debe realizarse sobre un conjunto de datos que no ha sido observado durante el entrenamiento. Tomemos el ejemplo de los datos en el Cuadro 1.8.

Núm. casa	m ²	Precio	Predicción	Error (y - f(x))
0	7	2	-3.1	5.1
1	15	7	7.19	-0.19
2	60	70	65.3	-4.7

Cuadro 1.8: Evaluación del modelo de regresión

A partir de estos datos podemos calcular el error cuadrático medio y el coeficiente R^2 . Para el error cuadrático medio tenemos:

$$MSE = \frac{1}{3}((5.1)^2 + (-0.19)^2 + (-4.7)^2) \approx 16.04$$

Para el coeficiente R^2 debemos obtener la media de y sobre los datos de evaluación;

ésta es $\mu_y = \frac{1}{3}(2 + 7 + 70) \approx 26,3$. De esta forma obtenemos:

$$R^2 = 1 - \frac{(5,1)^2 + (-0,19)^2 + (-4,7)^2}{(2 - 26,3)^2 + (7 - 26,3)^2 + (70 - 26,3)^2} \approx 0,982$$

Notamos que el coeficiente R^2 es cercano a 1, lo que nos indica que el modelo lo hace bien. De igual forma, el error cuadrático medio nos muestra que la media del error no es demasiado grande.

1.7. Regresión logística

Otro modelo lineal que será relevante para la discusión posterior es el modelo de regresión logística. La regresión logística estima la densidad probabilística de los datos con base en su pertenencia a un conjunto de clases. Por tanto, si bien su salida es un valor continuo en el intervalo $[0, 1]$, suele usarse como un método de clasificación. Asimismo, la estimación de esta densidad probabilística se hace mediante una aproximación lineal. Aquí revisamos el caso donde la clasificación es binaria, contamos sólo con las clases 0 y 1 de salida y la probabilidad que estimamos es $p(y = 1|x)$, esto es, la probabilidad que tiene un elemento de pertenecer a la clase 1. La regresión lineal se basa en el concepto de logit, que definimos a continuación.

Definición 38 (Logit). Sea p la probabilidad de un evento. El logit de p es la diferencia logarítmica:

$$\text{logit}(p) = \ln(p) - \ln(1 - p) = \ln \frac{p}{1 - p}$$

El logit determina la capacidad predictiva de una medida de probabilidad (binaria). Por ejemplo, si $p = 0,5 = 1 - p$, entonces el evento es completamente aleatorio y el logit es 0. Por su parte, si $p > 1 - p$ el logit crece y es positivo, mientras que si $1 - p > p$, el logit es negativo y decrece. La regresión logística asume que el logit de la distribución es lineal, esto es que:

$$\ln \frac{p}{1 - p} = wx + b$$

Donde x es el dato que condiciona la probabilidad $p = p(y = 1|x)$ y w y b son los pesos que parametrizan la función lineal. Bajo esta suposición, tenemos el siguiente resultado.

Proposición 4. Sea $\mathcal{S} = \{(x, y) : x \in \mathbb{R}^d, y \in \{0, 1\}\}$. Si $p = p(y = 1|x)$ es la probabilidad que tiene x de pertenecer a la clase 1 y se asume que el logit es lineal, entonces:

$$p = \frac{e^{wx+b}}{e^{wx+b} + 1}$$

Demostración. Ejercicio. □

A la función de probabilidad que surge de la suposición de un logit lineal se le conoce como función logística, aunque también es común llamarle función sigmoide, principalmente en el área de aprendizaje profundo; por tanto, es la terminología que usamos aquí. Es fácil notar que $\frac{e^a}{e^a + 1} = \frac{1}{1 + e^{-a}}$ al multiplicar por el uno $\frac{e^{-a}}{e^{-a}}$.

Definición 39 (Función sigmoide). *La función sigmoide (o logística) es la función $\sigma : \mathbb{R} \rightarrow (0, 1)$ definida como:*

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (1.14)$$

Aquí hemos definido la función sigmoide en un dominio real. Debemos notar que los valores de 0 y 1 sólo se obtienen en el límite, por lo que no los consideramos dentro del intervalo de la imagen de la función. Finalmente, cuando tratamos con la regresión logística, nuestra función objetivo será la entropía cruzada con base en las clases binarias.

Definición 40 (Regresión logística). *Un modelo de regresión logística está definido por una función $f : \mathbb{R}^d \rightarrow (0, 1)$ que asigna una probabilidad de clase y que se define como:*

$$f(x) = \sigma(wx + b) = \frac{1}{1 + e^{-wx - b}}$$

Con $w \in \mathbb{R}^d$ y $b \in \mathbb{R}$ pesos a aprender. La función objetivo del problema está dada como:

$$R(w, b) = - \sum_{(x, y)} \left(y \ln(\sigma(wx + b)) + (1 - y) \ln(1 - \sigma(wx + b)) \right)$$

La función objetivo es la entropía cruzada entre la probabilidad $p(y = 1|x) = \sigma(wx + b)$ y la probabilidad $p(y = 0|x) = 1 - p(y = 1|x)$. Puede comprobarse sin mucha dificultad que esta última probabilidad cumple que:

$$p(y = 0|x) = 1 - \sigma(wx + b) = \sigma(-wx - b)$$

Para obtener los pesos óptimos, se utiliza el método de gradiente descendiente en el aprendizaje. Este método se revisará en la Sección 4.1 por lo que omitimos aquí la ejemplificación del proceso de aprendizaje.

Ejemplo 8. *Para ejemplificar la aplicación de la regresión logística considérese los datos en la Figura 1.13 (a), donde los datos del lado derecho pertenecen a la clase 1 y los datos del lado izquierdo pertenecen a la clase 2. El objetivo de la regresión logística es estimar la probabilidad de que un punto pertenezca a la clase 1 (a la sección izquierda).*

Observando los datos se puede inferir que la información que determina la pertenencia de un dato a una clase es el valor sobre el eje de las x : si este es mayor a 0 el dato tendrá mayor probabilidad de pertenecer a la clase 1, mientras que si es menor a 0, la probabilidad será más baja, pues el dato pertenecerá a la clase 0. Podemos definir la siguiente regla:

$$cls(x) = \begin{cases} 1 & \text{si } x_1 > 0 \\ 0 & \text{si } x_1 < 0 \end{cases}$$

Donde x_1 es la primera entrada del vector x . Por tanto, la función cls clasifica los datos en la clase a la que pertenecen.

Para expresar esto como un modelo de regresión logística podemos observar que la probabilidad de pertenecer a la clase 1 está dada por $\sigma(x_1)$, por lo que necesitamos

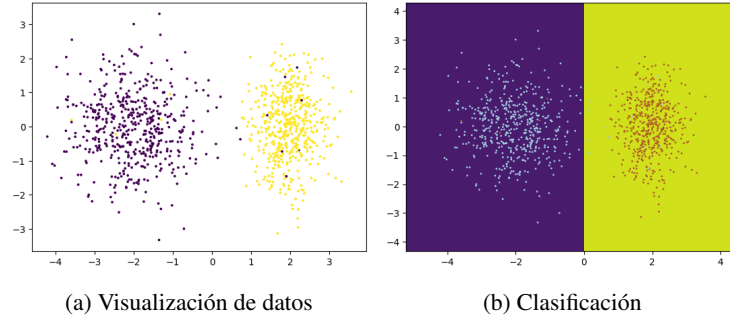


Figura 1.13: Ejemplo de aplicación de regresión logística a un problema de clasificación

los pesos $w^T = (1 \ 0)$ y $b = 0$. Por ejemplo, para el dato $x^T = (3 \ -5)$ tendremos el siguiente resultado en la regresión logística:

$$f(X) = \sigma\left((1 \ 0) \begin{pmatrix} 3 \\ -5 \end{pmatrix}\right) = \frac{1}{1 + e^{-3}} \approx \frac{1}{1,049} = 0,95$$

Es decir, tiene una alta probabilidad de pertenecer a la clase 1, por lo que el modelo asumirá que pertenece a esta clase. Puede comprobarse que para el vector $x = (-3 \ 5)$ la regresión lineal predice el valor $f(x) = 0,04$, por lo que se asumirá que el dato no pertenece a la clase 1, sino a la clase 0.

El modelo de regresión lineal asume una recta que separa el espacio de datos en dos regiones como se puede ver en la Figura 1.13 (b). Esta recta pasa por el valor 0 y mientras más alejados estén hacia la derecha los datos, mayor la probabilidad de pertenecer a la clase 1. Por ejemplo, se puede comprobar que $x = (0 \ 0)$ tiene probabilidad $f(x) = 0,5$, por lo que su clasificación es complicada, aunque en la práctica los puntos con probabilidad 0.5 suelen clasificarse en la clase 0.

Para evaluar podemos tomar un conjunto de datos de evaluación. Asumiendo el esquema 70-30 podemos tomar 300 datos que tienen la misma distribución que se muestra en la Figura 1.13. La matriz de confusión se muestra en el Cuadro 1.9.

Real / Predicha	Clase 0	Clase 1
Clase 0	150	0
Clase 1	3	147

Cuadro 1.9: Matriz de confusión de clasificación con regresión lineal

A partir de los datos de la matriz de confusión es fácil calcular la precisión, la exhaustividad, la medida F_1 y la exactitud. Estas métricas se muestran en el Cuadro 1.10.

Clase	Precisión	Exhaustividad	F_1
0	0.98	1	0.99
1	1	0.98	0.99
Exactitud	0.99		

Cuadro 1.10: Resultados de la evaluación de la regresión logística

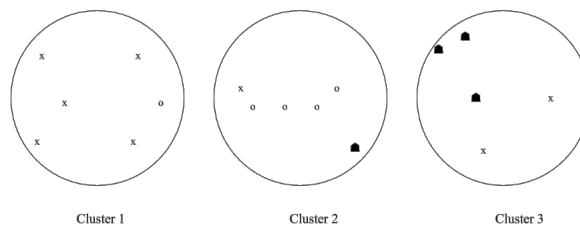
Ejercicios

- De los siguientes ejemplos determine cuáles son los elementos: 1) tarea T , 2) medida de desempeño P , y 3) experiencia E .
 - Un sistema que aprende a detectar spam.
 - Detectar y etiquetar con una clase los objetos que aparecen en una imagen.
 - Un sistema para predecir una enfermedad a partir de los síntomas de un paciente.
 - Predecir el precio de un televisor dado los años de uso.
 - Generar voz sintetizada a partir de una entrada textual.
- Dado el siguiente cuadro con las predicciones hechas por una máquina de aprendizaje para predecir colores con los valores esperados de la supervisión:

Predicción	v	v	a	a	r	r	v	a	r	r	v	v	a	v	v
Supervisión	a	a	a	a	a	r	r	r	r	r	v	v	v	v	v

obtener la matriz de confusión, los valores de precisión, recall (sensitivity), F_1 , así como el *micro* y *macro average* por clase.

- A partir de los siguientes datos de un agrupamiento:



Calcular la pureza de los clústers propuestos.

- A partir del siguiente dataset:

X_1	X_2	X_3	X_4	Y
0.5	1	0	1	1
0	0	1	0.8	0
0.7	1	0.5	0	1
0.5	1	0	1	0
0.1	1	0	0	1
1	0	1	1	0
1	1	0.1	0.2	1

Hacer el análisis de cada variable (media, varianza) así como obtener los valores de covarianza y calcular la correlación de las variables X_1, X_2, X_3 y X_4 con respecto a Y .

5. A partir de los datos del ejercicio anterior, asumiendo que se tienen los pesos $w^T = (1 \quad 1 \quad -1 \quad -1)$ y $b = 0.5$, estimar las probabilidades y las predicciones de clase en base al modelo de regresión lineal. Generar la matriz de confusión y las métricas de evaluación de precisión, exhaustividad, F_1 y exactitud.
6. A partir de los siguientes datos de entrenamiento:

x	y
0	0
1	2
1.5	2
3	3
3	2.5
3.5	4.5
5	6

Estimar la recta de regresión que minimice el error cuadrático medio usando el algoritmo de mínimos cuadrados (nótese que $b = 0$). Evaluar el modelo usando MSE y MAE obtenido en los siguientes datos:

x	y
2.5	3.5
4	4.8
6	8

7. Demostrar que la divergencia KL, $D = \mathbb{E}[\log \frac{p(x)}{f(x)}]$, es siempre mayor a 0 y sólo igual a 0 si $f(x) = p(x)$ para toda x .
8. Demostrar que la solución al problema de la regresión lineal está dada por los parámetros:

$$(w, b) = (X^T X)^{-1} X^T Y$$

donde X es la matriz de datos y Y el vector de valores de supervisión.

9. Demostrar que si se asume que el logit es lineal, i.e. $\ln \frac{p}{1-p} = wx + b$, entonces la probabilidad p está dada por $p = p(x) = \frac{1}{1+e^{-(wx+b)}}$
10. Calcular los siguientes valores:
- a) Entropía cruzada de una variable con distribución real $Ber(\frac{3}{4})$ y distribución empírica $Ber(\frac{1}{2})$.
 - b) La divergencia KL de las distribuciones anteriores.
11. Implementa los modelos de regresión lineal y regresión logística.

Capítulo 2

El Perceptrón

Una de las primeras propuestas sobre un modelo matemático del funcionamiento neuronal fue el de McCulloch and Pitts (1943). Estos autores proponían un modelo de cálculo lógico (capaz de procesar compuertas lógicas) inspirado por el trabajo de las neuronas biológicas. Sin embargo, una de las desventajas que presentaba este trabajo es que no se planteaba la forma de obtener los valores indicados (pesos) para resolver problemas específicos. Es decir, no se conocía un método de aprendizaje sobre estas neuronas. No fue hasta finales de los años 50s en que se propuso un modelo de aprendizaje basado en el funcionamiento de las neuronas. Este modelo matemático de una neurona fue llamado perceptrón y presentado por Rosenblatt (1958).

El perceptrón es un modelo lineal inspirado por el funcionamiento de una neurona. En la Figura 2.1 se muestra una neurona biológica que, como puede observarse, se compone de los siguientes elementos.

- **Dendritas.** Ramificaciones de la neurona que se dedican a la recepción de estímulos, así como a la alimentación celular.
- **Cuerpo celular o soma.** Contiene el núcleo celular, además de otros organelos.
- **Núcleo.** Cuenta con el material genético (ADN y ARN), y se puede decir que es aquí donde se coordinan las funciones de la neurona.
- **Axón.** Es una prolongación de la neurona que se enfoca en conducir los impulsos nerviosos de una a otra neurona. El axón se recubre de células de Schwann, con producción de mielina, la cual protege el axón y permite la transmisión del impulso nervioso. En el axón se presentan los nódulos de Ranvier, que son las interrupciones que se dan en el axón. Finalmente, los terminales del axón se comunican con otras neuronas.

De esta forma, la neurona puede pensarse en tres partes que cumplen tres funciones esenciales: 1) la recepción de estímulos (dendritas); 2) el procesamiento de estos estímulos para producir un impulso nervioso; y 3) la emisión del impulso nervioso hacia otras neuronas.

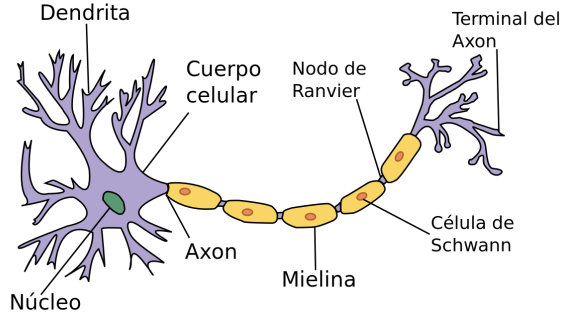


Figura 2.1: Diagrama de una neurona biológica

Como es natural en matemáticas, un modelo busca abstraer una estructura para enfocarse en sus componentes más relevantes. El modelo del perceptrón es una abstracción de una neurona que deja de lado varios de los componentes para simplificar su implementación. En el perceptrón se busca que, dado un conjunto de estímulos que llamaremos $x_1, x_2, \dots, x_d$, se emita una señal de salida $f(x) \in \{0, 1\}$ que sea adecuada a una respuesta esperada. Por tanto, dado un conjunto de estímulos, se busca emitir un valor $f(x)$ que sea una respuesta adecuada al estímulo.

Visto así, el perceptrón busca emular la reacción de una neurona ante un estímulo. El modelo del procesamiento que hace el núcleo de la neurona de los estímulos para obtener la respuesta se basa en la idea de que los estímulos que tengan mayor influencia para que la neurona mande la señal (se obtenga el valor $f(x) = 1$), serán conexiones más fuertes, mientras que aquellos estímulos que no influyen o que influyen de forma negativa tendrán conexiones débiles. La fuerza de las conexiones se expresa como valores numéricos $w_1, w_2, \dots, w_d$. Estos valores numéricos multiplican al valor de los estímulos y se suman para obtener el valor total de los estímulos. El procesamiento del núcleo, por tanto, se expresará en la ecuación:

$$\sum_{i=1}^d w_i x_i \quad (2.1)$$

Este es un valor continuo. Para obtener el valor de la señal, 1 ó 0, se planteará la regla: si el valor de la suma de los estímulos ponderado por los pesos de las conexiones es mayor a un umbral b , la neurona se activa, mandando una señal $f(x) = 1$. Es decir, si $\sum_{i=1}^d w_i x_i > b$, entonces el perceptrón tendrá la salida 1, en caso contrario (valor menor a b) su salida será 0.

Ya que los pesos de las conexiones deben adaptarse a los estímulos específicos al problema, éstos serán parámetros que se aprenderán durante un proceso de entrenamiento. De igual forma, el umbral b será algo que se deba aprender. Dado esto, no importa si b tiene un valor negativo o positivo, por lo que podemos expresar la decisión del perceptrón con base en $\sum_{i=1}^d w_i x_i + b > 0$. Visto como un problema de aprendizaje, los estímulos se representan en un vector $x \in \mathbb{R}^d$, de igual forma los pesos de las conexiones conforman un vector $w \in \mathbb{R}^d$. Finalmente, al umbral b lo llamaremos **bias** o sesgo. De esta forma, el perceptrón se puede definir de la siguiente forma:

Definición 41 (Perceptrón). *El perceptrón es un modelo de clasificación lineal, que estima una función $f : \mathbb{R}^d \rightarrow \{0, 1\}$, definida como:*

$$f(x) = \begin{cases} 1 & \text{si } wx + b > 0 \\ 0 & \text{si } wx + b \leq 0 \end{cases} \quad (2.2)$$

Una forma común de representar al perceptrón es por medio de una gráfica que se presenta en la Figura 2.2. Los estímulos conforman nodos de entrada que se conectan con un nodo (el núcleo de la neurona) que contiene el valor $wx + b$. Con base en este valor, se realiza una suma de los estímulos ponderados por los pesos de conexiones. Finalmente, el perceptrón realiza una decisión con base en la función escalonada definida en la Ecuación 2.2. Este modelo del perceptrón cuenta con los siguientes elementos:

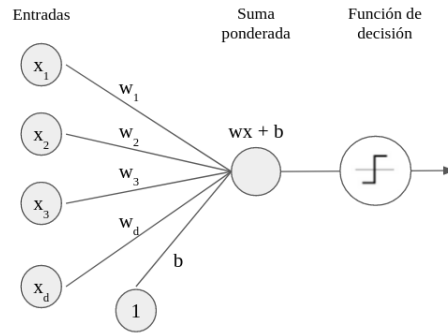


Figura 2.2: Representación gráfica del Perceptrón

- Entradas $x_1, \dots, x_d$ que emulan a las dendritas que reciben los estímulos codificados en un vector.
- Los pesos $w_1, \dots, w_d$ que representan la intensidad de las conexiones de los estímulos de entrada con la neurona.
- El sesgo o bias b que representa el umbral en el que la neurona se activa o no.

Como lo señalamos en la Definición 41, el perceptrón es un modelo lineal; esto es, realiza una clasificación en base a una función lineal, la cual define un hiperplano (o una recta en el caso de $\mathbb{R}^2$) que separa los datos en dos clases, 0 y 1. Este hiperplano está parametrizado por los pesos $w \in \mathbb{R}^d$ y el bias $b \in \mathbb{R}$ como se puede ver en la Figura 2.3.

El hiperplano que separa los datos está definida por el conjunto de puntos que cumplen $wx + b = 0$, es decir, este hiperplano es ortogonal a w y presenta una traslación dada por b . El entender al perceptrón bajo esta perspectiva geométrica resulta importante para comprender sus limitaciones. Es, precisamente, a partir de una argumentación geométrica que Minsky and Papert (1969) pudieron demostrar que el perceptrón no puede resolver problemas complejos como el problema XOR (véase la Sección 2.2.1).

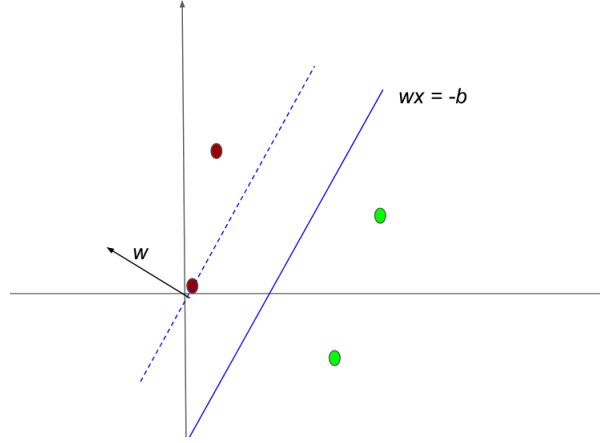


Figura 2.3: Separación lineal de los datos en clases binarias que realiza el perceptrón

2.1. Aprendizaje en el perceptrón

Una vez explicado el funcionamiento del perceptrón, nos preguntamos cómo podemos estimar los pesos $w \in \mathbb{R}^d$ y el bias $b \in \mathbb{R}$ para que realicen una clasificación correcta dado un conjunto de datos supervisados $\mathcal{S} = \{(x, y) : x \in \mathbb{R}^d, y \in \{0, 1\}\}$. Denotemos a los pesos como $\theta = \{w \in \mathbb{R}^d, b \in \mathbb{R}\}$. Supongamos que tenemos un perceptrón $f(x; \theta)$ que depende de un conjunto de pesos específico. Sabiendo que este perceptrón debe clasificar un conjunto de datos en dos clases, 0 y 1, podemos observar que se realizan dos tipos de errores:

- Un tipo de error se da cuando se predice $f(x) = 1$, pero la clase real es $y = 0$; en este caso, notando que $f(x) - y = 1 - 0 = 1$, tenemos que disminuir los pesos usando un factor de cambio (denotado Δw) negativo:

$$\Delta w = -1 = -(f(x) - y)$$

- Otro error se da cuando se predice $f(x) = 0$, pero la clase real es $y = 1$; en este caso, notando que $f(x) - y = 0 - 1 = -1$, tenemos que aumentar los pesos usando un factor de cambio positivo:

$$\Delta w = +1 = -(f(x) - y)$$

Es decir, podemos expresar el factor de cambio de los pesos con base en el mismo error $f(x) - y$. Con base en esta diferencia, podemos proponer que el factor de cambio sobre los pesos sea:

$$\Delta w = (f(x) - y)$$

Este factor de cambio modificará los pesos del perceptrón de forma iterativa hasta que el perceptrón clasifique los datos de manera correcta. En particular, llamaremos **época**

a cada proceso del algoritmo en que los pesos se actualizan después de observar todos los datos del conjunto de datos. Se debe notar que si hay ciertos estímulos que son 0 o que tiene valores pequeños, entonces estos valores tendrán poca o nula influencia en la decisión del perceptrón. Por ejemplo, si el estímulo es 0, ese peso no debe alterarse. Por tanto, podemos modificar cada uno de los pesos de las conexiones tomando en cuenta el estímulo de entrada:

$$\Delta w_i = (f(x) - y)x_i$$

Para moderar el factor de cambio, podemos multiplicar por un hiperparámetro, $\eta \in \mathbb{R}$, conocido como **tasa de aprendizaje**. Esta tasa de aprendizaje se encarga de controlar la magnitud del cambio en cada época en que se modifican los pesos del perceptrón. De tal forma que tenemos la regla para actualización de pesos del perceptrón:

$$w_i \leftarrow w_i - \eta (f(x) - y)x_i$$

Es decir, cada peso w_i se modifica restando al peso anterior el factor de cambio $\eta (f(x) - y)x_i$. En el caso del bias, ya que podemos considerarlo como una entrada con valor 1 (Figura 2.2), éste se actualizará como:

$$b \leftarrow b - \eta (f(x) - y)$$

El proceso de actualización de los pesos se repetirá hasta conseguir la clasificación deseada, o bien hasta que se complete un número de épocas dado. Esto último porque no siempre podemos garantizar que el perceptrón clasifique de manera correcta los datos.

Con base en esta regla de actualización de los pesos, podemos proponer el algoritmo de aprendizaje en el perceptrón. Este algoritmo tomará como entradas los siguientes elementos:

- El conjunto de datos de entrenamiento $\mathcal{S} = \{(x, y) : \mathbb{R}^d, y \in \{0, 1\}\}$.
- Un número máximo de épocas T .
- La tasa de aprendizaje η .

Tanto el número de épocas T , como la tasa de aprendizaje η son **hiperparámetros**, es decir, deben ser establecidos por el programador del algoritmo. El algoritmo se divide en dos pasos: 1) el paso forward donde predice las clases en base a los pesos con los que cuenta; y 2) el paso backward en donde estima el error y actualiza los pesos. Si el error es 0 (si ha clasificado todos los puntos correctamente) el algoritmo se detiene, de lo contrario continúa actualizando los pesos hasta acabar las épocas. El algoritmo se describe a continuación:

```

1: procedure FIT-PERCEPTRON( $\mathcal{S} = \{(x, y)\}, T, \eta$ )
2:    $w, b \leftarrow \text{RANDOM}(w, b)$                                      Inicializa aleatoriamente
3:   for  $t$  from 1 to  $T$  do
4:     for  $x, y$  in  $\mathcal{S}$  do
5:        $f(x) = \begin{cases} 1 & \text{si } wx + b > 0 \\ 0 & \text{si } wx + b \leq 0 \end{cases}$                                      Forward

```

```

6:          $w_i \leftarrow w_i - \eta (f(x) - y)x_i$                                 Backward
7:          $b \leftarrow b - \eta (f(x) - y)$ 
8:     end for
9:     if  $\sum_{(x,y)} (f(x) - y)^2 = 0$  then
10:         returns  $f(\cdot; w, b)$                                 Termina el proceso
11:     end if
12: end for
13: returns  $f(\cdot; w, b)$ 
14: end procedure

```

Ejemplo 9. Supongamos que tenemos un problema de clasificación binaria con el conjunto de datos que se muestra en el Cuadro 2.1. Para aplicar el algoritmo de aprendizaje del perceptrón a este problema, en primer lugar debemos elegir, de forma aleatoria unos valores iniciales para los pesos $w \in \mathbb{R}^d$ y $b \in \mathbb{R}$.

x_1	x_2	y
0	1	0
0.2	1	0
0.7	0	1
1	0.1	1

Cuadro 2.1: Datos de un problema de clasificación binaria

Para simplificar tomemos todos los valores de inicio en 1. Es decir, iniciamos $w^T = (1 \ 1)$ y $b = 1$. En el paso 1, tenemos que obtener los valores de predicción $f(x)$ para cada uno de los ejemplos; en la primera época, el perceptrón está dado por la función $f(x) = x_1 + x_2 + 1$. Se puede comprobar con facilidad que este perceptrón asumirá que todos los datos pertenecen a la clase 1. El resultado de las predicciones en el paso forward es el siguiente:

$f(x)$	1	1	1	1
--------	---	---	---	---

Pasemos entonces a actualizar los pesos del perceptrón de acuerdo a la regla establecida en el paso backward del algoritmo. Elijamos un $\eta = 1$, de tal forma que consideraremos la tasa de aprendizaje en el entrenamiento. Para el primer dato, por ejemplo, tenemos que:

$$w \leftarrow \begin{pmatrix} 1 \\ 1 \end{pmatrix} - (1 - 0) \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

Hemos actualizado todo el vector de pesos w en un sólo paso, en lugar de actualizar entrada por entrada como se indica en el algoritmo. Para el bias tenemos que la actualización se da como $b \leftarrow 1 - (1 - 0) = 0$.

La actualización de w debe hacerse por cada uno de los ejemplos. Sin embargo, para los últimos dos ejemplos, en donde la clasificación es correcta, no se realiza ninguna actualización, puesto que $f(x) - y = 1 - 1 = 0$, por lo que el vector w no sufre

cambios. Para el segundo ejemplo, dada la actualización ya realizada, tenemos:

$$w \leftarrow \begin{pmatrix} 1 \\ 0 \end{pmatrix} - (1-0) \begin{pmatrix} 0,2 \\ 1 \end{pmatrix} = \begin{pmatrix} 0,8 \\ -1 \end{pmatrix}$$

Con esto, terminamos la primera época, obteniendo que los nuevos valores para los pesos del perceptrón son $w^T = (0,8 \quad -1)$ y $b = 0$. Es decir, nuestra función del perceptrón con los pesos actualizados está dada como:

$$f(x) = 0,8x_1 - x_2$$

Este perceptrón con los pesos dados clasifica de forma correcta cada uno de los datos de entrenamiento, por lo que el algoritmo terminaría después de esta primera época.

El algoritmo de aprendizaje del perceptrón permitió el uso del perceptrón como un método de aprendizaje para su aplicación a problemas reales. Sin embargo, como veremos en la Sección 2.2.1, este método tiene fuertes limitaciones. A pesar de esto, el perceptrón se puede utilizar en problemas sencillos de manera eficiente.

Proposición 5. La complejidad del algoritmo del perceptrón es de $O(TN)$, donde T es el número máximo de épocas y N el número de ejemplos de entrenamiento.

Demostración. El algoritmo del perceptrón recorre cada uno de los ejemplos en el paso forward y el el paso backward, por lo que realiza $2N$ operaciones. En el peor de los casos, se realizará este proceso T veces. Por tanto su complejidad es de orden $O(TN)$. □

La proposición anterior muestra que el tiempo que tomará entrenar un perceptrón dependerá del tamaño del conjunto de datos. El número de épocas T también es un factor importante, pues elegir un número de épocas muy pequeño podría llevar al algoritmo a no converger a pesar de que exista un perceptrón que solucione el problema. En la práctica se suelen elegir números grandes de épocas, buscando asegurar que la convergencia se dé. Sin embargo, esto también implica un tiempo largo de entrenamiento sin la certeza de encontrar una solución. El siguiente resultado determina una cota para el número máximo de épocas para asegurar la convergencia del algoritmo.

Teorema 4. Sea $\mathcal{S} = \{(x, y) : x \in \mathbb{R}^d, y \in \{0, 1\}\}$ un problema de clasificación binaria que pueda resolverse correctamente por un perceptrón. Sea w^* los pesos de la solución óptima del perceptrón. Si para todo x , $\|x\| \leq \rho$, y además:

$$\frac{|w^* \cdot x|}{\|w^*\|} \geq \gamma$$

Entonces el número de épocas, T en que el perceptrón converge está acotado como:

$$T \leq \frac{\rho}{\gamma^2}$$

Demostración. Ejercicio. □

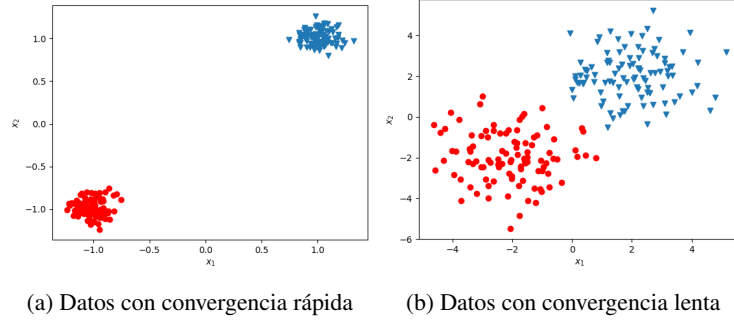


Figura 2.4: Dos conjuntos de datos con diferentes rangos de convergencia

El teorema anterior puede demostrarse con conceptos de álgebra lineal básicos. Se ignora el bias pues se puede suponer que el hiperplano separador pasa por el origen (si no es así, se puede aplicar una traslación a los datos). Las cotas ρ y γ determinan la cota superior del número de épocas: ρ representa la concentración de los datos, un valor de ρ alto implica que los datos se encuentran con mayor dispersión en el espacio $\mathbb{R}^d$. Por su parte, γ determina la separación entre los dos conjuntos de datos entre cada clase. En la Figura 2.4 (a) se puede observar un conjunto de datos que tienen un valor de ρ pequeño y un valor γ alto; es decir, están separadas las clases y los conjuntos de clases están concentrados en una distancia amplia. La convergencia en estos datos será rápida; intuitivamente, se encontrará con facilidad una recta que separe los datos: visualmente podemos ver que podemos trazar un gran número de rectas que resuelvan el problema. Por el contrario en la Figura 2.4 (b) encontrar una recta es más difícil. Aquí los datos tienen un valor ρ mayor (tienen mayor dispersión) mientras que el valor γ es más pequeño (la distancia entre ambos conjuntos es corta). Por tanto, requerirán de un mayor número de épocas para converger.

En ambos ejemplos de la Figura 2.4 podemos garantizar que existe un perceptrón que soluciona el problema, lo cual de hecho es una de las condiciones del Teorema 4. Si no existe dicho perceptrón, es claro que por más épocas que se asignen al algoritmo nunca se resolverá el problema. A continuación discutimos cuáles son las condiciones que pueden llevarnos a garantizar que un problema pueda ser resuelto por un perceptrón.

2.2. Separabilidad lineal

Se dice que el perceptrón, en tanto un modelo lineal, puede solucionar sólo aquellos problemas que son linealmente separables. El concepto de *linealmente separable* se utiliza principalmente en problemas de clasificación y es de gran importancia dentro de los modelos lineales y del aprendizaje profundo. De forma intuitiva, un problema de clasificación binaria es linealmente separable si existe una función lineal que puede asignar las clases correctamente. Esta definición, sin embargo, parece circular, pues podemos decir que una función lineal puede clasificar correctamente un problema si

este último es linealmente separable.

Para dar una definición formal de la separabilidad lineal y poder determinar cuándo un modelo lineal como el perceptrón puede resolver correctamente un problema, primero introduciremos algunos conceptos relevantes: el primero de ellos, el concepto de un conjunto convexo.

Definición 42 (Conjunto convexo). *Un conjunto $X \subseteq \mathbb{R}^d$ es convexo si para todo $x, x' \in X$ se cumple que:*

$$tx + (1 - t)x' \in X$$

con $t \in [0, 1]$.

La ecuación $tx + (1 - t)x'$, con $t \in [0, 1]$ define una línea en $\mathbb{R}^d$ con extremos x y x' , por lo que podemos decir que un conjunto convexo es aquel en donde para todos dos puntos del conjunto, la línea que va de uno a otro punto sólo contiene puntos que son también parte del conjunto. En la Figura 2.5 se puede ver de manera gráfica el contraste entre un conjunto convexo (a la izquierda) y un conjunto no convexo (a la derecha), en donde se puede ver que si se traza una línea entre los dos puntos elegidos, parte de esta línea queda fuera del conjunto.

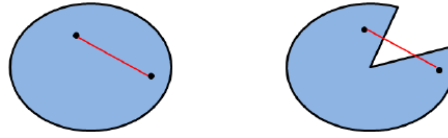


Figura 2.5: Un conjunto convexo (izquierda) y un conjunto no convexo (derecha)

A partir del concepto de conjunto convexo podemos definir el envolvente convexo, que nos será de gran ayuda para determinar cuando es posible que el perceptrón encuentre una solución.

Definición 43 (Envolvente convexo). *Sea $X \subseteq \mathbb{R}^d$ un conjunto. El envolvente convexo de X es el conjunto definido como:*

$$\text{conv}(X) = \bigcap \{C : X \subseteq C \text{ y } C \text{ es convexo}\}$$

En otras palabras, el envolvente convexo de un conjunto X es el conjunto convexo más pequeño que contiene a X . Claramente, si un conjunto X es convexo, entonces $\text{conv}(X) = X$. En la Figura 2.5 el envolvente convexo del conjunto a la derecha es justamente el conjunto a la izquierda; es decir, su envolvente convexo es el conjunto que incluye a todos los puntos de las líneas que antes no estaban en ese conjunto.

Con el concepto de envolvente convexo podemos comenzar a elucidar cuándo existe un perceptrón que sea capaz de solucionar un problema de clasificación. En primer lugar, debemos recordar la conjetura de la variedad (Sección 1.5.4), que nos dice que los datos pertenecen a un conjunto cerrado y acotado (una variedad). Por tanto, si bien los datos de entrenamiento son una muestra finita de esta variedad, podemos asumir que el conjunto al que pertenecen es infinito e incluso continuo. Por tanto, podemos pensar que los datos pertenecen o no a conjuntos convexos o con envolventes convexas.

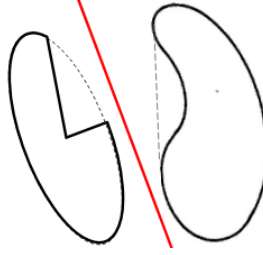


Figura 2.6: Dado dos conjuntos cuyas envolventes convexas sean disjuntas podemos encontrar un separador lineal

Teorema 5. Sea $X = X_0 \cup X_1 \subseteq \mathbb{R}^d$ un conjunto de datos de un problema de clasificación, tal que X_0 y X_1 son los conjuntos de cada una de las clases. Entonces, existe un perceptrón $f(x; \theta)$ con pesos $\theta = \{w^*, b^*\}$ que puede clasificar correctamente los datos si y sólo si se cumple que:

$$\overline{\text{conv}(X_0)} \cap \overline{\text{conv}(X_1)} = \emptyset$$

Demostración. En primer lugar, supongamos que existe un perceptrón $f(x; \theta)$ con pesos $\theta = \{w^* b^*\}$ que clasifica correctamente los datos $X = X_0 \cup X_1$. Sean los puntos de X_0 pertenecientes a la clase 0 y los de X_1 de la clase 1. Realizamos la demostración por contradicción. Por tanto, supondremos que $\overline{\text{conv}(X_0)} \cap \overline{\text{conv}(X_1)} \neq \emptyset$, entonces existe un $x \in \overline{\text{conv}(X_0)} \cap \overline{\text{conv}(X_1)}$. Debemos notar que para todo $x \in \overline{\text{conv}(X_0)}$, $f(x) = 0$, mientras que si $x \in \overline{\text{conv}(X_1)}$, entonces $f(x) = 1$, ya que el hiperplano separador crea regiones convexas. Sin pérdida de generalidad, podemos suponer que $f(x) = 1$, por lo que $w x + b > 0$, pero como x también es parte de X_0 , entonces $f(x) = 0$, por lo que $w x + b \leq 0$, pero esto es una contradicción que proviene de asumir que los conjuntos no son disjuntos.

Ahora, si suponemos que $\overline{\text{conv}(X_0)} \cap \overline{\text{conv}(X_1)} = \emptyset$, podemos garantizar que existe un hiperplano que separa a ambos conjuntos. De forma intuitiva, existe una separación entre las envolturas convexas de ambos conjuntos (pues son disjuntas). Podemos elegir el punto medio entre la distancia entre ambos conjuntos para trazar el hiperplano garantizando que los puntos de la clase 0 queden por debajo de éste, mientras que los de la clase 1 queden por encima (la Figura 2.6 da un ejemplo gráfico de esto). Una demostración formal se obtiene como consecuencia del teorema de Hahn-Banach,¹ el cual mencionamos en la Sección 3.2. $\square$

El Teorema 5 da condiciones necesarias y suficientes para que un problema de clasificación binaria sea resuelto por el perceptrón. La Figura 2.6 da un ejemplo de esta separación en $\mathbb{R}^2$. Con base en el resultado expuesto, podemos dar la siguiente definición:

Definición 44 (Linealmente separable). Un conjunto de datos $X = X_0 \cup X_1$ de un problema de clasificación binaria es linealmente separable si existe un perceptrón $f(x)$ que puede clasificarlo correctamente.

¹ Puede revisarse la segunda forma geométrica del teorema de Hahn-Banach en Brézis (1984).

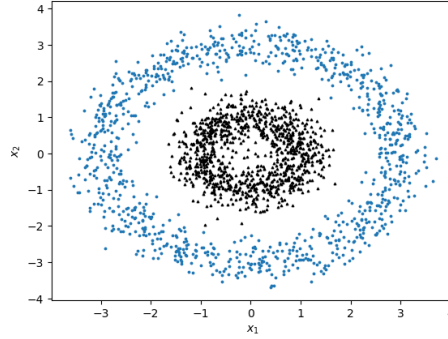


Figura 2.7: Un problema de clasificación binaria que no es linealmente separable

Es decir, X es linealmente separable si los envolventes convexos de los conjuntos de sus clases son disjuntos.

Por tanto, podemos encontrar una equivalencia entre conjuntos linealmente separables y conjuntos que un perceptrón puede clasificar. Esta discusión también nos muestra los límites del perceptrón. Por ejemplo, la Figura 2.7 muestra un problema que no es linealmente separable, es decir, que no puede ser resuelto por el perceptrón. En este ejemplo notamos que el envolvente convexo de la clase representada por el anillo superior contiene a la clase que se encuentra en su interior. Si decimos que el conjunto del círculo interior es X_0 , notamos que $\text{conv}(X_0) \subseteq \text{conv}(X_1)$. El tomar al envolvente convexo es de especial importancia, pues en este mismo ejemplo notamos que $X_0 \cap X_1 = \emptyset$, pero no podemos encontrar ninguna recta que separe los datos, pues sus envolventes convexos no son disjuntos.

2.2.1. Perceptrón y problemas lógicos

Los modelos neuronales como el de McCulloch and Pitts (1943) se propusieron con la intención de solucionar problemas lógicos. El perceptrón, de hecho, tiene la capacidad de solucionar los problemas lógicos básicos: AND, OR y NOT.² Para solucionar el problema lógico cada una de las proposiciones en los operadores binarios AND y OR representa una entrada de un vector x . Estos problemas lógicos conforman, entonces, un espacio 2 dimensional (véase Figura 2.8).

Si bien los vectores son binarios, puede pensarse que los datos conforman variedades que separan las regiones del espacio $\mathbb{R}^2$ de tal forma que la solución del perceptrón, además, es sensible al ruido. De esta forma, se puede ver que los siguientes pesos solucionan cada uno de los problemas lógicos básicos:

1. Para el problema AND, $w^T = (1 \ 1)$ y $b = -1,5$.
2. Para el problema OR, $w^T = (1 \ 1)$ y $b = -0,5$.

²Generalmente, el problema lógico unario NOT se plantea en dos dimensiones, aplicándose a alguna de las entradas.

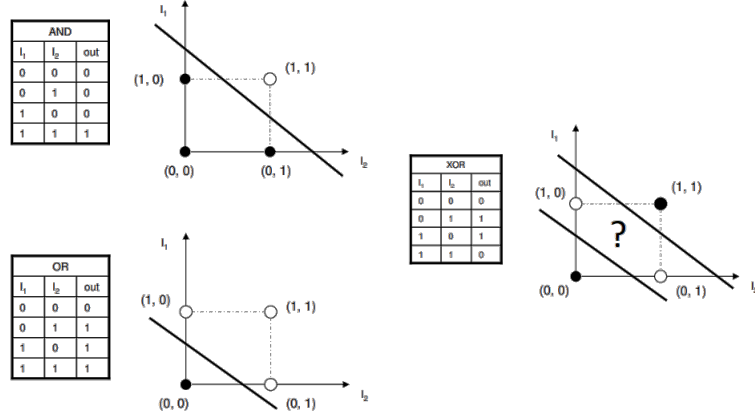


Figura 2.8: Solución del perceptrón al problema AND y OR, así como la incapacidad de solucionar el problema XOR

3. Para el problema NOT en la primera entrada, $w = (-1 \ 0)^T$ y $b = 0,5$.

Sin embargo, como hemos señalado en la sección anterior, el perceptrón tiene limitaciones. Una de los problemas que han representado con mayor claridad las limitaciones del perceptrón es el problema lógico XOR (véase Figura 2.8 derecha). El problema XOR es un contraejemplo que muestra que el perceptrón no es capaz de lidiar con problemas complejos, o no linealmente separables.

Proposición 6. *El perceptrón no es capaz de realizar una clasificación correcta en el problema XOR.*

Demostración. El problema XOR asocia vectores en $\mathbb{R}^2$ a las salidas de la siguiente forma:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

En otras palabras, el problema lógico XOR está determinado por una función binaria $y : \{0, 1\}^2 \rightarrow \{0, 1\}$ dada como:

$$y(x_1, x_2) = \begin{cases} 1 & \text{si } x_1 \neq x_2 \\ 0 & \text{si } x_1 = x_2 \end{cases}$$

Podemos suponer que existe un conjunto de parámetros $w^T = (w_1 \ w_2)$, b que es capaz de solucionar el problema XOR. Entonces debe de realizar las asignaciones anteriores en base a la regla de clasificación que hemos establecido para el perceptrón. De las asignaciones a la clase 1, obtenemos las desigualdades:

$$\begin{aligned}w_1 + b &> 0 \\w_2 + b &> 0\end{aligned}$$

Como el vector $(0 \ 0)$ se clasifica en la clase 0, entonces $b \leq 0$. De estas tres desigualdades podemos obtener:

$$b \leq 0 < w_1 + w_2 + 2b$$

Y restando b , es claro que:

$$0 < w_1 + w_2 + b = w_1(1) + w_2(1) + b$$

Pero esto quiere decir que el vector $(1 \ 1)$ es clasificado en la clase 1, lo que contradice la suposición de que los pesos escogidos clasifican correctamente el problema XOR, puesto que en este caso, tendríamos $w_1 + w_2 + b \leq 0$.

De esta contradicción podemos concluir que no existe un conjunto de parámetros del perceptrón que resuelva correctamente el problema XOR; por tanto, el perceptrón no puede solucionar el problema XOR. $\square$

Si bien el problema XOR no puede solucionarse por un perceptrón simple, nos puede dar una intuición de cómo resolver este problema. Recordemos que la compuerta XOR es un problema lógico compuesto, el cual puede definirse a partir de compuertas lógicas básicas (AND, OR y NOT) de la siguiente forma:

$$\text{XOR}(x_1, x_2) = \text{OR}\left(\text{AND}(\text{NOT}(x_1), x_2), \text{AND}(x_1, \text{NOT}(x_2))\right)$$

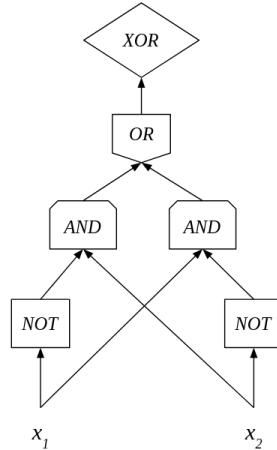


Figura 2.9: Posible solución al problema XOR a partir de aplicación consecutiva de perceptrones para problemas básicos

También existen otras formas de definir al problema XOR, pero lo importante es que podemos definir este problema a partir de compuertas lógicas básicas. Esta es la base de los circuitos lógicos. Sabiendo que el perceptrón puede solucionar estos problemas lógicos. Podemos entonces pensar que aplicar de forma consecutiva el perceptrón para problemas lógicos básicos específicos, podemos obtener una función compuesta de perceptrones que compute el problema XOR. En la Figura 2.9 se muestra de forma gráfica esta propuesta: cada nodo que es representado por una compuerta lógica debe pensarse como un perceptrón particular. Esta idea, como veremos en el Capítulo 3, se puede extender para resolver problemas no linealmente separables. En este caso, tendremos que desarrollar un método para poder aprender los pesos de cada uno de estos perceptrones en conjunto.

2.3. Modelos de una neurona

Antes de profundizar en las redes multicapa, ahondaremos en el concepto de neurona artificial. Ya hemos visto que el perceptrón es un modelo de una neurona. En esta sección buscamos ampliar estos modelos para abarcar un número más grandes de neuronas artificiales que podrán utilizarse dentro de las redes neuronales profundas. En primer lugar, retomaremos los modelos de regresión lineal (Sección 1.6) y regresión logística (Sección 1.7). En primer lugar, observamos que el perceptrón y la regresión logística, a pesar de ser modelos que suelen implementarse de forma distinta, representan familias de funciones equivalentes.

Teorema 6. Sea $f_{perc} : X \rightarrow \{0, 1\}$ un perceptrón con los pesos $\theta = \{w, b\}$. Entonces la función de regresión logística $f_{log} : X \rightarrow [0, 1]$ con los mismos pesos θ realiza la misma clasificación que f_{perc} .

Demostración. Sea $f_{perc}(x; \theta)$ un perceptrón con pesos $w \in \mathbb{R}^d$ y $b \in \mathbb{R}$. Sea $f_{log}(x; \theta)$ una función de regresión logística con los mismos pesos del perceptrón. La predicción de una clase $\hat{y}$ en la regresión logística se da como:

$$\hat{y} = \arg \max_y p(Y = y|x)$$

En donde $p(Y = 1|x) = f_{log}(x; \theta)$ y $p(Y = 0|x) = 1 - f_{log}(x; \theta)$. Claramente, se elige la clase 1 si $f_{log}(x; \theta) > 0,5$ y la clase 0 en otro caso. Es decir, podemos definir una regla de clasificación en la regresión logística como:

$$\hat{y} = \begin{cases} 1 & \text{si } f_{log}(x; \theta) > 0,5 \\ 0 & \text{si } f_{log}(x; \theta) \leq 0,5 \end{cases}$$

Dado que $f_{log}(x; \theta) > 0,5$ si y sólo si $w x + b > 0$ (véase Ejercicio 9), podemos ver que la regresión lineal asignará la clase 1 cuando se cumpla esta condición, mientras que asignará la clase 0 si $w x + b \leq 0$. Justamente esta es la regla de asignación del perceptrón. Por lo que f_{perc} y f_{log} realizarán la misma asignación de clases bajo el mismo conjunto de pesos.

□

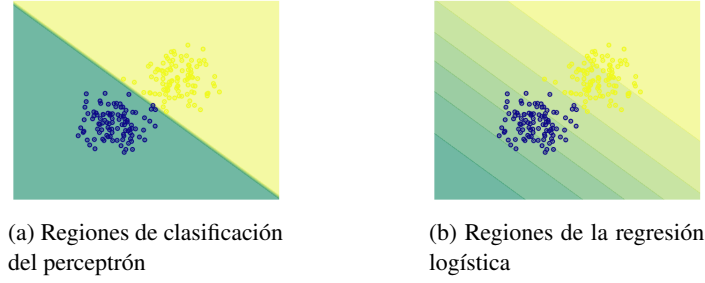


Figura 2.10: Visualización de las regiones de clasificación del perceptrón y la regresión logística

Cabe señalar que si bien tanto el perceptrón como la regresión logística realizan la misma clasificación de los datos bajo el mismo conjunto de pesos, el aprendizaje en ambos modelos puede llevar a que, dado un conjunto de datos, no se obtengan los mismos valores de los pesos. La diferencia entre ambos modelos radica en su función de decisión, pues el perceptrón se basa en una función escalonada, mientras que la regresión logística es una función suave. En la Figura 2.10 se puede observar en (a) que el perceptrón separa los datos en dos regiones bien definidas, mientras que la regresión logística (b) muestra un cambio gradual: entre más alejado esté un dato del hiperplano definido por w y b , mayor es su probabilidad de pertenecer a alguna de las clases. El entrenamiento de ambos modelos también será distinto, en cuanto el algoritmo para el perceptrón se detendrá en cuanto encuentre una clasificación adecuada para los datos, mientras que en la regresión logística se detendrá únicamente al completar un número de épocas dado. A pesar de esto, podemos pensar que dado un conjunto de pesos aprendido durante el entrenamiento de alguno de los modelos, estos pesos se pueden utilizar en el otro modelo obteniendo la misma clasificación.

Por tanto, podemos concluir que la diferencia entre perceptrón y regresión logística se da en el valor que se obtiene de salida. Con esto en mente, generalizaremos el concepto de modelo de una neurona observando que podemos separar la función de la neurona artificial en dos partes.

Definición 45 (Pre-activación). *Una función de pre-activación en un modelo de una neurona será la función $a : \mathbb{R}^d \rightarrow \mathbb{R}$ definida como:*

$$a(x) = wx + b$$

Con $w \in \mathbb{R}^d$ y $b \in \mathbb{R}$ los pesos de las conexiones.

Cuando no haya confusión, denotaremos a la función de pre-activación solamente como $a = wx + b$.

Definición 46 (Función de activación). *Una función de activación $g : \mathbb{R} \rightarrow \mathbb{R}$ es una función que toma como entrada el valor de una pre-activación.*

De estas dos definiciones podemos observar que tanto el perceptrón, como la regresión lineal se componen de una pre-activación y una activación; es justamente la función de activación la que diferencia a ambos modelos.

Definición 47 (Modelo de una neurona). *Un modelo de una neurona es un modelo matemático que representa la función de una neurona biológica en base a una función de pre-activación $a : \mathbb{R}^d \rightarrow \mathbb{R}$ y una activación $g : \mathbb{R} \rightarrow \mathbb{R}$. Formalmente se define como:*

$$f(x) = (g \circ a)(x) = g(wx + b)$$

donde $w \in \mathbb{R}^d$ representan los pesos de las conexiones y $b \in \mathbb{R}$ el bias o umbral de activación.

Con base en esto, podemos extender el concepto de neurona artificial modificando la función de activación. En particular, los tres modelos que hemos estudiado hasta ahora, regresión logística, lineal y el perceptrón, son modelos de neuronas:

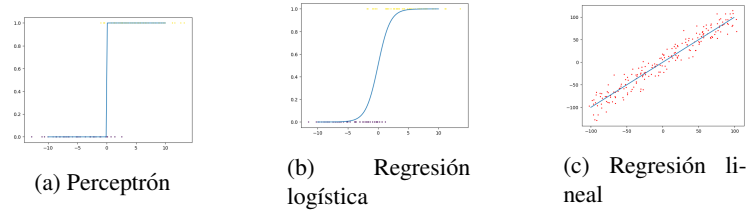


Figura 2.11: Funciones de activación en diferentes modelos de neuronas

- El **perceptrón** está definido por la función:

$$f(x) = 1_{>0}(wx + b)$$

Donde $1_{>0}$ es la función indicadora definida como:

$$1_{>0}(a) = \begin{cases} 1 & \text{si } a > 0 \\ 0 & \text{en otro caso} \end{cases}$$

- La **regresión logística** está definida por la función:

$$f(x) = \sigma(wx + b)$$

Con σ función de activación sigmoide.

- La **regresión lineal** está definida por la función:

$$f(x) = Id(wx + b)$$

Donde Id es la función identidad.

La Figura 2.11 muestra las gráficas de las tres funciones de activación. Cada modelo tiene aplicaciones diferentes: la regresión lineal se utiliza en problemas de regresión, donde se busca obtener el valor esperado dado un valor de entrada. El perceptrón se aplica a problemas de clasificación. La regresión logística también suele aplicarse a problemas de clasificación pero estima una probabilidad. Esto muestra la flexibilidad de las redes neuronales para resolver diferentes tipos de problemas. Existen otras funciones de activación que se revisarán más adelante.

Ejercicios

1. Demostrar que, dada w^* la solución óptima de un perceptrón, y si para todo x , $\|x\| \leq \rho$, y además:

$$\frac{|w^* \cdot x|}{\|w^*\|} \geq \gamma$$

entonces el número de épocas, T en que el perceptrón converge cumple:

$$T \leq \frac{\rho}{\gamma^2}$$

2. Si se asume que $\cos(\angle w, x) \propto w^T x$, se puede reformular la función del perceptrón como:

$$f(x) = \begin{cases} 1 & \text{si } \angle w, x < \frac{\pi}{2} \\ 0 & \text{si } \angle w, x \geq \frac{\pi}{2} \end{cases}$$

Derivar la fórmula de aprendizaje $w - (f(x) - y)x$ a partir de argumentar los errores en base a los ángulos entre el vector w y los elementos x mal clasificados (nota que el ángulo cambia respecto a $\cos(\angle w, x) \pm x^T w$).

3. Desarrollar un perceptrón para el operador lógico de implicación $x_1 \rightarrow x_2$, así como para los operadores de NAND y NOR.
4. A partir de los siguientes datos de entrenamiento:

X_1	X_2	X_3	Y
1	1	1	1
1	0	1	1
0	1	1	1
1	0	0	0
0	1	0	0
0	0	0	0

Aplicar el algoritmo del perceptrón por 5 iteraciones, iniciando con los pesos $w^T = (1 \ 1 \ 1)$ y $b = 1$ con rango de aprendizaje $\eta = \frac{1}{2}$. ¿Cuál es el valor final de los pesos?

5. Determinar un perceptrón que compute la siguiente función $f: \mathbb{R}^2 \rightarrow \mathbb{R}^2$:

X_1	X_2	Y_1	Y_2
1	1	1	1
1	0	0	1
0	1	1	0
0	0	0	0

6. Comprueba que el conjunto de pesos $w^T = (1 \ 1)$ y $b = -1,5$ solucionan el problema lógico AND.

7. Comprueba que el conjunto de pesos $w^T = (1 \ 1)$ y $b = -0,5$ solucionan el problema lógico OR.
8. Comprueba que el conjunto de pesos $w^T = (-1 \ 0)$ y $b = 0,5$ solucionan el problema lógico OR sobre la primera entrada y propón una solución para este problema sobre la segunda entrada.
9. Demuestra que en un modelo de regresión logística $f(x)$, si $wx + b > 0$, entonces $f(x) > 0,5$, mientras que si $wx + b < 0$, entonces $f(x) < 0,5$.
10. Implementa el perceptrón para un problema linealmente separable.

Capítulo 3

Redes feedforward

Los modelos de una sola neurona como el perceptrón o la regresión logística no son capaces de solucionar un problema lógico complejo como el XOR. En tanto modelos lineales, sólo son capaces de resolver un problema de clasificación cuando el problema es linealmente separable. Lo mismo podemos decir de la regresión lineal, que puede estimar correctamente los datos de los valores esperados sólo cuando existe una correlación lineal entre los datos. Sin embargo, en la práctica la mayoría de los problemas tienen comportamientos que no son lineales. Debemos, entonces, enfocarnos en modelos que sean capaces de solucionar problemas complejos. En particular veremos que las redes feedforward son capaces de solucionar problemas con datos no necesariamente lineales y revisaremos los teoremas de aproximador universal que señalan que este tipo de redes neuronales son capaces de aproximar cualquier función.

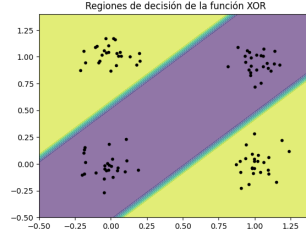
3.1. Capas ocultas y redes feedforward

Como se señaló en la Sección 2.2.1, podemos tomar un conjunto de neuronas que, de forma conjunta, resuelvan un problema complejo como el XOR. La dificultad es que no siempre podremos saber qué tipo de neuronas utilizar para cada problema particular. De hecho, dentro del aprendizaje automático lo que buscamos es justamente aprender los tipos de neuronas que puedan resolver el problema. Lo que haremos, entonces, es procesar la entrada en diferentes niveles hasta ser capaz, en un último nivel, de resolver el problema. Como hemos mencionado varias veces, el aprendizaje profundo es parte del aprendizaje representacional, por lo que lo que haremos es aprender una representación de los datos que sea linealmente separable.

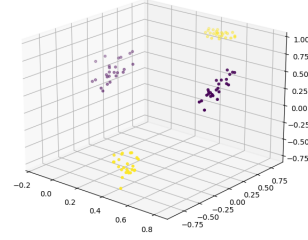
Para hacer esto, la idea esencial es aplicar una transformación a los datos de entrada $x \in \mathbb{R}^d$ que los lleve hacia un espacio en el que sean linealmente separables y, entonces, ahí aplicar un modelo lineal de una neurona. Es decir, debemos encontrar una transformación $h : \mathbb{R}^d \rightarrow \mathbb{R}^m$ que tome los datos de entrada y los lleve hacia un espacio $\mathbb{R}^m$, para un m arbitrario, donde exista una solución por medio del modelo de una neurona.

En la Figura 3.1 vemos una posible solución al problema XOR bajo este esquema. En el lado derecho (b) se observa una transformación de los datos al espacio $\mathbb{R}^3$ donde

existe un hiperplano separador que clasifica adecuadamente los datos. Las regiones de clasificación que se obtiene bajo esta transformación se observan en el lado izquierdo (a). Para obtener esta clasificación se aplica una neurona que toma los datos en $\mathbb{R}^3$ y los clasifica. El modelo final es la composición de la transformación más la neurona de salida que nos da la clasificación.



(a) Regiones de clasificación



(b) Transformación de los datos

Figura 3.1: La solución al problema XOR por medio de una red feedforward con una capa oculta

La idea esencial que radica en las redes profundas es el aprendizaje de representaciones de los datos de entrada que permitan la resolución del problema. Estas representaciones pueden realizarse de forma consecutiva por diferentes niveles en la red neuronal. A cada nivel en esta transformación de los datos lo llamaremos una capa oculta.

Definición 48 (Capa oculta). *Una capa oculta en una red neuronal profunda es una transformación $h : \mathbb{R}^d \rightarrow \mathbb{R}^m$ de los datos que tiene la forma:*

$$h(x) = g(Wx + b)$$

Con $W \in \mathbb{R}^{m \times d}$, $b \in \mathbb{R}^m$ y $g : \mathbb{R} \rightarrow \mathbb{R}$ una función de activación que se aplica punto a punto.

Escribiremos en lo que sigue sólo $h = g(Wx + b)$. Una capa oculta, entonces, puede verse como un conjunto de neuronas que toman una misma entrada. En la Figura 3.2 podemos ver una red con una capa oculta. La capa oculta se compone de cuatro neuronas. A las neuronas de las capas ocultas podemos llamarles unidades (ocultas). Las unidades de una capa oculta se corresponden a la dimensión del vector resultante $h \in \mathbb{R}^m$. En esta figura se ha resaltado las conexiones de la primera neurona para hacer notar que podemos ver que una capa oculta se compone de un conjunto de neuronas conectadas a la misma entrada.

También en la Figura 3.2 se puede ver que la capa oculta está conectada a otra neurona; esta última neurona puede verse como el modelo de neurona que se encargará de dar la respuesta con base en la representación de la capa oculta. Llamaremos a esta parte de la red la **capa de salida**. Si la neurona en la capa de salida es un perceptrón, podemos hablar que nuestra red es un perceptrón multi-capas.

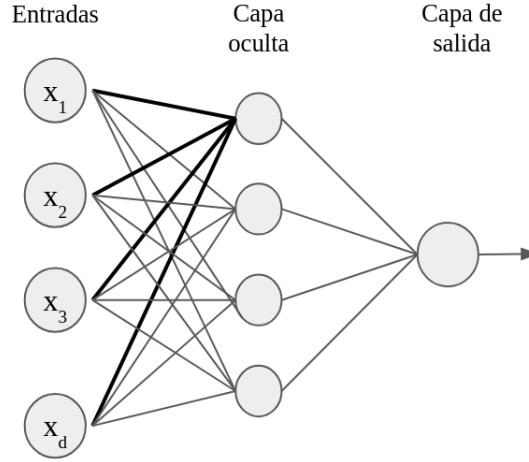


Figura 3.2: Ejemplo de una red feedforward con una capa oculta con 4 unidades

Lo que estamos haciendo al introducir capas ocultas es hacer que los datos de entrada se procesen por otras neuronas antes de que se obtenga la respuesta de las neuronas de salida. Podemos introducir más de una capa oculta. Por ejemplo, se puede introducir una segunda capa oculta que tome como entrada a la representación obtenida por una capa oculta previa.

Definición 49 (Red neuronal feedforward). *Una red neuronal del tipo feedforward con K capas ocultas es una función $f : \mathbb{R}^d \rightarrow \mathbb{R}^{m_{K+1}}$ definida como:*

$$f(x) = \phi(W^{(K+1)}h^{(K)} + b^{(K+1)})$$

Tal que $W^{(K+1)} \in \mathbb{R}^{m_{K+1} \times m_K}$ y $b^{(K+1)} \in \mathbb{R}^{m_{K+1}}$ son los pesos de la capa de salida, ϕ es la activación en la capa de salida y $h^{(K)} \in \mathbb{R}^{m_K}$ es la representación oculta obtenida recursivamente como:

$$h^{(1)} = g_1(W^{(1)}x + b^{(1)})$$

$$h^{(k)} = g_k(W^{(k)}h^{(k-1)} + b^{(k)})$$

Para toda $k \in \{2, 3, \dots, K\}$, con $x \in \mathbb{R}^d$ entrada, $W^{(k)} \in \mathbb{R}^{m_k \times m_{k-1}}$ y $b^{(k)} \in \mathbb{R}^{m_k}$ los pesos en cada capa y g_k las funciones de activación.

Como nos podemos percatar de la definición anterior, las redes feedforward se basan en la idea de los modelos de neuronas, pero en lugar de tomar como entrada los estímulos (datos) de manera cruda, procesan los datos para obtener una representación adecuada para la solución del problema. Esta representación se obtiene por la aplicación consecutiva de capas ocultas. Las capas ocultas también se basan en neuronas. Todas las capas, incluyendo las capas de salida tienen una forma similar, pero cabe señalar que los pesos de las conexiones y el bias no son los mismos en cada capa. Las funciones de activación también pueden variar entre capa y capa, aunque en algunos casos prácticos se suele usar la misma función de activación en todas las capas para

simplificar la construcción de la red, excepto en la capa de salida, la cual tiene que ser elegida dependiendo del problema que se busque resolver (por esto hemos denotado la función de activación de la capa de salida de forma distinta).

Ya que cada capa está definida como un modelo de neurona, podemos también separar las capas en pre-activación y activación. La **pre-activación** en cada capa será denotada como:

$$a^{(k)} = W^{(k)}h^{(k-1)} + b^{(k)} \quad (3.1)$$

Por su parte, la **activación** en cada capa será denotada como:

$$h^{(k)} = g_k(a^{(k)}) \quad (3.2)$$

Usaremos una notación similar para la capa de salida, teniendo $f(x) = \phi(a^{K+1})$, donde a^{K+1} es la pre-activación de esta capa. Esta notación nos permitirá simplificar algunos cálculos sobre las redes feedforward.

El resultado de las capas profundas son vectores $h^{(k)} \in \mathbb{R}^{m_k}$ cuyas entradas representan las unidades o neuronas. Cada capa oculta cuenta con m_k neuronas. Los valores de cada m_k , $k = \{1, \dots, K\}$, para cada capa oculta son hiperparámetros. Asimismo, el número de capas ocultas K es un hiperparámetro.

Definición 50 (Anchura). *La anchura de una red neuronal está definida como:*

$$width(f) = \max\{m_k : k = 1, \dots, K\}$$

Donde m_k es el número de unidades ocultas en la capa k . Es decir, la anchura es el máximo número de unidades con las que cuenta una red neuronal.

Definición 51 (Profundidad). *La profundidad de una red feedforward es el número K de capas ocultas con las que cuenta.*

Determinar la anchura y la profundidad de una red feedforward influye en su capacidad para resolver un problema. Podemos decir, por ejemplo, que una red feedforward con profundidad $K = 0$ define un modelo lineal que, dependiendo de la activación, puede ser el perceptrón, regresión logística o regresión lineal. Es decir, los modelos de una sola neurona, modelos lineales, son un tipo de red neuronal feedforward sin capas ocultas. El número de capas y unidades ocultas definirá la arquitectura.

Definición 52 (Arquitectura). *La arquitectura de una red feedforward es el conjunto $\{K, m_1, \dots, m_K\}$; es decir, el número de capas ocultas y las unidades ocultas en cada capa.*

La arquitectura de una red feedforward define su forma. Asimismo, influirá en su capacidad, pues sabemos que una red sin capas ocultas no es capaz de resolver problemas no linealmente separables. De hecho, mostraremos que basta una capa oculta para aproximar cualquier función, aunque el número de unidades ocultas en esta capa puede ser muy grande. Podemos notar que el número de parámetros o pesos de conexiones de una red neuronal está dada por la fórmula:

$$|\theta| = \sum_{k=1}^{K+1} m_k(m_{k-1} + 1)$$

donde $m_0 = d$ es el número de entradas o la dimensión de los datos y m_{K+1} es el número de neuronas de la capa salida.

En la arquitectura también se pueden especificar las funciones de activación. Generalmente las funciones de activación g tienen ciertas propiedades, como no ser polinómicas o ser sigmoideas. La función en la salida responde al problema que se quiera resolver. Las funciones de activación tienen especial importancia en garantizar las capacidades de aproximación de las redes neuronales, por los que las revisamos a detalle.

3.1.1. Funciones de activación

Las funciones de activación representan la forma en que las neuronas emiten una señal de salida. Dependiendo de que tipo de función de activación escojamos, podemos tener diferentes respuestas a problemas particulares. Por ejemplo, en el perceptrón se utiliza una función indicadora $1_{>0}$ que emite un valor 1 si la pre-activación es positiva y 0 en otro caso. Como utilizaremos métodos basados en gradiente para el entrenamiento de las redes neuronales, las funciones de activación deben ser derivables; por esta razón es que en la práctica no se suele usar esta función indicadora. Sin embargo, como hemos visto, podemos realizar una clasificación con una función sigmoide.

En general, existen ciertos criterios para elegir una función de activación. En primer lugar, debemos considerar si la activación es en las capas ocultas o en la capa de salida. En las capas ocultas debemos garantizar que la función de activación elegida nos garantice la aproximación universal. Algunos de los criterios para esto es que la función no sea polinomial. De hecho, funciones como el coseno pueden usarse como funciones de activación en las capas ocultas. En la práctica suelen usarse funciones de activación que además tengan una derivada sencilla. Una función de activación clásica es la función sigmoide.

Definición 53 (Función sigmoide). *Dada una pre-activación $a \in \mathbb{R}$ de una unidad neuronal, la función de activación sigmoide se define como:*

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

La función sigmoide define una probabilidad de una variable binaria, como lo hemos visto en la Sección 1.7. Los valores 0 y 1 sólo se alcanzan en el límite. La derivada de la función sigmoide es sencilla, pues se expresa en términos de sí misma, lo que permite que la implementación de la función y su derivada sea eficiente, pues no deben hacerse nuevos cálculos para la obtención del gradiente.

Proposición 7. *La derivada de la función sigmoide está dada como:*

$$\sigma'(a) = \sigma(a)(1 - \sigma(a))$$

Definición 54 (Tangente hiperbólica). *Dada una pre-activación $a \in \mathbb{R}$ de una unidad neuronal, la función de activación de tangente hiperbólica se define como:*

$$\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}$$

La tangente hiperbólica comparte propiedades con la función sigmoide pues ambas son acotadas y tienen forma de S. Pero la tangente hiperbólica toma valores entre -1 y 1, llegando a estos valores sólo en sus límites. La tangente hiperbólica se ha utilizado en extenso para la activación en las capas ocultas pues permite separar valores negativos, que suelen tener una influencia negativa en la decisión de la salida, de los valores positivos. Asimismo, su derivada se expresa en términos de sí misma.

Proposición 8. *La derivada de la tangente hiperbólica está dada como:*

$$\tanh'(a) = 1 - \tanh^2(a)$$

Las funciones sigmoide y tangente hiperbólica fueron de las primeras funciones de activación utilizadas para las capas ocultas en las redes profundas. De hecho, las primeras demostraciones sobre la capacidad de aproximación de funciones de las redes feedforward se basaba en las características de estas funciones (véase la Sección 3.2). Sin embargo, estas funciones suelen mostrar lo que se conoce como el problema del desvanecimiento del gradiente, esto es, pueden presentar un gradiente 0 en la implementación debido a que con valores grandes las funciones se aplastan.

Una alternativa a las funciones acotadas como la sigmoide y la tangente hiperbólica son funciones que tengan valores altos para los valores positivos, mientras que los valores negativos les asignen valores bajos. Una de ellas es la función softplus.

Definición 55 (Función softplus). *Dada una pre-activación $a \in \mathbb{R}$ de una unidad neuronal, la función de activación softplus se define como:*

$$\text{softplus}(a) = \ln(1 + e^a)$$

La función softplus hace los valores negativos pequeños, mientras que conserva los valores positivos. Se puede observar que en los valores positivos grandes la función es cercana a la identidad, mientras que en los valores negativos grandes es cercano a 0. La función softmax también muestra una derivación simple.

Proposición 9. *La derivada de la función softplus es la función sigmoide; esto es:*

$$\text{softplus}'(a) = \sigma(a)$$

La derivada de la softplus no se define en términos de sí misma, pero se puede aprovechar el cálculo de la exponencial en ésta para su derivada. En la práctica, sin embargo, la softplus no tiene tan buenos resultados como su versión dura, la función ReLU. La función ReLU es actualmente la función de activación estándar para las redes profundas. Como se puede ver en la parte superior de la Figura 3.3, la función ReLU es una versión rígida de la softplus. La softplus, por tanto, es una versión suave de esta última. A pesar de esto, la función ReLU muestra mejor desempeño en los ejemplos reales.

Definición 56 (Función ReLU). *Dada una unidad de pre-activación $a \in \mathbb{R}$, la función ReLU (Rectified Linear Unit) es la función de activación definida como:*

$$\text{ReLU}(a) = \max\{0, a\}$$

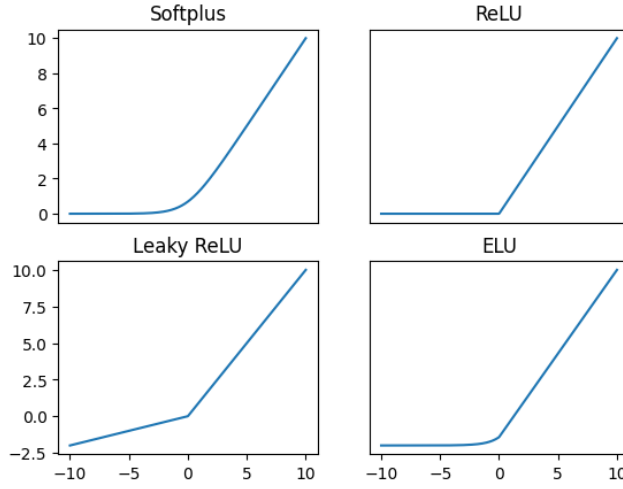


Figura 3.3: Funciones de activación basadas en rectificación

La función ReLU, a pesar de su sencillez, es capaz de solucionar problemas complejos cuando se utiliza como función de activación en las capas ocultas. Su derivada también se expresa en términos simples.

Proposición 10. *La derivada de la función ReLU está dada por:*

$$ReLU'(a) = \frac{1}{|a|} \max\{0, a\}$$

Excepto en 0, donde no está definida.

Es decir, la derivada de la función ReLU es 0 en los valores negativos y 1 en los positivos. En el 0, la derivada no está definida, pero asumimos que los valores de pre-activación no tomarán este valor. La función ReLU es la base de muchas arquitecturas actuales, puesto que también define redes que son aproximadores universales (Huang, 2020), además que son más simples que otras funciones de activación. A partir de la función ReLU se han buscado otras funciones similares, donde los valores negativos tengan salidas pequeñas, mientras que la parte positiva sea lineal. Una generalización de la función ReLU es la función leaky ReLU:

Definición 57 (Leaky ReLU). *Dada una unidad de pre-activación $a \in \mathbb{R}$, la función leaky ReLU es la función de activación definida como:*

$$LeakyReLU(a) = \max\{0, a\} + \beta \min\{0, a\}$$

La función leaky ReLU no lleva los valores negativos a 0, sino que los valores negativos van a estar escalados por una constante β , generalmente pequeña (véase Figura 3.3). Si $\beta = 0$, la función leaky ReLU es la función ReLU. La derivada de esta función es 1 si los valores de entrada son positivos y β en otro caso.

Basada en la función ReLU, existen varias funciones; mencionamos dos más aquí, la función ELU y la función SiLU.

Definición 58 (Función ELU). *Dada una unidad de pre-activación $a \in \mathbb{R}$ la función ELU (Exponential Linear Unit) es la función de activación definida como:*

$$ELU(a) = \begin{cases} a & \text{si } a > 0 \\ \beta(e^a - 1) & \text{si } a \leq 0 \end{cases}$$

La función ELU es cercana a 0 en los valores negativos y, al igual que leaky ReLU, su pendiente en estos valores está ligada a la constante β . A diferencia de las funciones ReLU y leaky ReLU, la función ELU sí es derivable en todos los puntos del dominio. En la Figura 3.3 se puede ver que esta función es suave cercano al 0. Asimismo, puede observarse que si $\beta = 0$, entonces la función ELU es la función ReLU. Finalmente, la derivada de la función ELU es sencilla.

Proposición 11. *La derivada de la función ELU está dada como:*

$$ELU'(a) = \begin{cases} 1 & \text{si } a > 0 \\ \beta e^a & \text{si } a \leq 0 \end{cases}$$

Definición 59 (Función SiLU). *Dada una unidad de pre-activación $a \in \mathbb{R}$ la función SiLU (Sigmoid Linear Unit) es la función de activación definida como:*

$$SiLU(a) = a\sigma(a)$$

La función SiLU es suave en todas partes y, al igual que ELU, tiene derivadas en todos sus puntos. Se basa en la función sigmoide, pero al multiplicar por la pre-activación a la parte positiva tiene un comportamiento casi lineal (la función sigmoide es cercana a 1 en valores grandes), mientras que con valores negativos es cercana a 0 (en valores negativos la función sigmoide es cercana a 0). Por tanto, la función SiLU tiene un comportamiento similar a la ReLU pero de manera suave.

Las funciones de activación tienen impacto en la resolución del problema al que nos estamos enfrentando. En las capas ocultas es común usar alguna de estas funciones de activación: sigmoide, tangente hiperbólica, softplus, ReLU, leaky ReLU, ELU, SiLU, o alguna otra función similar. Debemos tomar en cuenta que la función sea derivable y que, además, pueda asegurar la propiedad de aproximador universal. La elección de una u otra función puede afectar la forma de aprendizaje. Ya mencionamos que las funciones sigmoide o tangente hiperbólica pueden dar pie a desvanecimiento del gradiente, lo que impedirá que la red neuronal aprenda a resolver el problema. El uso de la función ReLU puede llevar a usar más unidades ocultas. Si bien es recomendada en la práctica el uso de la función ReLU o sus derivadas, es claro que existen diferencias entre cada función.

En la Figura 3.4 se muestra las regiones de clasificación del mismo problema con una red con la misma arquitectura variando las funciones de activación de la capa oculta. Como se puede observar, la función de tangente hiperbólica encuentra regiones suaves, mientras que las regiones de clasificación de la ReLU son rígidas. La función

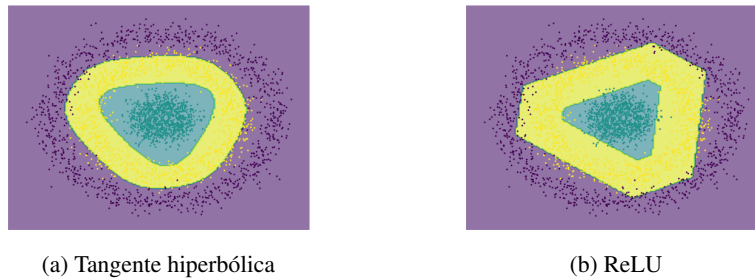


Figura 3.4: Regiones de clasificación de la función tangente hiperbólica y la función ReLU

ReLU define las regiones de clasificación en base a politopos (generalizaciones de los polígonos a dimensiones altas). En la práctica la selección de las funciones de activación muchas veces se hace de manera experimental.

Las funciones de activación que hemos revisado se utilizan principalmente en las capas ocultas. En la capa de salida, la selección de una función de activación debe responder a la tarea específica que buscamos resolver. Si bien podemos usar cualquier función derivable en la capa de salida según la tarea que busquemos resolver, la selección de la activación dependerá del problema.

- **Regresión:** Un problema de regresión tiene como salida un valor real continuo, por lo que se suele usar una activación lineal o identidad; es decir, no se aplica una función de activación, por lo que nuestra red neuronal será de la forma:

$$f(x) = Wh + b$$

Con h la última capa oculta. Aunque si se busca acotar los valores reales se puede usar alguna función de activación, como la ReLU que sólo obtendrá valores positivos.

- **Clasificación:** El problema de clasificación da como salida valores discretos y finitos bien definidos. Debido a que el aprendizaje en redes profundas requiere de funciones derivables, la clasificación se basa en maximizar las probabilidades. Por lo que las funciones de activación pueden ser la sigmoide (en caso de una clasificación binaria en una red con una sola neurona de salida) o la función Softmax (Definición 60).
- **Estimación de probabilidad:** Los problemas de estimación de probabilidad, como los de modelos generativos, estiman una probabilidad de salida, por lo que también utilizan funciones de activación basadas en probabilidad, principalmente la función Softmax.

Hasta ahora no hemos revisado la función Softmax. esta es una de las funciones con las que estaremos trabajando a lo largo del texto, ya que suele ser una de las más utilizadas dado que es útil tanto en la clasificación, como en la estimación de probabilidad. De hecho, es común que, a pesar de tener un problema de clasificación binaria,

se utilice la función Softmax (con dos neuronas de salida) en lugar de la sigmoide (con una sola neurona de salida).

Definición 60 (Función softmax). *Dada un vector de neuronas $a^T = (a_1 \ \dots \ a_m)$, la función de activación Softmax para la i -ésima neurona se define como:*

$$\text{Softmax}(a_i) = \frac{e^{a_i}}{\sum_{j=1}^m e^{a_j}}$$

La función Softmax se interpreta como una probabilidad cuando tenemos múltiples valores de salida. Esta función depende de todos los valores de salida por lo que, a diferencia de las funciones de activación previas, no se aplica punto a punto, si no que requiere del conocimiento de los valores de pre-activación de todas las otras neuronas en la capa. Cabe mencionar que la función Softmax se utiliza en la capa de salida y no en las capas ocultas. Como hemos mencionado, una parte importante de estas funciones es que todas son derivables y que, además, sus derivadas son fáciles de expresar. La función Softmax cumple esto, pues su derivada se expresa en términos de la misma función.

Proposición 12. *Las derivadas parciales de la función Softmax están dadas por:*

$$\frac{\partial \text{Softmax}(a_j)}{\partial a_i} = \text{Softmax}(a_j)(\delta_{i,j} - \text{Softmax}(a_i))$$

Demostración. Expresemos la función Softmax sobre la pre-activación de salida a_i como ϕ_i , esto es $\phi_i = \frac{e^{a_i}}{\sum_j e^{a_j}}$. Tómese el logaritmo $\ln \phi_i$ de tal forma que tenemos que:

$$\frac{\partial \ln \phi_j}{\partial a_i} = \frac{1}{\phi_j} \frac{\partial \phi_j}{\partial a_i}$$

Y de aquí, podemos ver que:

$$\frac{\partial \phi_j}{\partial a_i} = \phi_j \frac{\partial \ln \phi_j}{\partial a_i}$$

Falta derivar el término de la derecha:

$$\begin{aligned} \frac{\partial \ln \phi_j}{\partial a_i} &= \frac{\partial}{\partial a_i} \ln \frac{e^{a_j}}{\sum_k e^{a_k}} \\ &= \frac{\partial}{\partial a_i} \ln e^{a_j} - \frac{\partial}{\partial a_i} \ln \sum_k e^{a_k} \\ &= \frac{\partial a_j}{\partial a_i} - \frac{1}{\sum_k e^{a_k}} \frac{\partial a_j}{\partial a_i} \sum_k e^{a_k} \\ &= \delta_{i,j} - \frac{e^{a_i}}{\sum_k e^{a_k}} \\ &= \delta_{i,j} - \phi_i \end{aligned}$$

Por tanto, tenemos que:

$$\frac{\partial \phi_j}{\partial a_i} = \phi_j(\delta_{i,j} - \phi_i)$$

□

La derivada de la función Softmax tiene similitudes con la de la función sigmoide. Esto no es casualidad, la función Softmax puede verse como una generalización de la función sigmoide. Supongamos que tenemos una red feedforward con dos neuronas de salida (por ejemplo en clasificación binaria). Tenemos entonces que la pre-activación en la salida es un vector $a^T = (a_1 \ a_2)$. Si tomamos $a_2 = 0$ para todo valor de entrada, podemos observar que la función Softmax para la neurona a_1 es:

$$\text{Softmax}(a_1) = \frac{e^{a_1}}{e^{a_1} + e^0} = \frac{e^{a_1}}{e^{a_1} + 1} = \sigma(a_1)$$

De igual forma podemos comprobar que para a_0 se da:

$$\text{Softmax}(a_0) = \frac{e^0}{e^{a_1} + e^0} = \frac{1}{1 + e^{a_1}} = \sigma(-a_1) = 1 - \sigma(a_1)$$

Es decir, cuando la salida responde a una variable binaria, y una de las neuronas de salida ha sido apagada, la estimación de las probabilidades de una red con Softmax son equivalentes a la estimación de una red con la función sigmoide. Por tanto, la función sigmoide se utiliza cuando la salida de la red es una sola neurona, siendo equivalente al uso de la función Softmax. Esta equivalencia también se puede ver en la derivada, pues al tener una sola neurona siempre tendremos que en la expresión $\phi_j(\delta_{i,j} - \phi_i)$, $i = j$, por lo que la derivada será $\phi_i(1 - \phi_i)$, que es la derivada del sigmoide.

3.2. Teoremas de aproximador universal

Los teoremas de aproximador universal son una serie de teoremas que garantizan que las redes neuronales son capaces de aproximar tanto como se quiera a cualquier función de interés. La idea de la aproximación universal se basa en la idea de poder definir funciones (más sencillas o manejables) que sean tan cercanas como se quiera a otro conjunto de funciones (complejas o no conocidas). Podemos definir cuándo un conjunto de funciones es un aproximador universal.

Definición 61 (Aproximador universal). *Una clase funciones $\mathcal{F}$ se dice que es un aproximador universal sobre un conjunto compacto S si para toda función objetivo g y para toda $\varepsilon > 0$, existe $f \in \mathcal{F}$ tal que:*

$$\sup_{x \in S} |f(x) - g(x)| \leq \varepsilon$$

En particular, las redes neuronales pueden aproximar funciones continuas o Borel medibles $f : \mathbb{R}^d \rightarrow \mathbb{R}^o$. Para denotar esta familia de funciones usaremos la siguiente notación:

- $C(\mathbb{R}^d)$ es el conjunto de funciones continuas de $\mathbb{R}^d \rightarrow \mathbb{R}$.
- $M(\mathbb{R}^d)$ es el conjunto de funciones Borel medibles de $\mathbb{R}^d \rightarrow \mathbb{R}$.

Un problema matemático de gran relevancia ha sido encontrar aproximaciones a funciones en estos conjuntos a partir de representaciones simples. Por ejemplo, el teorema de representación de Arnold-Kolmogorov nos dice que toda función continua $f : [0, 1]^d \rightarrow \mathbb{R}$ puede ser aproximada por las sumas finitas:

$$f(x) = \sum_{j=0}^{2d} \phi_j \left(\sum_{i=1}^d g_{i,j}(x_i) \right)$$

Donde $\phi_j, g_{i,j} : \mathbb{R} \rightarrow \mathbb{R}$ son funciones continuas para toda i y toda j . En este caso, debemos encontrar estas funciones. En el caso de las redes neuronales, tenemos varios resultados que nos dicen que éstas son capaces de aproximar funciones. Aquí demostramos únicamente el caso de la aproximación de funciones continuas con funciones sigmoideas que presenta Cybenko (1989). Sin embargo, enunciamos también otros resultados sobre aproximaciones de funciones Borel medibles, y aproximación usando funciones de activación ReLU.

El resultado de Cybenko (1989) señala que se pueden aproximar funciones continuas a partir de una arquitectura de red neuronal con una capa oculta que tenga como función de activación una función sigmoideal. Si bien ya hemos hablado de la función sigmoide, el concepto de función sigmoideal de Cybenko puede verse como un caso más general.

Definición 62 (Función sigmoideal). *Una función $g : \mathbb{R} \rightarrow [0, 1]$ es una función sigmoideal si cumple:*

1. *Es no-decreciente.*
2. $\lim_{x \rightarrow \infty} g(x) = 1$
3. $\lim_{x \rightarrow -\infty} g(x) = 0$

Claramente, la función sigmoide $\sigma(x) = \frac{1}{1+e^{-x}}$ cumple las propiedades anteriores. Tomaremos ahora una forma de representar funciones a partir de sumas finitas con funciones sigmoideas. Definimos la función $f : I_d \rightarrow \mathbb{R}$, donde $I_d = [0, 1]^d$ es el hipercubo unitario, como sigue:

$$f(x) = \sum_{j=1}^N \theta_j g(w_j \cdot x + b_j) \quad (3.3)$$

Donde $\theta_j, b_j \in \mathbb{R}$ son escalares y $w_j \in \mathbb{R}^d$ es un vector. Claramente, esta representación define una red neuronal con una capa oculta, donde w_j son los pesos de la capa, b_j los bias, y θ_j los pesos de salida. Asimismo, $g : \mathbb{R} \rightarrow \mathbb{R}$ es una función continua no lineal. En particular, si tomamos la matriz $W \in \mathbb{R}^{N \times d}$, $b \in \mathbb{R}^N$ y $\theta \in \mathbb{R}^N$ podemos expresar estas sumas finitas en una forma más familiar como:¹

$$f(x) = \theta g(Wx + b) \quad (3.4)$$

Donde la función g se aplica sobre el vector $Wx + b$ punto a punto. Este tipo de funciones definen redes neuronales con una sola capa oculta y con una sola neurona de

¹Dado a que en esta sección sólo abarcaremos redes con una capa oculta, distinguimos los pesos de la capa de salida con el uso de la notación θ en lugar del uso de índices como lo hacemos a lo largo del texto.

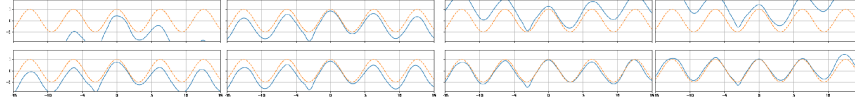


Figura 3.5: Aproximación de la función coseno por medio de redes neuronales

salida. Además, se puede ver que las redes neuronales que estas funciones definen determinan una arquitectura de regresión, pues no tiene una función de activación en la capa de salida. Esto es así puesto que, en primer lugar, nos interesa demostrar que una red neuronal con una sola capa oculta puede aproximar funciones continuas con valores en todo el espacio real $\mathbb{R}$. Este es un problema más amplio que, como veremos, después puede traducirse a un problema de clasificación. En particular, cabe notar que las funciones de probabilidad son funciones continuas.

Cabe señalar que aquí hemos delimitado el dominio de la función a entradas en el hipercubo $[0, 1]^d$, pero este dominio puede extenderse a cualquier hipercubo $X = [a, b]^d$ o incluso a otros espacios reales multidimensionales, siempre y cuando sean estos cerrados y acotados (en general, el dominio debe ser compacto); siguiendo la demostración de Cybenko, nos limitaremos al dominio I_d .

Una propiedad importante de estas sumas finitas es que la función $g : \mathbb{R} \rightarrow \mathbb{R}$ es una función continua **discriminatoria**, la cual entendemos de la siguiente forma:

Definición 63 (Función discriminatoria). *Una función $g : \mathbb{R} \rightarrow \mathbb{R}$ es discriminatoria cuando para una medida de Borel finita, regular y con signo μ se tiene que, si se da:*

$$\int_{I_d} g(wx + b) d\mu(x) = 0$$

para toda $w \in \mathbb{R}^d, b \in \mathbb{R}$, entonces $\mu \equiv 0$.

Resaltamos que las funciones sigmoideas son funciones discriminatorias. De esta forma, lo primero que debemos notar es que las funciones $f(x)$ determinadas por la representación en la Ecuación 3.4 pertenecen a $C(I_d)$, las funciones continuas con dominio en I_n . Claramente, ya que no tenemos ninguna función de activación, los valores de $f(x)$ son reales. Más aún, $f(x)$ es la composición de funciones continuas (g que por definición es continua, la función $Wx + b$ y la función $\theta \cdot x$), por tanto, toda función $f(x)$ es continua. A partir de esto, entonces podemos enunciar el siguiente resultado.

Teorema 7 (Aproximador universal (Cybenko, 1989)). *Sea $g : \mathbb{R} \rightarrow \mathbb{R}$ una función continua discriminatoria, y sea $\mathcal{F} = \{f : I_d \rightarrow \mathbb{R} : f(x) = \sum_{j=1}^N \theta_j g(w_j \cdot x + b_j)\}$. Entonces $\overline{\mathcal{F}} = C(I_d)$. Esto es, si $\phi \in C(I_d)$, entonces para toda $\varepsilon > 0$, existe $f \in \mathcal{F}$ tal que $\sup_{x \in I_d} |f(x) - \phi(x)| < \varepsilon$.*

El teorema de aproximador universal de Cybenko nos dice que una red neuronal puede aproximar a cualquier función continua de $C(I_d)$ con una sola capa oculta cuando la función de activación de esta capa es discriminatoria, o bien sigmoideal. Este resultado nos garantiza que las redes neuronales son herramientas de gran importancia,

pues a partir de ellas podemos obtener funciones desconocidas siempre que tengamos los datos que las representen.

En la Figura 3.5 se puede ver cómo una red neuronal aproxima una función coseno en varias iteraciones. La red neuronal se va ajustando poco a poco a la función modificando los pesos de sus conexiones. El dominio de la función no es sobre todos los reales sino sobre un intervalo cerrado, que cumple la condición de ser compacto.

En la siguiente sección se demuestra este teorema de aproximación universal. Su demostración requiere de ciertos conocimientos técnicos, principalmente requiere de conceptos del análisis funcional. Esta sección puede omitirse.

Demostración del teorema de aproximador universal

La demostración del teorema de aproximación universal de Cybenko (1989) se hace por reducción al absurdo. En primer lugar, denotemos que un conjunto A es denso en B como $\bar{A} = B$. Recuérdese que el concepto de densidad se puede pensar como en la Definición 61: los elementos del conjunto denso se acercan a los del conjunto objetivo. Antes de pasar a la demostración, debemos notar los siguientes resultados:

1. $\mathcal{F} \subset C(I_d)$; es decir, las funciones de la forma $\sum_{j=1}^N \theta_j g(w_j x + b_j)$ son funciones continuas. Más aún, este tipo de funciones son cerradas bajo la suma, por lo que conforman un subespacio de $C(I_d)$.
2. $\mathcal{F}$ es un subconjunto propio de $C(I_d)$, pues conocemos funciones continuas que no son de esta forma; por ejemplo, las funciones trigonométricas (donde $d = 1$).
3. Las funciones discriminatorias cumplen $g(wx + b) \in \overline{\mathcal{F}}$; en particular, podemos ver que $g(wx + b) \in \mathcal{F}$ cuando $N = 1 = \theta_1$.

Ahora bien, podemos suponer que $\overline{\mathcal{F}} \neq C(I_d)$; es decir, que las sumas finitas no son densas en $C(I_d)$ y que por tanto, existen funciones continuas en $C(I_d)$ que no pueden ser aproximadas por funciones en $\mathcal{F}$. Para continuar con la demostración, debemos recurrir a un resultado de gran importancia del análisis funcional: el teorema de Hahn-Banach. El teorema de Hahn-Banach trata sobre la extensión y separabilidad en espacios lineales y suele usarse para demostrar densidad de subespacios.

Teorema 8 (Hahn-Banach). Sea ρ un funcional definido sobre un espacio normado X que cumpla: i) $\forall x, y \in X, \rho(x + y) \leq \rho(x) + \rho(y)$; y ii) $\forall x, y \in X, \rho(\lambda x) = \lambda \rho(x)$ con $\lambda \in \mathbb{R}$. Sea G un funcional lineal definido en un subespacio M de X , tal que $g \leq \rho$. Entonces, existe un funcional lineal F en X que extiende G , es decir, para los puntos $x \in M \subset X$ se tiene que $G(x) = F(x)$, y además $F(x) \leq \rho(x)$ para toda $x \in X$.

El teorema de Hahn-Banach señala que si en un subespacio M de un espacio X tenemos un funcional $G : M \rightarrow \mathbb{R}$, podemos extender este funcional a todo el espacio X ; es decir, podemos encontrar el funcional F definido sobre todo el espacio X y que se comporte como G cuando evalúa puntos de M . Este funcional está acotado, pero esta parte del teorema no será de gran relevancia para nuestros propósitos.

La idea de la demostración de este teorema consiste en crear un subespacio M_1 que contiene a M agregando un nuevo punto x_0 que no está en M . Este nuevo subespacio se define como:

$$M_1 = \{x + \lambda x_0 : x \in M, \lambda \in \mathbb{R}\}$$

Claramente $M \subset M_1$, pues basta tomar $\lambda = 0$ para obtener los elementos de M . Entonces se define un nuevo funcional $G_1(x + \lambda x_0) = G(x) + \lambda c$ para algún $c \in \mathbb{R}$. El funcional G_1 extiende a G pues claramente cuando $x \in M$, se tiene que $\lambda = 0$, por lo que $G_1(x) = G(x)$. La idea es crear otro subespacio M_2 agregando otro punto que no estaba en M_1 , luego de igual forma extender M_2 a otro subespacio M_3 y así consecutivamente. De igual forma se crean extensiones de los funcionales $G_1, G_2, \dots$. Se comprueba que existe un elemento maximal² y que este maximal es todo el espacio X ; por tanto, la extensión de G en este maximal es precisamente el funcional F que estamos buscando.

El teorema de Hahn-Banach, entonces, señala la extensibilidad de funcionales lineales. Una de las aplicaciones relevantes de este teorema tiene que ver con la separabilidad. Enunciamos este resultado en el siguiente corolario:

Corolario 1 (Separabilidad por funcional lineal). *Sea M un subespacio de un espacio lineal normado X , tomemos $x' \in \overline{M}$, entonces existe un funcional lineal $F : X \rightarrow \mathbb{R}$ que cumple:*

1. Para todo $x \in M$, $F(x) = 0$,
2. $F(x') = 1$
3. $\|F\| = \frac{1}{d}$ donde d es la distancia de x' al subespacio M .

El tercer punto no nos interesa para la demostración del aproximador universal, pues tiene que ver con la norma del funcional. Lo que nos interesa aquí es señalar que lo que este resultado nos dice es que podemos encontrar un funcional que separe el punto x' de los elementos en M asignándole 0 a todos los elementos de M y 1 al elemento de x' . Este resultado está ligado con la separabilidad lineal de conjuntos convexos, pero aquí lo usaremos para mostrar que una red con una capa oculta puede separar conjuntos no necesariamente convexos.

Para mostrar este resultado basta generar un subespacio nuevo definido como:

$$M_1 = \{x + \lambda x' : x \in M, \lambda \in \mathbb{R}\}$$

Definimos entonces un funcional lineal que cumpla la siguiente igualdad:

$$F(x + \lambda x') = \lambda$$

El cual, precisamente, por tratarse de un funcional lineal cumple que $F(x + \lambda x') = F(x) + \lambda F(x')$ para todo $x \in M$. Por tanto, podemos ver que $F(x) = 0$ para todo $x \in M$, y $F(x') = 1$.

²Esta demostración utiliza el lema de Zorn, que es un resultado que fue polémico en matemáticas, pues asume el axioma de elección.

Ahora bien, podemos usar el teorema de Hahn-Banach para extender este funcional a todo el espacio X . De tal forma que F se anule en M , pero el funcional no es 0, pues al menos para el punto x' es diferente de 0.

Podemos ahora usar el resultado de la separabilidad para el caso que nos interesa. En este caso, el espacio lineal será el espacio de funciones continuas $C(I_d)$. Se trata de un espacio normado en el que podemos definir la norma:

$$\|f\| = \sup_x |f(x)|$$

Para toda $f \in C(I_d)$. Entonces el subespacio de $C(I_d)$ que nos interesa será $\mathcal{F}$, el subespacio de las sumas finitas (o redes neuronales con una capa oculta). Por los resultados que acabamos de ver, sabemos que existe un funcional lineal F en $C(I_d)$ tal que $F \neq 0$, pero $F(h) = 0$ para toda función $h \in \mathcal{F}$. Ya que $\mathcal{F} \subseteq \overline{\mathcal{F}}$ es claro que F también se anula en el espacio que nos interesa. Nótese que, como hemos asumido que $\overline{\mathcal{F}} \neq C(I_d)$, entonces $F \neq 0$ en $C(I_d) \setminus \overline{\mathcal{F}}$; es decir, hay funciones en $C(I_d)$ que no están en $\mathcal{F}$ para los que F es diferente de 0.

Ahora, usaremos otro resultado de importancia en el análisis funcional: el teorema de representación de Riesz que nos dice que todo funcional lineal puede expresarse en la forma:

$$F(h) = \int_{I_d} h(x) d\mu(x)$$

Para toda $h \in C(I : d)$ y con μ es una medida finita, regular y con signo. Ahora bien, sabemos que la función discriminatoria $g \in \overline{\mathcal{F}}$ (de hecho, podemos pensar a esta función como una red neuronal con una sola neurona). Por tanto, podemos aplicar el funcional F a g obteniendo:

$$F(g) = \int_{I_d} g(wx + b) d\mu(x) = 0$$

Y ya que g es una función discriminatoria, tenemos necesariamente que $\mu \equiv 0$; pero si la medida para esta representación es nula, entonces $F \equiv 0$ sobre todo el espacio $C(I_d)$; es decir, debe pasar que $F(h) = 0$ para toda $h \in C(I_d)$. Pero esto contradice el Corolario 1.

La contradicción viene de asumir que $\overline{\mathcal{F}} \neq C(I_d)$, por lo que debe ser que $\overline{\mathcal{F}} = C(I_d)$. Por tanto, las funciones en $\mathcal{F}$ aproximan tanto como se quiera a las funciones continuas en $C(I_d)$.

3.2.1. Otros teoremas de aproximador universal

El teorema de Cybenko (1989) es uno de los primeros teoremas en demostrar la propiedad de aproximación de las redes neuronales. A partir de este teorema, se han desarrollado otros que amplían al de Cybenko. Algunos de estos teoremas, por ejemplo, se basan no en funciones continuas sino en funciones Borel medibles. Otros determinan cotas para la anchura de la red neuronal. Recordemos que si $\overline{\mathcal{F}} = \mathcal{G}$ ($\mathcal{F}$ es denso en $\mathcal{G}$) entonces para cada función $g \in \mathcal{G}$ podemos encontrar una función $f \in \mathcal{F}$ tal que para toda ε se cumple $\sup_x |f(x) - g(x)| \leq \varepsilon$. Como en el caso de Cybenko, asumiremos que el dominio de las funciones es compacto. Denotaremos al conjunto de las

redes neuronales como:

$$\mathcal{F} = \{f \in C(X, Y) : f \equiv \theta g(wx + b)\}$$

Con g una función de activación, θ los pesos de salida y W y b los pesos de la capa oculta. Consideremos las redes con una sola capa. De esta forma, enunciamos el teorema de Hornik et al. (1989):

Teorema 9 (Aproximador Universal (Hornik et al., 1989)). *Sea $\mathcal{F}$ el conjunto de redes feedforward con una sola capa oculta con función de activación sigmoideal, entonces si $X \subseteq \mathbb{R}^d$ es un conjunto compacto, $\overline{\mathcal{F}} = C(X, \mathbb{R}^o)$, siempre que la red tenga la anchura suficiente.*

El teorema de Hornik et al. (1989) extiende las funciones que se pueden aproximar a funciones con un co-dominio en $\mathbb{R}^o$, por lo que las redes neuronales tendrán o neuronas de salidas. Más adelante, en el trabajo de Hornik (1991), además las funciones que se aproximan se extienden a funciones Borel medibles; es decir, $\overline{\mathcal{F}} = M(\mathbb{R}^d, \mathbb{R}^o)$.

El trabajo de Leshno et al. (1993) muestra que las funciones de activación deben de cumplir la condición de ser no polinomiales para que la red sea un aproximador universal sobre funciones continuas.

Teorema 10 (Aproximador Universal (Leshno et al., 1992)). *Dada funciones de activación no-polinomiales, tenemos que $\overline{\mathcal{F}} = C(\mathbb{R}^d, \mathbb{R}^o)$.*

Recientemente, trabajos como el de Park et al. (2020), se han enfocado en redes con activación ReLU, mostrando además cotas para la anchura mínima de una red que pueda ser un aproximador universal.

Teorema 11 (Aproximador Universal (Park et al., 2020)). *Dada funciones de activación ReLU, tenemos que $\overline{\mathcal{F}} = M(\mathbb{R}^d, \mathbb{R}^o)$ siempre que $\text{width}(f) \geq \max\{d + 1, o\}$, $f \in \mathcal{F}$.*

Muchos trabajos actuales se han enfocado a mostrar la propiedad de aproximador universal de las redes neuronales, utilizando principalmente funciones de activación ReLU o de la misma familia. Principalmente se enfocan en determinar las cotas de una anchura mínima de las redes para que cumplan la aproximación universal. El trabajo de Duan et al. (2023) resume algunos de los trabajos recientes sobre aproximación universal.

3.3. Redes neuronales profundas

En las secciones precedentes se ha definido a las redes feedforward y se ha mostrado que son aproximadores universales, es decir, que pueden aproximar funciones de familias más grandes. En general, se ha visto que basta con una sola capa oculta para que las redes neuronales sean aproximadores universales. En la práctica, sin embargo, es común que el cómputo se distribuya a partir de un número mayor de capas ocultas. Esto busca que el tamaño de las capas ocultas no sea demasiado grande, pero además permite que las capas ocultas se especialicen en representaciones particulares. Como

veremos más adelante, podemos construir redes neuronales que tengan capas ocultas con arquitecturas particulares enfocadas a procesar cierto tipo de datos. Algunas de estas capas son las capas recurrentes, las capas convolucionales o las capas de atención.

Por tanto, las redes neuronales suelen tener una profundidad considerable. Esta profundidad conlleva que las representaciones aprendidas en las capas ocultas sea difícil de interpretar. Estas representaciones son cada vez más abstractas, pues pasan por diferentes transformaciones consecutivamente. Es aquí cuando hablamos de aprendizaje profundo. Al hablar de redes neuronales profundas el término puede referirse a dos casos:

- Una red neuronal se dice que es profunda si cuenta con un número grande de capa ocultas.
- Una red neuronal se dice que es profunda si las representaciones de los datos en sus capas ocultas es abstracta.

Ambas nociones están ligadas, pues a mayor profundidad, las representaciones obtenidas suelen ser más abstractas (por tanto, menos interpretables). Sin embargo, no es necesario tener una gran profundidad para tener representaciones abstractas. Un ejemplo de esto último son el aprendizaje de encajes o *embeddings* (véase la Sección 1.5.1), donde las representaciones aprendidas no son fáciles de interpretar aunque se utilice una sola capa oculta. En general, hablaremos de aprendizaje profundo para referirnos al uso de redes neuronales con capas ocultas, sea una o varias.

Debe tomarse en cuenta que el número de capas ocultas, así como de sus unidades tiene influencia en cómo se resuelve el problema. Algunos problemas requerirán de un número mayor de unidades ocultas que pueden estar en una o varias capas. Un mayor número de pesos (anchura y profundidad grande) hará que la red neuronal tenga mayor capacidad. Esto le permitirá adaptarse a problemas complejos, pero también será más susceptible al sobreajuste. Por tanto, la elección de número de parámetros es importante para el desempeño de nuestro modelo neuronal.

Ejemplo 10. *Supongamos que queremos construir una red neuronal que sea capaz de aproximar una función polinomial dada por:*

$$f^*(x) = x^4$$

Supongamos que en la red neuronal usaremos activaciones basadas en el logaritmo $\ln(x)$. Podemos entonces definir una red neuronal con una capa oculta de la siguiente forma:

- *Capa oculta:* $h = \ln(wx)$ con $w^T = (1 \quad 1 \quad 1 \quad 1)$. *Recuérdese que $\ln$ se aplica punto a punto. Por lo que el resultado de la capa oculta será*

$$h^T = (\ln x \quad \ln x \quad \ln x \quad \ln x)$$

- *Capa de salida:* La capa de salida se dará por la función $f(x) = e^{w'h}$ donde $w'^T = (1 \quad 1 \quad 1 \quad 1)$. *Por tanto, notamos que $w'h = 4\ln x$ de donde obtenemos que $f(x) = x^4$ como queríamos.*

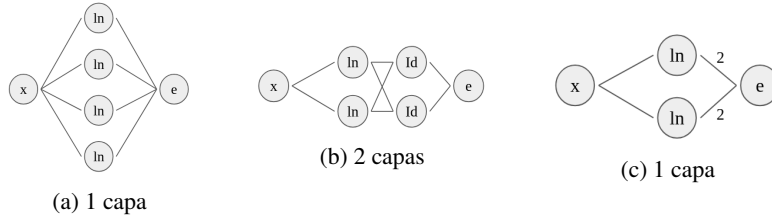


Figura 3.6: Diferentes redes neuronales para computar la función x^4

La red anterior es la más intuitiva, pero no es única. Podemos definir una red con dos capas ocultas que nos dé el mismo resultado de la siguiente forma:

- Capa oculta 1: Definida como $h^{(1)} = \ln \left(\begin{pmatrix} 1 & 1 \end{pmatrix} x \right)$, cuyo resultado será:

$$h^{(1)} = (\ln x \quad \ln x)$$

- Capa oculta 2: La definimos como:

$$h^{(2)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} h^{(1)}$$

Cuyo resultado es el vector $h^{(2)} = (2\ln x \quad 2\ln x)$

- Capa de salida: Por último, definimos la salida como $f(x) = e^{wh^{(2)}}$ con $w^T = \begin{pmatrix} 1 & 1 \end{pmatrix}$, lo que nos da el resultado esperado $f(x) = x^4$

En este último caso, la segunda capa oculta no tiene función de activación, por lo que podemos reducir esta capa con la siguiente capa tomando en cuenta el producto de sus pesos:

$$\begin{pmatrix} 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$$

Es decir, podemos tomar una red con una sola capa oculta, la cual esté definida como:

$$h = \begin{pmatrix} 1 & 1 \end{pmatrix}$$

Y la capa de salida esté dada como:

$$f(x) = e^{\begin{pmatrix} 2 & 2 \end{pmatrix} h} = e^{2h_1 + 2h_2}$$

La activación exponencial nos daría la salida esperada $f(x) = x^4$. En la Figura 3.6 se muestran gráficamente las diferentes redes que pueden resolver este problema. Incluso se puede simplificar todavía más la red.

Las redes neuronales profundas deben de obtener los pesos adecuados para resolver cada problema particular. La arquitectura de la red (anchura, profundidad, funciones de activación) debe ser especificada en cada caso por quien construya la red. Por tanto, sin

importar la arquitectura, la red debe aprender a resolver el problema. En la práctica es usual utilizar arquitecturas que cuenten con un número suficiente de pesos (profundidad y anchura) y nos enfocamos en aprender adecuadamente con base en los datos. Para evitar el sobreajuste, se utilizan técnicas de regularización (Capítulo 5), sin enfocarse en modificar la arquitectura. Aquí hemos ejemplificado redes donde conocemos pesos que son adecuados para el problema. Las redes neuronales pueden aprender estos pesos; para esto es necesario profundizar en el aprendizaje en las redes profundas, lo que abordamos en el siguiente capítulo.

Ejercicios

1. Definir una red neuronal que obtenga cada una de las siguientes funciones de $\mathbb{R}^2$ a $\mathbb{R}$ (usando activaciones $\ln$ y exponencial):

$$a) f(x, y) = x \cdot y$$

$$b) f(x, y) = x \cdot y + \frac{x}{y}$$

$$c) f(x, y) = x^2 + 2xy + x^2$$

Describir la arquitectura de cada una de las redes.

2. Demuestra que la derivada de la función sigmoide es $\sigma'(x) = \sigma(x)(1 - \sigma(x))$.
3. Demuestra que la derivada de la tangente hiperbólica es $\tanh'(x) = 1 - \tanh^2(x)$.
4. Demuestra que la derivada de la función softplus es la función sigmoide.
5. Obtener la derivada de la función de activación ELU.
6. Demostrar la identidad:

$$\frac{e^{a_i+c}}{\sum_j e^{a_j+c}} = \frac{e^{a_i}}{\sum_j e^{a_j}}$$

para una constante arbitraria $c \in \mathbb{R}$.

Capítulo 4

Aprendizaje en redes multicapa

El aprendizaje en las redes neuronales se realiza por medio de la minimización de la función de riesgo a partir de métodos basados en el gradiente. El problema de aprendizaje consiste en encontrar una función que resuelva el problema a tratar de forma correcta. Podemos plantear esta ecuación de la siguiente forma:

El proceso de aprendizaje consiste en encontrar la función óptima $\hat{f}$ dentro de un conjunto de funciones $\{f : X \rightarrow Y\}$, tal que $R(\hat{f}) \leq R(f)$.

Donde $R : \Theta \rightarrow \mathbb{R}$ es una función de riesgo u función objetivo que determina la calidad de la función dada para resolver el problema. Como se revisó en la Sección 1.4, el objetivo del aprendizaje es la minimización de la función objetivo. De forma general, podemos entender la función objetivo como:

$$R(f) = \int L(y, f(x)) dF(x)$$

En las redes neuronales, la búsqueda de la función óptima depende de un conjunto de parámetros (o pesos) θ ; los parámetros son parte del espacio de hipótesis Θ , por lo que podemos definir la familia de funciones dada una arquitectura de red neuronal como:

$$\{f(\cdot; \theta) : \theta \in \Theta\}$$

De tal forma que la función de pérdida $R : \Theta \rightarrow \mathbb{R}^+$ toma como dominio el espacio de parámetros. El aprendizaje consiste en encontrar los parámetros que minimicen dicha función:

$$R(\theta) = \int L(y, f(x; \theta)) dF(x)$$

Los parámetros θ están determinados por las matrices de pesos y los bias de cada una de las capas ocultas. Nos referiremos a este conjunto como los pesos de la red y lo definiremos como:

$$\theta = \{W^{(k)}, b^{(k)} : k = 1, 2, \dots, K, K + 1\}$$

Donde K es el número de capas ocultas. Se considera los pesos y el bias de la capa de salida. Dado que el espacio Θ es continuo, la minimización puede realizarse por medio de métodos basados en gradiente. Por tanto, buscamos el conjunto de parámetros:

$$\theta^* = \arg \min_{\theta \in \Theta} R(\theta)$$

Como en otros problemas de aprendizaje, el que una red neuronal aprenda se convierte en un problema de optimización que busca estimar los pesos que minimicen la función objetivo. Es decir, las conexiones de la red neuronal deben adaptarse con base en la observación de los datos, para poder resolver el problema en base a estos. La selección de la función objetivo depende en gran medida del tipo de problemas que se buscan resolver. Algunas de estas son:

- **Divergencia KL:** Se suele utilizar en arquitecturas como los AutoEncoders variacionales o las redes generativas adversariales (o GANs por *Generative Adversarial Networks*), donde se tienen dos distribuciones probabilísticas. Esta función está dada como:

$$R(\theta) = \sum_x q(x) \ln \frac{q(x)}{f(x; \theta)}$$

- **Entropía cruzada:** La entropía cruzada es una de las funciones objetivo más comunes, utilizada en redes básicas como la regresión logística, y también en redes feedforward, así como en redes recurrentes, convolucionales, atencionales, y de otro tipo, siempre y cuando estas redes se enfoquen a un problema de clasificación o de estimación de probabilidad. Esta definida como:

$$R(\theta) = - \sum_{y,x} \delta_{x,y} \ln f_y(x; \theta)$$

donde $\delta_{x,y}$ determina si un dato x pertenece a la clase y como:

$$\delta_{x,y} = \begin{cases} 1 & \text{si } x \text{ es clase } y \\ 0 & \text{en otro caso} \end{cases}$$

- **Error cuadrático:** El error cuadrático es utilizado en redes que trabajan problemas de regresión que pueden ser feedforward o de algún otro tipo. También es común utilizarla en AutoEncoders, donde se predice una salida que sea similar a la entrada, por lo que se requiere determinar la similitud entre entrada y salida. Se define como:

$$R(\theta) = \frac{1}{2} \sum_x \|y - f(x; \theta)\|_2^2$$

Nos enfocaremos principalmente en la entropía cruzada pues nos interesan por ahora las redes feedforward para estimación de probabilidades y clasificación. Utilizaremos algunos ejemplos del error cuadrático para problemas de regresión. La divergencia KL no será tema de interés, pero su derivación se presenta como ejercicio. Aprender sobre estas funciones objetivo implica derivarlas sobre los pesos de la red. La dificultad

que se presenta es que tenemos una red con múltiples capas ocultas. Es decir, tenemos una función compuesta sobre la que tenemos que obtener sus derivadas.

Recordemos que una red feedforward es una función de la forma:

$$f(x) = \phi(W^{(K+1)}h^{(K)} + b^{(K+1)})$$

Donde $h^{(K)}$ es la última capa oculta. Consideraremos, en principio el caso en que el problema a resolver es una clasificación. Considérese, entonces, un conjunto supervisado:

$$\mathcal{S} = \{(x, y) : x \in \mathbb{R}^d, y \in Y\}$$

En general, $f(x) \in \mathbb{R}^o$ es un vector de probabilidades cuyas entradas corresponden a la probabilidad de que la entrada x pertenezca a la clase y . La función objetivo que se define en las redes feedforward para este tipo de problemas es la **entropía cruzada**, por lo que el problema que buscamos resolver es el siguiente:

$$\arg \min_{\theta \in \Theta} R(\theta) = \arg \min_{\theta \in \Theta} - \sum_x \sum_y \delta_{x,y} \ln f_y(x; \theta) \quad (4.1)$$

Donde f_y representa la y -ésima neurona de salida (la cual representa a la clase y), y la función $\delta_{x,y}$ es 1 si el ejemplo x pertenece a la clase y , o 0 en otro caso. Nuestro objetivo es minimizar la función objetivo $R(\theta)$ al observar el conjunto de ejemplos $\mathcal{S}$. Minimizar esta función corresponde a encontrar los pesos $\theta = \{W^{(k)}, b^{(k)} : k = 1, \dots, K, K+1\}$ de las conexiones de la red que haga que ésta pueda resolver el problema con un margen de error lo más pequeño posible. La optimización de la función objetivo sobre los pesos de la red define una función compuesta de la forma:

$$(R \circ \phi \circ h^{(K)} \circ \dots \circ h^{(1)})(\theta)$$

Encontrar los pesos que minimicen esta función compuesta requiere derivarla sobre un número arbitrario de capas ocultas. En las siguientes secciones abordamos los métodos que nos permitirán aprender sobre la red neuronal, comenzando por el método de descenso por gradiente. Veremos después la incorporación de la retro-propagación para poder derivar sobre las diferentes capas ocultas.

4.1. Descenso por gradiente

El objetivo del aprendizaje en redes neuronales es encontrar los pesos que minimicen la función de riesgo; para esto se vale de la optimización de la función objetivo, la cual determina la calidad del aprendizaje de la red. Un método sencillo para optimizar una función es el descenso por gradiente, el cual busca el mínimo de la función con base en la información del gradiente de la misma. Antes de pasar al método de descenso por gradiente, introducimos el concepto de mínimo de una función.

Definición 64 (Mínimo global). Si $R : \Theta \rightarrow \mathbb{R}$ es una función (objetivo), diremos que su mínimo global es el valor $\theta^* \in \Theta$ que cumple que para todo $\theta \in \Theta$ se tiene,

$$R(\theta^*) \leq R(\theta)$$

El objetivo del aprendizaje en redes neuronales es encontrar los pesos θ^* que sean el mínimo de la función objetivo. Esto, sin embargo, no es tarea fácil y en la práctica es raro encontrar el mínimo global de una función objetivo durante el aprendizaje. En algunos casos, la función puede contar con los llamados mínimos locales.

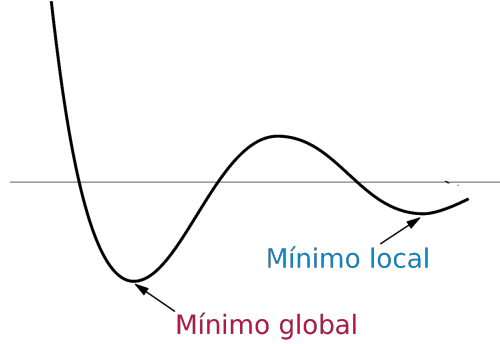


Figura 4.1: Mínimos global y local en una función

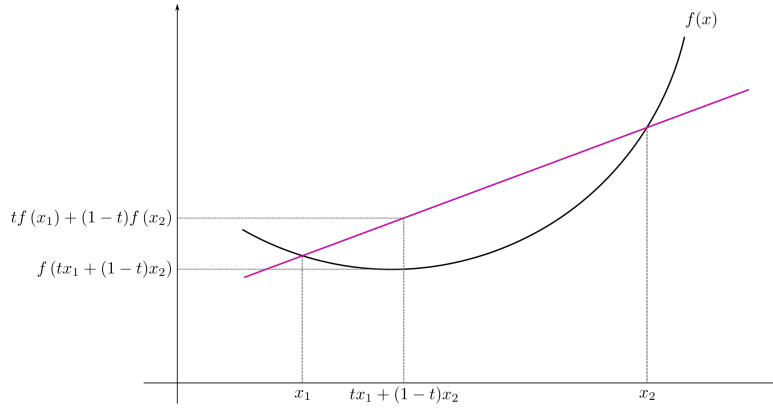
Definición 65 (Mínimo local). Si $R : \Theta \rightarrow \mathbb{R}$ es una función (objetivo), diremos que $\theta^* \in \Theta$ es un mínimo local si para una región $\mathcal{B} = \{\theta : \|\theta - \theta^*\| \leq r\}$ se cumple que $R(\theta^*) \leq R(\theta)$, con $\theta \in \mathcal{B}$.

Los mínimos locales son valores mínimos sólo en una región limitada de la imagen de la función. Los mínimos locales pueden ser problemáticos, pues pueden resultar en soluciones subóptimas. En la Figura 4.1 se puede ver la gráfica de una función con un mínimo global y un mínimo local. Entre más alejado esté el mínimo local del global, el desempeño del modelo de aprendizaje tenderá a ser peor. Podemos preguntarnos cuándo es posible garantizar que alcanzamos un mínimo global. En la práctica del aprendizaje desearíamos obtener en todos los casos un mínimo global, pero esto sólo lo podremos alcanzar bajo algunas restricciones.

Definición 66 (Función convexa). Una función $f : \mathbb{R}^d \rightarrow \mathbb{R}$ se dice que es convexa si para todo x_1 y x_2 y para $t \in [0, 1]$ se cumple que:

$$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2)$$

Las funciones convexas presentan el mejor de los panoramas en un problema de optimización. Un ejemplo claro de una función convexa es la función $f(x) = x^2$. La convexidad puede interpretarse como que la recta entre los puntos $f(x_1)$ y $f(x_2)$ está por encima siempre de la imagen de la función en la recta de x_1 a x_2 . En la Figura 4.2 se puede ver esta idea: la recta que va del punto $f(x_1)$ hasta $f(x_2)$ está por encima de la imagen de la recta en el dominio que va de x_1 a x_2 . Las funciones convexas permiten una optimización sencilla. El perceptrón, la regresión lineal y logística tienen una función de riesgo convexa (siempre que los datos sean linealmente separables o dependientes), por tanto, podemos encontrar mínimos en estas funciones. El siguiente resultado señala la relevancia de las funciones convexas.

Figura 4.2: Función convexa de $\mathbb{R}$ a $\mathbb{R}$

Proposición 13. Si R es una función convexa, entonces un mínimo local en la función es un mínimo global.

Demostración. Ejercicio. □

En otras palabras, las funciones convexas tienen siempre un sólo mínimo que es el mínimo global. Entonces encontrar el mínimo en estas funciones corresponde a encontrar el punto en que el gradiente sea igual a 0; esto es:

$$\nabla_{\theta} R(\theta) = 0$$

Y podremos comprobar que el valor θ^* que solucione esta ecuación es un mínimo global, pues en este tipo de funciones no podríamos caer más que en este mínimo.

Sin embargo, las funciones de riesgo en redes neuronales suelen ser más complejas y raramente convexas. Por tanto se propone otro método para su optimización. En el gradiente de la función podemos encontrar información que nos ayude a minimizar, pues observamos que:

- Si $\nabla_{\theta} R(\theta)$ es positiva, la función asciende; por tanto, para acercarnos a un mínimo debemos movernos en sentido contrario, esto es, disminuir el valor de θ :

$$\theta \leftarrow \theta - \Delta\theta$$

- Si $\nabla_{\theta} R(\theta)$ es negativa, la función desciende; por tanto, para acercarnos a un mínimo debemos seguir por este camino, aumentando el valor de θ :

$$\theta \leftarrow \theta + \Delta\theta$$

El vector gradiente nos da información sobre la dirección hacia dónde se dirige la función. Notamos también que el comportamiento del gradiente cumple que:

- Si el gradiente es positivo, entonces:

$$\Delta\theta = -\nabla_{\theta}R(\theta) < 0$$

- Si el gradiente es negativo, entonces:

$$\Delta\theta = -\nabla_{\theta}R(\theta) > 0$$

Por tanto, podemos decir que el factor de cambio sobre los parámetros de la función puede estar determinado por el gradiente:

$$\Delta\theta = -\nabla_{\theta}R(\theta)$$

Y una primera propuesta es que se modifiquen los pesos durante el aprendizaje restando este factor de cambio al valor anterior de éstos. Pero podemos introducir un factor que escale este cambio, de tal forma que podamos controlar qué tanto va a cambiar el valor de θ en cada iteración. Este factor de cambio es la llamada tasa de aprendizaje η .

Definición 67 (Descenso por gradiente). *Dada una función $R : \Theta \rightarrow \mathbb{R}^+$ el método de descenso por gradiente busca optimizar la función encontrando el mínimo de ésta a partir de la regla de actualización:*

$$\theta \leftarrow \theta - \eta \nabla_{\theta}R(\theta)$$

donde $\eta \in \mathbb{R}$ es un hiperparámetro conocido como la tasa de aprendizaje.

Con base en esta definición, podemos definir un algoritmo que tome como entrada una función $R : \Theta \rightarrow \mathbb{R}^+$, una tasa de aprendizaje η y que dado un número máximo de iteraciones T busque optimizar R .

```

1: procedure GD( $R : \Theta \rightarrow \mathbb{R}^+, \eta \in \mathbb{R}, T > 0$ )
2:    $\theta \leftarrow \text{RANDOM}(\Theta)$  Inicialización
3:   for  $t$  from 1 to  $T$  do Inducción
4:     Calcula  $R(\theta)$  y  $\nabla_{\theta}R(\theta)$ 
5:     if  $\nabla_{\theta}R(\theta) = 0$  then
6:       return  $\theta$ 
7:     else
8:       Actualizar el valor de  $\theta$  con respecto a la regla:
          
$$\theta \leftarrow \theta - \eta \nabla_{\theta}R(\theta)$$

9:     end if
10:  end for
11:  returns  $\hat{\theta} := \theta$  Terminación
12: end procedure

```

El algoritmo de descenso por gradiente es el fundamento para la optimización y, por tanto, para el aprendizaje en redes neuronales. Este algoritmo no garantiza la convergencia a un mínimo global, pero en muchos casos bastará con encontrar un mínimo

local. Asimismo, existen métodos que buscan lidiar con este problema. La única forma en que podremos garantizar la convergencia al mínimo global es si la función es convexa.

Teorema 12. *El algoritmo de descenso por gradiente encuentra el valor mínimo, dado el suficiente número de iteraciones y un valor $\eta > 0$ adecuado, de una función $R : \Theta \rightarrow \mathbb{R}^+$ si ésta función es convexa y diferenciable.*

Demostración. En primer lugar, se debe notar que el método del descenso por gradiente siempre minimiza la función, es decir, si θ_1 y θ_2 son dos valores consecutivos obtenidos por este método, entonces $R(\theta_2) \leq R(\theta_1)$. Dado que la función R es convexa, tenemos que:

$$R(\theta_1) - R(\theta_2) \geq \nabla R(\theta_2)^T (\theta_1 - \theta_2)$$

Ya que θ_2 se obtiene mediante la regla de asignación del descenso por gradiente en un paso, entonces $\theta_2 = \theta_1 - \eta \nabla R(\theta_1)$, de donde tenemos que:

$$\nabla R(\theta_2)^T (\theta_1 - \theta_2) = \eta \nabla R(\theta_2)^T \nabla R(\theta_1)$$

Notamos que $\nabla R(\theta_2)^T \nabla R(\theta_1) \geq 0$ siempre y cuando ambos gradientes tengan la misma dirección, lo cual se puede garantizar tomando un valor η adecuado.¹ De aquí concluimos que $R(\theta_1) \geq R(\theta_2) \geq 0$, por lo que:

$$R(\theta_1) \geq R(\theta_2)$$

Y ya que R tiene un sólo mínimo global, $\nabla R(\theta) > 0$ para todo valor diferente del mínimo global y sólo 0 en este mínimo. Por lo que siempre se hará una actualización de los pesos fuera del mínimo, dando una secuencia $R(\theta_1) \geq R(\theta_2) \geq \dots \geq R(\theta_t) \geq \dots$ que alcanzará el mínimo con un número de iteraciones suficiente. $\square$

La argumentación del teorema anterior asume que tenemos un valor de la tasa de aprendizaje adecuado. Esto tiene especial importancia en la optimización por medio del descenso por gradiente, pues una mala selección del hiperparámetro η puede llevar a que no se aprenda adecuadamente, alejando los valores del mínimo. Ponemos un ejemplo simple para mostrar estos casos.

Ejemplo 11. *Considérese la función $R(\theta) = \theta^2$ la función que se busca optimizar mediante el descenso por gradiente. Recuérdese que la derivada es 2θ . Si tomamos $\eta = 1$ e iniciamos el valor $\theta_1 = 1$ tenemos que en la primera iteración la actualización se da como:*

$$\theta_2 = 1 - 1(2(1)) = -1$$

Por lo que el nuevo valor es -1 el cual no es todavía el mínimo (que sabemos que es 0). Si procedemos a realizar una segunda iteración tenemos lo siguiente:

$$\theta_3 = -1 - 1(2(-1)) = 1$$

Es decir, volvemos al valor con que hemos iniciado por lo que en la siguiente iteración repetiríamos el valor 1, después -1 y estos dos valores se repetirían continuamente sin llegar a converger al mínimo.

¹El caso contrario se daría si el método de descenso por gradiente se salta el mínimo, por ejemplo al tomar un valor η muy alto.

Del ejemplo anterior notamos que una buena selección de la tasa de aprendizaje es de especial relevancia para el descenso por gradiente, de otra forma, podría pasar que el método nunca converja, a pesar de que la función sea convexa. Un valor óptimo para el ejemplo sería tomar $\eta = \frac{1}{2}$, se puede comprobar que con esta tasa de aprendizaje, bajo las mismas condiciones del ejemplo, el algoritmo converge al mínimo en la primera iteración. Valores más pequeños pueden garantizar la convergencia al mínimo pero con un mayor costo temporal. De tal forma que elegir una tasa de aprendizaje muy alta podría llevar al algoritmo a no converger, mientras que una tasa de aprendizaje pequeña podría demorar demasiado para converger. Debido a las dificultades que presenta la tasa de aprendizaje, se han desarrollado métodos que buscan adaptar esta tasa durante el aprendizaje, los cuales revisaremos en la Sección 4.2.

El uso del gradiente puede adaptarse para los fines de la optimización. Por ejemplo, si lo que queremos hacer es encontrar el máximo (local o global) de una función, podemos utilizar el **ascenso por gradiente**, el cual al igual que descenso por gradiente se basa en el vector gradiente para mover los pesos hacia la dirección del óptimo. Ya que en este caso nos movemos en sentido contrario al descenso por gradiente, debemos cambiar el signo del cambio:

$$\theta \leftarrow \theta + \eta \nabla_{\theta} R(\theta)$$

El método de ascenso por gradiente es muy similar al descenso. Aunque en el aprendizaje profundo lo común es utilizar el descenso del gradiente, se podría adaptar un método de ascenso. En lo que sigue nos enfocaremos únicamente en la minimización.

Antes de profundizar en otras nociones del descenso por gradiente, ejemplificaremos su uso para derivar el método de aprendizaje en el modelo de regresión logística.

Ejemplo 12. *Supongamos que tenemos un problema de clasificación binaria $Y = \{0, 1\}$ en el que utilizaremos el modelo de regresión logística. Recordemos que en la regresión logística estimamos una función f de probabilidad, i.e. $f(x) = p(y|x)$, donde la probabilidad está determinada por la función sigmoide. En términos estadísticos, buscamos maximizar la función de máxima verosimilitud dada por:*

$$\prod_{(x,y)} p(y|x) = \prod_{x \in Y_1} p(y = 1|x) \prod_{x \in Y_0} p(y = 0|x)$$

Donde Y_0 y Y_1 son los datos que pertenecen a la clase 0 y la clase 1, respectivamente. Si aplicamos el logaritmo para escalar los valores bajos nos queda:

$$\sum_{x \in Y_1} \ln p(y = 1|x) + \sum_{x \in Y_0} \ln p(y = 0|x)$$

Para evitar utilizar dos sumas, podemos utilizar las clase y asignadas en el conjunto supervisado, de tal forma que tenemos:

$$\sum_{(x,y)} y \ln p(y = 1|x) + (1 - y) \ln p(y = 0|x)$$

Finalmente, para convertir el problema de maximización en un problema de minimización basta con cambiar el signo:

$$- \sum_{(x,y)} y \ln p(y = 1|x) + (1 - y) \ln p(y = 0|x)$$

Podemos observar que esta es la entropía cruzada para una distribución binaria. Como sabemos que la probabilidad se estima por medio de la función de regresión $f(x; \theta)$ que depende del conjunto de parámetros $\theta = \{w, b\}$, entonces la función objetivo estará dada como:

$$\arg \min_{\theta \in \Theta} R(\theta) = \arg \min_{\theta \in \Theta} - \sum_{(x,y)} y \ln f(x; \theta) + (1 - y) \ln 1 - f(x; \theta)$$

Podemos proceder a derivar esta función. En primer lugar, notemos que podemos considerar dos casos: cuando $y = 0$ y cuando $y = 1$. Si $y = 0$, tenemos las derivadas parciales:

$$\begin{aligned} \frac{\partial}{\partial \theta_i} - \sum_{(x,y)} \ln(1 - f(x; \theta)) &= - \sum_{(x,y)} \frac{\partial}{\partial \theta_i} \ln(1 - f(x; \theta)) \\ &= - \sum_{(x,y)} \frac{1}{1 - f(x; \theta)} \left(- \frac{\partial}{\partial \theta_i} f(x; \theta) \right) \end{aligned}$$

Recordemos que la derivada del sigmoide es $\sigma(x)(1 - \sigma(x))$ y la derivada de $wx + b$ sobre w_i es x_i y 1 si derivamos sobre b . Por lo que tenemos:

$$\begin{aligned} - \sum_{(x,y)} \frac{1}{1 - f(x; \theta)} \left(- \frac{\partial}{\partial \theta_i} f(x; \theta) \right) &= \sum_{(x,y)} \frac{1}{1 - f(x; \theta)} f(x; \theta)(1 - f(x; \theta))x_i \\ &= f(x; \theta)x_i \\ &= (f(x; \theta) - 0)x_i \end{aligned}$$

Ahora consideremos el caso en que $y = 1$. En este caso nos quedamos sólo con el primer término, por lo que tenemos:

$$\begin{aligned} \frac{\partial}{\partial \theta_i} - \sum_{(x,y)} \ln f(x; \theta) &= - \sum_{(x,y)} \frac{1}{f(x; \theta)} \frac{\partial}{\partial \theta_i} f(x; \theta) \\ &= - \sum_{(x,y)} \frac{1}{f(x; \theta)} f(x; \theta)(1 - f(x; \theta))x_i \\ &= \sum_{(x,y)} (f(x; \theta) - 1)x_i \end{aligned}$$

Considerando ambos casos, podemos escribir las derivadas de la función objetivo utilizando la clase y de la siguiente forma:

$$\frac{\partial R(\theta)}{\partial w_i} = \sum_{(x,y)} (f(x; \theta) - y)x_i$$

En el caso de la derivada sobre el bias b , simplemente ignoramos el factor x_i . De esta forma, el método de aprendizaje con descenso de gradiente en la regresión logística está determinado por la regla de actualización:

$$w_i \leftarrow w_i - \eta \sum_{(x,y)} (f(x; \theta) - y)x_i$$

Si tomamos un sólo ejemplo de entrenamiento, podemos quitar la suma de la regla de actualización, obteniendo:

$$w_i \leftarrow w_i - \eta (f(x; \theta) - y) x_i$$

Se notan las similitudes de esta regla de actualización, que obtuvimos derivando la función de la regresión logística y aplicando el descenso por gradiente, con la regla de aprendizaje del perceptrón (Sección 2.1).

4.1.1. Lotes y mini lotes

Cuando se calcula el gradiente de la función objetivo sobre todos los ejemplos, el gradiente resulta en una suma sobre los datos. Esta suma puede hacer el gradiente más grande, por lo que la actualización de los pesos puede cambiar muy rápido, impidiendo una convergencia adecuada. De manera más general, podemos considerar una función objetivo determinada como:

$$R(\theta) = \sum_{(x,y) \in \mathcal{S}} L(y, f(x; \theta))$$

Aquí $\mathcal{S}$ representa el conjunto de entrenamiento completo. Por las propiedades de linealidad del gradiente tenemos que:

$$\begin{aligned} \nabla_{\theta} R(\theta) &= \nabla_{\theta} \sum_{(x,y) \in \mathcal{S}} L(y, f(x; \theta)) \\ &= \sum_{(x,y) \in \mathcal{S}} \nabla_{\theta} L(y, f(x; \theta)) \end{aligned}$$

Esto apunta a que en el método de descenso por gradiente se deben obtener los gradientes por cada uno de los ejemplos y sumarlos para poder así actualizar los pesos. Esto implica que la actualización se hará como:

$$\theta \leftarrow \theta - \eta \sum_{(x,y) \in \mathcal{S}} \nabla_{\theta} L(y, f(x; \theta)) \quad (4.2)$$

Esta actualización tiene la particularidad de revisar todos los gradientes de cada uno de los ejemplos para realizar la actualización. Sin embargo, como mencionábamos, esto puede presentar un problema en el aprendizaje en algunos casos. Por tanto, se proponen métodos que actualicen los pesos de la red en diferentes subconjuntos del conjunto total de datos de entrenamiento. A estos subconjuntos los llamaremos **lotes**. El caso que hemos revisado es el más general.

Definición 68 (Descenso por gradiente (por lote)). *Decimos que el descenso por gradiente es por lote (batch gradient descent) cuando la actualización de los pesos se realiza después de revisar todos los datos como en la Ecuación 4.2.*

Con base en esta forma de actualización de los pesos con el descenso por gradiente, se puede definir el algoritmo de la siguiente forma:

```

1: procedure (BATCH) GD( $R(\theta) = \sum_{(x,y) \in \mathcal{S}} L(y, f(x; \theta))$ ),  $\eta \in \mathbb{R}, T > 0$ )
2:    $\theta \leftarrow \text{RANDOM}(\Theta)$  Inicialización
3:   for  $t$  from 1 to  $T$  do Inducción
4:     Actualizar el valor de  $\theta$  con respecto a la regla:

$$\theta \leftarrow \theta - \eta \sum_{(x,y) \in \mathcal{S}} \nabla_{\theta} L(y, f(x; \theta))$$

5:   end for
6:   return  $\theta$  Terminación
7: end procedure

```

En la práctica es poco común utilizar un sólo lote (batch) como lo hemos mostrado, pues no suele presentar buenos resultados. Además, este método tiene dos principales problemas:

- Puede ser intratable si el conjunto de entrenamiento es muy grande, debido a limitaciones de memoria.
- Si se desea actualizar *online* (cada vez que llegan nuevos ejemplos) se tendría que recalcular el gradiente sobre todos los ejemplos, y no sólo sobre los nuevos ejemplos obtenidos.

En muchos casos, es preferible utilizar pequeños subconjuntos de datos o lotes para actualizar los pesos de manera más reservada. Un caso extremo de esto es el **descenso por gradiente estocástico** (Ruder, 2016), en este caso se actualizan los pesos por cada uno de los ejemplos:

```

1: procedure STOCHASTIC GD( $R(\theta) = \sum_{(x,y) \in \mathcal{S}} L(y, f(x; \theta))$ ),  $\eta \in \mathbb{R}, T > 0$ )
2:    $\theta \leftarrow \text{RANDOM}(\Theta)$  Inicialización
3:   for  $t$  from 1 to  $T$  do Inducción
4:     for  $(x, y) \in \text{SHUFFLE}(\mathcal{S})$  do
5:       Actualizar el valor de  $\theta$  con respecto a la regla:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(y, f(x; \theta))$$

6:     end for
7:   end for
8:   return  $\theta$  Terminación
9: end procedure

```

Este tipo de descenso por gradiente actualiza cada vez que revisa un dato, garantizando una mejor convergencia pero a un costo mucho mayor. La función `SHUFFLE` aleatoriza el orden en que se toman los ejemplos en cada iteración, esto garantiza que en cada época el orden de los ejemplos sea aleatorio, y por tanto no se introduzca algún sesgo que dependa de este orden. En este caso, se calcula la derivada con respecto a cada uno de los ejemplos. El problema con el gradiente descendiente estocástico es

que toma más tiempo en converger, pues necesita actualizar ejemplo por ejemplo. Esto hace que su implementación sea más lenta.

Una opción intermedia propone utilizar subconjuntos del total de datos de entrenamiento que contengan un número limitado de ejemplos. A esto se le conoce como **Descenso por Gradiente por Mini lotes** o **Mini-Batch Gradient Descent**.

```

1: procedure MINI-BATCH GD( $R(\theta) = \sum_{(x,y) \in \mathcal{S}} L(y, f(x; \theta))$ ,  $size, \eta \in \mathbb{R}, T > 0$ )
2:    $\theta \leftarrow \text{RANDOM}(\Theta)$  Inicialización
3:   for  $t$  from 1 to  $T$  do Inducción
4:      $B_i \leftarrow \text{PARTITION}(\mathcal{S}, size)$ 
5:     for  $B_i$  from  $i = 1$  to  $i = \lceil \frac{|\mathcal{S}|}{size} \rceil$  do
6:       Actualizar el valor de  $\theta$  con respecto a la regla:

$$\theta \leftarrow \theta - \eta \sum_{(x,y) \in B_i} \nabla_{\theta} L(y, f(x; \theta))$$

7:     end for
8:   end for
9:   return  $\theta$  Terminación
10: end procedure

```

La función PARTITION particiona el conjunto total de datos en lotes de tamaño $size$. De esta forma, el descenso por gradiente por mini lotes utiliza subconjuntos del conjunto de entrenamiento, los mini lotes, para obtener un cambio en los pesos que no sea el total de los datos, pero que tampoco sea dato por dato. En la práctica, este es el método más utilizado en el aprendizaje de redes neuronales pues puede garantizar una mejor convergencia, sin arriesgar demasiado el costo en tiempo. Los mini lotes también se aleatorizan en cada iteración para garantizar que no existan sesgos en los datos que puedan afectar el entrenamiento.

Cabe señalar que el descenso por gradiente por mini lotes generalizan los casos anteriores. Si tomamos un lote de tamaño 1, obtenemos un descenso por gradiente estocástico donde la actualización se realiza por cada dato. Si se toma el tamaño del lote al tamaño total del conjunto de entrenamiento, obtenemos una actualización al revisar todos los ejemplos. Por tanto, es común que durante el entrenamiento se usen lotes. Estos se administran por medio de los **cargadores de datos**, los cuales son programas que permiten cargar y administrar los datos de manera sencilla y eficiente (este tipo de cargadores se pueden encontrar en bibliotecas como Pytorch o Tensorflow).

4.2. Optimizadores basados en gradiente

El descenso por gradiente es un método simple que tiene ciertas limitaciones. Por ejemplo, uno de los problemas que presenta es la elección correcta del hiperparámetro de la tasa de aprendizaje. Esta elección es importante en tanto influye en la convergencia de los pesos de la red neuronal. Podemos tener dos casos problemáticos:

- Si el rango de aprendizaje es muy grande, el algoritmo puede no converger.

- Si el rango de aprendizaje es muy pequeño, el algoritmo tardará mucho en converger.

Para lidiar con esto se proponen diferentes métodos que lidian de forma distinta con el rango de aprendizaje o que introducen factores en la actualización de los pesos para mejorar la convergencia. Se han propuesto diferentes métodos para lidiar con los problemas del descenso por gradiente. Presentamos aquí algunos de los más populares. Una revisión más completa puede verse en Ruder (2016).

Método de momentum

Una propuesta para mejorar la convergencia del descenso por gradiente es tomar en cuenta la influencia de los cambios anteriores sumándolos como un momento en la regla de actualización. Esto en busca de acelerar la convergencia al mínimo, de tal forma que se obtenga una convergencia en menor tiempo y que se evite los mínimos locales con mayor facilidad.

Definición 69 (Descenso por gradiente por momento). *Sea $R : \Theta \rightarrow \mathbb{R}$ una función objetivo. El método de descenso por gradiente con momento optimiza la función con base en la tasa de cambio:*

$$\mu_t = \eta \nabla_{\theta} R(\theta) + \gamma \mu_{t-1}$$

Donde t es la época actual, $\mu_0 = 0$ y $\gamma \in \mathbb{R}$ es un hiperparámetro. La actualización se realiza por medio de la regla:

$$\theta \leftarrow \theta - \mu_t$$

Generalmente, $\gamma = 0,9$. El método de momentum acumula los gradientes en las épocas anteriores para que estos tengan una influencia en la actualización de los pesos en la época actual. La influencia de los gradientes se hace más pequeña con el tiempo.

Proposición 14. *El método de descenso por gradiente con momentum realiza las actualizaciones de la forma:*

$$\theta_{t+1} \leftarrow \theta_t - \sum_{i=0}^t \gamma^i \eta \nabla R(\theta_{t-i})$$

Es decir, en cada época la tasa de cambio está dada como $\mu_t = \sum_{i=0}^t \gamma^i \eta \nabla R(\theta_{t-i})$

Demostración. Por inducción sobre el número de épocas t . Cuando tomamos $t = 2$ tenemos que:

$$\begin{aligned} \theta_2 &= \theta_1 - \eta \nabla R(\theta_1) - \gamma \mu_0 \\ &= \theta_1 - \eta \nabla R(\theta_1) \end{aligned}$$

Ya que $\mu_0 = 0$. Por lo que $\mu_1 = \gamma^0 \eta \nabla R(\theta_1)$ sólo toma en cuenta el gradiente inmediatamente anterior.

Ahora supongamos que la proposición se cumple para $t - 1$; es decir que: $\mu_{t-1} = \sum_{i=0}^{t-1} \gamma^i \eta \nabla R(\theta_{t-1})$. Entonces, para obtener μ_t en la actualización de la época $t + 1$ obtenemos:

$$\begin{aligned} \theta_{t+1} &= \theta_t - \gamma \mu_t \\ &= \theta_t - \eta \nabla R(\theta_t) - \gamma \mu_{t-1} \\ &= \theta_t - \eta \nabla R(\theta_t) - \gamma \sum_{i=0}^{t-1} \gamma^i \eta \nabla R(\theta_{t-i}) \\ &= \theta_t - \sum_{i=0}^t \gamma^i \eta \nabla R(\theta_{t-i}) \end{aligned}$$

Esto demuestra el resultado. □

El factor γ generalmente se encuentra entre 0 y 1, específicamente $0 < \gamma < 1$, de tal forma que el gradiente más antiguo tenga una influencia pequeña determinada por γ^i en la época t . Es decir, el descenso tomará en cuenta la dirección determinada por los gradientes anteriores, pero con un factor determinado por una potencia de γ , esto ayudará a acelerar la convergencia, pero evitando mínimos locales que estarían determinados por el gradiente inmediato.

Impulso de Nesterov

Similar al método del momentum, se propone otro optimizador, el impulso de Nesterov (Lin et al., 2019), que toma en cuenta un momento el cual se suma al gradiente de la función, pero además el gradiente se evalúa en el punto $\theta - \gamma \mu_{t-1}$. Esto busca que el gradiente se obtenga no desde el punto actual, sino desde la dirección donde previamente se han acumulado los gradientes, buscando así evitar de mejor forma los mínimos locales, al mismo tiempo que se acelera el descenso por gradiente.

Definición 70 (Impulso de Nesterov). *El método de impulso (o momentum) de Nesterov es un optimizador basado en descenso por gradiente que, dada la función objetivo $R : \Theta \rightarrow \mathbb{R}$, busca minimizar la función en base a una tasa de cambio:*

$$\mu_t = \eta \nabla_{\theta} R(\theta - \gamma \mu_{t-1}) + \gamma \mu_{t-1}$$

De tal forma que la actualización se realiza de la forma:

$$\theta \leftarrow \theta - \mu_t$$

Se pueden observar las similitudes entre el impulso de Nesterov y el de momentum, pues se acumulan los gradientes con base a un factor γ . La diferencia aquí es que el gradiente no parte del punto θ , sino del punto trasladado por la acumulación $\gamma \mu_{t-1}$. Esta traslación puede llevar a explorar mayor parte del espacio de hipótesis Θ , por lo que parecería más efectivo ante posibles mínimos locales.

Adagrad

Los dos métodos anteriores basados en momentos buscan introducir un factor direccional, basado en los gradientes acumulados, en cada época de la actualización de los pesos. En cada época, de hecho, la influencia del gradiente actual no se ve modificada. En ambos casos el último gradiente calculado se multiplica directamente por la tasa de aprendizaje η . Sin embargo, como habíamos mencionado al estudiar el descenso por gradiente estándar, la elección de la tasa de aprendizaje puede afectar la convergencia del aprendizaje. Por tanto, se han propuesto métodos que adapten la tasa de aprendizaje para que la influencia del gradiente en cada época se de con base en la magnitud de dicho gradiente. En particular, el método de Adagrad (Duchi et al., 2011), llamado así por *Adaptive gradient*, busca adaptar la tasa de aprendizaje con respecto a la información del gradiente.

Definición 71 (Adagrad). *El método de Adagrad es un método basado en descenso por gradiente que modifica el rango de aprendizaje con base en un factor estimado como:*

$$\mu_t \leftarrow \mu_t + (\nabla_{\theta} R(\theta_t))^2$$

Tal que $\mu_0 = 0$. La regla de actualización para Adagrad está dada como:

$$\theta_t \leftarrow \theta_t - \frac{\eta}{\sqrt{\mu_t + \varepsilon}} \nabla_{\theta} R(\theta_t)$$

Donde ε es un hiperparámetro que evita las divisiones entre 0, y generalmente $\varepsilon = 1e-8$.

Como se observa, el método de Adagrad rescala la tasa de aprendizaje con respecto al factor μ_t . El valor ε evita la división entre cero. Este factor está relacionado con el tamaño del gradiente. Vemos que en general bajo el método de Adagrad la tasa de aprendizaje se adapta como:

$$\frac{\eta}{\sqrt{\sum_{i=1}^t \nabla R(\theta_i)^2 + \varepsilon}}$$

Para ver el efecto que tiene el método de Adagrad retomamos el ejemplo anterior donde el método de descenso por gradiente estándar no converge.

Ejemplo 13. Supóngase la función objetivo $R(\theta) = \theta^2$ cuya derivada está dada por $\nabla R(\theta) = 2\theta$. Considérese que $\theta_1 = 1$ y que la tasa de aprendizaje es $\eta = 1$. Bajo estos parámetros el método de descenso por gradiente no llega a converger, pues vacila entre los valores 1 y -1. Con el método de Adagrad obtenemos en primer lugar que:

$$\mu_1 = \nabla R(\theta_1)^2 = (2(1))^2 = 4$$

Si tomamos $\varepsilon = 0$, la primera época se actualiza como:

$$\begin{aligned}\theta_2 &= \theta_1 - \frac{\eta}{\sqrt{\nabla R(\theta)}} \nabla R(\theta_1) \\ &= 1 - \frac{1}{\sqrt{4}} 2 \\ &= 1 - \frac{2}{2} \\ &= 0\end{aligned}$$

En este caso, hemos convergido a la solución en la primera época utilizando el método de Adagrad.

RMSProp y Adadelta

El método de de Adagrad toma en cuenta la suma de los cuadrados de los gradientes de forma directa. El método de RMSProp (Mukkamala and Hein, 2017) se fundamenta en el método de Adagrad, pero considerando una ponderación en el cuadrado de los gradientes de los tiempos anteriores por medio de un hiperparámetro $\gamma \in \mathbb{R}$. Para definir el método de RMSProp, primero, definiremos la raíz del promedio al cuadrado o *Root Mean Squared*.

Definición 72 (Raíz del promedio cuadrado). *La raíz de la media al cuadrado o RMS (de Root Mean Squared) del gradiente de una función objetivo $R : \Theta \rightarrow \mathbb{R}$ se define como:*

$$RMS(\nabla R(\theta))_t = \sqrt{\mathbb{E}[\nabla R(\theta)^2]_t + \varepsilon}$$

Donde la media $\mathbb{E}[\nabla R(\theta)^2]_t$ en la época t se estima por medio de una combinación convexa:

$$\mathbb{E}[\nabla R(\theta)^2]_t \leftarrow \gamma \mathbb{E}[\nabla R(\theta)^2]_{t-1} + (1 - \gamma) \nabla R(\theta)^2$$

con $\gamma \in [0, 1]$.

Proposición 15. *Dada una función objetivo $R : \Theta \rightarrow \mathbb{R}$ con gradientes $\nabla R(\theta_t)$ en cada época, el factor RMS en la época t está determinado como:*

$$RMS(\nabla R(\theta))_t = \sqrt{\sum_{i=0}^{t-1} (\gamma^i - \gamma^{i+1}) \nabla R(\theta_{t-i})^2 + \varepsilon}$$

Demostración. La demostración se puede realizar por inducción y se deja como ejercicio. $\square$

El factor RMS determina la influencia de los gradientes previos por medio de un factor $\gamma^i - \gamma^{i+1}$; ya que $\gamma \in [0, 1]$, este factor siempre es positivo. El gradiente actual siempre está determinado por $1 - \gamma$, por lo que la elección de γ determinará su influencia. Asimismo, se puede observar que si $\gamma = 0$, entonces $RMS(\nabla R(\theta))_t = \nabla R(\theta_t)^2$. Por

su parte, tomar $\gamma = 1$ genera que todos los gradientes se anulen, por lo que el factor sólo será $\sqrt{\epsilon}$. Podemos comprobar que esta forma de actualizar el RMS se aproxima a una esperanza sobre una distribución que pondera con mayor probabilidad a los gradientes recientes.

Proposición 16. *La forma de actualizar el factor $\mathbb{E}[\nabla R(\theta)^2]_t$ como se muestra en la Definición 72 converge a una esperanza sobre $\nabla R(\theta)^2$ cuando t tiende a infinito y $\gamma < 1$.*

Demostración. Utilizando la Proposición 15 obtenemos que el valor acumulado en la variable $\mathbb{E}[\nabla R(\theta)^2]_{t+1}$ es de la forma:

$$\mathbb{E}[\nabla R(\theta)^2]_{t+1} = \sum_{i=0}^t (\gamma^i - \gamma^{i+1}) \nabla R(\theta_{t-i})^2$$

En primer lugar, notamos que la suma de los factores $(\gamma^i - \gamma^{i+1})$ hasta el tiempo t está dada como:

$$\begin{aligned} \sum_{i=0}^t (\gamma^i - \gamma^{i+1}) &= (\gamma^0 - \gamma^1) + (\gamma^1 - \gamma^2) + \dots + (\gamma^t - \gamma^{t+1}) \\ &= \gamma^0 + (\gamma^1 - \gamma^1) + (\gamma^2 - \gamma^2) + \dots + (\gamma^t - \gamma^t) - \gamma^{t+1} \\ &= 1 - \gamma^{t+1} \end{aligned}$$

Ahora bien, obteniendo el límite cuando t tiende a infinito y considerando que el hiperparámetro $\gamma < 1$, obtenemos:

$$\begin{aligned} \lim_{t \rightarrow \infty} \sum_{i=0}^t (\gamma^i - \gamma^{i+1}) &= \lim_{t \rightarrow \infty} 1 - \gamma^{t+1} \\ &= 1 - \lim_{t \rightarrow \infty} \gamma^{t+1} \\ &= 1 \end{aligned}$$

Es decir, en el límite los factores $(\gamma^i - \gamma^{i+1})$ definen una distribución de probabilidad. Por tanto, $\mathbb{E}[\nabla R(\theta)^2]_t$ es un valor esperado cuando t tiende a infinito. $\square$

La forma de obtener los valores de RMS, por tanto, se acercan a obtener un valor esperado sobre los gradientes al cuadrado, de allí la notación utilizada para este factor. En la práctica, se espera que $\gamma < 1$, puesto que si $\gamma = 1$, entonces el factor RMS siempre será 0. El método de RMSProp toma este factor RMS y lo utiliza de forma similar que Adagrad para dividirlo en cada época por la tasa de aprendizaje, adaptando esta tasa.

Definición 73 (RMSProp). *El método de RMSProp es un método de optimización basado en descenso de gradiente que, dado una función objetivo $R : \Theta \rightarrow \mathbb{R}$, tiene la regla de actualización:*

$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta}{\text{RMS}(\nabla R(\theta))_t} \nabla R(\theta_t)$$

Al igual que el método de Adagrad, el método de RMSProp busca acelerar la convergencia al mínimo y evitar los mínimos locales. Bajo una elección adecuada del hiperparámetro γ , el método de RMSProp puede tener un mejor desempeño que el método de Adagrad y que los métodos basados en momentum. Sin embargo, en RMSProp las unidades del factor de cambio de los pesos no corresponde a las unidades de los pesos. Para solventar esta dificultad, se propone el método de Adadelta (Zeiler, 2012), que utiliza un factor delta para realizar el cambio.

Definición 74 (Factor delta). *Dada una función objetivo $R : \Theta \rightarrow \mathbb{R}$ su factor delta se define en base al RMS de la siguiente forma.*

$$\Delta\theta_t = -\frac{RMS(\Delta\theta_{t-1})}{RMS(\nabla R(\theta))_t} \nabla R(\theta)$$

Tal que $\Delta\theta_0 = 0$.

El valor de $RMS(\Delta\theta_t)$ se puede calcular mediante la fórmula expuesta en la Proposición 15. De manera iterativa se puede utilizar la regla de actualización de la Definición 72, de tal forma que tendríamos:

$$\mathbb{E}[\Delta\theta^2]_t \leftarrow \gamma \mathbb{E}[\Delta\theta^2]_{t-1} + (1 - \gamma) \Delta\theta_t^2$$

Por lo tanto, el método de Adadelta no sólo acumula el cuadrado de los gradientes anteriores, sino que los valores de $\Delta\theta$ acumulan los cambios que se producen en épocas anteriores. Para obtener $RMS(\Delta\theta_t)$ basta sumar ε y sacar raíz cuadrada. A partir de este valor, podemos definir el método de Adadelta.

Definición 75 (Adadelta). *El método de Adadelta es un método de optimización basado en descenso por gradiente que optimiza una función objetivo $R : \Theta \rightarrow \mathbb{R}$ a partir de la regla de actualización:*

$$\theta \leftarrow \theta + \Delta\theta$$

Donde el factor $\Delta\theta$ se define como anteriormente.

Adam

Hasta ahora hemos revisado dos métodos que se basan en momentum (el método de momentum y el impulso de Nesterov), así como tres métodos adaptativos (Adagrad, RMSProp y Adadelta). Todos estos métodos buscan que el aprendizaje se realice de forma más eficiente: con mayor rapidez y buscando evitar mínimos locales. Ambas estrategias, el momentum y la adaptividad, muestran diferentes formas de alcanzar este objetivo. En la actualidad, el método más utilizado es el de Adam (Kingma, 2014); se trata de un método que utiliza un momento y una adaptación. Su nombre proviene de *Adaptive Moment Estimation*. Para definir el método de Adam, definiremos primero los vectores de momento.

Definición 76 (Vectores de momento). *Dada una función objetivo $R : \Theta \rightarrow \mathbb{R}$ definimos el primer vector de momento como:*

$$\mu \leftarrow \beta_1 \mu + (1 - \beta_1) \nabla_{\theta} R(\theta)$$

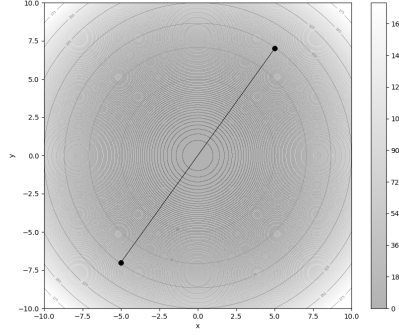
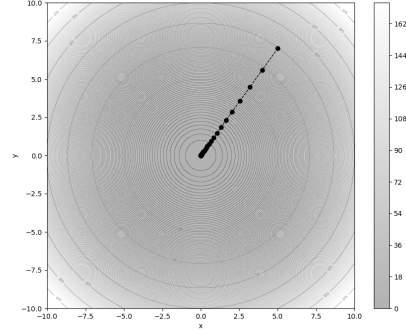
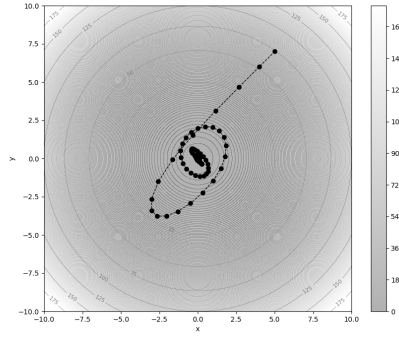
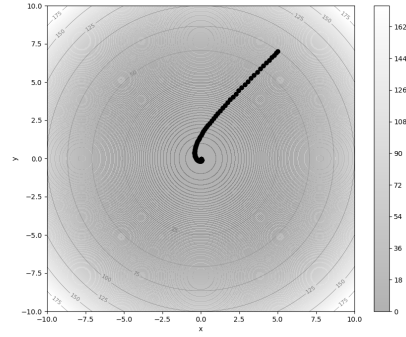
(a) DG, $\eta = 1$ (b) DG, $\eta = 0,1$ (c) Adam, $\eta = 1$ (d) Adam, $\eta = 0,1$

Figura 4.3: Comparación de la convergencia del descenso por gradiente y Adam. En la figura a) se muestra el descenso por gradiente con una tasa de aprendizaje $\eta = 1$, el método no llega a converger, vacila entre los valores positivos y negativos de inicio. En b) se da la convergencia con una tasa de aprendizaje $\eta = 0,1$, el método converge siempre descendiendo hacia el mínimo. En las figuras inferiores se muestra el método de Adam. Adam converge tanto con una tasa de aprendizaje grande ($\eta = 1$) como con una menor ($\eta = 0,1$). Cuando la tasa de aprendizaje inciaa en un valor mayor la convergencia es menos directa.

Y el segundo vector de momento como:

$$\mathbf{v} \leftarrow \beta_2 \mathbf{v} + (1 - \beta_2) (\nabla_{\theta} R(\theta))^2$$

En ambos casos tenemos que $\beta_1, \beta_2 \in [0, 1)$ son hiperparámetros y tanto μ como $\mathbf{v}$ se inicializan en 0. Los vectores de momento corregidos por sesgo se calculan con base en estos como:

$$\hat{\mu} = \frac{\mu}{1 - \beta_1}$$

Para el primer momento, y:

$$\hat{v} = \frac{v}{1 - \beta_2}$$

Para el segundo momento.

La forma en que se calculan los vectores de momento recuerda a la forma de estimar los valores para la función RMS. De hecho, podemos observar que en general se pueden expresar de una forma similar a lo apuntado en la Proposición 15.

Proposición 17. Sea $R : \Theta \rightarrow \mathbb{R}$ una función objetivo. Sus vectores de momento para la época $t + 1$ son de la forma:

$$\mu_{t+1} = \sum_{i=0}^t (\beta_1^i - \beta_1^{i+1}) \nabla R(\theta_{t+1-i})$$

Para el primer momento, mientras que para el segundo se tiene:

$$v_{t+1} = \sum_{i=0}^t (\beta_2^i - \beta_2^{i+1}) \nabla R(\theta_{t+1-i})^2$$

Ya que $\beta_1, \beta_2 \in [0, 1)$ entonces el factor $\beta_k^i - \beta_k^{i+1}$, $k \in \{1, 2\}$, siempre será mayor a 0.

Proposición 18. En los vectores de momento μ y v se tiene que para $\beta \in [0, 1)$ se cumple que:

$$\beta^i - \beta^{i+1} \geq \beta^{i+1} - \beta^{i+2}$$

Demostración. Denotemos como β a cualquiera de los dos hiperparámetros. Como $\beta \in [0, 1)$ es claro que $\beta^i - \beta^{i+1} \geq 0$ para cada época i . Ahora tomemos la diferencia de estos factores entre dos épocas (la época $i + 1$ y la época i); tenemos:

$$\begin{aligned} (\beta^{i+1} - \beta^{i+2}) - (\beta^i - \beta^{i+1}) &= \beta^{i+1} - \beta^{i+2} - \beta^i + \beta^{i+1} \\ &= 2\beta^{i+1} - \beta^{i+2} - \beta^i \\ &= (2\beta - \beta^2 - 1)\beta^i \\ &= -(\beta - 1)^2\beta^i \\ &\leq 0 \end{aligned}$$

De esta desigualdad se deduce el resultado. $\square$

De el resultado anterior podemos concluir que los gradientes de las épocas anteriores influyen menos en las actualizaciones de la época actual, de forma similar al caso de RMSProp. De hecho, vemos que en el límite, al igual que pasaba con RMSProp, estos vectores convergen al primer y segundo momento sobre el gradiente.

Teorema 13. Dada la función $R : \Theta \rightarrow \mathbb{R}^+$ con gradiente $\nabla R(\theta)$, el vector μ y v convergen al primer y segundo momento del gradiente, respectivamente. Esto es:

$$\lim_{t \rightarrow \infty} \mu_t = \mathbb{E}[\nabla R(\theta)]$$

y

$$\lim_{t \rightarrow \infty} v_t = \mathbb{E}[\nabla R(\theta)^2]$$

Demostración. Al igual que en el caso de RMSProp, sólo debemos probar que los factores $\beta_j^i - \beta_j^{i+1}$ de la Proposición 17 son una medida de probabilidad. Para el caso de μ , tomando μ_t como el momento acumulado hasta el valor actual, tenemos que:

$$\lim_{t \rightarrow \infty} \sum_{i=0}^t (\beta_1^i - \beta_1^{i+1}) = \lim_{t \rightarrow \infty} (\beta_1^0 - \beta_1^1) + (\beta_1^1 - \beta_1^2) + \dots + (\beta_1^t - \beta_1^{t+1}) \quad (4.3)$$

$$= \lim_{t \rightarrow \infty} 1 + (\beta_1^1 - \beta_1^1) + \dots + (\beta_1^t - \beta_1^t) + \beta_1^{t+1} \quad (4.4)$$

$$= \lim_{t \rightarrow \infty} 1 + \beta_1^{t+1} \quad (4.5)$$

$$= 1 + \lim_{t \rightarrow \infty} \beta_1^{t+1} = 1 + 0 \quad (4.6)$$

$$= 1 \quad (4.7)$$

Ya que $\beta_1 < 1$. Por tanto, tomando el límite sobre todo el vector μ_t , podemos observar que define el primer momento del gradiente:

$$\lim_{t \rightarrow \infty} \mu_t = \lim_{t \rightarrow \infty} \sum_{i=0}^{t-1} (\beta_1^i - \beta_1^{i+1}) \nabla R(\theta_{t-i}) = \mathbb{E}[\nabla R(\theta)]$$

De igual forma, se puede comprobar que v define el segundo momento bajo el mismo argumento. □

Al corregir sobre el sesgo que divide entre el factor $1 - \beta_k$, observamos que los vectores de momento resultan en:

$$\hat{\mu}_{t+1} = \nabla R(\theta_{t+1}) + \sum_{i=1}^t \frac{\beta_1^i - \beta_1^{i+1}}{1 - \beta_1} \nabla R(\theta_{t+1-i})$$

Para el primer momento, y para el segundo momento:

$$\hat{v}_{t+1} = \nabla R(\theta_{t+1})^2 + \sum_{i=1}^t \frac{\beta_2^i - \beta_2^{i+1}}{1 - \beta_2} \nabla R(\theta_{t+1-i})^2$$

Los primeros gradientes en ambos casos se dan sin modificación alguna, mientras que los gradientes en las épocas pasadas se ponderan por un factor que depende de β_1 y β_2 . A partir de estos vectores de momento se puede definir el método de Adam.

Definición 77 (Adam). *El método de Adam es un método de optimización basado en gradiente que actualiza los parámetros de una función objetivo con base a la siguiente regla:*

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{\hat{v}} + \epsilon} \hat{\mu}$$

Donde $\hat{\mu}$ y $\hat{v}$ son los vectores del primer y segundo momento corregido por sesgo, respectivamente.

Podemos notar que el método de Adam generaliza otros casos. Por ejemplo, si tomamos $\beta_1 = 0$ y tomamos un β_2 arbitrario, entonces tenemos un método similar a RMSProp (excepto por la corrección de sesgo). Ahora bien si tomamos un valor de $\beta_2 = 0$ y un valor de β_1 arbitrario obtenemos un método cercano al de momentum (excepto por una adaptación al dividir entre el cuadrado del último gradiente). Finalmente, si tomamos $\beta_1 = 0$ y $\beta_2 = \frac{1}{2}$ obtenemos el método de Adagrad.

En la Figura 4.3 se puede ver una comparación entre el descenso de gradiente estándar y el método de Adam con parámetros $\beta_1 = 0,9$ y $\beta_2 = 0,99$, los cuales son recomendados en la literatura. Como se puede observar, el método de Adam converge a pesar de iniciar con una tasa de aprendizaje alta, mientras que el descenso por gradiente no converge en estos casos. También es interesante ver que en la Figura 4.3 c), el método de Adam vacila al llegar al mínimo. Si la función tuviera mínimos locales, este comportamiento ayudaría a librarlos.

En la actualidad, el método de Adam es el más utilizado para el aprendizaje en redes neuronales. Existen variantes de este algoritmo, que pueden revisarse en Kingma (2014). En Vaswani et al. (2017) se propone otra variación que adopta el optimizador Adam como base, el llamado optimizador Noam (de *normalized Adam*), el cual se utiliza principalmente para el entrenamiento de los Transformers. La elección del optimizador juega un papel importante en el aprendizaje y los resultados que obtendremos.

4.3. Retro-propagación

Hemos revisado diferentes optimizadores basados en descenso de gradiente para poder optimizar una función objetivo y aprender a resolver un problema. En el Ejemplo 12 hemos utilizado este método para derivar una regla de aprendizaje sobre el modelo de regresión logística. Sin embargo, en las redes profundas no es sencillo escribir una regla cómo lo hemos hecho para la regresión logística pues éstas cuentan con un número arbitrario de capas ocultas. Esto implicaría que cada vez que tuviéramos una arquitectura nueva, tendríamos que calcular su gradiente para poder optimizar los pesos.

Una red feedforward, y en general una red neuronal profunda, puede verse como la composición de diferentes funciones caracterizadas como capas ocultas que se aplican de forma consecutiva a una entrada de la forma:

$$f(x; \theta) = \phi \left(h^{(K)} \left(\dots h^{(1)}(x; \theta_1); \theta_K \right); \theta_{K+1} \right)$$

Cada función que hemos denotado como $h^{(k)}$ depende de un conjunto de parámetros θ_k , $k \in \{1, \dots, K, K+1\}$. La función ϕ es la capa de salida, que toma la última capa oculta para obtener una respuesta. Que la red neuronal aprenda implica optimizar los parámetros θ_k bajo una función objetivo. La función objetivo evalúa el resultado de la red neuronal con base en un criterio L (que llamaremos función de pérdida) y considerando el conjunto de datos de entrenamiento (x, y) . Podemos, entonces, escribir la función objetivo como:

$$R(\theta) = \sum_{(x,y)} L(y, f_y(x; \theta))$$

La función objetivo depende de los parámetros $\theta = \{\theta_k : k = 1, \dots, K, K+1\}$. La notación $f_y(x; \theta)$ denota la y -ésima neurona de salida de la red bajo los parámetros θ .

Entonces, tenemos que optimizar una función objetivo compuesta por una red neuronal con diferentes capas:

$$\nabla_{\theta} R(\theta) = \nabla_{\theta} (R \circ \phi \circ h^{(K)} \circ \dots \circ h^{(1)})(\theta) \quad (4.8)$$

$$= \nabla_{\phi} R \nabla_{h^{(K)}} \phi \dots \nabla_{\theta_k} h^{(1)} \quad (4.9)$$

En la ecuación anterior hemos utilizado la regla de la cadena, que apunta a que el gradiente de una función compuesta es el producto de los gradientes que componen a la función. Debemos observar dos cosas: en primer lugar, que debemos obtener un gradiente para cada conjunto de parámetros en las diferentes capas; en segundo lugar, que los datos de entrenamiento $x \in \mathbb{R}^d$ se consideran como constantes en la función objetivo; al momento de obtener los gradientes, estos no se modifican.

Como mencionábamos, ya que el número de capas ocultas (y el tipo de éstas) es arbitrario, desarrollaremos un método de diferenciación automática que nos permita obtener los gradientes correspondientes para cada arquitectura de red neuronal. Para motivar este algoritmo, antes revisaremos un ejemplo simple de una red con una capa oculta.

Ejemplo 14. *Supongamos que tenemos una red neuronal feedforward con una sola capa oculta y que está definida de la siguiente forma:*

$$f(x; \theta) = \phi(W^{(2)} \tanh(W^{(1)}x))$$

En esta red el conjunto de parámetros $\theta = \{W^{(1)}, W^{(2)}\}$ son los pesos de las conexiones de la red de dimensiones arbitrarias (no hemos considerado el bias por simplicidad). Como se puede observar se tiene una capa oculta $h = \tanh(W^{(1)}x)$ con pre-activación $a^{(1)} = W^{(1)}x$. La pre-activación de la capa de salida la denotaremos como $a^{(2)} = W^{(2)}h$. Finalmente, consideremos la función objetivo:

$$R(\theta) = \sum_x \sum_y L(y, f_y(x))$$

Entonces tenemos que obtener los gradientes sobre el conjunto de pesos. Ya que $W^{(1)}$ y $W^{(2)}$ son matrices, obtendremos las derivadas parciales de la función de riesgo sobre los parámetros. Más aún, podemos denotar $\theta_{k,i,j} = W_{ij}^{(k)}$. Comencemos obteniendo las derivadas parciales sobre los pesos en la capa de salida (ignoramos la suma sobre x , pues ésta depende de los lotes):

$$\begin{aligned} \frac{\partial R}{\partial W_{ij}^{(2)}} &= \frac{\partial}{\partial W_{ij}^{(2)}} \sum_y L(y, f_y(x)) \\ &= \sum_y \frac{\partial}{\partial W_{ij}^{(2)}} L(y, f_y(x)) \end{aligned}$$

Gracias a la linealidad de la derivada podemos enfocarnos sólo en derivar la función $L(y, f_y(x))$ en cada una de las neuronas de salida de la red. Derivando esta función

sobre los pesos de salida, tenemos:

$$\begin{aligned}\frac{\partial L}{\partial W_{ij}^{(2)}} &= \frac{\partial L}{\partial f_y} \frac{\partial f_y}{\partial a_y^{(2)}} \frac{\partial a_y^{(2)}}{\partial W_{ij}^{(2)}} \\ &= \frac{\partial L}{\partial f_y} \frac{\partial f_y}{\partial a_y^{(2)}} h_j\end{aligned}$$

Es decir, obtenemos que la derivada de la función de pérdida sobre el peso $W_{i,j}^{(2)}$ es el producto de las derivadas de L , f y $a^{(2)}$. Esta última derivada es h_j . Pues puede observarse que:

$$\begin{aligned}\frac{\partial a_i^{(2)}}{\partial W_{ij}^{(2)}} &= \frac{\partial}{\partial W_{ij}^{(2)}} W_{i,\cdot}^{(2)} h \\ &= \frac{\partial}{\partial W_{ij}^{(2)}} \sum_k W_{i,k}^{(2)} h_k \\ &= \sum_k \frac{\partial}{\partial W_{ij}^{(2)}} W_{i,k}^{(2)} h_k \\ &= h_j\end{aligned}$$

Con esta derivada podemos actualizar los pesos de la capa de salida. De tal forma que para cada neurona y en la salida, tendremos que sus derivadas parciales serán de la forma $\frac{\partial L}{\partial f_y} \frac{\partial f_y}{\partial a^{(2)}} h_j$. Si ahora nos fijamos en el Jacobiano de la función R , notaremos que al derivar por cada neurona de salida, obtenemos un vector columna que multiplica a los valores de la neurona de la capa oculta previa. Por lo que el Jacobiano de R será:

$$\begin{aligned}\nabla_{W^{(2)}} R(\theta) &= \begin{pmatrix} \frac{\partial L}{\partial f_1} \frac{\partial f_1}{\partial a^{(2)}} h_1 & \frac{\partial L}{\partial f_1} \frac{\partial f_1}{\partial a^{(2)}} h_2 & \cdots & \frac{\partial L}{\partial f_1} \frac{\partial f_1}{\partial a^{(2)}} h_m \\ \frac{\partial L}{\partial f_2} \frac{\partial f_2}{\partial a^{(2)}} h_1 & \frac{\partial L}{\partial f_2} \frac{\partial f_2}{\partial a^{(2)}} h_2 & \cdots & \frac{\partial L}{\partial f_2} \frac{\partial f_2}{\partial a^{(2)}} h_m \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial L}{\partial f_o} \frac{\partial f_o}{\partial a^{(2)}} h_1 & \frac{\partial L}{\partial f_o} \frac{\partial f_o}{\partial a^{(2)}} h_2 & \cdots & \frac{\partial L}{\partial f_o} \frac{\partial f_o}{\partial a^{(2)}} h_m \end{pmatrix} \\ &= \begin{pmatrix} \frac{\partial L}{\partial f_1} \frac{\partial f_1}{\partial a^{(2)}} \\ \frac{\partial L}{\partial f_2} \frac{\partial f_2}{\partial a^{(2)}} \\ \vdots \\ \frac{\partial L}{\partial f_o} \frac{\partial f_o}{\partial a^{(2)}} \end{pmatrix} \otimes h\end{aligned}$$

Donde $\otimes$ es el producto externo de los vectores. Hemos obtenido entonces el gradiente para la capa de salida, lo que nos permitirá actualizar los pesos en esta capa por medio de algún optimizador. En el caso del descenso por gradiente la actualización será $W^{(2)} = W^{(2)} - \eta \nabla_{W^{(2)}} R(\theta)$. Para la capa oculta, podemos observar que derivar

sobre los pesos $W_{i,j}^{(1)}$ implica aplicar la regla de la cadena sobre las capas superiores:

$$\begin{aligned}
 \frac{\partial R}{\partial W_{ij}^{(1)}} &= \frac{\partial}{\partial W_{ij}^{(1)}} \sum_y L(y, f_y(x)) \\
 &= \sum_y \frac{\partial}{\partial W_{ij}^{(1)}} L(y, f_y(x)) \\
 &= \sum_y \frac{\partial L}{\partial f_y} \frac{\partial f_y}{\partial a_y^{(2)}} \frac{\partial a_y^{(2)}}{\partial h_i} \frac{\partial h_i}{\partial a_i^{(2)}} \frac{\partial a_i^{(2)}}{\partial W_{ij}^{(1)}} \\
 &= \left(\sum_y \frac{\partial L}{\partial f_y} \frac{\partial f_y}{\partial a_y^{(2)}} W_{y,i}^{(2)} \right) \frac{\partial h_i}{\partial a_i^{(2)}} h_j
 \end{aligned}$$

Lo interesante es que el factor $\frac{\partial L}{\partial f_y} \frac{\partial f_y}{\partial a_y^{(2)}}$ lo hemos calculado para obtener las derivadas de la salida, y ahora este factor lo utilizamos de nuevo aquí, sólo que multiplicando a la matriz de pesos de la capa de salidas. Por tanto, podemos guardar una variable $d_2(y)$ que definiremos como:

$$d_2(y) = \frac{\partial L}{\partial f_y} \frac{\partial f_y}{\partial a_y^{(2)}}$$

De tal forma que la derivada en los siguientes casos se puede expresar como:

$$\frac{\partial R}{\partial W_{ij}^{(1)}} = \left(\sum_y d_2(y) W_{y,i}^{(2)} \right) \frac{\partial h_i}{\partial a_i^{(2)}} h_j$$

Más aún, si denotamos al vector de las variables $d_2(y)$ como $\mathbf{d}_2$, tenemos que:

$$\frac{\partial R}{\partial W_{ij}^{(1)}} = (\mathbf{d}_2 \cdot \mathbf{W}_{:,i}^{(2)}) \frac{\partial h_i}{\partial a_i^{(2)}} h_j$$

Donde $\mathbf{d}_2 \cdot \mathbf{W}_{:,i}^{(2)}$ es el producto punto entre el i -ésimo vector columna de $W^{(2)}$ y el vector de variables $d_2(y)$ $\mathbf{d}_2$. De esta forma, hemos obtenido las derivadas para las capas ocultas y las capas de salida, por lo que aplicando un optimizador como el descenso por gradiente nos permitirá aprender los pesos de la red neuronal que minimicen la función objetivo dada.

El ejemplo anterior nos muestra que al derivar sobre la capa oculta, la derivada se puede expresar en términos de derivadas que ya se obtuvieron al derivar las capas superiores. Incluir más capas ocultas nos llevaría a conclusiones similares. Observaríamos que hay valores que se repiten y que ya hemos calculado. De hecho, requeriríamos de las derivadas en la capa superior para obtener las derivadas de la capa inmediatamente anterior. Las derivadas se van acumulando y se propagan hacia las capas previas. La idea de propagar hacia atrás las derivadas es lo que radica detrás del algoritmo de retro-propagación o *backpropagation*.

Por tanto, podemos pensar el algoritmo de retro-propagación como un algoritmo iterativo que inicia obteniendo las derivadas en la primera capa y posteriormente obtiene derivadas en las siguientes capas con base en las derivadas previas. Podemos describir el algoritmo de la siguiente forma:

1. **Entrada:** Una red neuronal $f(x; \theta)$ con K capas ocultas y con los parámetros:

$$\theta = \{W^{(k)}, b^{(k)} : k = 1, \dots, K, K+1\}$$

que depende de datos de entrenamiento supervisados (x, y) . Una función objetivo $R(\theta) = \sum L(y, f(x; \theta))$.

2. **Salida:** Las derivadas o gradientes $\frac{\partial R}{\partial \theta_{k,i,j}}$ para todo conjunto de pesos $\theta_{k,i,j} = W_{i,j}^{(k)}$.
3. **Inicialización:** Se obtienen las derivadas para la capa de salida que se almacenan en una variable $d_{K+1}(i)$, para toda neurona de salida i :

$$d_{K+1}(i) = \frac{\partial L}{\partial f_i} \frac{\partial f_i}{\partial a_i^{(K+1)}}$$

4. **Inducción:** Se recorren las capas ocultas en sentido inverso, obteniendo las variables $d_k(i)$ para toda neurona i de la capa k :

$$d_k(i) = \frac{\partial h_i^{(k)}}{\partial a_i^{(k)}} \sum_q d_{k+1}(q) W_{q,i}^{(k+1)}$$

Para $k \in \{K, K-1, \dots, 1\}$.

5. **Finalización:** Las derivadas parciales para cada conjunto de pesos estarán dadas de la forma:

$$\frac{\partial R(\theta)}{\partial W_{i,j}^{(k)}} = d_k(i) h_j^{(k-1)}$$

Donde $k = 1, \dots, K, K+1$ y $h_j^{(k-1)}$ es la j -ésima neurona de la capa anterior, considerando que $h^{(0)} = x$ el vector de entrada.

El algoritmo de retro-propagación es un método que nos permite obtener las derivadas sobre una estructura de red neuronal. En la Figura 4.4 se muestra de forma gráfica la intuición detrás del método. Se trata de una red con o neuronas de salida de la forma $f_i(x)$, $i \in \{1, \dots, o\}$ y con una función objetivo de error cuadrático. La retro-propagación obtendrá las derivadas de las pérdidas y las salidas que guarda en las variables $d_{out}(i)$ y utiliza esas variables para obtener las variables $d_K(j)$ en las capas anteriores para las neuronas de esta capa. Puede observarse que las variables de la capa superior $d_{K+1}(i)$ multiplican a los pesos. De tal forma que se retro-propaga $W_{q,i}^{(k+1)} d_{k+1}(i)$, los cuáles son los términos de la suma en $d_k(i)$. Esto es similar a la forma en que las entradas

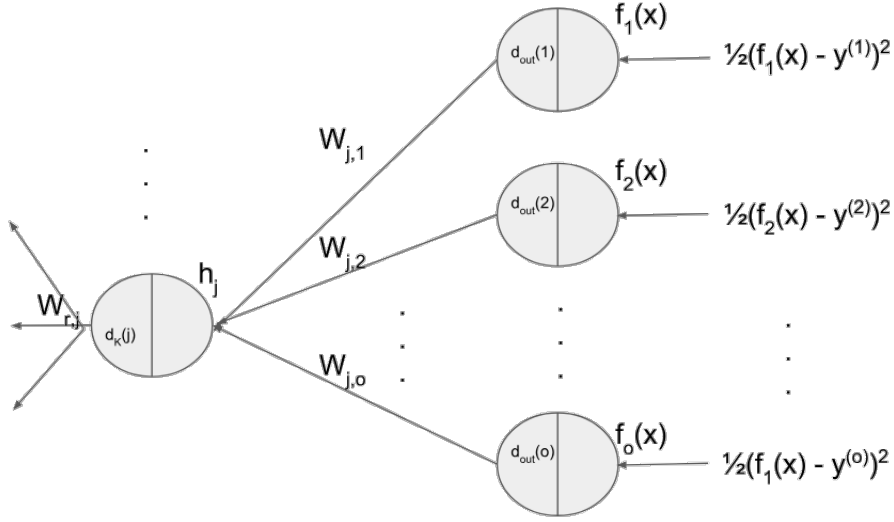


Figura 4.4: Retro-propagación sobre una red con función de riesgo MSE

se propagan hacia adelante en la red. Por lo que podemos expresar un vector $\mathbf{d}_k$ de variables como:

$$\mathbf{d}_k = \nabla h^{(k)} \odot (\mathbf{d}_{k+1}^T W^{(k+1)})$$

Donde $\mathbf{d}_k^T = (d_k(1) \ d_k(2) \ \dots \ d_k(m_k))$ es el vector de variables para las m_k neuronas de la capa k , y $\nabla h^{(k)}$ es el vector gradiente que guarda las derivadas parciales de la activación en ésta capa. De esta forma, podemos expresar también el algoritmo de retro-propagación en términos de productos entre matrices y vectores. Por lo que los gradientes en cada capa k se definen como un producto externo de la forma:²

$$\nabla_k R(\theta) = \mathbf{d}_k \otimes h^{(k-1)}$$

De tal forma que podremos actualizar los pesos en cada capa con alguno de los optimizadores discutidos en la Sección 4.2. Por ejemplo, para el descenso por gradiente estándar tenemos la regla de actualización:

$$W^{(k)} \leftarrow W^{(k)} - \eta \nabla_k R(\theta)$$

El algoritmo de retro-propagación junto con el método de optimización basado en descenso por gradiente nos permitirá aprender sobre las redes neuronales en general. De esta forma, podemos dar una definición más funcional del algoritmo de retro-propagación.

Definición 78 (Retro-propagación). *El algoritmo de retro-propagación para una red neuronal $f : X \rightarrow Y$ con una función objetivo $R : \Theta \rightarrow \mathbb{R}^+$ y un conjunto de datos*

²Recuérdese que el producto externo se define como $(a \otimes b)_{i,j} = a_i b_j$.

$\{(x, y) : x \in \mathbb{R}^d, y \in Y\}$ obtiene los gradientes con base al cálculo de las variables $\mathbf{d}_k$ como:

$$\nabla_{W^{(k)}} R(\theta) = \mathbf{d}_k \otimes h^{(k-1)}(x) \quad (4.10)$$

Considerando que $h^{(0)} = x$. Estas variables se obtienen de forma recursiva con regla de inicialización:

$$\mathbf{d}_{K+1} = \nabla_f L(f(x), y) \odot \nabla_{a^{(K+1)}} f(x) \quad (4.11)$$

Y regla inductiva dada como:

$$\mathbf{d}_k = \nabla_{a^{(k)}} h^{(k)}(x) \odot \mathbf{d}_{k+1} \cdot W^{(k+1)} \quad (4.12)$$

para roda $k \in \{1, 2, \dots, K\}$.

Antes de continuar, demostraremos que, en efecto, este algoritmo obtiene las derivadas sobre las capas ocultas.

Teorema 14. Dada una red feedforward $f(x; \theta)$ con profundidad K y parámetros $\theta = \{W^{(k)}, b^{(k)} : k = 1, 2, \dots, K, K+1\}$ y la función de riesgo:

$$R(\theta) = \sum_x \sum_y L(y, f_y(x; \theta))$$

el algoritmo de retro-propagación obtiene las derivadas evaluadas en cada uno de los parámetros. Es decir:

$$d_k(i) h_j^{(k-1)} = \frac{\partial R(\theta)}{\partial W_{i,j}^{(k)}}$$

Para todo $k \in \{1, 2, \dots, K, K+1\}$ y con $h^{(0)} = x$.

Demostración. Lo demostramos por inducción sobre la profundidad de la red. En el caso base, tenemos una red de profundidad 0; este caso corresponde a un modelo similar al perceptrón y la regresión logística. Debemos fijarnos en la inicialización del algoritmo de retro-propagación, donde tenemos que:

$$d_{K+1}(i) = \frac{\partial L}{\partial f_i} \frac{\partial f_i}{\partial a_i^{(K+1)}}$$

Por otra parte, la derivada parcial de la función de pérdida L sobre $W_{i,j}$, aplicando la regla de la cadena, es:

$$\begin{aligned} \frac{\partial R(\theta)}{\partial W_{i,j}^{(K+1)}} &= \sum_i \frac{\partial L(y_i; f_i(x; \theta))}{\partial W_{i,j}^{(K+1)}} \\ &= \frac{\partial L(y_i; f_i(x; \theta))}{\partial W_{i,j}^{(K+1)}} \\ &= \frac{\partial L}{\partial f_i} \frac{\partial f_i}{\partial a_i^{(K+1)}} \frac{\partial a_i^{(K+1)}}{\partial W_{i,j}^{(K+1)}} \\ &= \frac{\partial L}{\partial f_i} \frac{\partial f_i}{\partial a_i^{(K+1)}} h_j^{(K)} \\ &= d_{K+1}(i) h_j^{(K)} \end{aligned}$$

En particular, si la profundidad es $K = 0$, $h_j^{(K)} = x_j$.

Ahora supongamos que el algoritmo obtiene la derivada para la capa $k + 1$; es decir, que tenemos que:

$$\frac{\partial R(\theta)}{\partial W_{i,j}^{(k+1)}} = d_{k+1}(i)h_j^{(k)}$$

Demostremos que también obtienen la derivada cuando la red aumenta en 1 la profundidad, es decir, para la capa k . En este caso, tenemos que:

$$\frac{\partial R(\theta)}{\partial W_{i,j}^{(k)}} = \sum_q \frac{\partial R(\theta)}{\partial h_q^{(k+1)}} \frac{\partial h_q^{(k+1)}}{\partial a_q^{(k+1)}} \frac{\partial a_q^{(k+1)}}{\partial h_i^{(k)}} \frac{\partial h_i^{(k)}}{\partial W_{i,j}^{(k)}}$$

La suma se realiza por cada una de las neuronas en la capa $k + 1$, como puede observarse, pues estamos derivando sobre la neurona i -ésima de la capa k , que está presente en todas las neuronas de la capa superior y a su vez cada neurona de la capa siguiente $k + 2$ suma sobre cada neurona. Tenemos entonces que para toda pre-activación en la neurona q de la capa $k + 2$ se da:

$$\begin{aligned} a_r^{(k+2)} &= W_{r,\cdot}^{(k+2)} h^{(k+1)} + b_r^{(k+2)} \\ &= \sum_q W_{r,q}^{(k+2)} h_q^{(k+1)} + b_r^{(k+2)} \end{aligned}$$

Por tanto, al derivar debemos considerar esta suma. En cada capa, hay una suma similar sobre las neuronas de la capa que sigue a esa capa; cada una de estas sumas está considerada en las derivadas de las capas superiores.

La hipótesis de inducción nos dice que:

$$\begin{aligned} d_{k+1}(q)h_j &= \frac{\partial R(\theta)}{\partial W_{q,j}^{(k+1)}} \\ &= \frac{\partial R(\theta)}{\partial h_q^{(k+1)}} \frac{\partial h_q^{(k+1)}}{\partial a_q^{(k+1)}} h_j^{(k+1)} \end{aligned}$$

De donde fácilmente se obtiene que:

$$d_{k+1}(q) = \frac{\partial R(\theta)}{\partial h_q^{(k+1)}} \frac{\partial h_q^{(k+1)}}{\partial a_q^{(k+1)}}$$

Finalmente, sustituyendo en la ecuación anterior obtenemos que:

$$\begin{aligned}
\frac{\partial R(\theta)}{\partial W_{i,j}^{(k)}} &= \sum_q \frac{\partial R(\theta)}{\partial h_q^{(k+1)}} \frac{\partial h_q^{(k+1)}}{\partial a_q^{(k+1)}} \frac{\partial a_q^{(k+1)}}{\partial h_i^{(k)}} \frac{\partial h_i^{(k)}}{\partial W_{i,j}^{(k)}} \\
&= \sum_q d_{k+1}(q) \frac{\partial a_q^{(k+1)}}{\partial h_i^{(k)}} \frac{\partial h_i^{(k)}}{\partial W_{i,j}^{(k)}} \\
&= \sum_q d_{k+1}(q) W_{q,i}^{(k+1)} \frac{\partial h_i^{(k)}}{\partial W_{i,j}^{(k)}} \\
&= \sum_q d_{k+1}(q) W_{q,i}^{(k+1)} \frac{\partial h_i^{(k)}}{\partial a_i^{(k)}} \frac{\partial a_i^{(k)}}{\partial W_{i,j}^{(k)}} \\
&= \left(\frac{\partial h_i^{(k)}}{\partial a_i^{(k)}} \sum_q d_{k+1}(q) W_{q,i}^{(k+1)} \right) \frac{\partial a_i^{(k)}}{\partial W_{i,j}^{(k)}} \\
&= d_k(i) h_j^{(k-1)}
\end{aligned}$$

Por tanto, el algoritmo de retro-propagación está dando los valores de las derivadas para cada una de las capas en una red neuronal. $\square$

Recalcamos que el algoritmo de retro-propagación es un algoritmo de diferenciación automática que nos permite derivar la función objetivo sobre la estructura de la red neuronal. La optimización la realiza el optimizador con base en las derivadas que se obtienen por este método.

Ejemplo 15. Supongamos que tenemos una red neuronal de tipo feedforward con dos capas ocultas, cuya salida es de la forma:

$$f(x) = \text{Softmax}(W^{(3)}h^{(2)} + b^{(3)})$$

Con 2 neuronas de salida y donde la última capa oculta está dada de la forma:

$$h^{(2)} = \sigma(W^{(2)}h^{(1)})$$

Con 3 unidades ocultas en esta capa. La primera capa es de la forma:

$$h^{(1)} = \tanh(W^{(1)}x)$$

Que cuenta con dos unidades. Finalmente, considérese que la función objetivo es la función de entropía cruzada; esto es, $L(y, f_y(x)) = \delta_{y,x} \ln f_y(x)$. Aplicando el algoritmo de retro-propagación obtenemos las derivadas. Para la capa de salida, calculamos la

variable:

$$d_3(i) = \frac{\partial L}{\partial f_y} \frac{\partial f_y}{\partial a_i^{(2)}} \quad (4.13)$$

$$= \frac{\partial -\ln f_y}{\partial f_y} \frac{\partial f_i}{\partial a_i^{(2)}} \quad (4.14)$$

$$= -\frac{1}{f_y(x)} f_y(x) (\delta_{y,i} - f_i(x)) \quad (4.15)$$

$$= f_i(x) - \delta_{y,i} \quad (4.16)$$

Hemos usado la derivada de la función Softmax, y la del logaritmo. El resultado es que la derivada es $f_i(x) - 1$ si la neurona de salida es la que corresponde a la clase de x , es decir, si $i = y$, mientras es $f_i(x)$ en otro caso. Como tenemos 2 neuronas de salida tenemos que:

$$\mathbf{d}_3 = \begin{pmatrix} f_1(x) - \delta_{y,1} \\ f_2(x) - \delta_{y,2} \end{pmatrix}$$

Por lo que el gradiente, considerando que la capa anterior tiene 3 neuronas, será de la forma:

$$\nabla_3 R(\theta) = \mathbf{d}_3 \otimes h^{(2)} = \begin{pmatrix} (f_1(x) - \delta_{y,1})h_1^{(2)} & (f_1(x) - \delta_{y,1})h_2^{(2)} & (f_1(x) - \delta_{y,1})h_3^{(2)} \\ (f_2(x) - \delta_{y,2})h_1^{(2)} & (f_2(x) - \delta_{y,2})h_2^{(2)} & (f_2(x) - \delta_{y,2})h_3^{(2)} \end{pmatrix}$$

En la segunda capa oculta, entonces, ocuparemos las variables ya obtenidas para calcular nuestra nueva variable:

$$\begin{aligned} d_2(i) &= \frac{\partial h_i^{(2)}}{\partial a_i^{(2)}} \sum_q d_3(q) W_{q,i}^{(3)} \\ &= \frac{\partial \sigma(a_i^{(2)})}{\partial a_i^{(2)}} \sum_q d_3(q) W_{q,i}^{(3)} \\ &= \sigma(a_i^{(2)}) (1 - \sigma(a_i^{(2)})) \sum_q d_3(q) W_{q,i}^{(3)} \\ &= \sigma(a_i^{(2)}) (1 - \sigma(a_i^{(2)})) \sum_q (f_q(x) - \delta_{y,q}) W_{q,i}^{(3)} \end{aligned}$$

Para obtener la derivada parcial, basta con multiplicar este término por $h_j^{(1)}$. Se puede deducir fácilmente el gradiente de esta capa. Finalmente, para la primera capa oculta obtendremos las variables:

$$\begin{aligned}
d_1(i) &= \frac{\partial h_i^{(1)}}{\partial a_i^{(1)}} \sum_q d_2(q) W_{q,i}^{(2)} \\
&= \frac{\partial \tanh(a_i^{(1)})}{\partial a_i^{(1)}} \sum_q d_2(q) W_{q,i}^{(2)} \\
&= (1 - \tanh^2(a_i^{(1)})) \sum_q d_2(q) W_{q,i}^{(2)} \\
&= (1 - \tanh^2(a_i^{(1)})) \sum_q \left(\sigma(a_q^{(2)}) (1 - \sigma(a_q^{(2)})) \sum_r (f_r(x) - \delta_{y,r}) W_{r,q}^{(3)} \right) W_{q,i}^{(2)}
\end{aligned}$$

De nuevo, al multiplicar por las neuronas de la capa anterior, esto es x_j , obtenemos las derivadas para los pesos en esa capa. También podemos obtener su gradiente y con base en esto actualizar los pesos.

En la implementación computacional del algoritmo de retro-propagación, las variables $d_k(i)$ guardarán valores numéricos que corresponden a la evaluación de las expresiones en los valores específicos que tomen las entradas x y los pesos de la red. De tal forma que lo que se retro-propaga son estos valores numéricos. A continuación detallamos algunos detalles de la implementación de una red feedforward y su entrenamiento.

4.4. Implementación del aprendizaje

Los algoritmos de retro-propagación y de optimización por descenso del gradiente son los métodos esenciales para el aprendizaje sobre redes neuronales. La implementación de las redes neuronales y de su entrenamiento se basa en estos dos métodos. Su implementación puede llevarse a cabo de forma directa, programando los métodos. El algoritmo de retro-propagación se describe a continuación:

```

1: procedure BACKPROP( $R(\theta) = \sum_x \sum_y L(y, f_y(x; \theta))$ )
2:    $f$  es una red con  $K$  capas ocultas  $h^{(k)}$ 
3:    $R$  toma como argumentos los pesos  $\theta = \{W^{(k)}, b^{(k)} : k = 1, 2, \dots, K+1\}$ 
4:    $d_{K+1}(i) \leftarrow \frac{\partial L}{\partial f_i} \frac{\partial f_i}{\partial a_i^{(K+1)}}$ ,  $i \in \{1, 2, \dots, o\}$  Inicialización
5:   for  $k$  from  $K$  to 1 do Inducción
6:      $d_k(i) \leftarrow \frac{\partial h_i^{(k)}}{\partial a_i^{(k)}} \sum_q d_{k+1}(q) W_{q,i}^{(k+1)}$ 
7:   end for
8:   returns  $\{d_k(i) h_j^{(k-1)} : k = 1, \dots, K+1\}$  Terminación
9: end procedure

```

El algoritmo de retro-propagación toma como entrada la función objetivo que se basa en la red neuronal feedforward f , y regresa las derivadas de esta función sobre los pesos de la red. La complejidad del algoritmo depende del número de capas de la red.

Para implementar el algoritmo de aprendizaje sobre una red neuronal específica y estimar los pesos adecuados para un problema dado, requerimos de un procedimiento que tome como entrada, además de la arquitectura de la red y la función objetivo, los datos de entrenamiento X e Y , los ejemplos y las clases. Este método para aprendizaje supervisado también tomará los hiperparámetros del tamaño de los mini lotes, la tasa de aprendizaje y el número de épocas.

```

1: procedure FIT-FEEDFORWARD( $X, Y, f, R, size, \eta, T$ )
2:    $f$  es una red con  $K$  capas ocultas  $h^{(k)}$ 
3:    $R$  toma como argumentos los pesos  $\theta = \{W^{(k)}, b^{(k)} : k = 1, 2, \dots, K+1\}$ 
4:    $\eta$  es la tasa de aprendizaje, y  $T$  el número de épocas
5:    $\theta \leftarrow \text{INITIALIZE}(\Theta)$  Inicialización
6:   for  $t$  from 1 to  $T$  do
7:     for  $x, y \in \text{BATCH}(X, Y; \text{size})$  do
8:        $f(x) = \phi(W^{(K+1)}h^{(K)} + b^{(K+1)})$  Forward
9:        $\frac{\partial R(\theta)}{\partial W_{i,j}^{(k)}} \leftarrow \text{BACKPROP}(R(\theta)), \quad k = K+1, \dots, 1$  Backward
10:       $W_{i,j}^{(k)} \leftarrow \text{OPTIMIZER}\left(\frac{\partial R(\theta)}{\partial W_{i,j}^{(k)}}, \eta\right)$ 
11:    end for
12:  end for
13:  returns  $f(\cdot; \hat{\theta})$  Terminación
14: end procedure

```

El algoritmo se divide en varios pasos y utiliza subrutinas, las cuales ya hemos discutido en este capítulo. En primer lugar, se realiza la inicialización que determina los valores iniciales de los parámetros. La función INITIALIZE hace referencia a la forma en que estos pesos θ de la red neuronal se obtienen. La inicialización puede tener un impacto en la convergencia del algoritmo de aprendizaje. Ante esto, se han propuesto diferentes estrategias para inicializar estos pesos. Algunos métodos más comunes para inicializar los pesos de forma aleatoria son:

Inicialización Xavier: Muestrea los pesos iniciales en base a una distribución uniforme de la forma:

$$\theta \sim \text{Unif}\left(-\frac{1}{\sqrt{m}}, \frac{1}{\sqrt{m}}\right)$$

Donde m son el número de unidades en la capa oculta de donde generamos los pesos.

Inicialización He: Muestrea los pesos en base a una distribución normal con media 0 y media basada en el inverso de la raíz del número de unidades de la capa:

$$\theta \sim \mathcal{N}\left(0, \frac{2}{\sqrt{m}}\right)$$

Donde m son el número de unidades en la capa oculta de donde generamos los pesos.

Una vez inicializados los pesos de la red se realizan actualizaciones por cada época. Las actualizaciones de los pesos se basan en mini lotes, donde la función BATCH genera estos mini lotes con base al hiperparámetro *size* y el conjunto de datos de entrenamiento. El manejo de mini lotes lo hemos revisado en la Sección 4.1.1. Dependerá de un cargador de datos y según el tamaño de los mini lotes podremos tener un mejor o peor desempeño. Podemos considerar tres casos particulares:

1. Si *size* es igual al número de datos, obtenemos (*Batch*) *Gradient Descent*.
2. Si *size* = 1 obtendremos el método *Stochastic Gradient Descent*.
3. Si *size* no es ninguno de los anteriores, tendremos *Mini-Batch Gradient Descent*.

El aprendizaje se basa en dos fases esenciales: el avance (*forward*) y el retroceso (*backward*). Durante el avance, la red neuronal procesa los datos de entrada para obtener los datos de salida con respecto a los pesos en la época actual. Este paso obtendrá los valores en cada capa, incluyendo la capa de salida, que posteriormente se utilizarán en el paso de retroceso para computar los gradientes.

El paso de retroceso computa los gradientes en los pesos de cada capa, usando la función BACKPROP que implementa la retro-propagación. Aquí se pueden aprovechar valores computados en el avance. Por ejemplo, la salida de la red en el avance puede usarse para calcular el gradiente en la salida. Como muchas de las derivadas de las redes neuronales se describen en términos de las funciones que utilizan, el cálculo numérico de estas funciones servirá para calcular los gradientes.

Finalmente, durante el mismo paso de retroceso se hará el paso de actualización, el cual aplica la función OPTIMIZER que implementa alguno de los optimizadores basados en descenso por gradiente vistos en la Sección 4.2. Como hemos visto, necesitamos especificar la tasa de aprendizaje en la mayoría de los optimizadores, por lo que consideramos la tasa de aprendizaje como un argumento de la subrutina. Este paso modifica los pesos de la red neuronal, pero se debe notar que las derivadas con las que estos pesos se actualizan dependen de la evaluación en esa época. Es hasta la siguiente época en la que, en el paso de avance, se utilizarán los nuevos pesos para evaluar si el aprendizaje ha mejorado. Cabe señalar la distinción entre el algoritmo de retro-propagación y el optimizador, pues se trata de dos procesos separados que muchas veces se confunden. La retro-propagación, como ya lo hemos señalado, se encarga de obtener los gradientes de la función, mientras que el optimizador actualiza los pesos de la red con base en estos gradientes.

El algoritmo terminará al repasar las T épocas especificadas. Podríamos indicar una condición que detenga el algoritmo cuando se alcance una pérdida igual a 0, pero en la práctica esto no es común, pues se aplica a problemas complejos en donde difícilmente se puede evaluar un error nulo. Por tanto, el número de épocas tiene una gran influencia en la convergencia a un mínimo (cuando se aseguran las condiciones necesarias para ésta). Tener un número de épocas suficientemente grande puede garantizar una mejor convergencia, pero esto también hará que el aprendizaje sea más tardado y puede pasar que en las últimas épocas la mejora en la función objetivo no sea realmente significativa. Por tanto, debe determinarse un número de épocas adecuado para que el error de generalización sea lo suficientemente pequeño.

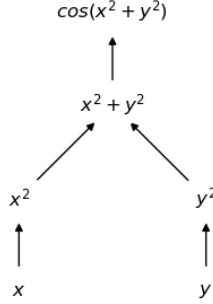


Figura 4.5: Gráfica computacional para la función $\cos(x^2 + y^2)$

4.4.1. Gráficas computacionales

Librerías especializadas en redes neuronales como Pytorch³ o Tensorflow⁴ utilizan gráficas computacionales para la implementación de arquitecturas específicas de redes profundas. Las gráficas computacionales permiten obtener los gradientes de manera eficiente por medio de métodos de diferenciación automática.

Definición 79 (Gráfica computacional). *Una gráfica computacional es una gráfica dirigida $G = (V, E)$ que estructura el orden en que se realizan un conjunto de procesos $f_1, \dots, f_K$. Cada proceso f_v , $v \in \{1, \dots, K\}$, está asociado a un nodo, y las arista $e_{u,v} \in E$ determina el orden de los procesos.*

Las gráficas computacionales no suelen tener ciclos. Un proceso f_v se ejecuta posteriormente de un proceso f_u si hay una arista que $e_{u,v} \in E$ que va de f_u a f_v . Generalmente, la salida de f_u será la entrada del nodo f_v . Las gráficas computacionales se utilizan para computar funciones compuestas, siendo que cada nodo representa una función. Además, para permitir la diferenciación de la función en la gráfica computacional, cada nodo guarda información de los gradientes.

En la Figura 4.5 se muestra una gráfica computacional que calcula la función $\cos(x^2 + y^2)$, cada nodo en la gráfica computa un valor particular. Los nodos de entrada son x e y que toman los valores específicos de estas variables. Estos nodos alimentan a los correspondientes valores de los nodos subsecuentes x^2 y y^2 , respectivamente. Posteriormente, estos dos últimos nodos se utilizan para calcular $x^2 + y^2$ y finalmente este para obtener la función $\cos(x^2 + y^2)$. La gráfica nos da el flujo de la información; algunos nodos, como x^2 y y^2 no son secuenciales por lo que pueden computarse paralelamente. Pero otros nodos requieren de la información de nodos previos para poder obtenerse.

La importancia de las gráficas computacionales en la diferenciación automática es que permiten el flujo de los gradientes al recorrer la gráfica en sentido inverso. Cada nodo debe contar con diferentes atributos que serán necesarios para poder computar la

³www.pytorch.org

⁴www.tensorflow.org

función y obtener los gradientes. Algunos de estos atributos y métodos de los nodos son:

1. **Entradas:** Los nodos que alimentan al nodo actual; es decir, aquellos nodos previos que le dan al nodo actual los valores para computar la función. Si el nodo es una variable no contará con nodo previos como entradas, sino que se les asignará valores específicos.
2. **Consumidores:** Aquellos nodos que se alimentan del nodo actual; es decir, la lista de nodos que toman como entrada al nodo actual.
3. **Avance:** Define la función que ejecuta el nodo y la computa. Se suele guardar el valor de la función.
4. **Retroceso:** El paso también llamado *backward* donde se calculan los valores de los gradientes de cada función del nodo actual con base en los gradientes de los nodos consumidores.

Para entrenar una red neuronal con base en una gráfica computacional se sigue el mismo procedimiento del algoritmo visto en la sección anterior. Se define un número de épocas y por cada mini lote se recorre la gráfica en dos sentidos:

1. **Avance (forward):** Computa la gráfica hacia adelante, recorriendo ésta para conseguir el valor de la red y de la función objetivo en esa época.
2. **Retroceso (backward):** Realiza propiamente la retro-propagación: calcula las derivadas del último nodo (el de la función objetivo) y envía este gradiente hacia atrás, al nodo de entrada, para que con base en éste se compute el gradiente del nodo multiplicando por el o los gradientes de los nodos consumidores.
3. **Actualización:** Cuando se llega a un nodo que corresponde a una variable de peso de red, se actualiza este peso con el gradiente obtenido y con el optimizador elegido.

Ejemplo 16. En la Figura 4.6 se puede observar la gráfica computacional para un procedimiento de entrenamiento en un modelo de regresión logística. Se cuenta con un conjunto de nodos de variables de entrada que corresponden a los parámetros w y b . Posteriormente se tiene el nodo de pre-activación $wx + b$ cuyas entradas son estos dos nodos previos y el nodo de datos x . A este nodo lo consume el nodo de la función sigmoide σ que a su vez es consumido por el nodo de la función objetivo $R(w, b)$.

Durante el paso forward se computarían las funciones de pre-activación $wx + b$, activación $\sigma(wx + b)$ que toma como entrada el valor del nodo previo, y finalmente la función objetivo, que tomaría como entrada la activación de la red. En el paso backward se computa primero el gradiente de la función de riesgo con respecto al valor del nodo de entrada, en este caso σ : $\frac{\partial R(w, b)}{\partial \sigma}$.

Con este valor se computa el gradiente o derivada de la activación sobre la pre-activación y además se utiliza el gradiente del consumidor para obtener la derivada total en este nodo al multiplicarlo:

$$\frac{\partial R(w, b)}{\partial \sigma} \frac{\partial \sigma}{\partial a} = \frac{\partial R(w, b)}{\partial a}$$

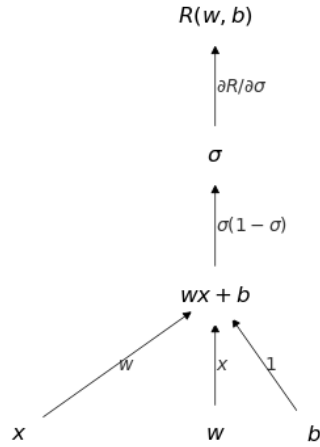


Figura 4.6: Gráfica computacional de la función de pérdida R sobre un modelo de regresión logística

Donde $a = wx + b$. Finalmente, se utiliza este gradiente para retro-propagar sobre los nodos de entrada; en particular, se propaga sobre w y b , pues x se considera como una constante y no nos interesa. De igual forma, el gradiente del consumidor se multiplica por el gradiente actual multiplicándose:

$$\frac{\partial R(w, b)}{\partial a} \frac{\partial a}{\partial w} = \frac{\partial R(w, b)}{\partial w}$$

En la gráfica de la Figura 4.6 los gradientes de cada función se han colocado en las aristas. Estos gradientes se acumulan en cada nodo para poder seguir retro-propagando y obtener los gradientes requeridos.

Las gráficas computacionales permiten la diferenciación automática por diferentes estrategias, además de la retro-propagación, que pertenece a los métodos de diferenciación llamados *reverse*, existen métodos *forward-forward*; estos pueden revisarse en el trabajo de Baydin et al. (2018). Las gráficas computacionales, por tanto, permiten la implementación de la retro-propagación de forma sencilla y dinámica, puesto que a partir de diferentes nodos previamente definidos, se puedan construir nuevas arquitecturas sin tener que definir de manera analítica el algoritmo. Como lo mencionamos, las bibliotecas especializadas incorporan las gráficas computacionales para la construcción de redes neuronales profundas.

En la Figura 4.7 se puede observar el resultado de entrenar una red neuronal feed-forward en un problema no linealmente separable. Esta fue una red con una sola capa

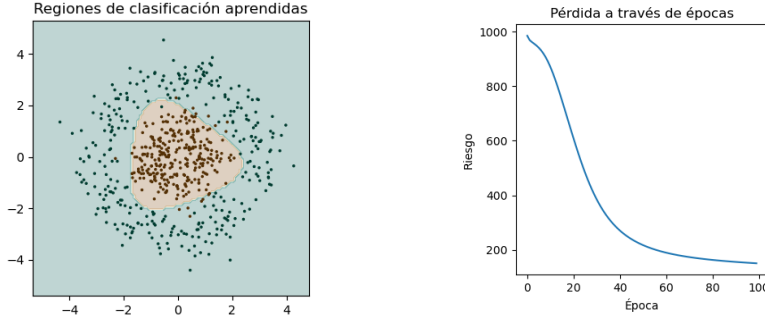


Figura 4.7: Resultado de la implementación de una red feedforward

oculta con 10 unidades y activación tangente hiperbólica. Para su entrenamiento se utilizaron 100 épocas y el optimizador de descenso por gradiente con una tasa de aprendizaje de 0.001. En el lado izquierdo se puede observar las regiones de clasificación aprendidas, las cuales son adecuadas para el problema, pues la exactitud fue de 0.97, al igual que la medida F_1 . Del lado derecho se observa el descenso de los valores de la función objetivo a través de las épocas.

Con lo visto en este capítulo, se tienen las nociones suficientes para implementar una red neuronal feedforward. Sin embargo, este tipo de redes, debido a su capacidad, son propensas al sobreajuste. En el siguiente capítulo revisamos las estrategias de regularización que ayudan a que las redes neuronales tengan mejores capacidades de generalización aunque no se cuente con gran cantidad de datos.

Ejercicios

1. Obtén las derivadas parciales sobre cada peso θ_i de la función objetivo de divergencia KL:

$$\frac{\partial}{\partial \theta_i} \sum_x q(x) \ln \frac{q(x)}{f(x; \theta)}$$

Asume que las derivadas de f sobre θ_i son conocidas.

2. Demuestra que, en una función convexa diferenciable, un mínimo local es mínimo global.
3. Demuestra que si $R : \Theta \rightarrow \mathbb{R}$ es convexa y diferenciable, entonces para todo $\theta, \theta' \in \Theta$ se cumple la desigualdad:

$$R(\theta) - R(\theta') \geq \nabla R(\theta')^T (\theta - \theta')$$

4. Utiliza el descenso por gradiente para obtener los pesos de un modelo de regresión logística para el problema AND, inicializa los valores como $w^T = (1 \ 1)$ y $b = 1$, utiliza una tasa de aprendizaje $\eta = \frac{1}{2}$ y un número máximo de 5 épocas.

5. Realiza la actualización de pesos del problema anterior pero utilizando el método de Adagrad (considera $\varepsilon = 0$).
6. Dada una función objetivo $R : \Theta \rightarrow \mathbb{R}$ con gradientes $\nabla R(\theta_t)$ en cada época, demuestra que el factor RMS en la época t está determinado como:

$$RMS(\nabla R(\theta))_t = \sqrt{\sum_{i=0}^{t-1} (\gamma^i - \gamma^{i+1}) \nabla R(\theta_{t-i})^2 + \varepsilon}$$

7. Utilizar el algoritmo de backpropagation para obtener las derivadas de la siguiente red neuronal:
 - Capa de entrada: $x \in \mathbb{R}^3$, $h^{(1)} = \tanh(W^{(1)}x + b^{(1)})$ con 10 unidades ocultas.
 - Segunda capa: $h^{(2)} = \text{ReLU}(W^{(2)}h^{(1)} + b^{(2)})$ con 20 unidades.
 - Tercera capa: $h^{(3)} = \sigma(W^{(3)}h^{(2)} + b^{(3)})$ con 10 unidades.
 - Capa de salida: $f(x) = \text{Softmax}(W^{(4)}h^{(3)} + b^{(4)})$ con 2 unidades.
 - Función de riesgo: $R(\theta) = \sum_x \sum_y \delta_{y,x} \ln f(x)$
8. Usar el método de backpropagation para obtener las derivadas de la siguiente red de regresión que utiliza regularización L_2 (revisar la Definición 81):
 - Capa de entrada: $x \in \mathbb{R}^2$, $h^{(1)} = \text{ReLU}(W^{(1)}x + b^{(1)})$ con 5 unidades ocultas.
 - Segunda capa: $h^{(2)} = \text{Softplus}(W^{(2)}h^{(1)} + b^{(2)})$ con 10 unidades.
 - Capa de salida: $f(x) = W^{(3)}h^{(2)} + b^{(3)}$ con 2 unidades.
 - Función de riesgo: $R(\theta) = \sum_x \sum_y \frac{1}{2} (y - f(x))^2$
9. Implementa cada uno de los optimizadores discutidos en la Sección 4.2.
10. Implementa los nodos de una gráfica computacional para construir una red feed-forward. Considera al menos las siguientes funciones:
 - Lineal: Un nodo que incorpore la función $wx + b$, inicializa los pesos con alguno de los métodos de inicialización.
 - ReLU: Función de activación en las capas ocultas, debe contener su derivada.
 - Softmax: La función de activación en la salida para un problema de clasificación.
 - Entropía cruzada: La función objetivo que debe optimizarse.

Toma un problema no linealmente separable (por ejemplo de la biblioteca `sklearn`) y encuentra una arquitectura que lo resuelva.

Capítulo 5

Regularización

Las redes neuronales son capaces de adaptarse a los datos de manera muy precisa debido a su gran capacidad, a la propiedad de ser aproximadores universales. Sin embargo, esto puede ser un problema cuando los datos con los que contamos no son lo suficientemente grandes. Si la red neuronal se ajusta demasiado a los datos de entrenamiento, puede pasar que su desempeño sobre nuevos datos sea malo. Es decir, podemos pensar que más que aprender, la red neuronal está memorizando. Ante esto, la red neuronal, a pesar de minimizar considerablemente el riesgo de entrenamiento, no será capaz de resolver el problema de forma general.

Para lidiar con estos problemas, se proponen los llamados métodos de regularización. Los métodos de regularización buscan reducir el sobre-ajuste durante el aprendizaje en redes neuronales. Goodfellow et al. (2016) entienden a los métodos de regularización como una modificación del algoritmo de aprendizaje en redes profundas que reduce el riesgo de generalización, sin afectar (o afectando lo menos posible) al error de entrenamiento.

Existen diversas técnicas de regularización, algunas de las más conocidas son la penalización L_2 , L_1 o el dropout. Aquí revisamos estas técnicas y algunos otros métodos de regularización. En la práctica será común usar modelos profundos de grandes capacidades incorporando métodos adecuados de regularización.

5.1. Regularización de Tychonoff

Uno de los tipos más comunes de regularización se conoce como la regularización de Tychonoff, comúnmente llamada en el área *weight decay*. La idea de este tipo de regularización es sumar a la función de riesgo original un factor que nos ayude a evitar el sobre-ajuste limitando el espacio de búsqueda a una región acotada. Abordamos la perspectiva de Vapnik (1998) y definimos, de forma general, la regularización de Tychonoff y posteriormente damos casos concretos de ésta: la regularización L_2 y L_1 .

Definición 80 (Regularización de Tychonoff). *Dado una función objetivo $R(\theta)$ para aprendizaje sobre redes neuronales, definimos la regularización de Tychonoff como la*

función $\hat{R} : \Theta \rightarrow \mathbb{R}^+$ dada por:

$$\hat{R}(\theta) = R(\theta) + \gamma\Omega(\theta) \quad (5.1)$$

Donde $\gamma \in \mathbb{R}^+$ es un hiperparámetro que controla la influencia de $\Omega(\theta)$ y $\Omega : \Theta \rightarrow \mathbb{R}$ es un funcional que cumple lo siguiente:

- $\Omega(\theta)$ es positivo para todo $\theta \in \Theta$.
- El conjunto $M = \{f(\cdot; \theta) : \Omega(\theta) \leq c\}$ para $c \geq 0$ es compacto.

Esta definición generaliza una familia de regularizadores basados en la restricción del espacio de búsqueda a una región compacta determinada por la función Ω . El factor $\gamma\Omega(\theta)$ modifica los valores de la función de riesgo; sin embargo, se puede mostrar que bajo ciertas condiciones, la convergencia a la solución adecuada se presenta al aplicar la regularización, evitando el sobre-ajuste. Esto se muestra en el siguiente teorema, cuyos detalles técnicos pueden omitirse en una lectura más técnica.

Teorema 15. Sea Θ un espacio de Hilbert y $\Omega(\theta) = \|\theta\|^2$ su norma. Supóngase que $R(\theta) \leq \delta$ y sea $\gamma(\delta)$ un hiperparámetro que depende de δ tal que $\gamma(\delta) \rightarrow_{\delta \rightarrow 0} 0$, y $\lim_{\delta \rightarrow 0} \frac{\delta^2}{\gamma(\delta)} = 0$, entonces la regularización de Tychonoff converge a la solución exacta (si es que existe) cuando $\delta \rightarrow 0$.

Demostración. En primer lugar, tómese $\gamma = \gamma(\delta)$, lo que el teorema nos dice es que $\gamma \rightarrow 0$ cuando $\delta \rightarrow 0$, es decir, cuando el error de la función de riesgo original $R(\theta)$ tiende a 0. Sea θ_δ^γ los parámetros obtenidos con la función de regularización con parámetro γ y alcanzando $R(\theta) \leq \delta$.

En primer lugar, el riesgo con regularización requiere que $\|\theta_\delta^\gamma\| \leq c$ sea compacto (esto se da por las propiedades de los espacios de Hilbert). La compacidad nos permite asegurar que la secuencia de parámetros θ_δ^γ converge a un punto dentro del subespacio compacto $M_\Theta = \{\theta : \|\theta\| \leq c\}$.

Sea θ^* los parámetros óptimos. Dado que la función de riesgo con regularización minimiza $R(\theta)$, podemos ver que en general $R(\theta^*) \leq R(\theta_\delta^\gamma)$. Geométricamente, podemos pensar que los valores θ_δ^γ se encuentran en una región de mayor radio que de los de θ^* (véase Figura 5.1). Por tanto, los valores θ_δ^γ cumplen que:

$$\|\theta^*\| \leq \liminf_{\delta \rightarrow \infty} \|\theta_\delta^\gamma\| \quad (5.2)$$

Por otra parte, tomando el cuadrado de las normas y agregando el factor $\frac{\delta^2}{\gamma}$ obtenemos que:

$$\limsup_{\delta \rightarrow 0} \|\theta_\delta^\gamma\|^2 \leq \lim_{\delta \rightarrow 0} \left(\|\theta^*\|^2 + \frac{\delta^2}{\gamma} \right)$$

Pero por la hipótesis del teorema tenemos que el $\lim_{\delta \rightarrow 0} \frac{\delta^2}{\gamma} = 0$; por lo que podemos concluir que:

$$\limsup_{\delta \rightarrow 0} \|\theta_\delta^\gamma\|^2 \leq \|\theta^*\|^2 \quad (5.3)$$

Podemos concluir de las Ecuaciones 5.2 y 5.3 la siguiente igualdad:

$$\lim_{\delta \rightarrow 0} \|\theta_\delta^\gamma\| = \|\theta^*\|$$

Más aún, dado que hemos asumido un espacio de Hilbert, tenemos que la convergencia en norma implica la convergencia fuerte, por lo que tendremos que:

$$\|\theta_\delta^\gamma - \theta^*\| \rightarrow 0$$

Lo que muestra que los parámetros de la función de riesgo con regularización convergen a la solución óptima.

□

Retomamos algunos puntos del teorema anterior; en primer lugar, el teorema nos asegura que la función de riesgo con regularización obtiene los parámetros que minimizan la función de riesgo. Por tanto, podemos encontrar la red neuronal adecuada usando $\hat{R}$ en lugar de R . Si nos fijamos en las condiciones del teorema, la condición de ser espacio de Hilbert se cumple de forma trivial, pues los pesos de una red neuronal son elementos de $\mathbb{R}^m$ (o isomorfos) que son espacios de Hilbert bajo las normas comunes.

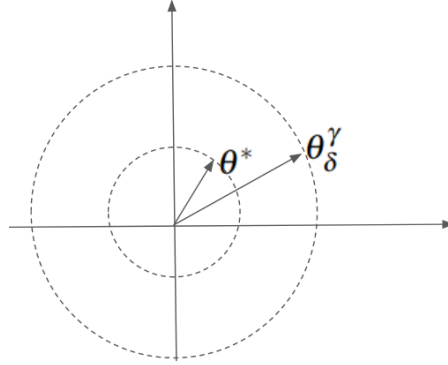


Figura 5.1: Regiones de búsqueda de la función de riesgo con regularización L_2

El hiperparámetro γ tiene algunas consideraciones en su elección; generalmente se elige un valor pequeño, menor a 1. Algunos valores comunes son 0.1, 0.01, etc. Sin embargo, un parámetro muy pequeño no será capaz de eliminar el sobre-ajuste, mientras que un valor muy grande puede llevar a sub-ajustes.

Finalmente, en el teorema hemos asumido que la función $\Omega(\theta)$ es la norma del espacio. Esta es una buena elección, pues las normas cumplen con todas las propiedades que estamos pidiendo para este regularizador. Además, al tomar las normas como las funciones de regularización, observamos que este método de regularización se basa en limitar la búsqueda a una región del espacio acotada por la norma. Podemos observar que al elegir alguna norma como regularizador, la función $\hat{R}(\theta)$ nunca será igual a 0, pues de otra forma $\Omega(\theta) = 0$, lo que implicaría que todos los pesos de la red son 0, por

lo que nuestra red será una función constante nula, y si esto pasa la función de riesgo original $R(\theta)$ no puede ser 0.

En la Figura 5.1 se puede ver de manera intuitiva cómo funciona el método de regularización. Bajo la norma usual (euclidiana) las regiones de búsqueda corresponde a discos. Por tanto, el factor de regularización buscará que la región de búsqueda sea lo más pequeña posible, pero la función de riesgo $R(\theta)$ evitará que la región sea más pequeña que la región óptima.

Esto evitará el sobre-ajuste en tanto que el factor de regularización impedirá que los parámetros se alejen demasiado, cayendo en regiones donde se consigan mínimos sobre los datos de entrenamiento. Este tipo de regularización es útil cuando tenemos pocos datos aplicando una red neuronal profunda. En la Figura 5.2 podemos ver cómo una red profunda para regresión se ajusta a los datos a partir de una muestra pequeña en el aprendizaje. Como se observa, la función sin regularización es más compleja, pues busca adaptarse mejor a los pocos datos que observa en el entrenamiento (sobre-ajuste), mientras que al introducir regularización evitamos este sobre-ajuste y la función aprendida representa mejor a los datos de generalización.

Finalmente, de manera concreta, definimos los tipos de regularizaciones específicas que caen dentro de lo que hemos llamado regularización de Tychonoff. La primera de ellas, y la más obvia, es la regularización con una norma euclidiana, llamada regularización L_2 :

Definición 81 (Regularización L_2). *La regularización L_2 es un método de regularización, cuya función de riesgo se determina como:*

$$\hat{R}(\theta) = R(\theta) + \gamma \|\theta\|_2^2$$

En este caso, cabe mencionar que la norma corresponde a la función: $\|\theta_k\|_2^2 = \sum_{i,j} (w_{i,j}^{(k)})^2$ para todas las capas ocultas $k \in \{1, \dots, K\}$. Por lo que es fácil observar que las derivadas sobre estas capas son de la forma:

$$\frac{\partial \hat{R}(\theta)}{\partial w_{i,j}^k} = \frac{\partial R(\theta)}{\partial w_{i,j}^k} + 2\gamma w_{i,j}^{(k)}$$

Por lo que la integración de estas derivadas al algoritmo de backpropagation consistirá únicamente en sumar el último factor a las derivadas que ya obtiene el algoritmo.

Además de la regularización L_2 , es común utilizar la regularización L_1 que se basa en una norma definida a partir del valor absoluto como $\|\theta_k\|_1 = \sum_{i,j} |w_{i,j}^{(k)}|$.

Definición 82 (Regularización L_1). *La regularización L_1 está determinada por la siguiente función de riesgo:*

$$\hat{R}(\theta) = R(\theta) + \gamma \|\theta\|_1$$

Dentro del área de topología, se sabe que las topologías determinadas por las normas L_1 y L_2 son equivalentes; esto nos garantiza que lo dicho sobre la norma L_2 se aplica de igual forma sobre la norma L_1 . Lo único que cambia es la forma de los dis-

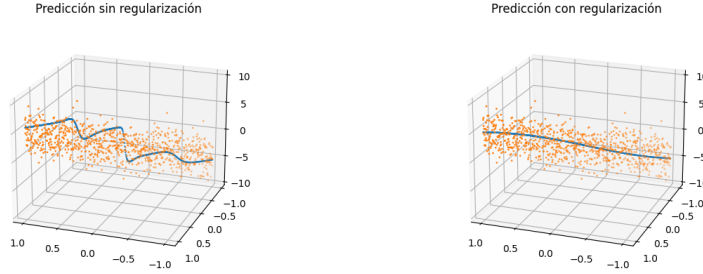


Figura 5.2: Ajuste de un método de regresión con y sin regularización

cos, pues en la norma L_1 estos se ven como rombos.¹ Más aún sabemos que toda norma de la forma $\|\theta\|_p^p = \sum_i \theta_i^p$ define una topología equivalente a la L_2 . A estas normas se les suele llamar L_p , aunque no son usualmente usadas como regularizadores.

El último método de regularización que observamos combina ambos métodos previos:

Definición 83 (Regularización Elastic Net). *El método de regularización Elastic Net define una función de riesgo dada por la ecuación:*

$$\hat{R}(\theta) = R(\theta) + \gamma_1 \|\theta\|_1 + \gamma_2 \|\theta\|_2^2$$

Las derivaciones en todos estos métodos son sencillas y permiten que el aprendizaje sobre la red neuronal se haga de manera efectiva con el método de backpropagation. Este tipo de regularizaciones son comunes en varios métodos de aprendizaje automático, y pueden aplicarse de manera sencilla a las redes neuronales. Pero no agotan todas las técnicas de regularización.

5.2. Aumento de datos

Uno de los problemas que llevan al sobre-ajuste es que no se cuenta con una cantidad suficiente de datos. Se suele sugerir que cuando un modelo presenta sobre-ajuste o se reduzca la capacidad del modelo o se aumente el número de datos. Un mayor número de datos, generalmente, implica un mejor desempeño de la red neuronal. Sin embargo, los datos son un recurso muchas veces escaso. Para solventar la ausencia de datos se propone crear nuevos datos de manera sintética, por medio de lo que se conoce como *data augmentation*.

Definición 84 (Aumento de datos). *El aumento de datos o data augmentation es el conjunto de técnicas utilizadas para obtener datos sintéticos a partir de un conjunto de datos dados.*

¹También debe notarse que la derivada en L_1 no está definido en el 0; en la práctica, suele ignorarse este caso asumiendo que los pesos nunca se anularán. Cuando no se utiliza retropropagación, se pueden implementar métodos de subgradientes como *Iterative Soft-Thresholding Algorithm* (Beck and Teboulle, 2009). Dado que las derivaciones en redes profundas suelen aumentar la complejidad, estos métodos no son comunes.

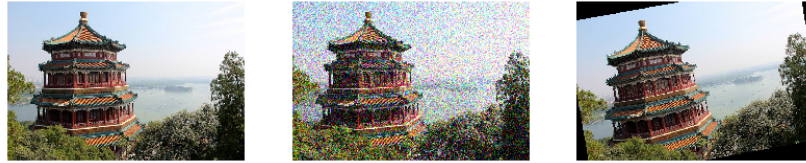


Figura 5.3: Diferentes transformaciones sobre una imagen: introducción de ruido y rotación

Las técnicas de aumento de datos dependen del tipo de datos con el que se esté trabajando y buscan generar datos artificialmente que respondan a las mismas características que los datos del problema. Una forma de generar estos datos es conocer las transformaciones que se pueden dar en los datos con el que estamos trabajando, sin que el resultado se salga del espacio de los datos originales.

Quizá el ejemplo más simple para hablar sobre el aumento de datos es el espacio de las imágenes. Sobre estas se pueden aplicar diferentes transformaciones cuyo resultado sigue siendo una imagen; algunos ejemplos son:

- Rotaciones: El rotar una imagen un número de grados da lugar a nuevas imágenes de una misma clase.
- Reflexiones: Hacer reflexiones o simetrías sobre las imágenes puede generar una nueva imagen de una misma clase, donde la imagen resultante es vista como por un espejo.
- Traslaciones: La traslación mueve la imagen trasladando sus píxeles por un valor fijo. El resultado sigue siendo una imagen de una misma clase o que puede usarse para entrenar modelos de detección de objetos.
- Ruido aditivo: Agregar ruido a una imagen puede generar una imagen del mismo tipo o clase. Aunque debe controlarse la cantidad de ruido que se puede agregar.

En la Figura 5.3 se puede observar del lado izquierdo la imagen original, en medio la imagen con ruido gaussiano y del lado derecho la imagen con una rotación. En el caso de señales de audio, por ejemplo, se puede agregar ruido para generar nuevas señales de forma sintética. Cuando hablamos de texto también existen diferentes estrategias que se pueden utilizar para generar datos sintéticos:

- Cambiar un sustantivo por otro en un texto puede dar un nuevo texto de la misma clase.
- Introducir ruido al eliminar ciertas palabras o enmascararlas.
- Utilizar métodos formales, como gramáticas libres de contexto, para generar cadenas de un lenguaje.

El generar nuevos datos sintéticos implica conocer adecuadamente los datos originales y el tipo de problema que se está trabajando. Algunos problemas no responden de

forma similar cuando se generan nuevos datos. Por ejemplo, si aplicamos una simetría a una tarea de reconocimiento óptico de caracteres podemos confundir ‘d’ con ‘b’ que pertenecen a dos clases distintas, lo que daría lugar a un mal desempeño del modelo. Por tanto, es importante que al utilizar técnicas de aumento de datos se considere el tipo de resultados que estamos obteniendo y si estos son convenientes para la tarea a resolver.

5.3. Robustecimiento por ruido

Como mencionamos en la sección anterior, el añadir ruido a los datos de entrada puede verse como una forma de aumentar los datos en el sentido de que estamos generando datos similares a los datos originales pero que muestran cierta alteración, generalmente pequeña, determinada por el tipo de ruido que introducimos. Por ejemplo, si $x \in \mathbb{R}^d$ es un dato de la clase y , podemos generar un dato sintético de esta clase como:

$$\hat{x} = x + \beta N$$

Donde $N \sim \mathcal{N}(\mu, \sigma)$ es ruido gaussiano (con media μ y varianza σ^2) y $\beta \in \mathbb{R}$ es un hiperparámetro, generalmente pequeño, que indica qué tanto ruido le agregamos al dato nuevo. En la Figura 5.4 se puede observar esta estrategia aplicada a un conjunto de datos pequeño (100 puntos) dentro de una tarea de clasificación binaria. Del lado derecho se ve la generación de diez mil datos nuevos a partir de añadir ruido gaussiano. Lo que se observa es que los datos con ruido están en las cercanías de su clase, por lo que, en este caso, al utilizar un clasificador como el perceptrón, el introducir ruido puede mejorar las fronteras de decisión.

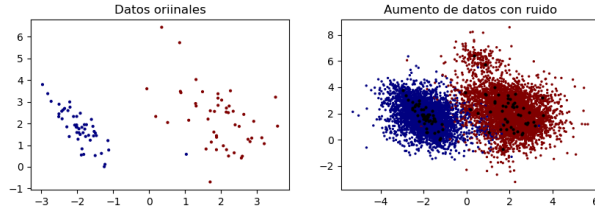


Figura 5.4: Generación de datos sintéticos a partir de ruido

El robustecimiento por ruido lleva la introducción del ruido más allá de los datos de entrada. Recordemos que en el caso de las redes neuronales profundas, lo que se aprende en las capas ocultas son representaciones de los datos. Por tanto, podemos regularizar el aprendizaje inyectando ruido a las capas ocultas y no sólo a los datos de entrada.

Definición 85 (Robustecimiento por ruido de capas ocultas). *Dada una red neuronal profunda $f : X \rightarrow Y$ con capas ocultas $h^{(k)}$, $k \in \{1, 2, \dots, K\}$, el robustecimiento por ruido consiste en añadir ruido $N \sim \mathcal{N}(0, \sigma)$ a las capas ocultas:*

$$\hat{h}^{(k)} = h^{(k)} + \beta N$$

donde $\beta \in \mathbb{R}$ es un hiperparámetro que controla la influencia del ruido.

Esta inyección de ruido en las capas ocultas puede hacerse en algunas o en todas las capas, asimismo puede realizarse de forma estocástica. De tal forma, las capas de la red serán de la forma:

$$\begin{aligned} h^{(k)} &= g(W^{(k)}\hat{h}^{(k-1)} + b^{(k)}) \\ &= g(W^{(k)}(h^{(k-1)} + \beta N) + b^{(k)}) \\ &= g(W^{(k)}h^{(k-1)} + \beta W^{(k)}N + b^{(k)}) \end{aligned}$$

Los pesos que se aprenden en la red también se ven influenciado por el ruido de las capas ocultas; esta influencia se controla por el hiperparámetro β . Esto permite que, en cada caso, la red observe una representación ‘nueva’, evitando que se presente un sobreajuste.

La regularización agregando ruido puede ir todavía más allá al incorporar no sólo el ruido a los datos y las representaciones de estos, si no también se puede añadir a los pesos de la red.

Definición 86 (Robustecimiento por ruido de los pesos). *Sea $f : X \rightarrow Y$ una red neuronal con pesos $\theta = \{W^{(k)}, b^{(k)} : k = 1, \dots, K, K+1\}$. El robustecimiento por ruido sobre los pesos es una técnica de regularización que añade ruido ruido $N \sim \mathcal{N}(0, \sigma)$ a estos:*

$$\hat{\theta} = \theta + \beta N$$

donde $\beta \in \mathbb{R}$ es un hiperparámetro que controla la influencia del ruido.

Este ruido puede añadirse a algunas de las capas o a todas. Este ruido tiene efecto en las representaciones y, por tanto, también en la forma en que se aprenden los pesos. Se puede observar que:

$$h^{(k)} = g(\hat{W}^{(k)}h^{(k-1)} + b^{(k)}) \quad (5.4)$$

$$= g((W^{(k)} + \beta N)h^{(k-1)} + b^{(k)}) \quad (5.5)$$

$$= g(W^{(k)}h^{(k-1)} + \beta Nh^{(k-1)} + b^{(k)}) \quad (5.6)$$

El ruido añade un factor $\beta Nh^{(k-1)}$ controlado por β que modifica la representación $h^{(k)}$. Este factor modifica la forma en que se aprenden los pesos

Proposición 19. *Supóngase que se tiene una red neuronal y un problema donde se ha tomado como función objetivo el error cuadrático medio. Entonces el robustecimiento por ruido cumple que:*

$$R(\hat{\theta}) \rightarrow \mathbb{E}[(f(x; \theta) - y)^2] + \beta \mathbb{E}[\nabla_{\theta}^2 R(\theta)]$$

Cuando σ , la desviación estándar del ruido, tiende a 0.

Demostración. Tomemos una red neuronal $f : X \rightarrow Y$ con pesos θ y supongamos que la función de riesgo es el error cuadrático medio:

$$R(\theta) = \mathbb{E}[(f(x; \theta) - y)^2]$$

Asúmase que el ruido que se agrega tiene una distribución normal con media 0 y desviación estándar σ , esto es $N \sim \mathcal{N}(0, \sigma)$, por lo que tenemos que:

$$\hat{\theta} = \theta + \beta N$$

Desarrollando la función objetiva del error cuadrático bajo el regimen de robustecimiento por ruido tenemos que:

$$\begin{aligned} R(\hat{\theta}) &= \mathbb{E} \left[(f(x; \hat{\theta}) - y)^2 \right] \\ &= \mathbb{E} \left[f(x; \hat{\theta})^2 - 2yf(x; \hat{\theta}) + y^2 \right] \\ &= \mathbb{E} \left[(f(x; \theta) - y)^2 \right] + \beta N \mathbb{E} \left[2(f(x; \theta) - y)x \right] \\ &= \mathbb{E} \left[(f(x; \theta) - y)^2 \right] + \beta N \mathbb{E} \left[\nabla_{\theta} R(\theta) \right] \end{aligned}$$

Esta última igualdad surge de desarrollar la ecuación con base en 5.6 y agrupar juntos los términos con y sin ruido. Entonces, haciendo tender la desviación σ a 0 obtenemos el resultado:

$$R(\hat{\theta}) \rightarrow \mathbb{E} \left[(f(x; \theta) - y)^2 \right] + \beta \mathbb{E} \left[\nabla_{\theta}^2 R(\theta) \right]$$

□

El resultado anterior nos deja ver que el robustecimiento por ruido sobre los pesos tiene un efector similar a la regularización L_2 , limitando las regiones de búsqueda sobre un espacio acotado, controlando está búsqueda por medio del hiperparámetro β . De tal forma, que introducir ruido con mayor valor de β llevará a una influencia mayor del factor del gradiente cuadrado en la función de riesgo. De forma inversa, un valor pequeño de β llevará a una influencia menor.

Hemos revisado la introducción de ruido en diferentes secciones de la red: los datos de entrada, las capas y los pesos. Otra estrategia de introducción de ruido se puede dar al agregarlo a la salida de la red; es decir, en las predicciones. A este tipo de estrategia se le conoce como suavicamiento de clases o *label smoothing*. Su efecto es que, al introducir ruido en una probabilidad de salida, los datos de entrada podrán ser asociados con una probabilidad distinta en cada época a una clase.

5.4. Aprendizaje multitarea

En el *label smoothing* se modifican los valores de salida de la red con ruido para que ésta reduzca la ‘memorización’ de las decisiones que toma con respecto a un dato de entrada. En general, una forma de reducir el sobre-ajuste es evitar que la red neuronal se especialice en una tarea específica al grado de memorizarla. Para esto, se propone que la red se enfoque en otras tareas sobre el mismo tipo de datos, pero que sean distintas.

Definición 87 (Aprendizaje multitarea). *El aprendizaje multitarea o multitask learning consiste en entrenar una representación h profunda en diferentes tareas. Para esto, se definen dos módulos de una red:*

- *Capas compartidas:* son aquellas capas profundas $h^{(k)}$ que resultan en la representación h y que se utilizan en todas las tareas. Suelen ser las capas más cercanas a la entrada y se entrenan como parte de todas las tareas.
- *Capas específicas a la tarea:* son las capas ocultas de una red que se enfocan a una tarea específica, y que son disjuntas a las capas específicas de otras tareas. El entrenamiento aquí se da sólo en una tarea específica.

Se puede decir que las capas específicas a la tarea toman como entrada la representación h de las capas compartidas. De esta forma, la representación h se pueda pensar como una representación ‘general’ que evite sobre-ajustar sobre una sola tarea, incorporando un conocimiento sobre otros aspectos de los datos.

En el aprendizaje multitarea, por tanto necesitamos que el conjunto de datos tenga varias respuestas posibles. Por ejemplo, si se cuenta con m tareas distintas, cada una debe contar con una supervisión Y_i , $i \in \{1, 2, \dots, m\}$. En este sentido, se puede pensar que se cuenta con m conjuntos de datos distintos, pero donde los ejemplos $x \in X$ se comparten. A partir de esto, se pueden definir m salidas, cada una para una tarea distinta; estas salidas pueden estar precedidas por capas ocultas especializadas. En cada una de estas tareas se debe tomar a la representación general h , la cual puede producirse también con varias capas ocultas previas. En la Figura 5.5 se muestra un esquema de una

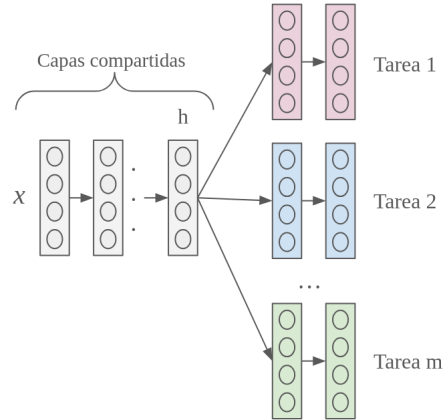


Figura 5.5: Esquema del aprendizaje multitarea

red profunda para aprendizaje multitarea. Primero los datos pasan por un conjunto de capas ocultas hasta conformar una representación general. Posteriormente, esta representación pasa a capas de tareas específicas para obtener una salida por cada problema. Esto generará m redes $f_1(x), \dots, f_m(x)$ que tienen un conjunto de capas compartidas. Entre estas tareas puede estar la tarea objetivo, de tal forma que, al generalizar el conocimiento en otras tareas, el aprendizaje en la tarea objetivo se puede ver enriquecido, evitando el sobre-ajuste.

Existen otras estrategias que caen dentro de lo que se conoce como transferencia de conocimiento o *transfer learning*. En estas estrategias se utiliza el conocimiento de tareas generales para resolver problemas específicos. Por ejemplo, si se tienen muchos

datos supervisados para una tarea general, se puede entrenar una red en esa tarea y después utilizar lo aprendido en una tarea específica. Por ejemplo, en el lenguaje natural se suele entrenar redes en tareas como predicción de la siguiente palabra (una tarea auto-supervisada, por lo que se puede aprender con texto plano). Posteriormente, lo aprendido se puede utilizar para tareas específicas en la misma lengua, como clasificación de spam, detección de sentimiento, agrupamiento de documentos, etc. En estos casos, se puede usar toda o parte de la red original. Por ejemplo, en el lenguaje natural se suelen usar representaciones de las palabras.

Asimismo, se puede entrenar una red en una tarea general y posteriormente usar ese conocimiento, los pesos aprendidos, para ajustarla a una tarea específica. En este caso, los pesos de la red original se modifican en el entrenamiento sobre el problema específico, como si los pesos se inicializaran con el conocimiento previo de la tarea general. A este tipo de procedimiento se le conoce como ajuste fino o *fine tuning*.

5.5. Early stopping

En el sobreajuste existe una tendencia en donde el error en el entrenamiento disminuye, pero el error en la evaluación aumenta. Esto no pasa durante todas las épocas, pues puede suceder que el error de evaluación y el de entrenamiento comiencen a disminuir hasta un punto en que se separan. Podemos pensar que el momento en que el error de evaluación comienza a crecer, aunque el error de entrenamiento siga disminuyendo, es un punto óptimo para detener el proceso de entrenamiento. En la Figura 5.6 se muestra gráficamente esta idea: la curva de entrenamiento continua disminuyendo, mientras el error de evaluación comienza a crecer. El valor óptimo se alcanza antes que el modelo termine de entrenar, por lo que se buscaría detener éste en ese punto.

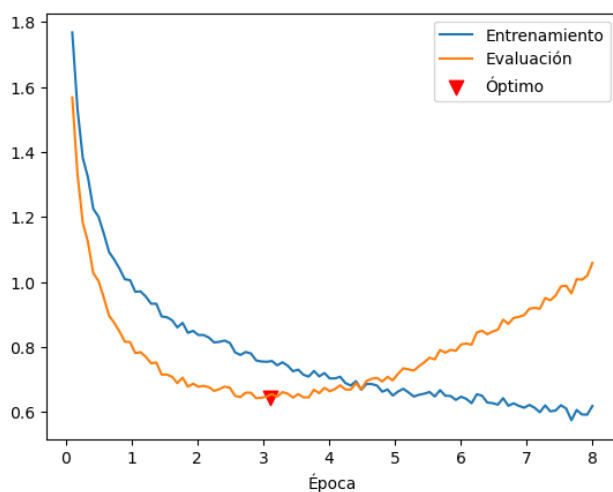


Figura 5.6: Distanciamiento de las curvas de entrenamiento y evaluación que se da en el sobreajuste

Para poder detener el algoritmo en un momento óptimo se ha propuesto el método de *early stopping* (Raskutti et al., 2014). Este método propone observar un error de generalización durante el entrenamiento. Para esto se utiliza un conjunto de datos de validación. Se trata de un subconjunto de los datos, generalmente pequeño (alrededor del 10% de los datos) el cual se integra en las épocas de entrenamiento para evaluar por cada época la capacidad de generalización del modelo y determinar que tanto el error de entrenamiento y el de evaluación se alejan. Los pesos de cada época se almacenan y se elige el conjunto de pesos que reduzcan el error de validación. El algoritmo de *early stopping* se muestra a continuación.

```

1: procedure EARLY-STOPPING( $n, p$ )
2:    $n$  número de pasos entre evaluaciones.
3:    $p$  ‘paciencia’ del modelo.
4:    $\theta \leftarrow \text{RANDOM}(\theta)$  Inicialización de pesos
5:    $i, j, err \leftarrow 0, 0, \infty$ 
6:    $\theta^* \leftarrow \theta$ 
7:    $i^* \leftarrow i$ 
8:   while  $j < p$  do
9:      $\theta \leftarrow \text{UPDATETRAIN}(n)$  Entrenar por  $n$  pasos.
10:     $i \leftarrow i + n$ 
11:     $err' \leftarrow \text{VALIDATIONRISK}(\theta)$  Error en la validación.
12:    if  $err' < err$  then
13:       $j, i^*, err \leftarrow 0, i, err'$ 
14:       $\theta^* \leftarrow \theta$  Guardar los pesos óptimos.
15:    else
16:       $j \leftarrow j + 1$ 
17:    end if
18:  end while
19: end procedure

```

El problema con el procedimiento de *early stopping* es que depende de un conjunto de datos de validación que, como en la evaluación, no es utilizado para el entrenamiento. Pero esto reduce la cantidad de datos de entrenamiento. Para solucionar esto, se integra el conjunto de datos de validación al entrenamiento. Esto se realiza siguiendo los siguientes pasos:

- La variable i^* del algoritmo guarda el número de épocas en que se ha conseguido el mejor error de validación y, por tanto, el mejor conjunto de pesos. Una solución es guardar este número i^* y entrenar el modelo, integrando el conjunto de validación, por i^* épocas, asumiendo que este es el número de épocas en que se alcanza el conjunto de pesos óptimo. Por tanto, se correría el método de *early stopping* para obtener el valor i^* y, posteriormente, se volvería a entrenar un modelo por i^* épocas pero integrando al conjunto de datos de entrenamiento el conjunto de validación.
- Otra opción es guardar el conjunto óptimo θ^* para entrenar un modelo que inte-

gre el conjunto de datos de validación iniciando con los pesos θ^* . Para esto, se puede integrar el siguiente algoritmo:

```

1: procedure INTEGRATE-VALIDATION( $\theta^*, X^{train}, Y^{train}, X^{val}, Y^{val}$ )
2:    $err \leftarrow R(\theta^*, X^{train}, Y^{train})$ 
3:   while  $R(\theta, X^{val}, Y^{val}) > err$  do
4:     TRAIN( $X^{train} \cup X^{val}, Y^{train} \cup Y^{val}$ )
5:   end while
6: end procedure

```

Por tanto, el algoritmo de *early stopping* nos permite obtener una mejor capacidad de generalización. Este método no garantiza un óptimo global, pero puede ayudar a reducir el sobre-ajuste durante el entrenamiento, teniendo mejor resultado que un entrenamiento estándar.

5.6. Métodos por conjuntos

En el aprendizaje multi-tarea se escogían diferentes tareas para que puedan ser resueltas por una sola máquina de aprendizaje; esto impide que la máquina de aprendizaje se especialice en un solo problema, evitando el sobre-ajuste. En un sentido contrario, podemos tomar una sola tarea para que sea resuelta por varias máquinas de aprendizaje, de tal forma que tengamos respuestas variadas al mismo problema buscando una mejor generalización. A este tipo de estrategias se les conoce como métodos por conjuntos (*ensemble methods*).

Definición 88 (Método por conjuntos). *Un método por conjuntos o ensambles es un método que incorpora diferentes máquinas de aprendizaje $f^{(i)}$, $i \in \{1, 2, \dots, M\}$ para resolver un problema, por ejemplo, supervisado $\mathcal{S} = \{(x, y) : x \in \mathbb{R}^d, y \in Y\}$.*

En este sentido, lo que se hace es entrenar diferentes modelos que provienen de diferentes máquinas de aprendizaje bajo el mismo conjunto de datos de entrenamiento. En particular, este tipo de métodos se puede aplicar a cualquier modelo de aprendizaje. Una revisión de este tipo de métodos se puede ver en Mienye and Sun (2022). En el caso del aprendizaje profundo, se pueden utilizar diferentes redes neuronales con diferentes capacidades para poder evitar una inferencia con sobre-ajuste; al contar con un número mayor de posibles salidas, podemos elegir la que sea mejor. En el tiempo de inferencia, entonces, se obtiene un conjunto de salidas $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_M$, donde M es el número de máquinas de aprendizaje y cada $\hat{y}_i$ es la salida correspondiente al i -ésimo modelo.

El entrenamiento de este tipo de métodos consiste en ajustar los modelos bajo el mismo conjunto de datos. La inferencia o predicción es la que presenta particularidades, pues se obtienen M salidas, de las cuales se debe tomar la más adecuada, esto es, aquella que muestre una mejor capacidad de generalización. Para esto, se proponen diferentes métodos como el promedio de modelos.

Definición 89 (Promedio de modelos). *Dado un método por conjuntos con salidas $\hat{y}_i$, $i \in \{1, 2, \dots, M\}$, el promedio de modelos (model averaging) obtiene una salida en base*

al valor medio del conjunto de salidas:

$$\hat{y} = \frac{1}{M} \sum_{i=1}^M \hat{y}_i$$

El promedio de modelos, como su nombre lo dice, busca únicamente de obtener un promedio de los resultados obtenidos entre todos los modelos; el uso de este tipo de predicción puede realizarse para métodos de regresión de manera directa, pues podemos obtener un valor real por medio del promedio de las salidas del conjunto de modelos. Por otra parte, si se desea obtener una clase necesitamos obtener un valor que se encuentre dentro de los valores permitidos en la salida, por lo que el promedio no será adecuado. En este caso, podremos obtener la máxima probabilidad.

Definición 90 (Probabilidad de clase). *Dado un método por conjuntos con salidas $\hat{y}_i$, $i \in \{1, 2, \dots, M\}$ sobre un problema de clasificación, podemos obtener una salida en base a la probabilidad de clases como:*

$$\hat{y} = \arg \max_i \{p(\hat{y}_i) : i = 1, 2, \dots, M\}$$

Donde $p(\hat{y}_i)$ es una medida de probabilidad sobre las salidas.

La probabilidad de la clase $p(\hat{y}_i)$ puede obtenerse a partir de la distribución sobre los diferentes modelos. De tal forma, si la mayoría de los modelos predijo la clase, por ejemplo, 1, entonces el modelo general habrá de predecir esta clase. Muchas veces, a los modelos que participan del método por conjunto se les conoce como jueces, pues su función es emitir un “voto” a favor de una clase; la clase final será la que cuente con un mayor número de votos.

Una estrategia que busca minimizar el error de generalización y cae dentro de los métodos por conjuntos es el llamado *Bootstrap Aggregation*, también llamados *bagging* (Breiman, 1996). Estos métodos se enfocan principalmente en problemas de clasificación, buscando minimizar el error de generalización a partir de entrenar diferentes modelos sobre muestras de un mismo conjunto de datos de entrenamiento.

Definición 91 (Bootstrap aggregation). *El método de Bootstrap aggregation o bagging es un método por conjuntos que, dado un conjunto de datos $\mathcal{S} = \{(x, y) : x \in \mathbb{R}^d, y \in Y\}$, genera M subconjuntos de datos $\mathcal{S}_1, \dots, \mathcal{S}_M$ por medio de un submuestreo uniforme y con reemplazo sobre $\mathcal{S}$.*

El *bagging*, entonces, primero realiza submuestras del conjunto original de datos que permiten que cada modelo se entrene de forma particular, buscando así una mejora en la generalización al ver diferentes configuraciones del conjunto de datos durante el entrenamiento. La idea de submuestrear los datos busca que las diferentes máquinas de aprendizaje generen modelos de aprendizaje variados. Por ejemplo, si se usan redes neuronales para una misma tarea, se puede asegurar la variación de éstas en base no sólo a cambiar el número de neuronas, sino también con respecto a la forma en que aprenden del conjunto de entrenamiento. Esta variación en los modelos utilizados por un método por conjuntos es de primordial importancia, pues asegura un menor error en la generalización.

Proposición 20. Dado un método por conjuntos con M máquinas de aprendizaje $f^{(i)}$, cada una con error ε_i , varianza $\sigma^2 = \mathbb{E}[\varepsilon_i^2]$, $i = 1, 2, \dots, M$, y que muestran covarianza $c = \mathbb{E}[\varepsilon_i \varepsilon_j]$, entonces el error esperado es:

$$\mathbb{E}\left[\left(\frac{1}{M} \sum_i \varepsilon_i\right)^2\right] = \frac{\sigma^2}{M} + \frac{M-1}{M}c$$

Demostración. Bajos los supuestos del teorema, esto es, asumiendo que cada máquina de aprendizaje tiene un error ε_i y una varianza $\sigma^2 = \mathbb{E}[\varepsilon_i^2]$, tenemos el siguiente desarrollo:

$$\begin{aligned} \mathbb{E}\left[\left(\frac{1}{M} \sum_i \varepsilon_i\right)^2\right] &= \frac{1}{M^2} \mathbb{E}\left[\sum_i (\varepsilon_i^2 + \sum_{j \neq i} \varepsilon_i \varepsilon_j)\right] \\ &= \frac{1}{M^2} \sum_i \mathbb{E}\left[\varepsilon_i^2 + \sum_{j \neq i} \varepsilon_i \varepsilon_j\right] \\ &= \frac{1}{M^2} \left(\sum_i \mathbb{E}[\varepsilon_i^2] + \sum_{j \neq i} \mathbb{E}[\varepsilon_i \varepsilon_j]\right) \\ &= \frac{1}{M^2} \left(\sum_i \sigma^2 + \sum_{j \neq i} c\right) \\ &= \frac{1}{M^2} (M\sigma^2 + M(M-1)c) \\ &= \frac{1}{M} \sigma^2 + \frac{M-1}{M}c \end{aligned}$$

□

De teorema anterior se observa que si los errores están perfectamente correlacionados, esto es $c = \sigma^2$ que implica que $\varepsilon_i = \varepsilon_j$, entonces el error esperado será simplemente σ^2 ; por lo que, en este caso, el modelo por conjuntos no tiene ningún efecto, pues todos los modelos actúan de la misma forma y la predicción final entre los modelos es la misma.

Por otro lado, si no hay correlación entre los errores de los modelos, esto es $c = 0$, el error esperado es $\frac{1}{M} \sigma^2$, lo que implica que entre más modelos se tenga, más pequeño será el error esperado. Es decir, si tenemos un método por conjuntos cuyos modelos de aprendizaje tengan una distribución completamente distinta, el error de generalización tenderá a 0.

En este sentido, para que una técnica por conjuntos surja efecto se espera que los modelos del conjunto estén lo menos correlacionados posible, cuando es así, ensamblar un mayor número de modelos implica reducir el error esperado; pero si la correlación del error entre los modelos es alta, entonces la técnica por conjunto tiene menos efecto en la regularización. Esto parece lógico, pues tenemos un conjunto de jueces con diferentes formas de juzgar, pero que asumimos juzgan siempre de la mejor manera según sus propios criterios. El problema con los métodos por conjuntos es el costo de entrenar varios modelos, principalmente cuando se trata de modelos neuronales, pues éstos son costosos de entrenar.

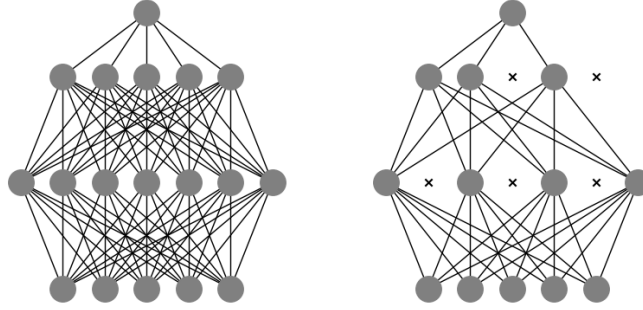


Figura 5.7: Generación de una nueva arquitectura a partir del método de dropout

5.7. Dropout

Una forma de simular los modelos por conjuntos a partir de las redes neuronales es por medio de generar diferentes arquitecturas variando el número de neuronas que cada red cuenta. En particular, se propone que las unidades ocultas en cada capa oculta varíen, siendo que el número de neuronas afecta a la capacidad de una red neuronal. El método de *dropout* (Srivastava et al., 2014) busca crear un conjunto de redes neuronales diferentes que se encarguen de procesar los datos en una sola tarea. Este método aprovecha la flexibilidad de las redes neuronales al generar redes independientes a partir de una arquitectura base por medio de reducir el número de neuronas en cada capa oculta. Las nuevas arquitecturas se determinan por medio de ignorar nodos dentro de la red base. Como se observa en la Figura 5.7, el método de dropout elimina nodos dentro de la red y, por tanto, ignora las conexiones de estos nodos. La red que resulta de aplicar el dropout puede verse como una nueva red. Repitiendo este método varias veces, eliminando de forma aleatoria un conjunto de nodos de la red, se crea un conjunto de redes que funcionan como un método por conjuntos.

Definición 92 (Dropout). Sea $f : X \rightarrow Y$ una red neuronal con arquitectura $\{K, m_i : i = 1, 2, \dots, K\}$ donde K son las capas ocultas. El modelo de regularización dropout modifica las capas ocultas como:

$$\bar{h}^{(k)} = h^{(k)} \odot \mu$$

Para cada capa oculta $k \in \{1, 2, \dots, K\}$, donde $\mu \sim \text{Ber}(p)$ es un vector binario con distribución Bernoulli con parámetro p .

Ya que μ es un vector aleatorio, cada capa oculta en el dropout tiene un número diferente de unidades ocultas cada vez que se recorre la red durante el entrenamiento. Es decir, se genera un conjunto de sub-redes con diferentes arquitecturas que dependen del número de unidades ocultas en cada capa. Es claro que en la salida no se aplica

el dropout pues, precisamente, buscamos que diferentes arquitecturas se entrenen para una misma tarea. El dropout se aplica a las capas ocultas y, algunas veces, a las neuronas de entrada (para modificar los datos como en el aumento de datos por ruido). Asimismo, este método suele combinarse con el entrenamiento por mini lotes para obtener en cada lote una arquitectura distinta sobre un subconjunto diferente de los datos de entrada.

Para aplicar el dropout, se utiliza lo que se conoce como enmascaramiento (*masking* en inglés). El enmascaramiento aplica una máscara a cada unidad oculta en una de las capas ocultas de la red base. Esta máscara genera valores aleatorios 0 ó 1 para cada unidad oculta. Si la máscara es 0, la unidad se ignora, si es 1 entonces la unidad se preserva. El enmascaramiento depende de una probabilidad p . Por ejemplo, si elegimos una probabilidad $p = \frac{1}{2}$ una neurona de la red se va a enmascarar con 0 (ignorar) la mitad de las veces, y la otra mitad será 1 (permanecer).

Dada una capa oculta $h \in \mathbb{R}^m$ con m unidades ocultas, podemos definir el vector de enmascaramiento $\mu \in \{0, 1\}^m$ que será un vector de 0s y 1s. Esto facilita la forma de enmascarar, pues el vector μ se genera en base a la probabilidad p , asignando un 0 o un 1 a cada entrada en base a la probabilidad, y para enmascarar la capa oculta h se realiza un producto punto a punto entre h y μ . Por ejemplo, pensemos que tenemos el vector $h^T = (-1 \ 0,5 \ 0,4 \ 0,3)$ en la red base, y generamos un vector de enmascaramiento en base a una probabilidad p que sea $\mu^T = (0 \ 1 \ 1 \ 0)$, entonces la capa en la sub-red será:

$$\bar{h} = \begin{pmatrix} -1 \\ 0,5 \\ 0,4 \\ 0,3 \end{pmatrix} \odot \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0,5 \\ 0,4 \\ 0 \end{pmatrix}$$

Es decir, se están enmascarando con 0 la primera y la última unidad oculta, por lo que en la sub-red estas unidades serán ignoradas, dando como resultado una sub-red con sólo dos unidades ocultas en lugar de cuatro. Estas unidades serán la segunda y tercera unidades de la red base.

Al genera un conjunto nuevo de sub-redes, es claro que durante el entrenamiento no se actualizarán aquellos pesos que provengan de unidades ignoradas o enmascaradas. Como hemos señalado, cada sub-red puede pensarse como un modelo distinto; por tanto, podemos pensar que cada sub-red depende de una función de riesgo particular. Denotemos a esta función de riesgo como $R(\theta; \mu)$ donde θ son los pesos y μ es el vector (o tensor) de enmascaramiento. Esta notación nos permite ver que la función de riesgo es similar para todas las sub-redes, pero que sólo varía en los pesos que considera. Es decir, podemos pensar que la función de riesgo depende los pesos $\theta \odot \mu$, que son los pesos que permanecen activos en esa sub-red.

De esta forma, la función de riesgo no podrá considerarse similar en todos los casos. Por tanto, lo que se sugiere es adoptar un régimen de entrenamiento en que se minimice no una sola función de riesgo, sino el valor esperado de las funciones de riesgos sobre los pesos de cada sub-red. Esto implica que nuestro problema de optimización se plantea ahora como:

$$\min \mathbb{E}_{\mu} [R(\theta; \mu)] = \min \sum_{\mu} p(\mu) R(\theta; \mu)$$

Donde $p(\mu)$ es la probabilidad de ese tensor de enmascaramiento o, lo que es lo mismo, la probabilidad de la sub-red. De igual forma que en los modelos por conjuntos, para obtener una predicción de la red, podemos usar las diferentes sub-redes y obtener una predicción $\hat{y}$ en base al valor esperado de las predicciones de cada una de las sub-redes:

$$\hat{y} = \arg \max_y \sum_{\mu} p(\mu) p(y|x, \mu)$$

Otra forma de obtener una predicción, propuesta por Warde-Farley et al. (2014), es por medio no del valor esperado, sino de la media geométrica de las probabilidades.

Definición 93 (Dropout boosting). *Se llama dropout boosting al método de entrenamiento con dropout basado en una función objetivo que toma como probabilidad de clases:*

$$p_{ens}(y|x) = \frac{\hat{p}_{ens}(y|x)}{\sum_{y'} \hat{p}_{ens}(y'|x)}$$

Donde las probabilidades $\hat{p}_{ens}(y|x)$ están dadas por la media geométrica como:

$$\hat{p}_{ens}(y|x) = \left(\prod_{\mu} p(y|x, \mu) \right)^{1/2^r}$$

Tal que r es el número de unidades que se enmascaran.

De esta forma, se puede predecir la probabilidad de clase con un modelo basado en dropout, de forma similar a los modelos por conjuntos. El *dropout boosting* puede aplicarse a cualquier método por conjuntos, pero es particularmente interesante cuando se usa con el método de dropout. Bajo el método de dropout, la probabilidad de clase de un vector de entrada puede aproximarse usando el modelo base, es decir, usando la red neuronal con todas las neuronas activas, pero con los pesos de la red multiplicados por la probabilidad de dropout. A esta forma de obtener las probabilidades de salida se le conoce como la regla de inferencia por escalamiento de pesos o *weight scaling inference rule*.

Teorema 16 (Weight scaling inference rule). *Sea $f = \text{Softmax}(W\bar{h} + b)$ una red neuronal con activación softmax en la capa de salida y en la que se ha aplicado el regularización dropout para obtener capas $\bar{h}^{(k)} = h^{(k)} \odot \mu$, con $k \in \{1, \dots, K\}$ y $\mu \sim \text{Ber}(p)$. Entonces:*

$$p_{ens}(y|h) = \text{Softmax}_y \left(\frac{1}{2^r} W h + b \right)$$

Donde r es el número de neuronas enmascaradas con μ .

Demostración. Sea $f : X \rightarrow Y$ la red neuronal cuya capa de salida define la función de probabilidad: $p(y|h) = \text{Softmax}(Wh + b)_y$. Si tomamos un vector máscara μ , entonces la probabilidad de cada sub-red estará dada como $p(y|h, \mu) = \text{Softmax}(W(h \odot \mu) + b)_y$, que al obtener la probabilidad del conjunto final resulta en:

$$p_{ens}(y|h) = \frac{\hat{p}_{ens}(y|h)}{\sum_{y'} \hat{p}_{ens}(y'|h)}$$

La cual es la probabilidad en la capa de salida. A partir de este valor podemos derivar lo siguiente, suponiendo que el número de neuronas enmascaradas es r :

$$\begin{aligned}
 \hat{p}_{ens}(y|h) &= \left(\prod_{\mu} p(y|h, \mu) \right)^{1/2^r} \\
 &= \left(\prod_{\mu} \text{Softmax}(W(h \odot \mu) + b)_y \right)^{1/2^r} \\
 &= \left(\prod_{\mu} \frac{1}{Z} \exp(W_{y,\cdot}(h \odot \mu) + b_y) \right)^{1/2^r} \\
 &\propto \left(\prod_{\mu} \exp(W_{y,\cdot}(h \odot \mu) + b_y) \right)^{1/2^r} \\
 &= \exp\left(\frac{1}{2^r} \sum_{\mu} W_{y,\cdot}(h \odot \mu) + b_y\right) \\
 &= \exp\left(\frac{1}{2^r} W_{y,\cdot} h + b_y\right)
 \end{aligned}$$

Aquí $Z = \sum_{y'} \hat{p}_{ens}(y'|h)$ representa la función de partición que es constante para cada salida de la red, por lo que podemos aplicar la proporcionalidad. De este desarrollo se puede obtener que la probabilidad con base en *dropout boosting* es:

$$\begin{aligned}
 p_{ens}(y|h) &= \frac{\hat{p}_{ens}(y|h)}{\sum_{y'} \hat{p}_{ens}(y'|h)} \\
 &= \frac{\exp\left(\frac{1}{2^r} W_{y,\cdot} h + b_y\right)}{\sum_{y'} \exp\left(\frac{1}{2^r} W_{y',\cdot} h + b_{y'}\right)} \\
 &= \text{Softmax}_y\left(\frac{1}{2^r} W h + b\right)
 \end{aligned}$$

□

En el desarrollo del problema anterior se puede asumir que $\frac{1}{2^r}$ es la probabilidad total del dropout. De esta forma, el método de dropout permite una forma eficiente de regularización, su costo durante el entrenamiento es de $O(r)$, donde r es el número de unidades que se enmascaran. El dropout puede interpretarse como un método por conjuntos donde la misma red se va adaptando a diferentes arquitecturas en diferentes épocas durante el entrenamiento. En la práctica, es común que en el tiempo de inferencia se ocupe la red completa, sin el factor $\frac{1}{2^r}$, para la predicción. De cualquier forma, el dropout evita el sobre-ajuste al entrenar sobre diferentes secciones de la red en cada época del entrenamiento. Actualmente, este es uno de los métodos de regularización más utilizados.

Ejercicios

1. Usar el método de retro-propagación para obtener las derivadas de la siguiente red de regresión que utiliza regularización L_2 :

- Capa de entrada: $x \in \mathbb{R}^2$, $h^{(1)} = \text{ReLU}(W^{(1)}x + b^{(1)})$ con 5 unidades ocultas.
 - Segunda capa: $h^{(2)} = \text{Softplus}(W^{(2)}h^{(1)} + b^{(2)})$ con 10 unidades.
 - Capa de salida: $f(x) = W^{(4)}h^{(3)} + b^{(4)}$ con 2 unidades.
 - Función objetivo: $R(\theta) = \sum_x \sum_y (y - f(x))^2 + L_2(\theta)$
2. Repite el ejercicio anterior pero utilizando la función objetivo con regularización L_1 .
 3. Repite el ejercicio utilizando la regularización de *elastic net*.
 4. Implementa una red neuronal que incorpore métodos de robustecimiento por ruido.
 5. Utiliza el conjunto de datos `linnerud` de la biblioteca `sklearn` para entrenar un modelo de aprendizaje multi-tarea sobre las 3 clases de este conjunto. Determina un número de capas compartidas. Evalúa los resultados en cada tarea.
 6. Implementa una red neuronal que entrene utilizando dropout y *early stopping*.

Capítulo 6

Redes neuronales recurrentes

Las redes feedforward procesan únicamente elementos individuales, pero no son capaces de procesar secuencias de estados dentro de un sistema. Si tenemos un proceso estocástico $X^{(1)}X^{(2)}\dots X^{(T)}$, una red feedforward tendría que procesar cada uno de las variables de estado $X^{(t)}$ de manera independiente, ignorando la dependencia con los estados anteriores. Se podría entrenar una red feedforward para cada uno de los estados $t \in \{1, 2, \dots, T\}$, sin embargo esto sería demasiado costoso.

Para trabajar con datos secuenciales se propone el uso de redes que incorporan recurrencia en sus estados. En particular estas redes comparten parámetros a través de los diferentes estados temporales de la secuencia, guardando información de los estados anteriores. La ventaja de compartir parámetros es que un mismo valor en diferentes estados va a ser tratado de forma similar, sólo considerando lo que ha pasado anteriormente. En este capítulo abordamos las redes neuronales recurrentes, los tipos de redes recurrentes que se pueden construir, así como estrategias para mejorar la transferencia de información a largo plazo.

6.1. Definición de red recurrente

Las redes neuronales recurrentes son una familia de redes neuronales enfocadas a procesar datos secuenciales. Como lo señalamos, usan un solo conjunto de parámetros a través de los diferentes estados temporales, pero incorporan capas recurrentes para transmitir la información de los estados anteriores al estado actual compartiendo los pesos en cada nuevo estado. Para entender, entonces, las redes recurrentes, debemos definir lo que entendemos por recurrencia por medio de funciones recurrentes.

Definición 94 (Función recurrente). *Una función recurrente definida sobre un estado $s^{(t)}$ que depende del tiempo t se define como:*

$$s^{(t)} = f(s^{(t-1)}; \theta)$$

Donde θ es un conjunto de parámetros de la función.

Para obtener el estado en el tiempo t se debe aplicar una función sobre el estado anterior, que para haber sido obtenida debió aplicar la función sobre un estado anterior y así consecutivamente. Todo el proceso debe iniciar en un estado inicial que denotamos como $s^{(1)}$. De esto podemos inferir que un estado en tiempo t se obtiene al aplicar repetidas veces la función de la siguiente forma:

$$s^{(t)} = f(f(f(\dots f(s^{(1)}); \theta); \theta); \theta); \theta)$$

La función f ha sido aplicada t veces, iniciando con el estado inicial $s^{(1)}$, el cual asumimos es conocido al inicio del proceso. La idea es, entonces, partiendo de un estado inicial $s^{(1)}$, poder obtener un conjunto de estados subsecuentes aplicando una recurrencia. Gráficamente, esto se puede plantear como un ciclo sobre un nodo estado, o bien como una secuencia de nodos (lo que se conoce como el despliegue de la gráfica). En la Figura 6.1 se puede ver estas dos formas de representar la recurrencia.

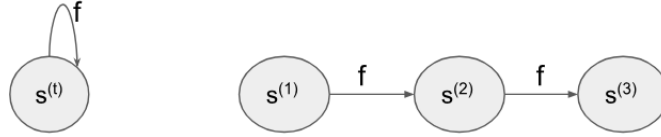


Figura 6.1: Visualización gráfica de una función de recurrencia

Un estado no sólo puede estar determinado por los estados anteriores, sino también por una entrada en cada uno de estos estados. Podemos definir una función de recurrencia que además de tomar en cuenta los estados anteriores se defina por entradas que dependen del tiempo.

Definición 95 (Función recurrente con entrada). *Dada una secuencia de vectores de entrada $x^{(1)}x^{(2)}\dots x^{(T)}$ que depende del tiempo, definimos un estado $s^{(t)}$ con base en una función recurrente como:*

$$s^{(t)} = f(s^{(t-1)}, x^{(t)}; \theta)$$

Donde θ son los parámetros de la función.

Esta función recurrente define al estado $s^{(t)}$ con base no sólo en los estados anteriores, sino también en un dato de entrada $x^{(t)}$ en el tiempo t . Más aún, queremos generar modelos de aprendizaje que además nos den una salida por cada estado del tiempo; de esta forma, la función arrojaría dos valores: la salida (la probabilidad de clase, por ejemplo) y el estado actual:

$$\begin{pmatrix} y^{(t)} \\ s^{(t)} \end{pmatrix} = f(s^{(t-1)}, x^{(t)}; \theta)$$

Esta función puede visualizarse gráficamente como se muestra en la Figura 6.2; en este caso, podemos ver que las salidas $y^{(t)}$ son las emisiones de la función, mientras que los estados $s^{(t)}$ se transmiten hacia el siguiente estado para poder así también obtener la

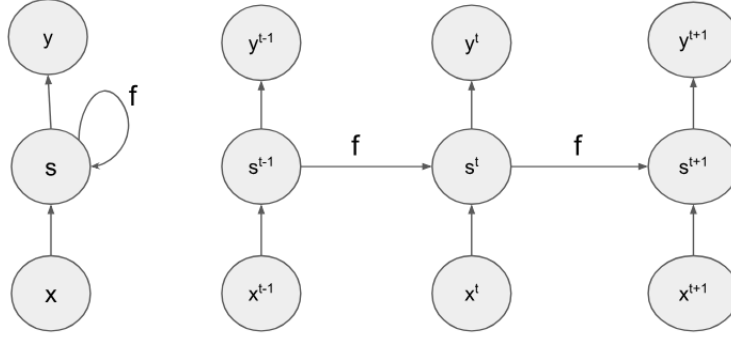


Figura 6.2: Representación gráfica de función recurrente con entradas y salidas

siguiente salida. Estos estados dependen de un estado inicial, sobre el que se aplica la función recurrente.

En este sentido, una función recurrente de este tipo es un mapeo de una secuencia de entrada a una secuencia de salida de la siguiente forma:

$$x^{(1)}x^{(2)}\dots x^{(T)} \mapsto y^{(1)}y^{(2)}\dots y^{(T)}$$

Si pensamos ahora que la función $f(\cdot; \theta)$ está definida como una red neuronal, entonces tendremos lo que se conoce como red neuronal recurrente. Observando la Figura 6.2 podemos ver que cada estado t es similar a una red feedforward con una capa oculta, por lo que podemos pensar que existe una capa de salida dada por:

$$\hat{y}^{(t)} = \phi(Vh^{(t)} + c)$$

Donde $V \in \mathbb{R}^{m \times o}$ es una matriz de pesos con o salidas, $c \in \mathbb{R}^o$ es el bias en la salida, y $\phi: \mathbb{R} \rightarrow \mathbb{R}$ es la función de activación. En general, con esta idea de función recurrente podemos definir la idea de una capa recurrente en una red neuronal.

Definición 96 (Capa recurrente). *Una capa recurrente en una red neuronal que toma como entrada una secuencia de elementos $x^{(t)}$, que dependen de $t \in \{1, 2, \dots, T\}$, es una capa que genera las representaciones $h^{(t)}$ como:*

$$h^{(t)} = f(h^{(t-1)}, x^{(t)}; \theta)$$

Tal que f es una función recurrente donde el estado inicial se define como $h^{(0)} = \bar{0}$, el vector nulo, y $\theta \in \Theta$ son los pesos de la capa.

Esta definición da una forma general de cómo concebir una capa recurrente en una red neuronal. De esta forma, podemos entender diferentes tipos de capas ocultas dentro de las redes recurrentes como serán las capas GRU (Sección 6.4.1) y las LSTM (Sección 6.4.2). Si tomamos en cuenta la forma típica de las capas en las redes neuronales, podemos generar conexiones del estado anterior $h^{(t-1)}$ hacia el estado actual $h^{(t)}$ por

medio de una matriz de pesos $W \in \mathbb{R}^{m \times m}$, como $Wh^{(t-1)}$, pero además debemos incluir las conexiones de la entrada. Por tanto, la función para la capa recurrente estará definida como:

$$h^{(t)} = g(Wh^{(t-1)} + Ux^{(t)} + b)$$

Donde $U \in \mathbb{R}^{m \times d}$ son los pesos de la entrada, $b \in \mathbb{R}^m$ es el bias, y $g : \mathbb{R} \rightarrow \mathbb{R}$ es la función de activación. Esta es la forma más simple de una red neuronal, que muchas veces se conoce como *vanilla* (Elman, 1990).

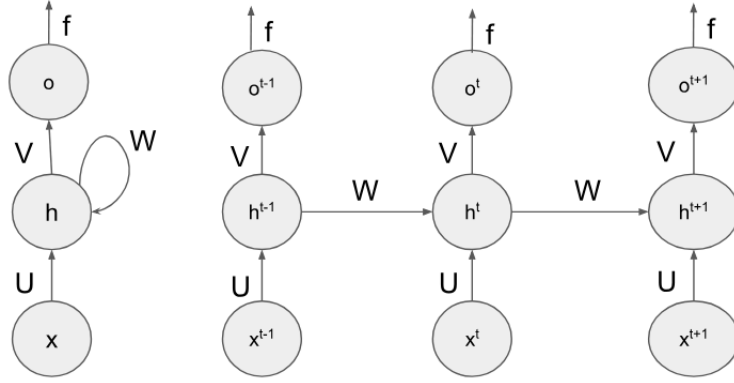


Figura 6.3: Visualización de una red recurrente

En la Figura 6.3 podemos ver una red neuronal recurrente, donde existe una capa oculta que funciona para conformar recurrencias y transmitir la información de los estados previos al estado actual. A partir de esto, podemos entender a una red recurrente como aquella que cuenta con capas recurrentes (*vanilla*).

Definición 97 (Red neuronal recurrente (*vanilla*)). *Una red neuronal recurrente (vanilla) es una red neuronal para procesar datos secuenciales $x^{(1)} \dots x^{(T)}$ que cuenta con al menos una capa recurrente:*

$$h^{(t)} = g(Wh^{(t-1)} + Ux^{(t)} + b)$$

Con $W \in \mathbb{R}^{m \times m}$, $U \in \mathbb{R}^{d \times m}$ y $b \in \mathbb{R}^m$. Y cuya capa de salida en un tiempo t está dada como:

$$\hat{y}^{(t)} = \phi(Vh^{(t)} + c)$$

Con $V \in \mathbb{R}^{m \times o}$ y $c \in \mathbb{R}^o$.

En esta definición los parámetros de la red son las matrices de pesos W, U y V , y los bias b y c . Como lo señala la definición, una red neuronal recurrente o RNN puede tener varias capas ocultas, pero al menos una de estas tiene que ser recurrente; pero también puede tener más de una capa recurrente; por ejemplo, puede trabajar con dos o más recurrencias, además de incorporar otras capas ocultas que no sean recurrentes. Podemos también tener profundidad a partir de redes recurrentes.

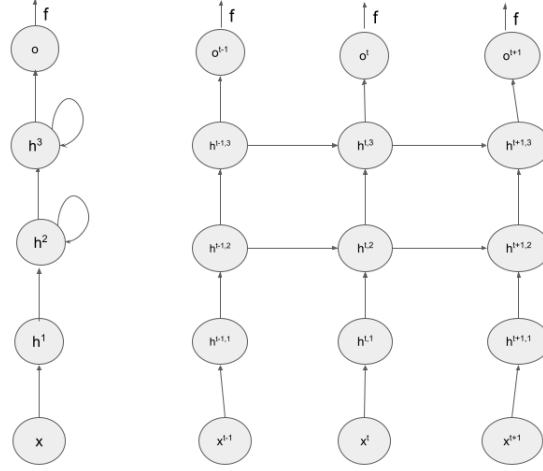


Figura 6.4: Red neuronal con tres capas ocultas, con dos recurrentes

Ejemplo 17. Consideremos el siguiente ejemplo, donde la notación $h^{(t,k)}$ indica que t es el estado de la recurrencia y k la profundidad de la capa oculta. Entonces, si definimos la siguiente red neuronal:

- Capa oculta: $h^{(t,1)} = \tanh(U^{(1)}x^{(t)} + b^{(1)})$
- Primera capa recurrente: $h^{(t,2)} = \sigma(W^{(2)}h^{(t-1,2)} + U^{(2)}h^{(t,1)} + b^{(2)})$
- Segunda capa recurrente: $h^{(t,3)} = \text{ReLU}(W^{(3)}h^{(t-1,3)} + U^{(3)}h^{(t,2)} + b^{(3)})$
- Capa de salida: $\hat{y}^{(t)} = \phi(Vh^{(t,3)} + c)$

Puede observarse que en una capa recurrente, las entradas pueden también ser representaciones recurrentes en el tiempo actual. En este ejemplo, la primera capa es una capa tipo feedforward que no guarda ningún tipo de recurrencia, la notación $h^{(t,1)}$ sólo indica que el dato de entrada corresponde al tiempo t en la secuencia de entrada. La siguiente capa incorpora una recurrencia, y toma la representación de la capa anterior como entrada (recuérdese que $h^{(0)} = \bar{0}$). Posteriormente aplicamos una capa recurrente más que toma como entrada la representación de la capa anterior. Por tanto, en la parte recurrente tomamos el estado anterior de la misma capa $h^{(t-1,3)}$ y el factor de entrada es el estado actual de la capa anterior $h^{(t,k-1)}$. Visualmente, la Figura 6.4 muestra cómo se vería esta red neuronal.

Como hemos señalado, las redes recurrentes son una familia de redes neuronales que se conforma por distintos tipos que esencialmente se basan en la Definición 96. Las de tipo *vanilla* son las más simples, pero existen otros tipos de redes recurrentes que revisaremos a continuación. En primer lugar, hablaremos de unas variaciones simples de las redes que ya hemos definido; posteriormente, introduciremos redes recurrentes que cuentan con celdas de memoria.

6.2. Tipos de redes recurrentes

Las redes recurrentes son una familia de redes que consta de diferentes arquitecturas dependiendo de los objetivos que se busquen alcanzar y los problemas que se busquen resolver. Existen diferentes tipos de redes recurrentes que, además, dependiendo de sus características, pueden llegar a combinarse entre sí. Entre una primera distinción sencilla que podemos hacer está la basada en el tipo de entradas y salidas que la red es capaz de procesar. En este aspecto, dividimos las redes recurrentes de la siguiente forma:

1. *Many-to-many*: Son redes recurrentes que cuentan con un número $T_x \geq 1$ de entradas y un con $T_y \geq 1$ de salidas. Es decir, toman una cadena de entrada y emiten una cadena de salida.
2. *Many-to-one*: Estas redes recurrentes toman en cuenta un número $T_x \geq 1$ de entradas, es decir, toman una cadena de entrada, pero sólo emiten una salida. Por ejemplo, se pueden usar para clasificar una cadena completa dentro de una sola clase.
3. *One-to-many*: Toman un solo elemento de entrada y emiten una secuencia de elementos de salida. Por ejemplo se pueden utilizar para generar la descripción (cadena de texto) de una imagen u otro objeto.
4. *One-to-one*: Las redes de uno a uno son equivalentes a las redes feedforward; por lo que se puede ver que las redes feedforward son un caso particular de las redes recurrentes.

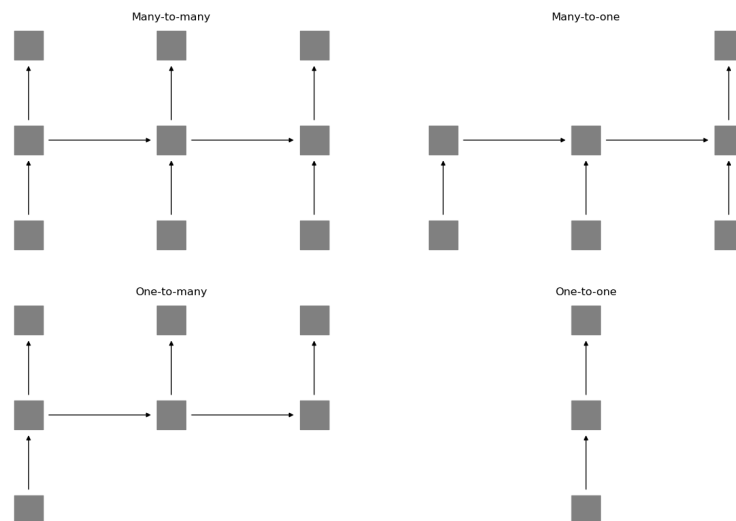


Figura 6.5: Tipos de RNNs según sus entradas y salidas

En la Figura 6.5 se puede ver una representación de este tipo de redes. En este caso, cada capa se representa con un cuadro, el cual contendrá un conjunto de neuronas. Representar las redes recurrentes por medio de cada neurona individual no es común; supondremos que cada nivel de la red tiene un conjunto de neuronas que se comunican por medio de las flechas. A partir de las celdas recurrentes, entonces, podemos construir diferentes tipos de arquitecturas que se enfoquen a diferentes tipos de problemas.

Las redes many-to-many son las redes que hemos revisado en la sección anterior que toman una cadena de entrada de longitud T y la llevan hacia una cadena de salida de la misma longitud. Asimismo, las redes one-to-one corresponden a una red que toma una cadena de longitud $T = 1$ y obtienen una salida de igual longitud. Bajo este supuesto, podemos ver que se trata justamente de redes de tipo feedforward.

Proposición 21. *Una red feedforward es una red recurrente donde la cadena de entrada y la cadena de salida tienen ambas longitud 1.*

Por tanto, las redes recurrentes generalizan a las redes feedforward y, en particular, en cada tiempo una red recurrente puede verse como una feedforward que se alimenta además de un estado previo.

Las redes many-to-one toman una cadena y la llevan hacia un sólo elemento de salida. Este tipo de redes se utilizan para obtener un sólo valor a partir de una secuencia; por ejemplo, en clasificación de texto, donde el texto es una secuencia y la salida es una clase (el tema del que trata, o si es un texto con opinión positiva o negativa). En este caso, se ignoran las salidas que no corresponden al último tiempo T , aunque la red puede emitir una salida en cada tiempo t , salida que depende de la secuencia hasta ese tiempo. Por tanto, las salidas previas serán salidas que no toman en cuenta toda la secuencia de entrada.

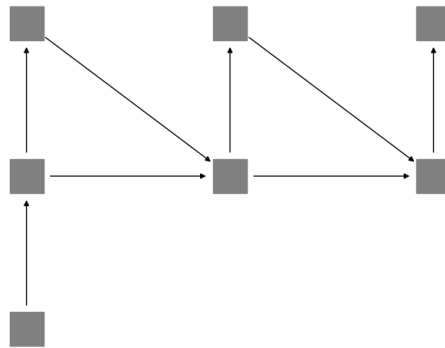


Figura 6.6: Red recurrente one-to-many con retroalimentación de las salidas

Las redes recurrentes de tipo one-to-many pueden tomar una sola entrada y arrojar diferentes salidas. Esto implica que la información de la entrada $x \in X$ se transmite a las salidas a partir de las capas recurrentes. Por ejemplo, esta red puede aplicarse para generar un texto descriptivo de una imagen, o una secuencia de imágenes a partir de una imagen inicial. En este caso, la primera capa recurrente está definida entonces por

esta entrada:

$$h^{(1)} = g(Wh^{(0)} + Ux + b) = g(Ux + b)$$

Ya que sabemos que $h^{(0)} = \bar{0}$ es el vector de ceros. Por tanto, la capa en este estado es similar a la capa oculta de una red feedforward, pero envía la información a las capas siguientes. Ya que no hay entradas en los siguientes estados podría definirse cada capa recurrente sólo a partir de la capa anterior, esto es: $h^{(t)} = g(Wh^{(t-1)} + b)$. Sin embargo, en la práctica es más usual utilizar las salidas anteriores, de tal forma que las salidas actuales sean dependientes de los elementos previos. De esta forma, podemos definir cada capa recurrente como:

$$h^{(t+1)} = g(Wh^{(t)} + U\hat{y}^{(t)} + b)$$

Donde $\hat{y}^{(t)}$ es el vector de probabilidades softmax de la salida. En este caso, debemos asegurarnos que las dimensiones de la capa recurrente (las unidades ocultas) sean las mismas que las de la capa de salida. Gráficamente, la red se puede representar como en la Figura 6.6.

6.2.1. Redes encoder-decoder

Las redes de tipo many-to-many toman como entrada una secuencia $x^{(1)}, x^{(2)}, \dots, x^{(T)}$ y obtienen una secuencia de salida $y^{(1)}, y^{(2)}, \dots, y^{(T)}$ de la misma longitud. En este tipo de redes, la cadena de entrada y la cadena de salida tienen el mismo número de elementos. De esta forma, cada vez que entra un elemento $x^{(t)}$ en un tiempo $t > 0$, se obtendrá una salida $y^{(t)}$. Sin embargo, muchas tareas basadas en secuencias no siempre buscan generar un mapeo del mismo número de elementos hacia otro igual. Las redes many-to-one y one-to-many son un ejemplo de esto. Pero también se pueden tener casos en que las secuencias de entrada y de salida difieran en longitud, pero ambas consistan de más de un elemento. Un ejemplo es la traducción automática, donde el número de palabras de una lengua no coincide necesariamente con el número de palabras de su traducción en otro lengua.

En estos casos, se utilizan redes recurrentes many-to-many especiales, las cuales son llamadas **encoder-decoder**. Este nombre proviene de que, precisamente, la red cuenta con dos módulos, un encoder o codificador que toma una cadena y la codifica en un vector; y un decoder o decodificador que toma el vector código y lo decodifica en una cadena de salida.

Definición 98 (Red recurrente codificadora). *Una red recurrente codificadora o encoder es una red recurrente de tipo many-to-one que, dada una cadena de entrada $x^{(1)}, x^{(2)}, \dots, x^{(T_x)}$, obtiene un vector de codificación $\hat{c} \in \mathbb{R}^m$:*

$$\hat{c} = \rho(h^{(1)}, h^{(2)}, \dots, h^{(T_x)})$$

Donde $h^{(t)} = f(h^{(t-1)}, x^{(t)})$ son las salidas de la capa recurrente en el tiempo t .

Definición 99 (Red recurrente decodificadora). *Una red recurrente decodificadora o decoder es una red recurrente de tipo one-to-many que toma como entrada un vector de codificación $\hat{c} \in \mathbb{R}^m$ y genera un conjunto de salidas $y^{(1)}, y^{(2)}, \dots, y^{(T_y)}$ como:*

$$y^{(t)} = \phi(Vh_d^{(t)} + c)$$

Donde $h_d^{(t)}$ es una representación de capa recurrente obtenida como:

$$h_d^{(t)} = f(h_d^{(t-1)}, y^{(t-1)})$$

Asumiendo que $y^{(0)} = \hat{c}$.

Hemos definido dos tipos de redes recurrentes complementarias. La red encoder es una red many-to-one que dada una secuencia la codifica en un sólo vector, mientras que la red decoder se encarga de decodificar un vector para obtener una salida. En el ejemplo de la traducción automática, una red encoder codificaría una cadena en una lengua para obtener una representación del ‘significado’ de la cadena. Esta representación se puede decodificar en una nueva lengua a partir de una red decoder. La red encoder-decoder combina ambas redes.

Definición 100 (Red recurrente encoder-decoder). *Una red recurrente encoder-decoder es una arquitectura de red recurrente que toma una entrada $x^{(1)}, x^{(2)}, \dots, x^{(T_x)}$ y lo lleva a una salida $y^{(1)}, y^{(2)}, \dots, y^{(T_y)}$ (con T_x no necesariamente igual a T_y), por medio de:*

1. Una red encoder que toma la cadena de entrada y la lleva a una codificación $\hat{c}$.
2. Una red decoder que toma la codificación $\hat{c}$ y lo lleva a una secuencia de salida.

La red recurrente encoder-decoder es una sola arquitectura, por lo que ambas partes de la red son complementarias y deben entrenarse juntas, para que los pesos del encoder correspondan a los del decoder y viceversa. La Figura 6.7 muestra la idea gráfica de una RNN encoder-decoder. Como se puede observar, el encoder toma una secuencia de elementos de entrada que emiten una codificación (el cuadro negro), esta codificación es la entrada de una nueva red que emite una secuencia de salida, el decoder. La red decoder puede retroalimentarse de las salidas previas.

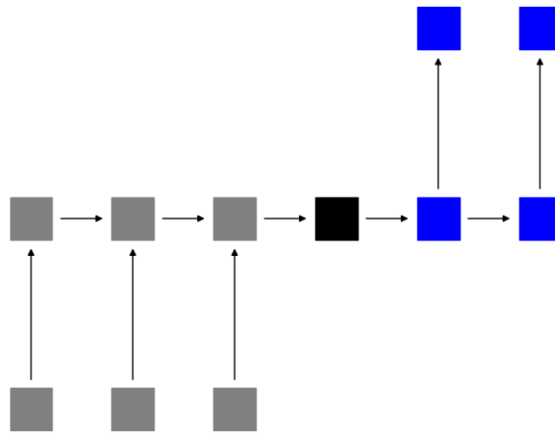


Figura 6.7: Red recurrente encoder-decoder

Las arquitecturas recurrentes encoder-decoder pueden conformarse de diferentes formas. En el caso más sencillo, tanto el encoder como el decoder son redes recurrentes vanilla (Definición 97). Y la codificación se puede definir de manera sencilla como:

$$\hat{c} = \rho(h^{(1)}, \dots, h^{(T_x)}) = h^{(T_x)}$$

Es decir, la codificación se correspondería únicamente al último estado oculto recurrente. De tal forma, que la red enviaría la información directamente del último estado sin pasar por un vector intermedio. Esto tiene sentido en cuanto en una red recurrente la información de los estados previos se pasa de estado a estado, por lo que el último estado guardaría la información de todos los estados previos. En la práctica, si la secuencia de entrada es muy larga, cierta información de los estados iniciales podría perderse en el último estado, es por eso que se proponen estrategias como GRU o LSTM que veremos en la Sección 6.4.

6.2.2. Redes recurrentes bidireccionales

Las redes recurrentes toman como entrada una secuencia que se asume avanza de izquierda a derecha en el tiempo. Es decir, se asume que hay una dependencia temporal secuencial. Sin embargo, en algunos problemas existe una relación bidireccional entre los diferentes estados del proceso. Es decir, los elementos precedentes influyen de igual manera en el estado actual que los elementos subsecuentes. En este caso, si bien se cuenta con una cadena, esta cadena es bidireccional, cada nodo se dirige hacia el nodo anterior y hacia el nodo subsecuente.

Para construir una red bidireccional, se pensará que existe por cada capa recurrente, otra capa casi idéntica pero que en lugar de almacenar la información de los precedentes, almacena la información de los subsecuentes. Denotemos a la capa recurrente de avance como:

$$\vec{h}^{(t)} = g(\vec{W}\vec{h}^{(t-1)} + \vec{U}x^{(t)} + \vec{b})$$

Las capas de avance son de capas recurrentes estándar, las cuales ya hemos revisado en las secciones anteriores. Para manifestar la direccionalidad inversa, definimos capas de retroceso. Estas capas toman en cuenta la información subsecuente y la envían hacia los estados anteriores de la cadena. Así, estando en el estado t debemos recuperar la información del estado $t + 1$. Por su parte, la información del estado t se enviará al estado $t - 1$. El procedimiento a seguir será muy similar a las capas de avance, pero tomando en cuenta el estado subsecuente y no el anterior. Nuestra capa recurrente de retroceso se definirá entonces como:

$$\overleftarrow{h}^{(t)} = g(\overleftarrow{W}\overleftarrow{h}^{(t+1)} + \overleftarrow{U}x^{(t)} + \overleftarrow{b})$$

Se puede observar que se recupera la información de la capa subsecuente $\overleftarrow{h}^{(t+1)}$ (y no la de la capa anterior). Hay que notar que los pesos de avance $\vec{W}$, $\vec{U}$ y $\vec{b}$ son diferentes (e independientes) de los pesos de retroceso $\overleftarrow{W}$, $\overleftarrow{U}$ y $\overleftarrow{b}$. De esta forma, se puede pensar que se computan de manera independiente las capas de avance y retroceso. De hecho, una red bidireccional se puede ver como dos redes que toman la misma entrada,

pero donde cada red hace sus propios cálculos y maneja la información en direcciones diferentes.

Las salidas de una red bidireccional se obtendrán al usar la información tanto de avance como de retroceso. Para hacer esto simplemente tomamos ambas capas para cada estado t y obtenemos la salida como:

$$\hat{y}^{(t)} = \phi(\vec{V} \vec{h}^{(t)} + \overleftarrow{V} \overleftarrow{h}^{(t)} + c)$$

Es decir, multiplicamos el avance y el retroceso por los pesos respectivos y sumamos esta información. Otra forma de verlo es a partir de la concatenación de las dos capas recurrentes. Esto es:

$$\hat{y} = \phi(V[\vec{h}^{(t)}; \overleftarrow{h}^{(t)}] + c)$$

Tal que $[\vec{h}^{(t)}; \overleftarrow{h}^{(t)}]$ es la concatenación del avance y el retroceso y $V = [\vec{V}; \overleftarrow{V}]$ es una matriz de $o \times 2m$ donde m es la dimensión de las capas ocultas y o las unidades de salida. Si la salida es una probabilidad de tipo softmax, la salida de una red de este tipo sería de la forma:

$$\begin{aligned} \hat{y}^{(t)} &= p(y^{(t)} | y^{(1)} \dots y^{(T_y)}, \mathbf{x}) \\ &= p(y^{(t)} | y^{(1)} \dots y^{(t-1)}, \mathbf{x}) p(y^{(t)} | y^{(t+1)} \dots y^{(T_y)}, \mathbf{x}) \end{aligned}$$

La probabilidad de una salida depende tanto de los elementos antecedentes como de los precedentes, así como de la cadena de entrada que denotamos como $\mathbf{x} = x^{(1)} \dots x^{(T_x)}$. Pero dado que las direcciones de la red recurrente son independientes, las probabilidades obtenidas también serán independientes.

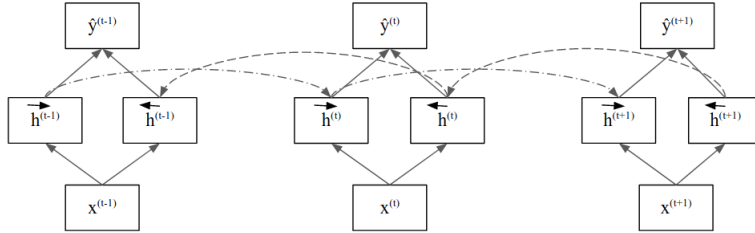


Figura 6.8: Visualización de una RNN bidireccional

En resumen, una red recurrente bidireccional se puede entender como sigue (Schuster and Paliwal, 1997):

Definición 101 (Red recurrente bidireccional). *Una red recurrente bidireccional es un tipo de red recurrente que toma la información precedente y la información subsecuente para calcular el estado actual. Se puede definir con los siguientes elementos:*

1. Un conjunto de $k \in \{1, 2, \dots, K\}$ capas ocultas recurrentes de **avance**:

$$\vec{h}^{(t,k)} = g(\vec{W}^{(k)} h^{(t-1,k)} + \vec{U}^{(k)} h^{(t,k-1)} + \vec{b}^{(k)})$$

2. Un conjunto de $k \in \{1, 2, \dots, K\}$ capas ocultas recurrentes de **retroceso**:

$$\overleftarrow{h}^{(t,k)} = g(\overleftarrow{W}^{(k)} h^{(t+1,k)} + \overleftarrow{U}^{(k)} h^{(t,k-1)} + \overleftarrow{b}^{(k)})$$

3. Capa de salida:

$$\hat{y} = \phi(V[\overrightarrow{h}^{(t,K)}; \overleftarrow{h}^{(t,K)}] + c)$$

La Figura 6.8 muestra el cómo se comunican las capas de avance y retroceso en una red bidireccional. Las redes recurrentes son útiles en tareas donde las cadenas de entrada no dependen de una sola dirección, como en el lenguaje natural. Es común que las aplicaciones de lenguaje natural se ocupen redes bidireccionales. Por ejemplo, en la traducción, que se realiza con modelos de encoder-decoder (Sección 6.2.1), incluyen en el decoder capas recurrentes bidireccionales, de tal forma que la codificación obtenida contenga una información no sesgada por una sola dirección de avance.

6.3. Aprendizaje en capas recurrentes

El aprendizaje en las redes feedforward dependía del algoritmo de retro-propagación el cuál permitía llevar la información del error desde las capas de salida hasta las capas ocultas, aprovechando la estructura de la red. En las redes recurrentes, debido a la dependencia del tiempo, se introduce una generalización de este algoritmo: retro-propagación a través del tiempo o *Backpropagation Throgh Time*, que abreviamos como BPTT (Werbos, 1990). La retro-propagación a través del tiempo es una generalización de la retro-propagación que hemos revisado en la Sección 4.3.

El método de BPTT tiene que retropropagar no sólo la información que viene de las capas superiores, sino también de la información que viene de las recurrencias, que guardan la información temporal en la red. Para comenzar, supongamos que nuestra red recurrente cuenta con una capa de entrada, una capa de salida y la capa recurrente. Además de esto, debemos definir la función objetivo $R: \Theta \rightarrow \mathbb{R}^+$. Esta función objetivo debe tomar en cuenta que está trabajando con secuencias, no con entradas y salidas independientes.

Definición 102 (Función objetivo para redes recurrentes). *Una función objetivo para redes recurrentes $R: \Theta \rightarrow \mathbb{R}^+$ es una función que busca los pesos óptimos θ^* en el espacio de hipótesis Θ evaluando los errores de la red. En su forma general está dada como:*

$$R(\theta) = \sum_t R_t(\theta) = \sum_t \sum_y L_t(y^{(t)}, f^{(t)}(\mathbf{x})) \quad (6.1)$$

Donde L_t es una función de pérdida que depende del tiempo, $f^{(t)}$ es la red recurrente en el tiempo t , $e y^{(t)}$ son los valores reales y las predicciones de la red respectivamente. Finalmente, $\mathbf{x} = x^{(1)}, x^{(2)}, \dots, x^{(t)}$ es una entrada secuencial.

Como se puede ver de esta definición, la función objetivo puede pensarse como una suma sobre los tiempos de funciones de riesgo que dependen del tiempo. En realidad, la función de riesgo (y la de pérdida) tienen siempre la misma regla de asociación, pero

varían dependiendo de las salidas de la red en los diferentes tiempos de la secuencia. Si pensamos en un problema de clasificación secuencial, las clases reales $y^{(t)}$ son las clases que clasifican a cada una de las entradas $x^{(t)}$, por lo que la clase depende del tiempo t en que se encuentra la secuencia.

En problema de clasificación de secuencias, por tanto, la función de riesgo que podemos definir es la entropía cruzada, considerando la dependencia en el tiempo. Por lo que esta función sería de la forma:

$$R(\theta) = - \sum_t \sum_y \delta_{y_t, x_t} \ln f_y^{(t)}(\mathbf{x})$$

Esta función es similar a la función de riesgo en problemas de clasificación con redes feedforward, pero considerando la dependencia del tiempo, por lo que no sólo nos fijamos en las neuronas de la capa de salida, sino también en el tiempo en que estas neuronas se activan. Por su parte, también las redes recurrentes nos permitirán tener problemas de regresión secuenciales, donde la salida sea una sucesión de números reales. En este caso, la función de riesgo puede definirse como:

$$R(\theta) = \frac{1}{2} \sum_t \sum_y (y^{(t)} - f^{(t)}(\mathbf{x}))^2$$

Esta función es una forma del error cuadrático medio, pero con dependencia del tiempo. El hecho de que las funciones de riesgo no cambien significativamente nos permite ver que la retro-propagación tampoco variará significativamente fuera de las capas recurrentes. De hecho es fácil ver que para derivar sobre los pesos en la capa de salida, que hemos denotada por la matriz de pesos V , obtenemos derivadas muy similares a la retro-propagación usual:

$$\begin{aligned} \frac{\partial R(\theta)}{\partial V_{i,j}} &= \sum_t \sum_y \frac{\partial L_t(y^{(t)}, f_y^{(t)}(\mathbf{x}))}{\partial V_{i,j}} \\ &= \sum_t \frac{\partial L_t}{\partial f_i^{(t)}} \frac{\partial f_i^{(t)}}{\partial a_i^{(t)}} \frac{\partial a_i^{(t)}}{\partial V_{i,j}} \\ &= \sum_t \frac{\partial L_t}{\partial f_i^{(t)}} \frac{\partial f_i^{(t)}}{\partial a_i^{(t)}} h_j^{(t)} \end{aligned}$$

Donde $a^{(t)} = Vh^{(t)} + c$ es la pre-activación en la capa de salida. Por tanto, las derivadas parciales obtenidas tienen una forma similar a las del algoritmo de retro-propagación, sólo debemos tomar en cuenta que aquí se está considerando al tiempo t como una variable en estas derivadas. Por tanto, tendremos tantas derivadas de esta forma como elementos en la secuencia procesada por la red neuronal.

Si observamos que es lo que pasa en las capas recurrentes, vemos que aquí se están procesando dos matrices de pesos: la matriz de pesos en la recurrencia W y la matriz de pesos de las capas previas U . Recordemos que esta capa está definida como:

$$h^{(t)} = g(W h^{(t-1)} + U x^{(t)} + b)$$

Los pesos en U tienen una derivación similar a la retro-propagación usual, pues la estructura que define (procesar capas previas para obtener valores de capas superiores) es similar a la estructura de las redes feedforward. Podemos observar que las derivadas son de la forma:

$$\begin{aligned} \frac{\partial R(\theta)}{\partial U_{i,j}} &= \sum_t \sum_y \frac{\partial L_t}{\partial U_{i,j}} \\ &= \sum_y \frac{\partial L_t}{\partial f_y^{(t)}} \frac{\partial f_y^{(t)}}{\partial a_y^{(t)}} \frac{\partial a_y^{(t)}}{\partial h_i^{(t)}} \frac{\partial h_i^{(t)}}{\partial \hat{a}_i^{(t)}} \frac{\partial \hat{a}_i^{(t)}}{\partial U_{i,j}} \\ &= \frac{\partial h_i^{(t)}}{\partial \hat{a}_i^{(t)}} \sum_y v_{i,y} \frac{\partial L_t}{\partial f_y^{(t)}} \frac{\partial f_y^{(t)}}{\partial a_y^{(t)}} x_j^{(t)} \end{aligned}$$

En la derivación, tomamos en cuenta que derivamos sobre un tiempo t específico, por lo que los valores en los otros tiempos son constantes. La forma que toma la derivación es similar a la de retro-propagación (Sección 4.3) con dependencia del parámetro temporal. Asimismo, notamos que los valores $\frac{\partial L_t}{\partial f_y^{(t)}} \frac{\partial f_y^{(t)}}{\partial a_y^{(t)}}$ han sido calculados en la capa superior, por lo que podemos usar la misma estrategia y almacenar estos valores en una variable para retro-propagarlos en las capas precedentes.

La derivación hasta aquí no ha cambiado mucho con respecto a lo visto previamente más allá de la dependencia del tiempo. La principal dificultad del BPTT se da justamente en los pesos de las recurrencias. En este caso, debemos obtener las derivadas parciales sobre los pesos W en el producto $Wh^{(t-1)}$, pero la capa en el tiempo $t-1$ también depende de los pesos W y, en general, todas las capas previas dependen de estos pesos, por lo que la derivación de estos pesos para cada $t > 1$ debe tomar en cuenta estas dependencias.

Para poder llegar a la derivada de las capas recurrentes, en primer lugar, nos enfocamos en las derivadas $\frac{\partial h^{(t)}}{\partial W_{i,j}}$ y su dependencia con los tiempos anteriores. Afirmamos el siguiente resultado.

Lema 2. *En una red recurrente, derivar los pesos en el tiempo t (dado que estos se comparten en los tiempos anteriores) se determina por medio de:*

$$\frac{\partial h^{(t)}}{\partial W_{i,j}} = \sum_{k=1}^t \left(\prod_{l=k}^{t-1} \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) \frac{\partial h^{(k)}}{\partial W_{i,j}}$$

Demostración. Demostramos esto usando inducción sobre la longitud de la secuencia. De tal forma que cuando $t = 1$ podemos observar que:

$$\frac{\partial h^{(1)}}{\partial W_{i,j}} = \frac{\partial h^{(1)}}{\partial W_{i,j}} + \frac{\partial h^{(0)}}{\partial W_{i,j}}$$

Como $h^{(0)}$ no depende de W (puede ser $\bar{0}$ o un vector de inicialización) se tiene que $\frac{\partial h_i^{(0)}}{\partial W_{i,j}} = 0$. Lo que demuestra la proposición para cadenas de longitud 1.

Ahora supongamos que la igualdad se cumple para t y mostraremos que también se cumple para $t+1$. En primer lugar, recordemos que $h_i^{(t+1)} = g(a_i^{(t+1)})$ con $a_i^{(t+1)} = W_{i,:}h^{(t)} + U_i x^{(t+1)} + b_i$ y que además $h_i^{(t)}$ también depende de los pesos en W . Pero notamos que el factor $Ux^{(t)} + b$ no depende de estos pesos, por lo que podemos considerarlos constantes; por lo que para la derivada podemos ignorarlos. Por tanto, tenemos que:

$$\begin{aligned} \frac{\partial h_i^{(t+1)}}{\partial W_{i,j}} &= \frac{\partial h_i^{(t+1)}}{\partial a_i^{(t+1)}} \frac{\partial a_i^{(t+1)}}{\partial W_{i,j}} \\ &= \frac{\partial h_i^{(t+1)}}{\partial a_i^{(t+1)}} \frac{\partial}{\partial W_{i,j}} W_{i,:}h^{(t)} \end{aligned}$$

Para obtener la derivada $\frac{\partial}{\partial W_{i,j}} W_{i,:}h^{(t)}$ debemos aplicar la regla del producto en tanto $h^{(t)}$ también depende de W , por lo que tenemos:

$$\begin{aligned} \frac{\partial h_i^{(t+1)}}{\partial W_{i,j}} &= \frac{\partial h_i^{(t+1)}}{\partial a_i^{(t+1)}} \frac{\partial}{\partial W_{i,j}} W_{i,:}h^{(t)} \\ &= \frac{\partial h_i^{(t+1)}}{\partial a_i^{(t+1)}} \left(h^{(t)} \frac{\partial}{\partial W_{i,j}} W_{i,:} + W_{i,:} \frac{\partial h^{(t)}}{\partial W_{i,j}} \right) \\ &= \frac{\partial h_i^{(t+1)}}{\partial a_i^{(t+1)}} \left(h_j^{(t)} + W_{i,:} \frac{\partial h^{(t)}}{\partial W_{i,j}} \right) \end{aligned}$$

De esta última igualdad, podemos expresar $h_j^{(t)} = \frac{\partial a_i^{(t+1)}}{\partial W_{i,j}}$ y $W_{i,:} = \frac{\partial a_i^{(t+1)}}{\partial h^{(t)}}$, por lo que tenemos:

$$\begin{aligned} \frac{\partial h_i^{(t+1)}}{\partial W_{i,j}} &= \frac{\partial h_i^{(t+1)}}{\partial a_i^{(t+1)}} \left(h_j^{(t)} + W_{i,:} \frac{\partial h^{(t)}}{\partial W_{i,j}} \right) \\ &= \frac{\partial h_i^{(t+1)}}{\partial a_i^{(t+1)}} \left(\frac{\partial a_i^{(t+1)}}{\partial W_{i,j}} + \frac{\partial a_i^{(t+1)}}{\partial h^{(t)}} \frac{\partial h^{(t)}}{\partial W_{i,j}} \right) \\ &= \frac{\partial h_i^{(t+1)}}{\partial a_i^{(t+1)}} \frac{\partial a_i^{(t+1)}}{\partial W_{i,j}} + \frac{\partial h_i^{(t+1)}}{\partial a_i^{(t+1)}} \frac{\partial a_i^{(t+1)}}{\partial h^{(t)}} \frac{\partial h^{(t)}}{\partial W_{i,j}} \\ &= \frac{\partial h_i^{(t+1)}}{\partial W_{i,j}} + \frac{\partial h_i^{(t+1)}}{\partial h^{(t)}} \frac{\partial h^{(t)}}{\partial W_{i,j}} \end{aligned}$$

Ahora podemos aplicar la hipótesis de inducción en $\frac{\partial h_i^{(t)}}{\partial W_{i,j}}$ lo que nos deja con:

$$\begin{aligned}
 \frac{\partial h_i^{(t+1)}}{\partial W_{i,j}} &= \frac{\partial h_i^{(t+1)}}{\partial W_{i,j}} + \frac{\partial h_i^{(t+1)}}{\partial h^{(t)}} \frac{\partial h^{(t)}}{\partial W_{i,j}} \\
 &= \frac{\partial h_i^{(t+1)}}{\partial W_{i,j}} + \frac{\partial h_i^{(t+1)}}{\partial h^{(t)}} \sum_{k=1}^t \left(\prod_{l=k}^{t-1} \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) \frac{\partial h^{(k)}}{\partial W_{i,j}} \\
 &= \sum_{k=1}^t \left(\frac{\partial h_i^{(t+1)}}{\partial h^{(t)}} \prod_{l=k}^{t-1} \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) \frac{\partial h^{(k)}}{\partial W_{i,j}} + \frac{\partial h_i^{(t+1)}}{\partial W_{i,j}} \\
 &= \sum_{k=1}^t \left(\prod_{l=k}^t \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) \frac{\partial h^{(k)}}{\partial W_{i,j}} + \frac{\partial h_i^{(t+1)}}{\partial W_{i,j}} \\
 &= \sum_{k=1}^t \left(\prod_{l=k}^t \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) \frac{\partial h^{(k)}}{\partial W_{i,j}} + \frac{\partial h^{(t)}}{\partial h^{(t)}} \frac{\partial h_i^{(t+1)}}{\partial W_{i,j}} \\
 &= \sum_{k=1}^{t+1} \left(\prod_{l=k}^t \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) \frac{\partial h^{(k)}}{\partial W_{i,j}}
 \end{aligned}$$

Por lo que el lema ha sido probado. $\square$

De esta forma, obtenemos que derivar las capas recurrentes sobre los pesos W implica la realización de una serie de productos por las capas previas. Esto, como veremos, puede ser problemático cuando las derivadas se hacen pequeñas. Por ahora, completamos la derivación de las capas recurrentes, lo que se muestra en el siguiente Teorema.

Teorema 17. *Dada una capa recurrente en una red neuronal, con función de riesgo dependiente del tiempo $R(\theta) = \sum_t \sum_y L_t(f_y^{(t)}(x), y_t)$, la derivada sobre los pesos en las recurrencias $W_{i,j}$ está dada como:*

$$\frac{\partial R(\theta)}{\partial W_{i,j}} = \sum_y \frac{\partial L_t}{\partial f_y^{(t)}} \frac{\partial f_y^{(t)}}{\partial a_y^{(t)}} \frac{\partial a_y^{(t)}}{\partial h_i^{(t)}} \sum_{k=1}^t \left(\prod_{l=k}^{t-1} \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) \frac{\partial h^{(k)}}{\partial W_{i,j}}$$

Demostración. Podemos derivar hasta llegar a la capa recurrente, considerando que en el tiempo t los otros tiempos de la primera suma de la función de riesgo son constantes:

$$\begin{aligned}
 \frac{\partial R(\theta)}{\partial W_{i,j}} &= \sum_t \sum_y \frac{\partial L_t}{\partial W_{i,j}} \\
 &= \sum_y \frac{\partial L_t}{\partial f_y^{(t)}} \frac{\partial f_y^{(t)}}{\partial a_y^{(t)}} \frac{\partial a_y^{(t)}}{\partial h^{(t)}} \frac{\partial h^{(t)}}{\partial W_{i,j}}
 \end{aligned}$$

Observamos que el último término corresponde a la derivación demostrada en el Lema anterior, por lo que sustituyendo obtenemos:

$$\frac{\partial R(\theta)}{\partial W_{i,j}} = \sum_y \frac{\partial R_t}{\partial f_y^{(t)}} \frac{\partial f_y^{(t)}}{\partial a_y^{(t)}} \frac{\partial a_y^{(t)}}{\partial h_i^{(t)}} \sum_{k=1}^t \left(\prod_{l=k}^{t-1} \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) \frac{\partial h^{(k)}}{\partial W_{i,j}}$$

□

Del Teorema anterior podemos simplificar un poco más la derivación, para observar que lo que obtenemos es de la forma:

$$\frac{\partial R(\theta)}{\partial W_{i,j}} = \sum_y V_{i,y} \frac{\partial R_t}{\partial f_y^{(t)}} \frac{\partial f_y^{(t)}}{\partial a_y^{(t)}} \sum_{k=1}^t \left(\prod_{l=k}^{t-1} \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) \frac{\partial h_i^{(k)}}{\partial a_i^{(k)}} h_j^{k-1}$$

Debemos notar que la derivada $\frac{\partial R_t}{\partial f_y^{(t)}} \frac{\partial f_y^{(t)}}{\partial a_y^{(t)}}$ ya se ha calculado en la capa superior, por lo que esta se retropropaga hacia esta capa. De hecho, puede observarse que el factor $\sum_y V_{i,y} \frac{\partial R_t}{\partial f_y^{(t)}} \frac{\partial f_y^{(t)}}{\partial a_y^{(t)}}$ se presenta tanto en la derivación de los pesos recurrentes W , como en la derivación de los pesos de entrada U . Por tanto, para aprovechar estos cálculos conviene guardar la información de esta derivación en una variable.

En resumen, el algoritmo de retro-propagación a través del tiempo (BPTT) se puede resumir en los siguientes pasos:

1. Computar el paso de avance (forward) de la red para obtener los valores $f^{(t)}(\mathbf{x})$ para todos los tiempos $t \in \{1, 2, \dots, T\}$.
2. Comenzar la retro-propagación con la capa de salida obteniendo:

$$\frac{\partial R(\theta)}{\partial V_{i,j}} = \frac{\partial L_t}{\partial f^{(t)}} \frac{\partial f^{(t)}}{\partial a_i^{(t)}} h_j^{(t)}$$

Donde V es la matriz de pesos de salida; se puede utilizar la variable $d_{K+1}(i) = \frac{\partial L_t}{\partial f^{(t)}} \frac{\partial f^{(t)}}{\partial a_i^{(t)}}$ que se retro-propaga.

3. Retro-propagar a través de las capas ocultas, si estas son recurrentes se retro-propaga el gradiente en dos direcciones:

- a) Sobre los pesos U que no son recurrentes se obtienen las derivadas parciales:

$$\frac{\partial L_t}{\partial U_{i,j}} = \frac{\partial h_i^{(t)}}{\partial a_i^{(t)}} \sum_y V_{i,y} d_{K+1}(y) x_j^{(t)}$$

Se puede guardar la variable $d_K(i) = \frac{\partial h_i^{(t)}}{\partial a_i^{(t)}} \sum_y V_{i,y} d_{K+1}(y)$. $x^{(t)}$ es la entrada que puede ser una capa oculta previa.

- b) Se obtiene la derivada sobre los pesos de la recurrencia W de la siguiente forma:

$$\frac{\partial L_t}{\partial W_{i,j}} = \sum_y V_{i,y} d_{K+1}(y) \sum_{k=1}^t \left(\prod_{l=k}^{t-1} \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) \frac{\partial h_i^{(k)}}{\partial a_i^{(k)}} h_j^{k-1}$$

- c) Se repite el paso anterior hasta abarcar todos los tiempos $t \in \{1, 2, \dots, T\}$

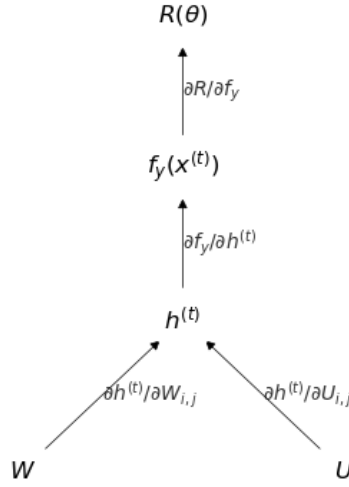


Figura 6.9: Esquema de una gráfica computacional para redes recurrentes

4. Si existen más capas se repite el paso 3 hasta alcanzar la capa de entrada.

Cuando hemos obtenido el gradiente en cada capa podemos actualizar los pesos correspondientes utilizando alguno de los optimizadores revisados en la Sección 4.2. De igual forma que vimos con las redes feedforward, el BPTT puede implementarse por medio de una gráfica computacional (véase Sección 4.3). La Figura 6.9 muestra la idea detrás de esta implementación. De forma todavía más general, toda la capa recurrente puede verse como un nodo de la gráfica computacional en la que el gradiente se obtiene según el Teorema 17.

Ejemplo 18. Supongamos que tenemos la siguiente red recurrente que procesará una cadena de longitud 3, con las siguientes especificaciones:

1. $\hat{a}^{(t)} = Wh^{(t-1)} + Ux^{(t)} + b$ para toda $t \in \{1, 2, 3\}$.
2. $h^{(t)} = \tanh(\hat{a}^{(t)})$ para toda $t \in \{1, 2, 3\}$.
3. $f^{(t)} = \text{Softmax}(Vh^{(t)} + c)$ para toda $t \in \{1, 2, 3\}$.
4. Función de riesgo de entropía cruzada: $R(\theta) = -\sum_t \sum_i \delta_{i,x_t} \ln f_i^{(t)}(\mathbf{x})$.

Entonces, aplicando el método de BPTT derivamos primero para las capas de salida por cada tiempo t , lo que resulta en:

$$\begin{aligned} \frac{\partial L_t}{\partial V_{i,j}} &= -\sum_i \sum_i \frac{\partial L_t}{\partial V_{i,j}} \delta_{y_i,i} \ln f_i^{(t)}(\mathbf{x}) \\ &= (f_i^{(t)} - y_i^{(t)}) h_j^{(t)} \end{aligned}$$

Con estos valores, podemos retropropagar por las capas ocultas recurrentes. Para los pesos sobre la matriz U tenemos:

$$\frac{\partial L_t}{\partial U_{i,j}} = (1 - (h_i^{(t)})^2) \sum_y (f_y^{(t)} - y^{(t)}) x_j^{(t)}$$

Finalmente, en los pesos de las recurrencias, tendremos que las derivadas para cada tiempo t son de la forma:

$$\begin{aligned} \frac{\partial L_t}{\partial W_{i,j}} &= \sum_y V_{i,y} (f_y^{(t)} - y^{(t)}) \sum_{k=1}^t \left(\prod_{l=k}^{t-1} \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) (1 - (h_i^{(k)})^2) h_j^{k-1} \\ &= \sum_y V_{i,y} (f_y^{(t)} - y^{(t)}) \sum_{k=1}^t \left(\prod_{l=k}^{t-1} (1 - (h^{(l+1)})^2) W_{i,:} \right) (1 - (h_i^{(k)})^2) h_j^{k-1} \end{aligned}$$

Por lo que hemos obtenido el gradiente en todas las capas ocultas, incluyendo los gradientes de los pesos en las recurrencias.

6.4. Capas de memoria

Cuando se obtiene el gradiente de una red recurrente, la función objetivo que depende del tiempo tiene la forma:

$$R(\theta) = \sum_x \sum_t \sum_y L_t(\hat{y}^{(t)}, y^{(t)})$$

Por lo que los gradientes deben calcularse tomando en cuenta el parámetro del tiempo t para toda t sobre la longitud de la cadena. Las derivadas parciales de la salida son de la forma:

$$\frac{\partial L_t}{\partial V_{i,j}} = \frac{\partial L_t}{\partial f_i^{(t)}} \frac{\partial f_i^{(t)}}{\partial a_i^{(t)}} \frac{\partial a_i^{(t)}}{\partial V_{i,j}}$$

Sin embargo, en las capas recurrentes se tiene que las capas de un estado depende de los estados anteriores. Si bien los pesos de la matriz recurrente W se comparten a través de los diferentes estados, podemos utilizar una variable “dummy” de tiempo en estas. De tal forma que describamos las capas recurrentes como:

$$h^{(t)} = g(W^{(t)} h^{(t-1)} + U x^{(t)} + b^{(t)})$$

Visto así, podemos pensar que derivar sobre un tiempo t dependerá del tiempo $t - 1$; esto quiere decir que derivar sobre la matriz de pesos en el tiempo t va a contar con los pesos en el tiempo $t - 1$, por lo que de cierta forma, se tiene una derivada que pasa por la capa $k < t$. Esto produce una suma de los productos de las derivadas entre las capas recurrentes (véase el Teorema 17):

$$\frac{\partial L_t}{\partial W_{i,j}} = \sum_y \frac{\partial L_t}{\partial f_y^{(t)}} \frac{\partial f_y^{(t)}}{\partial a_y^{(t)}} \frac{\partial a_y^{(t)}}{\partial h_i^{(t)}} \sum_{k=1}^t \left(\prod_{j=k}^{t-1} \frac{\partial h_i^{(j+1)}}{\partial h_i^{(j)}} \right) \frac{\partial h_i^{(k)}}{\partial W_{i,j}}$$

El problema que se presenta en el cálculo de dichos gradientes radica en el producto $\prod_{j=k}^{t-1} \frac{\partial h_i^{(j+1)}}{\partial h_i^{(j)}}$ que puede hacerse demasiado pequeño si t es grande. Ya que las derivadas durante el proceso de entrenamiento por BPTT se hacen de forma numérica, si los valores son demasiado pequeños, estos valores pueden interpretarse como ceros. A este problema se le conoce como el **desvanecimiento del gradiente**, y se da cuando la derivada empírica se vuelve 0 debido a que las aproximaciones hechas por la máquina se hacen demasiado pequeñas.

Para solventar este problema se han propuesto diferentes estrategias basadas en definir capas recurrentes que integren mecanismos que reduzcan el impacto del desvanecimiento del gradiente. Al mismo tiempo, estas capas permiten que el flujo de la información en secuencias de entrada de longitud considerable se propague mejor a través de los diferentes estados. Por tanto, consideramos estas capas como capas de memoria. En esta sección revisamos dos mecanismos clásicos para las redes recurrentes, las capas de tipo GRU y las de tipo LSTM.

6.4.1. Gated Recurrent Unit (GRU)

Una de las propuestas de memoria para lidiar con los problemas antes mencionados son las llamadas unidades de compuerta recurrente, del inglés *Gated Recurrent Unit* (GRU) (Cho et al., 2014). Históricamente, se desarrollaron después que las LSTM que revisamos en la siguiente sección, pero debido a que son más simples las explicamos primero. Se trata de capas que utilizan puertas de memoria para preservar la información que se transmite por las recurrencias de una red. Su idea fundamental está basada en la estructura de los circuitos eléctricos y cómo estos transmiten información; de allí el nombre de compuertas (*gates*)

Las capas GRU, entonces, se basan en dos compuertas fundamentales que permiten el flujo de información hacia los estados siguientes (de igual forma que compuertas lógicas secuenciales). Estas compuertas son la compuerta de actualización (*update*) y la compuerta de *reset*.

Definición 103 (Compuerta de actualización). *Una compuerta de actualización (en una capa GRU) es un vector $z^{(t)} \in \mathbb{R}^m$ que depende del tiempo t , definida como:*

$$z^{(t)} = \sigma(W^z h^{(t-1)} + U^z x^{(t)} + b^z)$$

Esta compuerta se utiliza para actualizar los pesos actuales y los anteriores.

Definición 104 (Compuerta reset). *Una compuerta de reset (en una capa GRU) es un vector $r^{(t)} \in \mathbb{R}^m$ que depende del tiempo t , definida como:*

$$r^{(t)} = \sigma(W^r h^{(t-1)} + U^r x^{(t)} + b^r)$$

Esta compuerta se utiliza para crear un candidato que contenga la información que se transmitirá a los estados siguientes.

Ambas compuertas son recurrentes, recuerdan a la definición de red recurrente vanilla en tanto toman en cuenta los estados anteriores y la entrada en el estado actual.

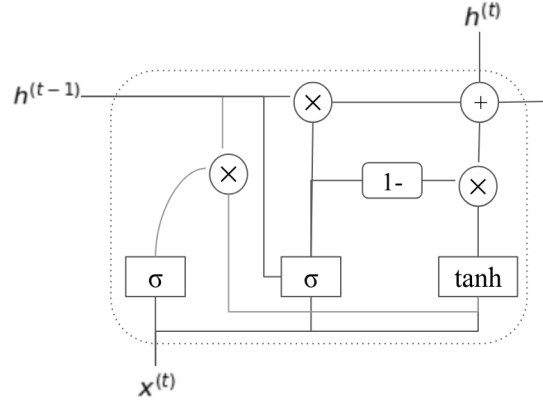


Figura 6.10: Capa recurrente de tipo GRU

Es importante notar que la activación que se utiliza es una función sigmoide, pues los valores de la compuertas van entre 0 y 1, de tal forma que los valores cercanos a 1 dejan pasar la información, mientras que los valores cercanos a 0 la inhiben.

Con la compuerta reset se propone lo que se conoce como un candidato, lo que es un vector que contiene la posible información que se enviará hacia los estados siguientes de la secuencia.

Definición 105 (Candidato). *Dentro de una capa GRU un candidato es un posible vector de información $\hat{h}^{(t)}$ que depende del tiempo t construido de forma recurrente utilizando la compuerta reset como:*

$$\hat{h}^{(t)} = \tanh \left(W(r^{(t)} \odot h^{(t-1)}) + Ux^{(t)} + b \right)$$

Como mencionábamos, el hecho de que la compuerta reset contenga valores entre 0 y 1 pondera los valores del estado anterior $h^{(t-1)}$, de tal forma que si la información es relevante los valores de la compuerta reset serán cercanos a 1 para dejar pasar a los siguientes estados esta información, mientras que la información poco relevante se anulará con valores cercanos a 0 en la compuerta reset.

Finalmente, el estado actual de la capa se estima usando la puerta de actualización $z^{(t)}$ que, al usar la función de activación sigmoide, puede entenderse como un vector de probabilidades de neuronas (independientes). La capa actual se obtiene ponderando la información del candidato por estas probabilidades; además, se suma la información de la capa anterior ponderado por el inverso de la probabilidad, es decir por $1 - z^{(t)}$.

Definición 106 (Capa recurrente GRU). *Una capa recurrente GRU es una capa dentro de una red recurrente que obtiene sus estados dependientes del tiempo $t \in \{1, 2, \dots, T\}$ a partir de la compuerta de actualización $z^{(t)}$ y de un vector candidato $\hat{h}^{(t)}$ como una suma convexa de la forma:*

$$h^{(t)} = (1 - z^{(t)}) \odot h^{(t-1)} + z^{(t)} \odot \hat{h}^{(t)}$$

Recuérdese que $\odot$ es el producto punto a punto. Por tanto, en una capa de tipo GRU se suman la información actual (vector candidato) y la información en el estado anterior ponderados por el vector de actualización. Esto permite que la información anterior y la actual se complementen. Claramente, cuando la información actual es poco relevante, la información anterior toma mayor relevancia. Por su parte, si la información actual es relevante, la información anterior tiene menos relevancia.

En la Figura 6.10 se muestra el flujo de la información en las capas GRU. Al igual que en las capas recurrentes (vanilla) el vector de salida $h^{(t)}$ se utiliza en la capa siguiente (sea ésta una capa de salida u otra capa oculta). Las capas GRU tienen un mayor número de pesos, pues cada compuerta, así como el candidato, tiene su propio conjunto de pesos; por tanto, tenemos que calcular tres veces más pesos que en una red vanilla. A pesar de esto, las redes GRU suelen mostrar un mejor desempeño, y son utilizadas principalmente cuando se quiere mejorar los resultados de la red sin elevar el costo tanto como con las capas LSTM.

6.4.2. Long-Short Term Memory (LSTM)

Las memorias de corto y largo plazo o LSTMs (por sus siglas en inglés *Long-Short Term Memory*) fueron introducidas por Hochreiter and Schmidhuber (1997) con el objetivo de almacenar la información relevante para transmitirla a los siguientes estados de una red recurrente. Éstas se basan en la idea de transmitir información a partir de un principio de escritura. El principio fundamental de las LSTMs es asegurar la integridad del mensaje original, que se transmite de $t = 1$ hasta $t = T$ (es decir, a partir de una secuencia temporal), a partir de la idea de escritura.

Definición 107 (Principio de escritura). *El principio de escritura se entiende como un cambio incremental aditivo que no se modifica por interferencia externa. Bajo este principio, las capas de recurrencia $h^{(t)}$ dependientes del tiempo $t \in \{1, \dots, T\}$ se determinan como:*

$$h^{(t)} = f(h^{(t-1)}, x^{(t)}) = h^{(t-1)} + \Delta h^{(t)}$$

Donde $h^{(t)}$ es el estado actual, $h^{(t-1)}$ es la información anterior y $\Delta h^{(t)}$ representa el cambio en la información actual.

El principio de escritura simplemente nos dice que el estado actual de una capa recurrente se debe determinar por una suma del estado anterior y los cambios del estado actual. De hecho, se puede comprobar que las capas de tipo GRU cumplen el principio de escritura, tomando como el cambio actual el candidato ponderado por el vector de probabilidades de actualización.

Al igual que en el caso de GRU, las capas de LSTM incorporan la idea de compuertas para poder asegurar el principio de escritura; esto es, las compuertas se aseguran de que la información relevante se transmita de manera adecuada a los siguientes estados de la capa.

Definición 108 (Compuertas en capa LSTM). *Las compuertas de una capa LSTM son vectores que controlan el flujo de información entre los estados de la capa. En una capa LSTM se utilizan tres tipos de compuertas:*

1. **Escritura:** Determina que información es relevante para escribir:

$$i^{(t)} = \sigma(W^i h^{(t-1)} + U^i x^{(t)} + b^i)$$

2. **Lectura:** Determina lo más informativo del estado anterior:

$$o^{(t)} = \sigma(W^o h^{(t-1)} + U^o x^{(t)} + b^o)$$

3. **Olvido:** Determina la información que se debe olvidar o borrar:

$$f^{(t)} = \sigma(W^f h^{(t-1)} + U^f x^{(t)} + b^f)$$

Al igual que con las compuertas de una capa GRU, las compuertas en LSTMs utilizan la función sigmoide como función de activación, de tal forma que las compuertas sean vectores con entradas entre 0 y 1; estos valores se encargan de dejar pasar o anular la información: entre más cercano a 0 la información se irá olvidando, mientras que entre más cercano a 1 la información se mantendrá transmitiéndose. Asimismo, cada compuerta está definida en forma recurrente y cuenta con su propio conjunto de pesos.

El nombre que le hemos dado a cada compuerta está relacionado con su aplicación dentro de la capa de LSTM. La compuerta $o^{(t)}$ se encarga de leer ‘leer’ el estado interno actual de la capa. De tal forma que antes de enviar la información a una salida (o justamente viene de *output*), este estado se lee para asegurar la integridad de la información que se transmite hacia la salida de la capa. Por tanto, la salida de la capa de LSTM será:

$$h^{(t)} = o^{(t)} \odot \tanh(c^{(t)})$$

El vector $c^{(t)}$ es el vector que representa el estado interno de la capa, a veces llamado contexto; es decir, es aquel vector que se encarga de almacenar la información íntegra de la capa LSTM a nivel interno. Este vector únicamente será transmitido a los diferentes estados dentro de la misma capa de LSTM. Lo primero que se hace es escalar este estado interno por medio de la función de tangente hiperbólica; posteriormente, se multiplica punto a punto por la compuerta de lectura. Este último producto hace que en la salida sólo prevalezca aquella información relevante para resolver el problema a tratar.

Cabe señalar que todo vector $h^{(t)}$ para todo tiempo t desde 1 hasta la longitud de la secuencia es de la forma anteriormente señalada. De tal forma, que las compuertas, en donde aparece $h^{(t-1)} = o^{(t-1)} \odot \tanh(c^{(t-1)})$ que depende de las compuertas y el estado interno de los estados anteriores. De tal forma que cada vez que se hace referencia a $h^{(t-1)}$ en las puertas de escritura, olvido y lectura se asume que estas ya han sido ‘leídas’.

Finalmente, para obtener los valores del estado interno $c^{(t)}$ en cada estado, estos se obtienen en dos pasos: primero se obtiene un candidato de escritura que propondrá la información que es relevante para escribirse. Este candidato está determinado de forma similar a las capas en una red recurrente (que es también la forma de las compuertas) y cumple la misma función que el candidato en las capas GRU, es decir, contener la información que se considera relevante enviar a los estados subsecuentes de la capa.

Definición 109 (Candidato en capa LSTM). *Un candidato dentro de una capa LSTM es un vector $\hat{c}^{(t)} \in \mathbb{R}^m$ que contiene información relevante para el contexto de los estados subsecuentes, y que se define como:*

$$\hat{c}^{(t)} = \tanh(W h^{(t-1)} + U x^{(t)} + b)$$

Este candidato suele ser similar al de las capas GRUs; recordemos que el vector $h^{(t-1)}$ está ponderado por la compuerta $o^{(t)}$ (que cumple una función similar al reset en GRU). La función de activación suele ser la tangente hiperbólica, aunque no necesariamente. El segundo paso consiste en obtener los valores de c a partir de utilizar las puertas de olvido y escritura en base a la función de escritura:

$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \hat{c}^{(t)}$$

Como se puede observar, la parte $f^{(t)} \odot c^{(t-1)}$ denota la información anterior y pasa por la puerta de olvido para borrar aquella información que no es relevante. Por otro lado, $\Delta h^{(t)} = i^{(t)} \odot \hat{c}^{(t)}$ es la información nueva que se va a escribir, y que pasa por la puerta de escritura para enviar la información relevante al estado siguiente. Por tanto, la capa de LSTM representa la idea de escritura como el cambio incremental aditivo, donde a un estado anterior (pasando por un “borrado” de información por medio de la compuerta de olvido) se agrega información nueva. Finalmente, podemos definir una capa LSTM como sigue:

Definición 110 (LSTM). *Una capa LSTM es una capa recurrente que utiliza las compuertas de escritura, lectura y olvido para obtener salidas dependientes del tiempo $t \in \{1, 2, \dots, T\}$ de la forma:*

$$h^{(t)} = o^{(t)} \odot \tanh(c^{(t)})$$

Donde el estado interno o contexto $c^{(t)}$ se estima en base al principio de escritura:

$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \hat{c}^{(t)}$$

En la Figura 6.11 se muestra la estructura de una capa LSTM. Un estado t de la LSTM toma una entrada $x^{(t)}$, un estado interno anterior $c^{(t-1)}$ y la salida del estado anterior $h^{(t-1)}$. Las casillas con los valores σ implican el cálculo de funciones recurrentes con activación sigmoide; lo mismo pasa con la casilla con símbolo de tangente hiperbólica. Los nodos con símbolos $\times$ y $+$ implican que los vectores cuyas aristas se cruzan se multiplican o suman, respectivamente. Por ejemplo, el símbolo $\times$ que aparece ligado a $c^{(t-1)}$ y $f^{(t)}$ implica que se realiza el producto $f^{(t)} \odot c^{(t-1)}$ para después sumarse con $i^{(t)} \odot \hat{c}^{(t)}$.

Las capas LSTM presentan una ventaja con respecto a las capas vanilla en cuanto permiten que la información se transmita de manera más adecuada a través de los diferentes estados a pesar de la longitud de la cadena. Sin embargo, requieren de un mayor número de pesos que estimar, lo que puede hacer más costosa su implementación. Como mencionábamos, las GRU surgieron posteriormente como una forma de simplificar la complejidad de las LSTM. Ambas capas con memoria son utilizadas ampliamente, y comparten similitudes como se muestra en Chung (2014), en este trabajo se puede profundizar más entre las diferencias y similitudes entre estos dos tipos de capas recurrentes.

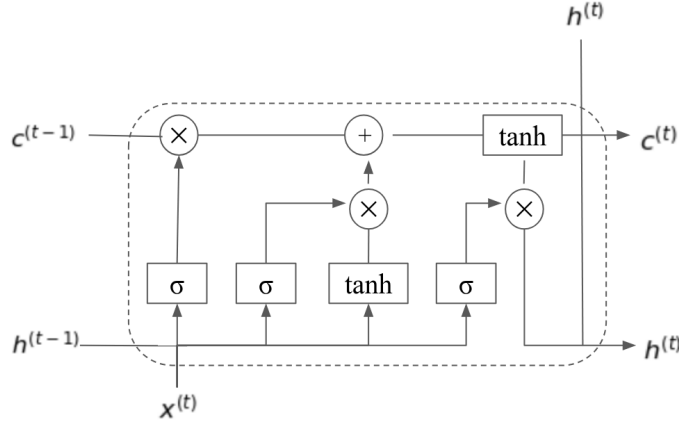


Figura 6.11: Capa recurrente con LSTM

Aprendizaje en LSTMs

La aplicación del método de retro-propagación en las capas recurrentes de tipo LSTM requiere de una función objetivo que muestre la dependencia en el tiempo de las salidas. Esta función objetivo es, por tanto, de la misma forma que en las redes recurrentes vanilla (Definición 102). Repetimos la función objetivo:

$$R(\theta) = \sum_t R_t(\theta) = \sum_t \sum_y L_t(y^{(t)}, f^{(t)}(\mathbf{x}))$$

La derivación en las capas de LSTM requiere de la actualización de un mayor número de pesos, ya que cada compuerta, así como el vector del candidato a escritura, contienen matrices de pesos independientes. Para poder derivar adecuadamente, debemos poner atención en el flujo de las funciones dentro de la capa recurrente. La capa más externa (la más cercana a la salida de la red) corresponde a $h^{(t)}$, función que a su vez depende de $o^{(t)}$. Por tanto, podemos enfocarnos en la derivación de los pesos de esta compuerta, que son W^o, U^o y b^o .

Proposición 22. Dada una capa recurrente de tipo LSTM con salida $h^{(t)}$, vector de contexto $c^{(t)}$ y compuerta de lectura $o^{(t)}$, la derivada en la compuerta de lectura está dada como:

$$\frac{\partial h_i^{(t)}}{\partial W_{i,j}^o} = \tanh(c_i^{(t)}) \sum_{k=1}^t \left(\prod_{l=k}^{t-1} \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) o_i^{(k)} (1 - o_i^{(k)}) h_j^{(k-1)}$$

para los pesos en la recurrencia y de la forma:

$$\frac{\partial h_i^{(t)}}{\partial U_{i,j}^o} = \tanh(c_i^{(t)}) o_i^{(t)} (1 - o_i^{(t)}) x_j^{(t)}$$

para los pesos sobre la capa anterior, donde $x^{(t)}$ es la entrada de la capa LSTM.

Demostración. Los pesos para la compuerta de lectura están dados por las matrices que hemos denotado como W^o y U^o . Para el caso de los pesos de las recurrencias, la matriz W^o , debemos observar que tenemos la siguiente derivación:

$$\frac{\partial L_t}{\partial W_{i,j}^o} = \sum_y \frac{\partial L_t}{\partial f_y} \frac{\partial f_y}{\partial h_i^{(t)}} \frac{\partial h_i^{(t)}}{\partial W_{i,j}^o}$$

Lo primero que debemos observar es que el factor $\sum_y \frac{\partial L_t}{\partial f_y} \frac{\partial f_y}{\partial h_i^{(t)}}$ corresponde a la retro-propagación en las capas superiores de la red, por lo que ahora no nos interesa. Considerando que la función de activación en la compuerta $o^{(t)}$ es sigmoide, y considerando el Lema 2, podemos enfocarnos en la última parte de la derivada con respecto a un tiempo fijo t ; tenemos que:

$$\begin{aligned} \frac{\partial h_i^{(t)}}{\partial W_{i,j}^o} &= \tanh(c_i^{(t)}) \frac{\partial o_i^{(t)}}{\partial W_{i,j}^o} \\ &= \tanh(c_i^{(t)}) \sum_{k=1}^t \left(\prod_{l=k}^{t-1} \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) o_i^{(k)} (1 - o_i^{(k)}) h_j^{(k-1)} \end{aligned}$$

Por su parte, para los pesos en la matriz U^o , la derivada es similar, sólo cambiando el último término por el vector de entrada, el cual denotamos como x . Ya que aquí no hay recurrencias el resultado se puede deducir fácilmente. □

Las derivadas para las compuertas de $f^{(t)}$, $i^{(t)}$ y $\hat{c}^{(t)}$ deben pasar por el cálculo de $c^{(t)}$. Su derivadas son similares, y en estos casos, dado que se suman con compuertas en estados previos, debemos considerar la retro-propagación en los tiempos anteriores. Consideremos, en primer lugar, las derivadas sobre la compuerta $f^{(t)}$.

Proposición 23. *Dada una capa recurrente de tipo LSTM con salida $h^{(t)}$, vector de contexto $c^{(t)}$ y compuertas $i^{(t)}$, $o^{(t)}$ y $f^{(t)}$, la derivada en la compuerta de olvido $f^{(t)}$ está dada como:*

$$\frac{\partial h_i^{(t)}}{\partial W_{i,j}^f} = o_i^{(t)} (1 - \tanh^2(c_i^{(t)})) c_i^{(t-1)} \sum_{k=1}^t \left(\prod_{l=k}^{t-1} \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) f_i^{(k)} (1 - f_i^{(k)}) h_j^{(k-1)}$$

para los pesos en la recurrencia y de la forma:

$$\frac{\partial h_i^{(t)}}{\partial U_{i,j}^f} = o_i^{(t)} (1 - \tanh^2(c_i^{(t)})) c_i^{(t-1)} f_i^{(t)} (1 - f_i^{(t)}) x_j^{(t)}$$

para los pesos sobre la capa anterior, donde $x^{(t)}$ es la entrada de la capa LSTM.

Demostración. Para esta compuerta debemos primero obtener la derivada sobre $h^{(t)} = o^{(t)} \odot \tanh(c^{(t)})$ sobre $c^{(t)}$, pues es aquí donde se ocupan tanto $f^{(t)}$ como $i^{(t)}$ y $\hat{c}^{(t)}$.

Tenemos entonces que:

$$\begin{aligned}\frac{\partial h^{(t)}}{\partial c^{(t)}} &= \frac{\partial o^{(t)} \odot \tanh(c^{(t)})}{\partial c^{(t)}} \\ &= o^{(t)} \odot (1 - \tanh^2(c^{(t)}))\end{aligned}$$

El factor $o^{(t)}$ no depende de la compuerta de olvido (ni la de entrada), por tanto se considera constante en esta derivación. Ahora, utilizando el Lema 2 y la derivada de la función sigmoide que activa la compuerta de olvido, tenemos que:

$$\begin{aligned}\frac{\partial h_i^{(t)}}{\partial W_{i,j}^f} &= \frac{\partial h_i^{(t)}}{c_i^{(t)}} \frac{\partial c_i^{(t)}}{\partial f^{(t)}_i} \frac{\partial f_i^{(t)}}{\partial W_{i,j}^f} \\ &= o_i^{(t)} (1 - \tanh^2(c_i^{(t)})) c_i^{(t-1)} \sum_{k=1}^t \left(\prod_{l=k}^{t-1} \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) f_i^{(k)} (1 - f_i^{(k)}) h_j^{(k-1)}\end{aligned}$$

Como $f^{(t)}$ multiplica punto a punto a $c^{(t-1)}$ este factor permanece como constante. Al igual que en el caso anterior, para derivar sobre $U_{i,j}^f$ sólo se sustituye $h_j^{(t-1)}$ por $x_j^{(t)}$. $\square$

El gradiente para la compuerta de escritura $i^{(t)}$ es muy similar al anterior, con la única diferencia que esta compuerta multiplica al candidato $\hat{c}^{(t)}$ y no a los estados anteriores. Por tanto, el siguiente resultado es inmediato.

Proposición 24. *Dada una capa recurrente de tipo LSTM con salida $h^{(t)}$, vector de contexto $c^{(t)}$ y compuertas $i^{(t)}$, $o^{(t)}$ y $f^{(t)}$, la derivada en la compuerta de lectura $i^{(t)}$ está dada como:*

$$\begin{aligned}\frac{\partial h_i^{(t)}}{\partial W_{i,j}^i} &= \frac{\partial h_i^{(t)}}{c_i^{(t)}} \frac{\partial c_i^{(t)}}{\partial i^{(t)}_i} \frac{\partial i_i^{(t)}}{\partial W_{i,j}^i} \\ &= o_i^{(t)} (1 - \tanh^2(c_i^{(t)})) \hat{c}_i^{(t)} \sum_{k=1}^t \left(\prod_{l=k}^{t-1} \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) i_i^{(k)} (1 - i_i^{(k)}) h_j^{(k-1)}\end{aligned}$$

para los pesos en la recurrencia y de la forma:

$$\frac{\partial h_i^{(t)}}{\partial U_{i,j}^i} = o_i^{(t)} (1 - \tanh^2(c_i^{(t)})) \hat{c}_i^{(t-1)} i_i^{(t)} (1 - i_i^{(t)}) x_j^{(t)}$$

para los pesos sobre la capa anterior, donde $x^{(t)}$ es la entrada de la capa LSTM.

Ya que ambas compuertas, olvido y lectura, se activan con sigmoide, las derivadas de estas son similares, sólo variando las compuertas. Asimismo, la compuerta por la que multiplican también varían, pues $f^{(t)}$ multiplica a $c^{(t-1)}$, mientras que $i^{(t)}$ multiplica a $\hat{c}^{(t)}$. De igual forma, para derivar sobre $U_{i,j}$ se debe cambiar el último producto $h_j^{(t-1)}$ por $x_j^{(t)}$.

Por último, queda obtener la derivada sobre el candidato a escritura $\hat{c}^{(t)}$. Esta compuerta también se utiliza en la ecuación del principio de escritura multiplicando punto

a punto a la compuerta de escritura, por lo que sus derivadas son prácticamente similares, aunque el candidato usa activación tangente hiperbólica y no sigmoide, por lo que esta derivación también varía. El siguiente resultado se puede obtener fácilmente.

Proposición 25. *Dada una capa recurrente de tipo LSTM con salida $h^{(t)}$, vector de contexto $c^{(t)}$ y compuertas $i^{(t)}$, $o^{(t)}$ y $f^{(t)}$, la derivada en el vector de candidato $\hat{c}^{(t)}$ está dada como:*

$$\begin{aligned} \frac{\partial h_i^{(t)}}{\partial W_{i,j}^c} &= \frac{\partial h_i^{(t)}}{c_i^{(t)}} \frac{\partial c_i^{(t)}}{\partial \hat{c}^{(t)} i} \frac{\partial \hat{c}_i^{(t)}}{\partial W_{i,j}^c} \\ &= o_i^{(t)} (1 - \tanh^2(c_i^{(t)})) i_i^{(t)} \sum_{k=1}^t \left(\prod_{l=k}^{t-1} \frac{\partial h^{(l+1)}}{\partial h^{(l)}} \right) (1 - (\hat{c}_i^{(k)})^2) h_j^{(k-1)} \end{aligned}$$

para los pesos en la recurrencia y de la forma:

$$\frac{\partial h_i^{(t)}}{\partial U_{i,j}^c} = o_i^{(t)} (1 - \tanh^2(c_i^{(t)})) i_i^{(t)} (1 - (\hat{c}_i^{(t)})^2) x_j^{(t)}$$

para los pesos sobre la capa anterior, donde $x^{(t)}$ es la entrada de la capa LSTM.

Podemos notar que el término $o_i^{(t)} (1 - \tanh^2(c_i^{(t)}))$ se repite en las compuertas $f^{(t)}$, $i^{(t)}$ y $\hat{c}^{(t)}$, por lo que se puede crear una variable para almacenar este valor en el BPTT. Asimismo, las derivadas sobre las recurrencias en cada tiempo se puede acumular en cada tiempo t en la implementación. Por lo que para obtener las derivadas sobre cada compuerta sólo bastaría calcular las derivadas de las funciones de activación con los valores numéricos correspondientes a éstas. Su implementación se puede pensar también como una gráfica computacional basada en el esquema de la Figura 6.11. Más aún, cuando implementamos una capa de tipo LSTM en una red profunda, una vez que podemos obtener la derivada sobre los pesos de la capa, toda la capa puede verse como un nodo más de la gráfica computacional correspondiente a la red neuronal completa. Esto nos permite aprender de forma eficiente en estos modelos.

Ejercicios

1. Demuestra que en una red bidireccional con una salida dada por la función softmax, las probabilidades de avance y retroceso son independientes, esto es:

$$\text{Softmax}(V[\vec{h}^{(t)}; \overleftarrow{h}^{(t)} + c]) = p(y^{(t)} | y^{(1)} \dots y^{(t-1)}, \mathbf{x}) p(y^{(t)} | y^{(t+1)} \dots y^{(T_y)}, \mathbf{x})$$

2. Demuestra que si bajo redes one-to-one (que procesan secuencias de longitud 1), el método de retro-propagación a través del tiempo BPTT, es equivalente al método de retro-propagación estándar.
3. Utiliza el algoritmo de BPTT para estimar los gradientes de una red con 2 capas ocultas, donde la segunda es recurrente, dado de la forma:

- a) $a^{(1,t)} = U^{(1)}x^{(t)} + b^{(1)}$ para toda $t \in \{1, 2, \dots, T\}$.
- b) $h^{(1,t)} = \sigma(a^{(1,t)})$ para toda $t \in \{1, 2, \dots, T\}$.
- c) $\hat{a}^{(2,t)} = Wh^{(t-1)} + Uh^{(1,t)} + b$ para toda $t \in \{1, 2, \dots, T\}$.
- d) $h^{(2,t)} = \tanh(\hat{a}^{(2,t)})$ para toda $t \in \{1, 2, \dots, T\}$.
- e) $f^{(t)} = \text{Softmax}(Vh^{(2,t)} + c)$ para toda $t \in \{1, 2, \dots, T\}$.
- f) Función de riesgo de entropía cruzada: $R(\theta) = -\sum_t \sum_i \delta_{i,x_t} \ln f_i^{(t)}(\mathbf{x})$.

4. Calcula la forma de los gradientes para cada peso dentro de las capas GRU.
5. Desarrolla de manera explícita la derivación para obtener las derivadas parciales de los pesos en el vector de candidato en una capa LSTM.
6. Elige un problema de clasificación secuencial (por ejemplo, etiquetado de palabras) e implementa una red recurrente vanilla, una de capa GRU y otra con LSTM. Compara el desempeño de estas redes.

Capítulo 7

Redes convolucionales

Como hemos visto en el capítulo anterior, las redes recurrentes se relacionan con las redes feedforward, en tanto las redes feedforward pueden verse como un caso particular de las primeras cuando se trabaja con secuencias de un sólo elemento. En ambos casos, las capas de la red en cada capa conectan todas su neuronas con las neuronas de la siguiente capa. Es por esto que a las capas de tipo feedforward se les llama a veces completamente conectadas o densas. Estas redes asumen que cada neurona en una capa oculta requiere de toda la información de la capa previa.

Sin embargo, en muchos problemas es importante considerar la información relevante de las capas anteriores y que esta información responda al problema a trabajar. Un ejemplo claro es el procesamiento de imágenes. Si tomamos una imagen cuyos píxeles pueden considerarse como valores numéricos que pasan a una neurona en la capa oculta, el hacer que la neurona tome toda a información directamente puede hacer que la red neuronal pierda la capacidad de generalizar ante transformaciones de estos datos, como una traslación o una rotación, ya que la neurona de la capa oculta se puede especializar en una característica particular, dando mayor peso a esta característica, pero cuando se aplica una transformación, como una traslación, a la imagen, esta característica puede desplazarse a otra posición en las neuronas de entrada, perdiendo así información relevante sobre la imagen.

Para solventar esta problemática, que no es particular a las imágenes, se ha propuesto la arquitectura de red convolucional (LeCun et al., 1998) que revisamos en este capítulo. Se trata de una arquitectura que resalta información local para enviar a las unidades ocultas. Para revisar el funcionamiento de las capas convolucionales revisamos, en primer lugar, el concepto de convolución y de correlación cruzada. Posteriormente definimos las capas convolucionales y de pooling, para finalmente hablar sobre el entrenamiento en este tipo de capas.

7.1. Convolución y correlación cruzada

Las redes convolucionales toman su nombre del concepto de convolución de funciones. Una convolución es un operador que toma una función y arroja una nueva función

en base a una medida, función o filtro (*kernel*). La convolución, entonces, toma dos funciones f y g para construir una tercera función que se suele denotar como $f * g$. La función resultante trata de reflejar el cómo la ‘forma’ de g es afectada por la de f . Esto nos servirá principalmente para darnos información acerca de la función g en términos de la función f (la que llamaremos el *kernel*).

Definición 111 (Convolución). *Una convolución es un operador sobre un espacio funcional $*$: $\mathbb{R}^{\mathbb{R}} \times \mathbb{R}^{\mathbb{R}} \rightarrow \mathbb{R}^{\mathbb{R}}$, donde $\mathbb{R}^{\mathbb{R}}$ denota el espacio de funciones de los reales hacia los reales, que se define como:*

$$(f * g)(x) = \int_{-\infty}^{\infty} f(t)g(x-t) dt$$

Llamamos a la función $f: \mathbb{R} \rightarrow \mathbb{R}$ el *kernel* o *filtro* de la convolución.

Ejemplo 19. Supongamos que tenemos una función principal $g \equiv \sigma$ dada por la función sigmoide que, ya conocemos, es de la forma $\sigma(x) = \frac{1}{1+e^{-x}}$. Si utilizamos como función *kernel* a la función escalonada $1_{>0}$ que toma el valor 1 cuando la entrada es positiva y 0 en otro caso, tenemos el siguiente resultado:

Proposición 26. Sea $\sigma(x)$ la función sigmoide y $1_{>0}(x)$ la función indicadora para valores positivos. Entonces, la convolución entre ambas es la función *softplus*; es decir:

$$(1_{>0} * \sigma)(x) = \ln(1 + e^x)$$

Demostración. Para demostrar el resultado de la convolución desarrollemos ésta con base en la definición. Tenemos:

$$\begin{aligned} (1_{>0} * \sigma)(x) &= \int_{-\infty}^{\infty} 1_{>0}(t) \sigma(x-t) dt \\ &= \int_{-\infty}^0 0 \sigma(x-t) dt + \int_0^{\infty} 1 \sigma(x-t) dt \\ &= \int_0^{\infty} \sigma(x-t) dt \\ &= \int_0^{\infty} \frac{1}{1 + e^{-x+t}} dt \end{aligned}$$

Podemos hacer un cambio de variable en la integración tomando $u = x - t$, por lo que tendremos que:

$$\begin{aligned} (1_{>0} * \sigma)(x) &= \int_{-\infty}^x \frac{1}{1 + e^u} du \\ &= \ln(1 + e^x) - \lim_{u \rightarrow -\infty} \ln(1 + e^u) \\ &= \ln(1 + e^x) \end{aligned}$$

El resultado es claro en tanto $\lim_{u \rightarrow -\infty} \ln(1 + e^u) = 0$. □

En la Figura 7.1 se puede ver gráficamente las dos funciones originales y el resultado de la convolución que es la función *softplus*. Este resultado es particularmente

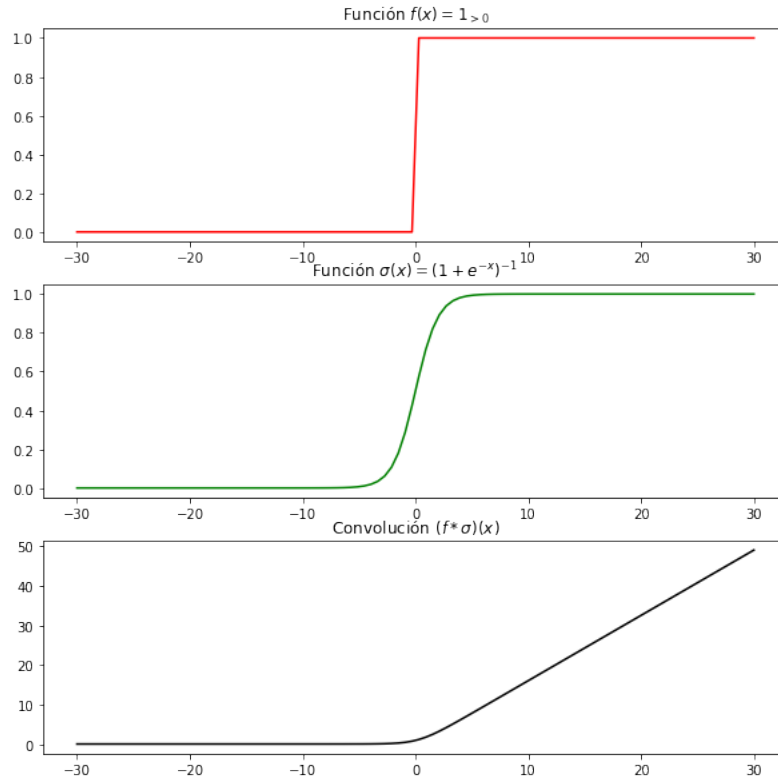


Figura 7.1: Convolución de las funciones escalón y sigmoide

interesante pues las funciones aquí implicadas se utilizan como activaciones de neuronas. También a partir de este resultado es fácil ver que la derivada de la función softplus es la función sigmoide.

Un operador común, similar a la convolución, es la llamada correlación cruzada, la cual tiene muchas similitudes con el operador de convolución, pues también busca expresar una función g en términos de una función kernel f , solamente que utiliza una suma de valores en lugar de la resta de la convolución.

Definición 112 (Correlación cruzada). Sean $f, g : \mathbb{R} \rightarrow \mathbb{R}$ funciones, la correlación cruzada entre ambas funciones está determinada como:

$$(f \star g)(x) = \int_{-\infty}^{\infty} f(t)g(x+t) dt$$

Decimos que $f : \mathbb{R} \rightarrow \mathbb{R}$ es la función kernel o el filtro.

Como se puede observar de la definición, la única diferencia con la convolución es el hecho de que la función g se aplica sobre la suma de los valores x y t y no la diferencia. Los resultados obtenidos con la convolución y con la correlación cruzada no

son iguales, pero ambos se pueden interpretar como una forma de obtener información de g en términos del kernel f .

Ejemplo 20. Consideremos el mismo ejemplo del caso anterior, tomando la función sigmoide $\sigma(x)$ y la función indicadora $1_{>0}(x)$. Tenemos el siguiente resultado:

Proposición 27. Sea $\sigma(x)$ la función sigmoide y $1_{>0}(x)$ la función indicadora para valores positivos. Entonces, la correlación cruzada entre ambas funciones se da como:

$$(1_{>0} \star \sigma)(x) = \ln(1 + e^{-x})$$

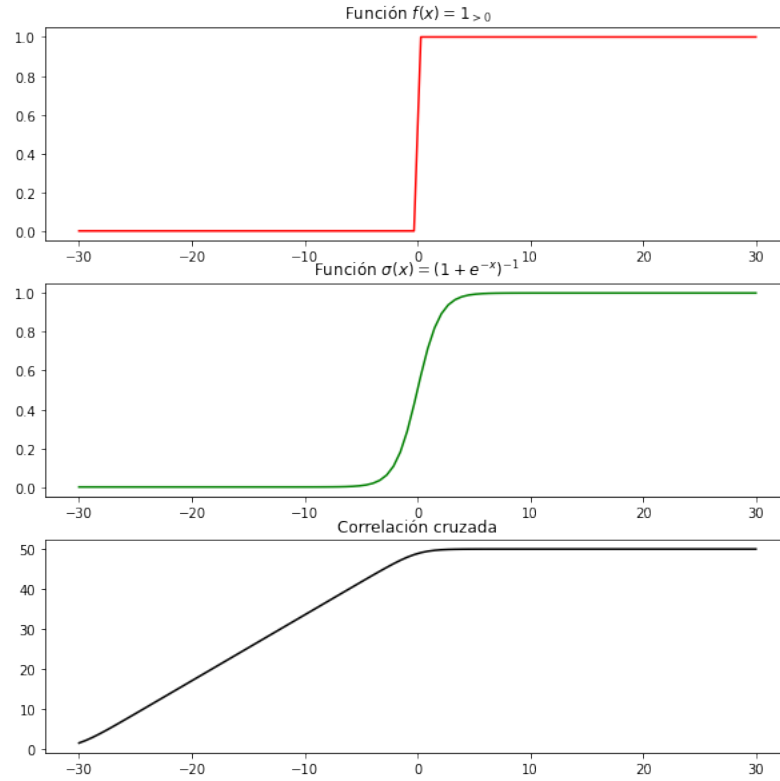


Figura 7.2: Correlación cruzada de las funciones escalón y sigmoide

En este caso, la función se integra sobre la suma de $x + t$ y no sobre la resta como en el caso de la convolución. La correlación cruzada se muestra en la Figura 7.2. El resultado de la convolución y la correlación cruzada son muy similares excepto por un cambio de signos.

Para los intereses del aprendizaje profundo, más que funciones continuas nos interesa aplicar una correlación cruzada o convolución sobre funciones definidas en dominios discretos (y comúnmente finitos). Podemos definir la correlación cruzada en términos discretos.

Definición 113 (Correlación cruzada discreta). Sean $f, g : \mathbb{Z} \rightarrow \mathbb{R}$ dos funciones con dominio discreto, la correlación cruzada (discreta) entre estas funciones está dada como:

$$(f \star g)(x) = \sum_{n=-\infty}^{\infty} f(n)g(x+n)$$

De forma general, la correlación cruzada se realiza sobre los valores negativos y positivos hasta infinito. Pero si el dominio está acotado, podemos tomar valores de inicio y término de la suma también acotados, permitiéndonos obtener la correlación cruzada de forma finita.

En las redes neuronales, de hecho, se utilizan funciones de correlación cruzada y no de convoluciones. Asimismo, ya que trabajamos con valores discretos, en las redes neuronales utilizaremos la correlación cruzada discreta y acotada sobre los valores de los datos de entrada. En lo que sigue nos referiremos a convoluciones (y capas convolucionales) cuando hablemos de la aplicación de operadores de correlación cruzada en las redes neuronales; esto con base a la terminología estándar dentro del área.

7.2. Capas convolucionales

Como lo hemos mencionado anteriormente, propiamente las capas convolucionales utilizan correlación cruzada para capturar información de los datos de entrada en base a las relaciones determinadas por un kernel. Las capas convolucionales por tanto aplicarán un operador de correlación cruzada entre los datos de entrada y un kernel, ambos determinados de manera discreta y finita. Generalmente, una capa convolucional típica toma como entrada una matriz, por ejemplo una imagen. En la correlación cruzada tenemos que $(f \star g)(x) = \int f(t)g(x+t)dt$. Si discretizamos esta fórmula obtendremos que:

$$(f \star g)(x) = \sum_t f(t)g(x+t)$$

Como mencionamos, la entrada es una matriz, por lo que en este caso, podemos ver a g como dicha matriz de entrada. Las matrices pueden verse como funciones sobre los números naturales. Supongamos el caso donde $x = (i, j)$ con $i, j \in \mathbb{N}$, es decir, los argumentos de la función son los pares (i, j) que determinan las coordenadas de la matriz, el renglón y la columna respectivamente. De igual forma, podemos pensar al kernel f como una función que toma pares de coordenadas como argumentos, esto es que $t = (l, l')$; por tanto, el kernel también será una matriz. Podremos reescribir la fórmula de la convolución de la siguiente forma:

$$(f \star g)(i, j) = \sum_l \sum_{l'} f(l, l')g(i+l, j+l')$$

Donde la doble suma surge del hecho de que las funciones toman dos argumentos de entrada y l, l' corren dentro del dominio de estas funciones, en particular dentro del dominio del kernel. Si tomamos f como un kernel donde los argumentos responden a las entradas de la matriz, $f(l, l') = W_{l, l'}$, y a g como la entrada donde de igual forma los argumentos responden a las entradas de la matriz, $g(i, j) = x_{i, j}$, entonces tenemos como resultado lo que en redes se conoce como convolución.

Definición 114 (Capa convolucional). *Una capa convolucional en una red neuronal es una capa que aplica un operador de convolución (correlación cruzada) finita a una matriz de entrada $x \in \mathbb{R}^{H' \times L'}$, cuyo resultado es una matriz cuyas entradas son de la forma:*

$$h_{i,j} = g(W \star x)_{i,j} = g\left(\sum_{l=0}^H \sum_{l'=0}^L W_{l,l'} x_{i+l,j+l'} + b\right)$$

Donde los pesos de la capa son $W \in \mathbb{R}^{H \times L}$ y b (el bias), tal que H y L son la altura y anchura del kernel. Finalmente, $g : \mathbb{R}$ es una función de activación.

En la notación de la definición, queda claro que nos referimos a la pre-activación en una capa convolucional como la aplicación de la convolución sin la función de activación. Esto es, la pre-activación será de la forma:

$$a_{i,j} = (W \star x)_{i,j} = \sum_{l=0}^H \sum_{l'=0}^L W_{l,l'} x_{i+l,j+l'} + b$$

La activación que se ocupa en las capas convolucionales suele ser la activación ReLU, pero puede ser cualquier función de activación que nos ayude a resolver el problema. Una red convolucional es una red que cuenta con una o más capas convolucionales que incluyen una operación de convolución.

Ejemplo 21. *Para ejemplificar como funciona una capa convolucional, tomemos como ejemplo un problema en donde los datos de entrada son matrices de 9×9 y tomemos un kernel de altura y anchura ambas igual a 3. Definamos nuestro kernel de la siguiente manera.*

$$W = \begin{pmatrix} 1 & -1 & -1 \\ -1 & 1 & -1 \\ -1 & -1 & 1 \end{pmatrix}$$

Por simplicidad, asumiremos que $b = 0$. Este kernel, se puede visualizar como se muestra en la Figura 7.3. Es decir, esta representando una diagonal. Ahora, tomemos un

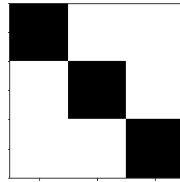


Figura 7.3: Visualización del kernel

dato de entrada, el cual es una matriz de 9×9 . Digamos que esta matriz está dada de

la siguiente forma:

$$x = \begin{pmatrix} -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 \\ -1 & 1 & -1 & -1 & -1 & -1 & -1 & 1 & -1 \\ -1 & -1 & 1 & -1 & -1 & -1 & 1 & -1 & -1 \\ -1 & -1 & -1 & 1 & -1 & 1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & 1 & -1 & -1 & -1 & -1 \\ -1 & -1 & 1 & -1 & -1 & 1 & -1 & -1 & -1 \\ -1 & 1 & -1 & -1 & -1 & -1 & -1 & 1 & -1 \\ -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 \end{pmatrix}$$

Esta matriz puede verse como una imagen en blanco y negro. En la parte izquierda de la Figura 7.4 se puede ver que esta matriz está definiendo una X, los valores de -1 son blancos y los valores de 1 negros. Podemos entonces comenzar a hacer la convolución entre x y el kernel W . Lo que nos dice esta operación es que tomemos el tamaño del kernel, multipliquemos por los elementos de x que lo abarcan y sumemos. Esto es, para cuando $i = 0 = j$, tenemos:

$$\begin{aligned} (W \star x)_{0,0} &= \sum_{l=0}^2 \sum_{l'=0}^2 W_{l,l'} x_{0+l,0+l'} \\ &= ((1)(-1) + (-1)(-1) + (-1)(-1)) + ((-1)(-1) + (1)(1) + (-1)(-1)) + \\ &\quad + ((-1)(-1) + (-1)(-1) + (1)(1)) \\ &= 1 + 3 + 3 = 7 \end{aligned}$$

Notemos que en este caso, hemos tomado el kernel W y hemos multiplicado por una región de la matriz de la imagen, en particular por los primeros 3 renglones y 3 columnas del extremo superior izquierdo. Para proceder con la convolución, ahora debemos movernos a la siguiente región de la imagen x , lo que correspondería a moverse hacia la derecha en un paso.

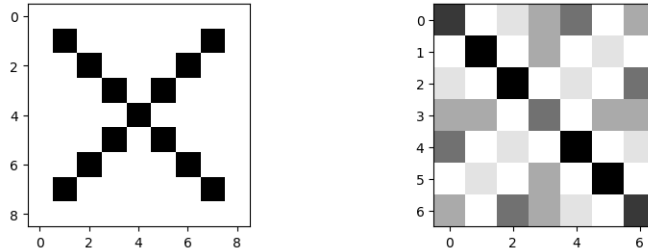


Figura 7.4: Imagen de la matriz de entrada y el resultado de aplicar la convolución

Procediendo de esta forma, el resultado es una nueva matriz que de hecho ha reducido su tamaño con respecto a la matriz de entrada, pues algunos de los píxeles en la convolución han conformado valores sin ser considerados en la nueva imagen. Por ejemplo, el píxel en la coordenada $(0,0)$ se ha usado para calcular un nuevo píxel,

pero no hay ninguno que lo reemplace. La matriz resultante es la siguiente:

$$(W \star x) = \begin{pmatrix} 7 & -1 & 1 & 3 & 5 & -1 & 3 \\ -1 & 9 & -1 & 3 & -1 & 1 & -1 \\ 1 & -1 & 9 & -3 & 1 & -1 & 5 \\ 3 & 3 & -3 & 5 & -3 & 3 & 3 \\ 5 & -1 & 1 & -3 & 9 & -1 & 1 \\ -1 & 1 & -1 & 3 & -1 & 9 & -1 \\ 3 & -1 & 5 & 3 & 1 & -1 & 7 \end{pmatrix}$$

Se trata ahora de una matriz de 7×7 con valores enteros. Esta nueva matriz también puede verse como una imagen, la cual se muestra en la parte derecha de la Figura 7.4. Se puede observar que la convolución de la imagen original con el kernel resalta la diagonal descendente de la imagen, pues justamente el kernel está pensado para hacer esto. De esta forma, la convolución con este kernel aporta información sobre esta característica de la imagen original.

Lo que hemos hecho con la capa de convolución es expresar funciones finitas por medio de matrices. Como vemos, la función de convolución en redes neuronales es similar a una la función que hemos definido en capas feedforward. Cada valor $g(W \star x)_{i,j}$ representa una neurona de la capa siguiente. Sin embargo, no se trata de conexiones densas, pues los pesos están limitados a una región específica de la entrada, determinada por la altura y anchura del kernel.

Las redes convolucionales, de hecho, reducen el número de parámetros que se aplican en la capa, pues en una capa feedforward o densa se utilizarían tantos pesos como el producto de los valores de entrada con los valores de salida, en la capa convolucional el número de pesos depende del tamaño del kernel, que es un hiperparámetro determinado por el programador. De esta forma, como hemos visto en el ejemplo, podemos tener una entrada de 9×9 (81 neuronas de entrada) y obtener una salida de 7×7 (49 unidades ocultas) con sólo 9 pesos correspondientes al kernel, en lugar de los 3969 pesos que requeriría una red densa.

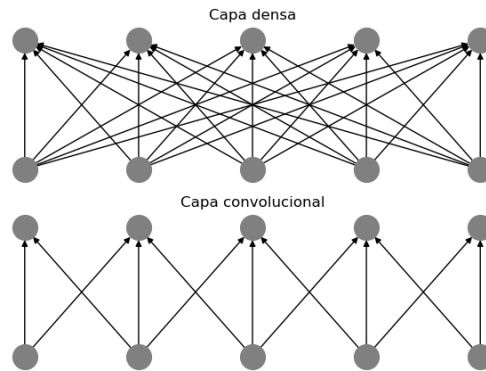


Figura 7.5: Una red densa o completamente conectada (arriba) y una red convolucional con conexiones dispersas (abajo)

Por tanto, podemos ver una capa convolucional como una capa en donde las unidades ocultas no conectan con todas las neuronas de la capa previa. A esto se le conoce como **conexión dispersa**. En la Figura 7.5 se puede ver una comparación gráfica entre una capa densa donde cada neurona de la capa previa conecta con cada unidad oculta, y la capa convolucional en donde estas conexiones son dispersas. Esto nos da una red más simple por la menor cantidad de pesos.

En el ejemplo, además, hemos notado que la aplicación de un kernel para obtener una unidad oculta $(W \star x)_{i,j}$ puede verse como multiplicar una región de la matriz de entrada x con el kernel. Es decir, para obtener una unidad oculta, tomamos el kernel y multiplicamos cada una de sus entradas por una región de la matriz x definida por el tamaño del kernel. Posteriormente, sumamos todos los valores del resultado para obtener un nuevo valor que conformará la unidad oculta. Esta idea se puede ver en la Figura 7.6; matemáticamente se puede expresar como:

$$(W \star x)_{i,j} = \sum_i^H \sum_j^L W \odot x[i : i+H, j : j+L]$$

Donde $x[i : H, j : L]$ denota la región de la matriz x acotada por el kernel y $\odot$ es el producto punto a punto. Finalmente, la suma se realiza sobre todos los valores tanto de la altura H y la anchura L de la matriz resultante.

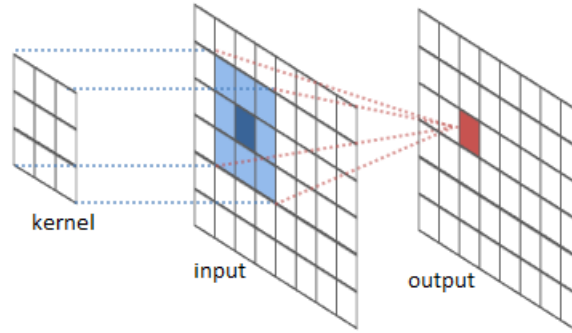


Figura 7.6: Una convolución equivale a multiplicar punto a punto el kernel por una ventana en la matriz original, para obtener así un nuevo valor de salida

La ventana tiene como centro un píxel y relaciona los otros píxeles con respecto a este como se ve en la Figura 7.6. Podemos interpretar que la convolución busca representar este píxel central a partir de su relación con los píxeles a su alrededor (los que caben en la ventana). En este caso, también las redes convolucionales toman en cuenta el **stride**, que es el número de píxeles por el que se desplaza la ventana. Por ejemplo, puede desplazarse dos píxeles, tanto a la derecha como hacia abajo, saltándose un píxel en cada caso. Lo común es que el stride sea igual a 1, pues de esta forma el desplazamiento de la ventana es píxel por píxel. Un stride más grande hará que la ventana se desplace mayores espacios, lo que generalmente resulta en una matriz nueva más pequeña.

También en el ejemplo pudimos ver que la aplicación de la convolución reduce el tamaño de la salida, esto porque hay píxeles que son ignorados para obtener la nueva imagen en la capa oculta. Para evitar que las imágenes se hagan más pequeñas conforme se aplican las capas convolucionales se suele utilizar una técnica de **padding**, que consiste simplemente en agregar 0's en la orilla de la matriz original (como formando un marco). En el ejemplo, si agregamos un padding al poner un 0 en todos los renglones tanto al inicio como al final, un marco, obtendremos que el resultado será del mismo tamaño que la imagen original. En particular tendríamos que:

$$\begin{pmatrix} 1 & -1 & -1 \\ -1 & 1 & -1 \\ -1 & -1 & 1 \end{pmatrix} \odot \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & -1 \\ 0 & -1 & 1 \end{pmatrix}$$

Donde los 0's arriba y a la izquierda definen el marco. Estos 0's anulan ciertos valores del kernel que correspondería a píxeles que no aportan información (los píxeles del marco). Aquí sólo tomamos el extremo superior izquierdo de la imagen, pero puede comprobarse fácilmente cuál es el resultado de aplicar la convolución a la imagen del ejemplo con este padding.

Como hemos señalado la convolución en redes neuronales busca conservar información relevante ante una modificación de los datos de entrada, principalmente ante las traslaciones. Las capas convolucionales se dicen equivariantes ante traslaciones; es decir, preservan la traslación aplicada a la entrada después de pasar por la capa. Para determinar esta propiedad de las capas convolucionales, damos primero una definición formal de la equivarianza.

Definición 115 (Función equivariante). *Sea T una transformación con dominio en los elementos f y sea ρ una función aplicada sobre este dominio. Decimos que ρ es equivariante si para toda f se cumple que:*

$$\rho(Tf) = T\rho(f)$$

En el caso específico de las capas convolucionales, el dominio son las matrices que representan los datos, tanto de entrada como de representación, y las transformaciones son las traslaciones. Recordemos que una traslación es una transformación que modifica las coordenadas de una entrada en una matriz. Esta transformación se puede ver como una suma en los índices:

$$Tx_{i,j} = x_{i+c_i, j+c_j}$$

Con base en esto, podemos señalar que las capas convolucionales son equivariantes ante traslaciones.

Teorema 18. *Sea W la matriz de pesos de un kernel, entonces las capas convolucionales $h_{i,j} = g(W \star x)_{i,j}$, con $x \in \mathbb{R}^{H' \times L'}$, son equivariantes ante traslaciones. Esto es:*

$$g(W \star Tx)_{i,j} = Th_{i,j}$$

Demostración. Sea W el kernel de la capa convolucional, para ver la equivarianza ante traslaciones sólo basta fijarse en la operación de convolución, pues la función de

activación se aplica punto a punto. Tenemos el siguiente desarrollo:

$$\begin{aligned}
 (W \star Tx)_{i,j} &= \sum_{l=1}^H \sum_{l'=1}^L W_{l,l'} T x_{i+l, j+l'} + b \\
 &= \sum_{l=1}^H \sum_{l'=1}^L W_{l,l'} x_{(i+c_l)+(l, (j+c_j)+l')} + b \\
 &= \sum_{l=1}^H \sum_{l'=1}^L W_{l,l'} x_{(i+l)+c_l, (j+l')+c_j} + b \\
 &= (W \star x)_{i+c_l, j+c_j} \\
 &= T(W \star x)_{i,j}
 \end{aligned}$$

Considerando la aplicación de la función de activación notamos que se obtiene $Th_{i,j}$, que es lo que buscábamos. □

Este resultado es de especial importancia, pues señala una propiedad características de las redes convolucionales que les permite trabajar adecuadamente con los datos de entrada. Por ejemplo, si tenemos una tarea de detección de objetos en imágenes, la equivarianza ante traslación nos permitirá seguir detectando los mismos objetos a pesar de que estos hayan sido trasladados en la imagen de entrada. Sin embargo, en tareas como clasificación de imágenes podríamos pensar que la red tendría que ser capaz de determinar que la imagen transformada y la original son las mismas. Para esto, introduciremos el concepto de pooling.

7.3. Pooling

Una vez que se ha obtenido la matriz de convolución y se ha aplicado la función de activación, en las capas convolucionales es común aplicar un pooling. El pooling trabaja de forma similar a una convolución, tomando regiones de la matriz h (la cual ha sido obtenida por la convolución) y aplicando una operación sobre estas regiones para obtener una nueva matriz que contenga valores representativos de la convolución anterior para que el resultado final sea localmente invariante. Es decir, queremos que después de aplicar el pooling tengamos un resultado donde las traslaciones no afecten a la representación h .

Definición 116 (Pooling). *El pooling es una estrategia para capas en una red neuronal, principalmente convolucional, que busca realizar un sub-muestreo de los datos, reduciendo la dimensión de los datos y conservando la mayor información de éstos.*

La capa de pooling realiza operaciones sobre las matrices de entrada que permiten cierto nivel de invarianza local, al mismo tiempo que reducen la dimensión de las matrices de datos; esto para hacer más eficiente el procesamiento y reducir el sobre-ajuste que pueda presentarse. Existen varias formas de realizar el pooling, las más comunes son el promedio y el máximo.

Definición 117 (Max-pooling). *El max-pooling es una estrategia de pooling que toma el valor máximo en una ventana de tamaño $m \times m$. Es decir, obtiene una representación h con entradas:*

$$h_{i,j} = \max_x \{x_{l,l'} : l = i, \dots, m; l' = j, \dots, m\}$$

Donde x es una matriz de entrada.

El max-pooling, por tanto, busca tomar las características con mayor valor dentro de una matriz de entrada. Esta matriz suele ser obtenida por medio de una convolución previa. Este máximo se toma sobre una ventana de tamaño $m \times m$ en la matriz de entrada, lo que reduce el tamaño de la matriz al tomar un sólo elemento entre los m^2 elementos originales. Otra forma de realizar el pooling es por medio de un promedio.

Definición 118 (Average-pooling). *El average-pooling es una estrategia de pooling que toma el valor promedio en una ventana de tamaño $m \times m$. Es decir, obtiene una representación h con entradas:*

$$h_{i,j} = \frac{1}{m \cdot m} \sum_{l=i}^m \sum_{l'=j}^m x_{l,l'}$$

Donde x es una matriz de entrada.

Al igual que el max-pooling, el average-pooling reduce m^2 neuronas en una sola a partir de tomar un valor promedio de los valores originales. Generalmente, el pooling toma una ventana que computa alguna de estas operaciones, pero en el caso del pooling la ventana no suele superponerse si no que se toma una partición de la matriz como se ve en la Figura 7.7 para el max-pooling.

2	1	3	4	1
5	0	0	1	2
3	0	1	7	6
0	1	0	4	0

→

5	4	2
3	7	6

Figura 7.7: El pooling toma una ventana, en este caso de 2×2 , y aplica la función, que es el máximo

En el ejemplo de la Figura 7.7 se toma un kernel de tamaño 2×2 , por lo que la matriz original se divide en regiones de este tamaño. El resultado son 6 regiones de las cuáles se elige el valor máximo, aplicando max-pooling, para generar ahora una nueva matriz con los 6 valores resultantes.

Ejemplo 22. *Para ejemplificar el uso del pooling retomemos el ejemplo anterior donde obtuvimos una matriz al aplicar una convolución a una matriz de una imagen de*

entrada. La matriz resultante fue:

$$(W \star x) = \begin{pmatrix} 7 & 0 & 1 & 3 & 5 & 0 & 3 \\ 0 & 9 & 0 & 3 & 0 & 1 & 0 \\ 1 & 0 & 9 & 0 & 1 & 0 & 5 \\ 3 & 3 & 0 & 5 & 0 & 3 & 3 \\ 5 & 0 & 1 & 0 & 9 & 0 & 1 \\ 0 & 1 & 0 & 3 & 0 & 9 & 0 \\ 3 & 0 & 5 & 3 & 1 & 7 & 0 \end{pmatrix}$$

Si elegimos el método de max-pooling con una ventana de 3×3 , tendremos como resultado una matriz de tamaño 3×3 . Recordemos que la matriz de entrada debe dividirse en sub-regiones del tamaño de la ventana. En este caso, el primer valor del max-pooling se aplicaría en la región superior izquierda de la matriz como:

$$\text{máx} \begin{pmatrix} 7 & 0 & 1 \\ 0 & 9 & 0 \\ 1 & 0 & 9 \end{pmatrix} = 9$$

En este caso, el primer valor del max-pooling es 9. Si continuamos de esta forma, el resultado será el siguiente:

$$\text{Pooling}(W \star x) = \begin{pmatrix} 9 & 5 & 5 \\ 5 & 9 & 3 \\ 5 & 7 & 0 \end{pmatrix}$$

Esta matriz sigue guardando cierta información sobre la diagonal de la imagen original en una matriz mucho más pequeña. Esto nos permitirá que la red procese esta información de forma más eficiente y preservando aquella información relevante. Puede realizarse como ejercicio este mismo ejemplo pero utilizando average-pooling.

La capa convolucional y el pooling son las dos partes esenciales de una convolución en redes neuronales. Como lo vimos, las convoluciones son equivariantes ante traslaciones, el pooling tiene sensibilidad también ante las traslaciones siendo invariante localmente ante pequeñas traslaciones. Estas propiedades permiten que la información relevante de las matrices de entrada resalte para que las decisiones de la salida sean mejores.

7.4. Redes convolucionales

Una vez que hemos revisado las convoluciones y los métodos de pooling podemos construir redes neuronales que incorporen este tipo de capas para que procesen datos de manera más efectiva. En particular, las redes convolucionales muestran un gran desempeño con imágenes como datos de entrada. Al igual que las capas densas o las capas recurrentes, las capas convolucionales se pueden aplicar de forma secuencial. Es común que una capa de este tipo siempre conlleve una convolución, una función de activación y pooling. El resultado del pooling es también una matriz que puede verse

como una representación abstracta de la imagen original con base en el kernel. Esta matriz puede pasar por otras capas convolucionales; por lo que de manera más general una capa convolucional de la red se podría definir como:

$$(W \star h^{(k-1)})_{i,j} = \sum_{l=1}^k \sum_{l'=1}^k W_{l,l'} h_{i+l,j+l'}^{(k-1)} + b$$

Donde $h^{(k-1)}$ es el resultado de aplicar la convolución, la activación y el pooling en la capa convolucional anterior. Cada capa oculta convolucional entonces será de la forma:

$$h^{(k)} = \text{Pooling}(g(W \star x))$$

Con $k = 1, \dots, K$ y $h^{(0)} = x$. Dependiendo de la tarea, pueden omitirse algunas capas de pooling. En la Figura 7.8 se puede ver el procesamiento de una imagen a través de dos capas convolucionales. Cuando se trata de imágenes, es común que cada punto esté representado por una píxel de tipo RGB (*Red, Green, Blue*) o bien RGB- α (donde α es el valor de transparencia). Es decir, se tienen que cada punto $x_{i,j}$ es un vector de 3 o 4 dimensiones. A estas dimensiones se les suele llamar **canales**. Por tanto, un dato de entrada será de la forma $x \in \mathbb{R}^{H' \times L' \times C'}$, donde H' representa las dimensiones de la altura, L' las de la anchura y C' los canales. Asimismo, el kernel será una matriz $W \in \mathbb{R}^{H \times L \times C}$. En este caso, en lugar de trabajar con matrices, trabajamos con tensores de rango 3.



Figura 7.8: Dos capas convolucionales aplicadas de forma secuencial a una imagen de entrada

El modificar el número de canales no altera la formulación de la operación de convolución. Se debe notar que aunque la entrada tenga un número de canales mayor a 1, el kernel puede tener un sólo canal, de tal forma que la ecuación de la convolución es una suma ponderada de vectores por un escalar. En cualquier caso, las capas convolucionales generan representaciones tensoriales de salida. En problemas de clasificación, por ejemplo, necesitamos que las salidas correspondan a probabilidades de clases que se representan con vectores. Para esto, las redes convolucionales, además de las capas convolucionales y de pooling, utilizan en la salida capas feedforward. Las capas feedforward toman como entrada un vector, por lo que se debe convertir el resultado de las convoluciones en un vector de entrada para la feedforward.

Definición 119 (Flattening). *El aplastamiento o flattening es el proceso que mapea un tensor a un vector. En este sentido, es una operación $\text{flat} : \mathbb{R}^{H \times L \times C} \rightarrow \mathbb{R}^{HLC}$ que convierte un tensor con dimensiones H , L y C en un vector de tamaño HLC .*

Sabemos que un tensor de cualquier rango es isomorfo a un vector. Aprovechando esto, podemos tomar la salida de las capas ocultas, que preserva la información relevante de la imagen, para que sea la entrada de capas feedforward con las que obtendremos las salidas. El uso de capas convolucionales presenta varias ventajas, pues genera representaciones que conservan la información más relevante de las entradas, y reduce el tamaño de estas para que su procesamiento por las capas sea más eficiente.

En general, entonces, podemos entender una red neuronal convolucional de la siguiente forma:

Definición 120 (Red neuronal convolucional). *Una red neuronal convolucional es una arquitectura neuronal f que cuenta con los siguientes elementos:*

1. *Capas convolucionales:*

$$h^{(k)} = \text{Pooling}(g(W \star h^{(k-1)}))$$

Con $k = 1, \dots, K$ y $h^{(0)} = x$.

2. *Capas feedforward para obtener salidas, dada la última capa convolucional:*

$$f(x) = \text{FFW}(\text{flat}(h^{(K)}))$$

Donde FFW es una arquitectura de red feedforward que toma el aplastamiento de $h^{(K)}$ como entrada.

En las redes neuronales se pueden combinar diferentes tipos de capas según el propósito de la red. Por ejemplo, si lo que se desea es procesar una secuencia de imágenes, se puede utilizar una red recurrente que tenga en las primeras capas operaciones convolucionales. A las redes convolucionales las abreviaremos CNN (de *Convolutional Neural Network*). Cabe mencionar que las redes convolucionales también generan representaciones que permiten la solución de los problemas de manera efectiva. Por ejemplo, en la Figura 7.9 se muestran las representaciones que obtiene una red convolucional al ser entrenada con datos de MNIST (LeCun et al., 1998), el cual consta de imágenes de dígitos (del 0 al 9) escritos a mano. El objetivo es clasificarlos en el número adecuado. Por tanto, la representación aprendida separa el espacio en 10 regiones en donde una capa de salida feedforward puede clasificarlos sin problemas.

7.5. Entrenamiento en capas convolucionales

En las redes convolucionales, los parámetros están dados por los kernels, que contienen los pesos que se deben aprender. Como mencionamos en la Sección 7.2, las capas convolucionales pueden verse como capas con conectividad dispersa. Por lo que su gradiente tendrá similitudes con el de las feedforward, pero, de hecho, será más simple

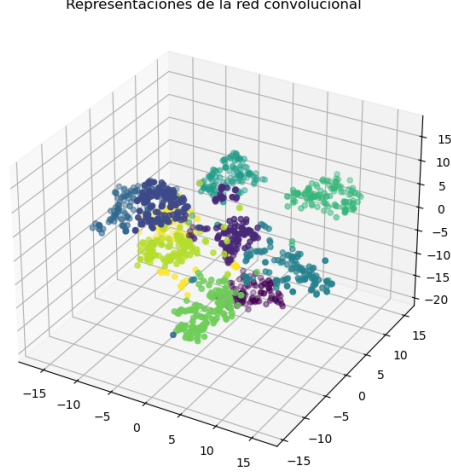


Figura 7.9: Representación de vectores aplastados en una red convolucional para el problema MNIST

en tanto se tienen menos conexiones. Para determinar la retro-propagación en las redes convolucionales considérese que nuestra función objetivo está dada como:

$$R(\theta) = \sum_x \sum_y L(y, f(x))$$

Donde $f(x)$ es la red con capas convolucionales. Por ahora, supongamos que tenemos una capa convolucional con parámetros $W \in \mathbb{R}^{H \times L}$. Entonces, derivando sobre la función de pérdida podemos observar:

$$\begin{aligned} \frac{\partial L(y, f(x))}{\partial W_{u,v}} &= \frac{\partial L(y, f(x))}{\partial h} \frac{\partial h}{\partial W_{u,v}} \\ &= \frac{\partial L(y, f(x))}{\partial h} \frac{\partial Pool}{\partial g(W \star x)} \frac{\partial g(W \star x)}{\partial W \star x} \frac{\partial W \star x}{\partial W_{u,v}} \end{aligned}$$

En este caso *Pool* denota la capa de pooling que toma como argumento a la salida de la capa de convolución. Podemos utilizar una notación como la que hemos venido usando, y llamar a la pre-activación $a = W \star x$. Por tanto, en la derivación de la red convolucional, las primeras derivadas parciales corresponden a la red feedforward, las cuales ya han sido estudiadas en la Sección 4.3. Propiamente, en la parte convolucional tenemos las derivadas sobre el pooling, la función de activación y la operación de convolución.

Para las capas de pooling, las derivadas son sencillas tanto para el caso de average-pooling como el de max-pooling. Se tienen los siguientes resultados.

Proposición 28. *La derivada del average-pooling para una ventana de $H \times L$ sobre la entrada $x_{u,v}$ está dada como:*

$$\frac{\partial \text{Pool}(x)}{\partial x_{u,v}} = \frac{1}{H \cdot L}$$

Donde H es la altura de la ventana del pooling y L su anchura.

Proposición 29. *La derivada del max-pooling sobre la entrada $g(a)_{u,v}$ está dada como:*

$$\frac{\partial \text{Pool}(g(a))}{\partial g(a)_{u,v}} = \delta_{\max(x_{u,v})}$$

Donde $\delta_{\max(x_{u,v})}$ es 1 si el valor es el máximo en la ventana del pooling, y 0 en otro caso.

En el caso del gradiente de la convolución, como mencionábamos, es similar al de las capas densas (lineales) pero con un número menor de pesos. Podemos comprobar fácilmente el siguiente resultado.

Teorema 19. *Sea $W \in \mathbb{R}^{H \times L}$ el kernel de una capa convolucional, entonces tenemos que las derivadas parciales sobre la operación de convolución son:*

$$\frac{\partial W \star x}{\partial W_{u,v}} = \sum_i \sum_j x_{i+u, j+v}$$

Donde $x_{i+u, j+v}$ son los valores que multiplican a $W_{u,v}$ durante la convolución.

Demostración. Supongamos que tenemos un kernel W de tamaño $H \times L$, entonces, en cada neurona del resultado de la convolución, tenemos.

$$\begin{aligned} \frac{\partial (W \star x)_{i,j}}{\partial W_{u,v}} &= \frac{\partial}{\partial W_{u,v}} \sum_{l=0}^H \sum_{l'=0}^L W_{l,l'} x_{i+l, j+l'} \\ &= \sum_{l=0}^H \sum_{l'=0}^L \frac{\partial}{\partial W_{u,v}} W_{l,l'} x_{i+l, j+l'} \\ &= x_{i+u, j+v} \end{aligned}$$

Ya que el kernel se mueve por diferentes regiones en x , la derivada parcial resulta de todos los elementos $x_{i+u, j+v}$ que multiplican el peso $W_{u,v}$ del kernel. □

Como se puede observar, las derivadas sobre una capa convolucional son sencillas y pueden realizarse de manera eficiente con menor costo que las redes feedforward o recurrentes. Asimismo, también se pueden aplicar gráficas computacionales para la retro-propagación. En este caso, la función de convolución y el pooling pueden verse como nodos distintos como se muestra en la Figura 7.10

$$W \xrightarrow{\partial W * x / \partial W_{i,j}} W * x \xrightarrow{\partial \text{Pool} / \partial W * x} \text{Pool} \xrightarrow{\partial f / \partial h} f(h) \xrightarrow{\partial R / \partial f_y} R(\theta)$$

Figura 7.10: Gráfica computacional para implementación de una red convolucional

Ejemplo 23. Supongamos que tenemos una red convolucional simple que cuenta sólo con una capa convolucional con max-pooling y función de activación ReLU que usaremos para clasificación. Entonces contamos con las siguientes capas:

1. Capa convolucional: Dada por la operación de convolución con un kernel $W \in \mathbb{R}^{3 \times 3}$ de la forma:

$$a = \sum_l \sum_{l'} W_{l,l'} x_{i+l, i+l'}$$

2. Una función de activación ReLU sobre el resultado de la convolución: $h = \text{ReLU}(a)$.
3. La capa de max-pooling sobre la convolución con activación.

4. Una capa de salida dada por una matriz de pesos W^{out} cuya pre-activación está dada como:

$$a^{\text{out}} = W^{\text{out}} h + b$$

Asumimos que h ha sido aplastado.

5. Capa de activación de salida dada por la función softmax.
6. Función de riesgo de entropía cruzada.

Sabemos que la derivada de la entropía cruzada sobre la pre-activación de la salida es de la siguiente forma:

$$\frac{\partial R(\theta)}{\partial f_y(x)} \frac{\partial f_y(x)}{\partial a^{\text{out}}} = f_y(x) - \delta_{x,y}$$

Asimismo, en las pre-activación de la salida tenemos que:

$$\frac{\partial a^{\text{out}}}{\partial h_{i,j}} = W_{i,j}^{\text{out}}$$

A partir de esto podemos desarrollar las derivadas sobre los pesos del kernel de la siguiente forma:

$$\begin{aligned} \frac{\partial R(\theta)}{\partial W_{u,v}} &= \frac{\partial R(\theta)}{\partial h_{u,v}} \frac{\partial h_{u,v}}{\partial g(a)} \frac{\partial g(a)}{\partial a} \frac{\partial W * x}{\partial W_{u,v}} \\ &= \left(\sum_q (f_q(x) - \delta_{y,q}) W_{q,y}^{\text{out}} \right) \delta_{\text{máx}} \delta_{>0} \sum_i \sum_j x_{i+u, j+v} \end{aligned}$$

Hemos denotado como $\delta_{>0} = \frac{1}{|a|} \text{máx}\{0, a\}$ a la derivada de la función ReLU. Aplicando un optimizador, podemos entonces actualizar los pesos sobre esta capa convolucional.

Ejercicios

1. Demuestra que la correlación cruzada entre la función sigmoide $\sigma(x)$ y la función indicadora $1_{>0}(x)$, que es 1 cuando x es positiva y 0 en otro caso, es la función:

$$(1_{>0} \star \sigma)(x) = \ln(1 + e^{-x})$$

2. Aplicar la operación de convolución sobre la matriz:

$$x = \begin{pmatrix} 5 & 3 & -1 & 2 & -5 & 0 & 3 \\ 3 & -1 & 5 & 7 & 0 & -1 & 0 \\ -1 & 8 & 2 & 5 & 1 & 6 & -5 \\ 4 & 3 & -3 & 6 & 2 & 3 & -3 \\ 7 & 2 & 5 & 9 & 0 & 0 & -1 \\ -9 & 1 & 0 & 3 & 0 & 9 & 0 \\ 8 & 2 & 3 & 2 & 0 & 1 & 2 \end{pmatrix}$$

Con el siguiente kernel:

$$W = \begin{pmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{pmatrix}$$

3. Del resultado de la convolución del ejercicio anterior, aplica la función ReLU y utiliza max-pooling con una ventana de 2×2 .
4. De la matriz en el ejercicio 2, aplica una operación de convolución con el siguiente kernel:

$$W = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -1 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

Aplica entonces activación ReLU y average-pooling en una ventana de 2×2 .

5. Demuestra que las derivadas parciales del average-pooling están dadas como: $\frac{\partial \text{Pool}(g(a))}{\partial g(a)_{u,v}} = \frac{1}{HL}$.
6. Implementa una red convolucional para el problema de clasificación de dígitos con MNIST (éste se puede encontrar en la biblioteca `sklearn`).

Capítulo 8

Redes atencionales

Hasta ahora, hemos revisado capas recurrentes y capas convolucionales, además de las capas densas o feedforward. La flexibilidad de las redes neuronales permite elaborar capas que procesen estructuras de entrada más complejas. Las capas de atención han tenido un gran impacto actualmente en todo el área de inteligencia artificial, y son la base de los llamados grandes modelos del lenguaje. Además de su impacto en el procesamiento del lenguaje natural, también han tenido éxito para procesar imágenes (Han et al., 2020).

El mecanismo de atención, como su nombre lo dice, busca poner “atención” a elementos que influyan más a la decisión que toma la red para una salida dada. Este tipo de capas se ha implementado con buenos resultados para el procesamiento de lenguaje natural. En principio, el mecanismo de atención se utilizó dentro de redes recurrentes de tipo encoder-decoder. Pero posteriormente, con el trabajo de Vaswani et al. (2017), se volvieron un mecanismo independientes que es el fundamento de las arquitecturas de transformadores.

En el presente capítulo introducimos los conceptos básicos sobre las capas de atención, comenzando por la aplicación de la atención en las redes recurrentes encoder-decoder. Posteriormente abordamos la llamada auto-atención (*self attention*) para, finalmente, abordar la arquitectura de transformadores.

8.1. Atención en redes recurrentes

Los mecanismos de atención se crearon en primera instancia para las redes recurrentes de tipo encoder-decoder (Bahdanau, 2014) en la tarea de traducción automática como un medio de generar un modelo probabilístico de traducción. En este caso, se busca asociar una probabilidad de traducción a una palabra en la lengua destino dada otra palabra en la lengua origen. El problema de un modelo de este tipo radica en que para decidir una salida $\hat{y}^{(t)}$, tomamos en cuenta una codificación c , proveniente del encoder de todos los elementos de entrada, sin tomar en cuenta ningún modelo que defina cuáles son palabras en la lengua destino que tenga mayor probabilidad de traducir una palabra en la lengua origen.

Supongamos entonces que las capas recurrentes son $h^{(1)}, h^{(2)}, \dots, h^{(T)}$ (sin importar si son capas bidireccionales GRU o LSTM). Pensemos entonces que podemos poner estos elementos como renglones dentro de una matriz:

$$h = \begin{pmatrix} h^{(1)} \\ h^{(2)} \\ \vdots \\ h^{(T)} \end{pmatrix}$$

Estos elementos provienen de la codificación que el encoder hace de las entradas. Para el decoder, tomaremos representaciones $s^{(t)}$ que se obtendrán como:

$$s^{(t)} = f(s^{(t-1)}, h)$$

Es decir, son funciones recurrentes que dependen del estado anterior $s^{(t-1)}$ y de las codificaciones h . Para incorporar el mecanismo de atención se propone que este forme parte de la codificación en cada salida en un tiempo t . Por tanto, en lugar de tener un vector de contexto como en los modelos típicos de encoder-decoder, se propone una codificación contextual en cada salida que depende del mecanismo de atención. Para la salida en el tiempo t se creará un vector:

$$c^{(t)} = \sum_{i=1}^T \alpha_{t,i} h^{(i)} \quad (8.1)$$

Donde $\alpha_{t,i} \in [0, 1]$ es una probabilidad que pondera la influencia de $h^{(i)}$ en el estado actual t . Para obtener este valor, que conoceremos como el peso de atención, utilizaremos la función softmax:

$$\alpha_{t,i} = \frac{e^{\varepsilon_{t,i}}}{\sum_r e^{\varepsilon_{t,i}}}$$

Donde $\varepsilon_{t,i}$ es un valor de relación entre $h^{(i)}$ y $s^{(t)}$, que puede calcularse de diferentes formas. Por ejemplo, Luong (2015) propone el cálculo de este valor como:

$$\varepsilon_{t,i} = s^{(t-1)T} W h^{(i)} \quad (8.2)$$

Mientras que Bahdanau (2014) sugiere usar:

$$\varepsilon_{t,i} = v^T \tanh(W[s^{(t-1)}; h^{(i)}]) \quad (8.3)$$

Donde $W \in \mathbb{R}^{d' \times d}$, y $v \in \mathbb{R}^{d'}$ son los pesos a aprender en la red. Lo que estas funciones buscan reflejar es una concepción de la relación entre la salida en el tiempo t y todas las entradas. Esta relación se traduce mediante la función softmax en una probabilidad (de la entrada dada la salida). Por tanto, de forma general se puede definir una capa de atención dentro de una red recurrente encoder-decoder de la siguiente forma:

Definición 121 (Capa de atención en RNNs). *Sea $h^{(1)}, h^{(2)}, \dots, h^{(T)}$ representaciones del encoder. En una RNN encoder-decoder, una capa de atención obtendrá las representaciones de decoder, $s^{(t)}$, de forma recurrente como:*

$$s^{(t)} = f(s^{(t-1)}, c^{(t)})$$

Donde $s^{(t-1)}$ es el vector de representación del estado anterior y $c^{(t)}$ es el vector de atención para la salida t obtenido como:

$$c^{(t)} = \sum_{i=1}^T \alpha_{t,i} h^{(i)}$$

Tal que los pesos de atención $\alpha_{t,i}$ se calculan por medio de la función softmax sobre algún valor de similitud entre $s^{(t-1)}$ y $h^{(i)}$.

En las capas de atención, las representaciones $s^{(t)}$, para cada tiempo t de la longitud de la salida, son las que se envían a una capa siguiente, que puede ser la capa de salida u otra capa intermedia. La Figura 8.1 muestra un diagrama de como funciona este esquema de atención, en donde los nodos del encoder son ponderados por los pesos de atención en una suma que después se envía, junto con el estado anterior $s^{(t-1)}$ al estado actual. Los pesos de atención $\alpha_{t,i}$ forman una matriz, la llamada **matriz de**

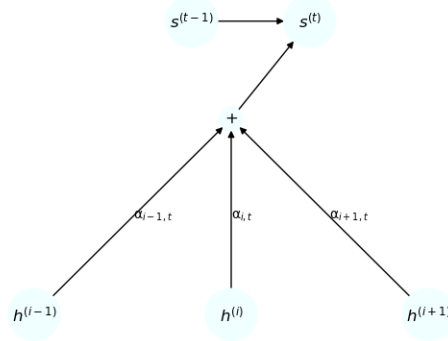


Figura 8.1: Diagrama de atención en redes recurrentes

atención, la cual es una matriz rectangular que contiene los pesos que relacionan a cada uno de los elementos de entrada $h^{(i)}$ con las salidas $s^{(t)}$. En la Figura 8.2 se puede ver gráficamente esta matriz de atención, donde los valores más oscuros implican valores cercanos a 0 (menor relación) y los más claros cercanos a 1 (mayor relación). La interpretación en este caso es clara, pues un valor de salida como la palabra ‘accord’ en francés (Figura 8.2 (a)) pone mayor atención (tiene mayor relación) con la entrada ‘agreement’ del inglés, al ser la traducción de ésta.

Ejemplo 24. Para ejemplificar el uso de atención en las redes recurrentes de tipo encoder-decoder, supongamos que ya contamos con un encoder que se ha encargado de codificar una cadena de longitud 3 en los siguientes vectores 2-dimensionales:

$$\begin{pmatrix} h^{(1)} \\ h^{(2)} \\ h^{(3)} \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 2 \\ 3 & 4 \end{pmatrix}$$

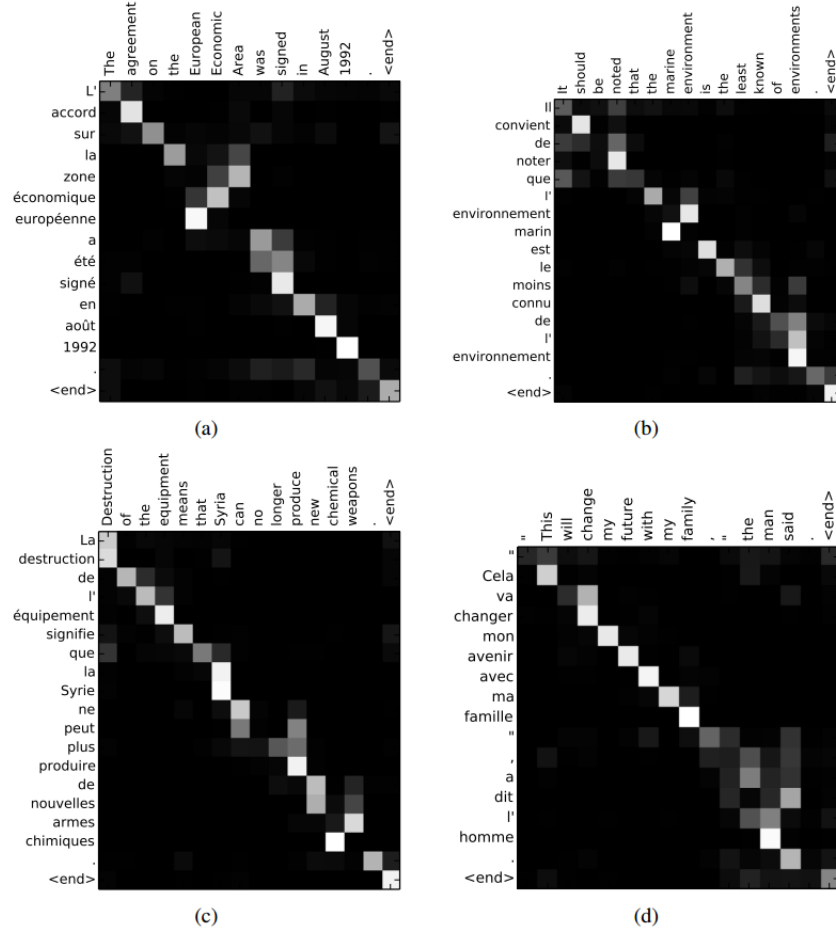


Figura 8.2: Matrices de atención para una tarea de traducción automática obtenidas de Bahdanau (2014)

Ahora, supongamos que para obtener la representación del estado actual del decoder $s^{(2)}$ contamos con la representación anterior $s^{(1)} = \begin{pmatrix} 1 & 1 \end{pmatrix}$. Para calcular los valores de atención, en primer lugar, debemos obtener los valores $\epsilon_{i,i}$. Por simplicidad, utilizemos la propuesta bilineal de Luong (2015), la cual señala que estos valores se obtienen como $\epsilon_{i,i} = s^{(t-1)} W h^{(i)}$, si tomamos W como la matriz identidad obtendremos un producto punto. Por tanto, para el primer vector del encoder obtenemos:

$$s^{(1)} h^{(1)} = \begin{pmatrix} 1 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = 1$$

En general notamos que, aplicando este producto a cada uno de los vectores de entra-

da, obtenemos los siguientes resultados:

$$\begin{pmatrix} h^{(1)} \\ h^{(2)} \\ h^{(3)} \end{pmatrix} s^{(t-1)} = \begin{pmatrix} 1 \\ 3 \\ 7 \end{pmatrix}$$

Ahora aplicando la función softmax a cada uno de estos valores resultantes, obtenemos que:

$$\alpha_t = \text{Softmax} \begin{pmatrix} 1 \\ 3 \\ 7 \end{pmatrix} = \begin{pmatrix} 0,002 \\ 0,017 \\ 0,97 \end{pmatrix}$$

Por tanto, vemos que el vector $c^{(t)}$ para obtener el estado $s^{(t)}$ del decoder será el siguiente:

$$c^{(t)} = \alpha_t \begin{pmatrix} h^{(1)} \\ h^{(2)} \\ h^{(3)} \end{pmatrix} = (0,002 \quad 0,017 \quad 0,97) \begin{pmatrix} 0 & 1 \\ 1 & 2 \\ 3 & 4 \end{pmatrix} = \begin{pmatrix} 2,9 \\ 3,9 \end{pmatrix}$$

Finalmente, la representación del estado actual será una función recurrente del estado anterior y de este vector $c^{(t)}$; esto es, $s^{(t)} = f(s^{(t-1)}, c^{(t)})$. Esta función puede ser de tipo vanilla, GRU o LSTM.

En el ejemplo anterior, hemos denotado a todos los pesos de atención para el estado t como un vector α_t , que correspondería a un renglón de una matriz de atención que llamaremos α . Bajo esta notación matricial, el mecanismo de atención se puede expresar como un producto de matrices:

$$s = \alpha h = \text{Softmax}(\varepsilon) h$$

Tal que ε es una matriz que contiene una operación entre los elementos $h^{(t)}$ y $s^{(t-1)}$. Cabe señalar que en estos caso se requiere conocer la longitud de la cadena de salida. En el entrenamiento esta longitud depende de los ejemplos. En la inferencia, se puede determinar un número máximo para la longitud de la salida.

Existen algunas variaciones sobre los mecanismos de atención en redes recurrentes. Para simplificar su cálculo, por ejemplo, se puede limitar el contexto de entrada que se observa en cada salida. En el caso general que hemos revisado, para obtener los pesos de atención se requiere hacer los cálculos sobre todos los elementos de entrada. Si la entrada tiene una longitud grande, estos cálculos tomarán un mayor tiempo. Además, en algunas tareas como la traducción (Figura 8.2) los pesos de atención recaen en la diagonal de la matriz de atención; es decir, en cada salida se pone mayor atención únicamente a los elementos que, en la entrada, están cercanos a la posición actual. Por tanto, podemos simplificar los cálculos asumiendo que sólo las palabras en una vecindad cercana a una posición serán influyentes en la salida actual.

Definición 122 (Atención local). Sea $h^{(1)}, h^{(2)}, \dots, h^{(T)}$ las representaciones de un encoder. En la atención local, la representación del estado t del decoder, $s^{(t)}$ busca sólo

los elementos de la entrada que están en una ventana con centro en la posición t . Esto es, el vector $c^{(t)}$ se obtiene como:

$$c^{(t)} = \sum_{i=\max\{t-k\}}^{\min\{t+k\}} \alpha_{t,i} h^{(i)}$$

Donde k es el tamaño de la ventana.

En el caso de la atención local, los elementos que influyen son valores locales. Sólo se busca aquellos valores que están k elementos antes y k elementos después. Esto limita el número de cálculos de la longitud de la cadena de entrada a sólo $2k$. En contraste, la atención que revisamos, donde se toma todo elemento de la cadena de entrada, se le conoce como **atención global**.

Asimismo, la atención que hemos revisado utiliza las probabilidades softmax para obtener un vector $c^{(t)}$ que considere una suma de todos los elementos de la cadena de entrada que se están tomando en consideración (sea de manera local o global). A este tipo de implementación se le conoce como **atención suave** (o *soft attention*). Como alternativa, podemos considerar la atención dura.

Definición 123 (Atención dura). Sea $h^{(1)}, h^{(2)}, \dots, h^{(T)}$ las representaciones de un encoder. La atención dura o *hard attention* obtiene el vector $c^{(t)}$ como:

$$c^{(t)} = \max \{ \alpha_{t,i} : i = 1, 2, \dots, T \}$$

Por lo que $c^{(t)}$ será la representación $h^{(i)}$ con mayor probabilidad softmax.

El caso de la atención dura no hace una suma ponderada de las representaciones del encoder, si no que toma sólo una representación que tenga mayor probabilidad. Por tanto, sólo habrá un elemento al que se pondrá atención. En la práctica, puede pasar que las probabilidades softmax en la atención se acumulen en ciertos elementos como se observa en la Figura 8.2, por lo que la atención dura puede mostrar un buen desempeño en algunas tareas. Sin embargo, habrá que considerar que en problemas específicos, donde más de un elemento de entrada influye a las salidas, la atención dura no podría funcionar adecuadamente.

Estas diferentes versiones de atención tienen diferentes propósitos: la atención local busca reducir la complejidad del número de cálculos limitándose a regiones cercanas; la atención dura busca poner atención a sólo un elemento de la entrada. De hecho, pueden combinarse atención local y dura (la atención estándar sería atención global suave). La atención tiene un impacto importante en tareas que buscan predecir cadenas a partir de otras cadenas de entrada, y actualmente han tenido un uso amplio en las arquitecturas de transformadores. Antes de pasar a ver estas arquitecturas abordamos los tipos de atención en los que dichas arquitecturas se fundamentan.

8.2. Auto-atención

La atención propuesta por Bahdanau (2014) para redes recurrentes tuvo un gran impacto en tareas enfocadas a secuencias. Sin embargo, el requerir de recurrencias conlleva

la necesidad de estimar cada estado en un tiempo t únicamente después de estimar los estados en tiempos anteriores. Esto puede hacer que las redes recurrentes sean costosas a nivel computacional, pues no pueden paralelizarse los procesos de cómputo de los estados. Por tanto, Vaswani et al. (2017) propusieron prescindir de las capas recurrentes y utilizar también un mecanismo de atención para obtener representaciones tanto en el encoder como en el decoder. A este tipo de atención que obtiene representaciones sobre una cadena le llamaron auto-atención (o *self-attention*). Para definir este tipo de capas, retomamos el concepto de pesos de atención.

Definición 124 (Pesos de auto-atención). Sean $x^{(1)}, x^{(2)}, \dots, x^{(T)} \subseteq \mathbb{R}^d$ un conjunto de vectores de entrada, los pesos de auto-atención entre estos vectores están dados como:

$$\alpha_{i,j} = \alpha(x^{(i)}, x^{(j)}) = \text{Softmax}\left(\frac{\psi_q(x^{(i)})\psi_k(x^{(j)})}{\sqrt{d}}\right)$$

Donde las funciones ψ_k y ψ_q son proyecciones al espacio de llaves (keys) y consultas (queries), respectivamente.

Esta definición, nos dice cómo se obtienen los pesos de atención de forma concreta, pero hay puntos dentro de la definición que deben aclararse. Un peso de atención $\alpha_{i,j}$ es una probabilidad que relaciona el elemento $x^{(i)}$ con el elemento $x^{(j)}$ ambos pertenecientes a la misma cadena de entrada. Incluso, entre los pesos de atención debemos considerar la relación de un elemento consigo mismo, los valores de la diagonal $\alpha_{i,i}$. La relación entre los pares de elementos se obtiene por medio de un producto punto de los elementos proyectados en espacios distintos. Aquí se habla de llaves y consultas, o *keys* y *queries*. Esta terminología refiere al uso que se hace de cada elemento dentro de la auto-atención. Las consultas son los elementos centrales de un peso de atención, los que condicionan la probabilidad, mientras que las llaves son los elementos que varían con respecto a una consulta. En particular, podemos ver que la función softmax se definiría como (ignoramos el factor $\sqrt{d}$):

$$\text{Softmax}(\psi_q(x^{(i)})\psi_k(x^{(j)})) = \frac{e^{\psi_q(x^{(i)})\psi_k(x^{(j)})}}{\sum_k e^{\psi_q(x^{(i)})\psi_k(x^{(k)})}}$$

Podemos observar que el factor que permanece constante en el denominador es la consulta. Ahora bien, las consultas y las llaves se distinguen a partir de proyectarse a diferentes espacios por medio de funciones que aquí hemos llamado ψ_q y ψ_k , respectivamente.

Definición 125 (Proyecciones). Sea $x^{(i)} \in \mathbb{R}^d$, $i \in \{1, 2, \dots, T\}$, un elemento de una cadena de entrada. Definimos la proyección al espacio de consultas como:

$$\psi_q(x^{(i)}) = W^Q x^{(i)} + b^Q$$

Con $W^Q \in \mathbb{R}^{d' \times d}$ y $b^Q \in \mathbb{R}^{d'}$ conjunto de pesos. De igual forma, la proyección al espacio de llaves se define como:

$$\psi_k(x^{(i)}) = W^K x^{(i)} + b^K$$

Con $W^K \in \mathbb{R}^{d' \times d}$ y $b^K \in \mathbb{R}^{d'}$ conjunto de pesos. Es común que $b^Q = \bar{0} = b^K$.

Como señala la definición, las proyecciones suelen ser lineales y son aprendidas durante el proceso de entrenamiento. De tal forma que el espacio de consultas y de llaves se aprenderán a partir de los datos. Finalmente, el factor $\sqrt{d}$, donde d se conoce como la dimensión del modelo (el tamaño de los vectores de entrada), se utiliza para reescalar los valores del producto punto para que el softmax no tome valores demasiado grandes. A la operación del producto punto dividido por este factor se le llama **producto punto escalado**.

Ejemplo 25. Consideremos que tenemos una cadena de entrada de longitud 3 conformada por vectores de dos dimensiones, esto es, $d = 2$. Sean estos datos los siguientes:

$$x = \begin{pmatrix} x^{(1)} \\ x^{(2)} \\ x^{(3)} \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Asimismo definamos las proyecciones a partir de las siguientes matrices:

$$\psi_q(x) = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} x$$

Para las consultas y para las llaves:

$$\psi_k(x) = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} x$$

Entonces, tomando como consulta al vector $x^{(1)}$ notamos que su proyección está dada como:

$$\psi_q(x^{(1)}) = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Por su parte, la proyección de los valores de las llaves las podemos realizar de la siguiente forma para todos los vectores de la cadena:

$$\psi_k(x) = xW^K = \begin{pmatrix} 0 & 1 \\ 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 2 & 1 \\ 4 & 3 \end{pmatrix}$$

Por tanto, el producto punto escalado entre el vector de consulta y los diferentes vectores de llaves está dado de la siguiente forma:

$$\frac{\psi_k(x)\psi_q(x^{(1)})}{\sqrt{2}} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 \\ 2 & 1 \\ 4 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 3 \\ 7 \end{pmatrix}$$

Es decir, la relación más alta se da entre el primer elemento y último, pues el valor aquí obtenido es de $\frac{7}{\sqrt{2}}$. Es de notar que la relación con el mismo elemento es pequeña, incluso la más baja de las tres. Con estos valores, podemos obtener las probabilidades de la función softmax como:

$$\text{Softmax}\left(\frac{\psi_k(x)\psi_q(x^{(1)})}{\sqrt{2}}\right)^T = (0,09 \quad 0,27 \quad 0,63)$$

De nuevo, notamos que la probabilidad más alta se da con el último elemento. Es decir, la auto-atención resaltará con mayor fuerza esta relación. Finalmente, notamos que si aplicamos la auto-atención entre todos los elementos, obtenemos una matriz de atención representada en la siguiente tabla:

	$x^{(1)}$	$x^{(2)}$	$x^{(3)}$
$x^{(1)}$	0.09	0.27	0.63
$x^{(2)}$	0.1	0.27	0.62
$x^{(3)}$	0.11	0.27	0.61

Cuadro 8.1: Matriz de atención con los pesos de auto-atención entre los elementos del ejemplo

Cada renglón de esta matriz representa las probabilidades de relación entre el elemento del renglón y los elementos de las columnas, es decir $\alpha_{i,j} = p(x^{(j)} | x^{(i)})$, donde esta probabilidad ha sido estimada mediante la probabilidad softmax.

En el ejemplo hemos observado que podemos realizar operaciones entre la matriz que contiene a los elementos de entrada y las matrices de proyecciones. Supóngase que se tiene como entrada un conjunto de vectores $x^{(1)}, x^{(2)}, \dots, x^{(T)}$ dentro de una matriz x :

$$x = \begin{pmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(T)} \end{pmatrix}$$

En lugar de hacer las proyecciones directamente sobre cada uno de los datos, podemos utilizar operaciones matriciales, lo cual va a contribuir a una implementación eficiente de la auto-atención. Consideremos entonces la matriz de datos x , las proyecciones al espacio de consultas, que llamaremos Q , y al espacio de llaves, que llamamos K , están dadas como:

$$\begin{aligned} Q &= xW^Q \\ K &= xW^K \end{aligned}$$

Donde W^Q y W^K son las matrices de pesos de tamaños $d' \times d$ (d dimensión del modelo). Una vez obtenidos estas copias, se procede a calcular los pesos de atención utilizando un producto punto escalado:

$$\alpha = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)$$

En este caso α es una matriz, la matriz de atención, cuyas entradas son los pesos $\alpha_{i,j}$ que, podemos ver, tienen la misma forma que en la Definición 124. Podemos notar las semejanzas que tiene esta forma de obtener la matriz de auto-atención con lo que veíamos en la atención en redes recurrentes. Sin embargo, en este caso sólo consideramos una cadena de entrada, no una relación entre cadenas del encoder y el decoder.

Proposición 30. Sea $W = (\frac{1}{\sqrt{d}}Id)W^Q(W^K)^T$, donde Id es la matriz identidad, entonces si las capas de auto-atención usan proyecciones lineales, tenemos que:

$$\alpha = \text{Softmax}(xWx^T)$$

Demostración. Tenemos el siguiente desarrollo:

$$\begin{aligned}\alpha &= \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right) \\ \alpha &= \text{Softmax}\left(\frac{(xW^Q)(xW^K)^T}{\sqrt{d}}\right) \\ \alpha &= \text{Softmax}\left(\frac{xW^Q(W^K)^T x^T}{\sqrt{d}}\right) \\ \alpha &= \text{Softmax}(xWx^T)\end{aligned}$$

Hemos tomado $W = (\frac{1}{\sqrt{d}}Id)W^Q(W^K)^T$. □

La proposición anterior nos muestra que bajo proyecciones lineales, la auto-atención tiene una forma bilineal similar a la que se propone en la atención de Luong (2015) utilizada en redes recurrentes y revisada en la sección anterior.

Sin embargo, hasta ahora sólo hemos definido los pesos de atención, lo que queremos obtener son representaciones de los datos de entrada que tomen en cuenta las relaciones con los otros elementos de la cadena. Por tanto, realizaremos una suma ponderada como lo hacíamos en las redes recurrentes, pero ahora entre elementos de una misma cadena.

Definición 126 (Auto-atención). Sea $x^{(1)}, x^{(2)}, \dots, x^{(T)} \subseteq \mathbb{R}^d$ una cadena de elementos de entrada, y sean $\alpha_{i,j}$, $i, j \in \{1, 2, \dots, T\}$, sus pesos de atención. Una capa de auto-atención obtiene representaciones nuevas a partir de la siguiente suma ponderada:

$$h^{(i)} = \sum_{j=1}^T \alpha_{i,j} \psi_v(x^{(j)})$$

Donde ψ_v es una proyección sobre el espacio de valores.

En esta definición hemos introducido el concepto de valor. Los valores son precisamente los vectores que se sumarán, ponderados por su peso de atención, para conformar la nueva representación dentro de las capas ocultas de la red. De igual forma que con las consultas y las llaves, la proyección de los valores suele ser una función lineal:

$$\psi_v(x) = W^V x + b^V$$

Que de forma general toma $b^V = \bar{0}$. Usualmente, W_V es una matriz cuadrada, proyectando los datos a un espacio de la misma dimensión de entrada; esto para que en

pasos posteriores del transformer se puedan realizar conexiones residuales. Asimismo, podemos utilizar toda la matriz de datos de entrada x para obtener una matriz de proyecciones de valores, V , de la siguiente forma:

$$V = xW^V$$

Finalmente, utilizando la notación matricial en lugar de la notación funcional de las Definiciones 126 y 124, podemos observar que la auto-atención obtiene una matriz de representaciones h a partir de una matriz de entadas x como:

$$h = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

Esta notación es la utilizada por Vaswani et al. (2017) en el artículo en el que por primera vez se introduce el concepto de auto-atención, pero podemos ver que es equivalente a la definición que hemos presentado de auto-atención.

Proposición 31. Sea x una matriz de datos de entrada, y sea $h^{(i)}$ el i -ésimo renglón de la matriz de representaciones obtenidas como $h = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$, entonces:

$$h^{(i)} = \sum_{j=1}^T \text{Softmax}\left(\frac{\psi_q(x^{(i)})\psi_k(x^{(j)})}{\sqrt{d}}\right)\psi_v(x^{(j)})$$

Tal que ψ_q , ψ_k y ψ_v son las proyecciones al espacio de consultas, llaves y valores, respectivamente.

Demostración. Se deja como ejercicio. □

Ejemplo 26. Del ejemplo anterior, teníamos que los pesos de atención son los siguientes:

	$x^{(1)}$	$x^{(2)}$	$x^{(3)}$
$x^{(1)}$	0.09	0.27	0.63
$x^{(2)}$	0.1	0.27	0.62
$x^{(3)}$	0.11	0.27	0.61

Cuadro 8.2: Matriz de atención con los pesos de auto-atención entre los elementos del ejemplo anterior

De esta forma, la representación obtenida para el primer elemento es un vector de la suma de todos los vectores ponderado por su peso de atención. Asumimos que la proyección en el espacio de valores es la identidad. Esto es:

$$h^{(1)} = 0,09 \begin{pmatrix} 0 \\ 1 \end{pmatrix} + 0,27 \begin{pmatrix} 1 \\ 2 \end{pmatrix} + 0,63 \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 2,16 \\ 3,15 \end{pmatrix}$$

Aplicando el mismo procedimiento para cada uno de los vectores de representación con sus correspondientes pesos de auto-atención, obtenemos la siguiente matriz de

salida:

$$h = \begin{pmatrix} 2,16 & 3,15 \\ 2,13 & 3,12 \\ 2,1 & 3,09 \end{pmatrix}$$

Donde cada renglón es la representación de un dato de entrada. Como se puede observar, las representaciones resultan bastantes similares, en tanto sus pesos de auto-atención son muy parecidos.

La auto-atención es un tipo de capas que, en primera instancia, se utilizó para reemplazar las capas recurrentes. En las capas de auto-atención no existe un orden específico en que los datos de entrada deban procesarse, por lo que estas capas pueden paralelizarse. Asimismo, obtienen una representación que guarda información de los elementos de las cadenas. Sin embargo, es claro que no hay información secuencial en las representaciones obtenidas. Para solventar esto se utilizará una codificación posicional, la cual introducirá en los datos de entrada la información de su posición en la cadena. Antes de abordar esto, repasaremos la obtención del gradiente en las capas de auto-atención, así como otras formas de auto-atención.

8.2.1. Derivación de una capa de auto-atención

Para comprender de mejor manera la derivación sobre la capa de auto-atención, conviene visualizar estas capas como gráficas computacionales; es decir, como aplicación de funciones que requieren del cálculo de funciones previas. En este caso, podemos enfocarnos en cada uno de los pasos que se siguen, o funciones que se aplican, para obtener la representación final de la auto-atención. En la sección anterior hemos revisado que en el cálculo de la auto-atención se aplican las siguientes operaciones:

1. **Capas lineales:** Dado el conjunto de datos de entrada $x^{(1)}, \dots, x^{(T)}$, lo primero que se aplica en la auto-atención son las proyecciones lineales a partir de las cuales se obtienen los vectores de consultas, llaves y valores. Estas proyecciones se obtienen a por medio de capas lineales: capas donde se multiplica por una matriz. En algunos casos se pueden ocupar capas más complejas, con bias o incluso con funciones de activación, pero aquí nos limitaremos al caso más sencillo. Por lo que las capas de este tipo son de la forma:

$$\psi_k(x^{(i)}) = Wx^{(i)}$$

Cada una de estas capas se debe estimar para los valores de consulta, llave y valor de manera independiente. Para facilitar el cálculo de estas capas, en esta sección utilizaremos una simplificación de la notación como $q(x^{(t)})$, $k(x^{(t)})$ y $v(x^{(t)})$ para las capas de consulta, llave y valor, respectivamente.

2. **Producto punto escalado:** Una vez que se han estimado los valores para las consultas y las llaves, se realiza el cálculo de los valores que entrarán en el cálculo de las probabilidades softmax por medio de un producto punto escalado, el cual es de la forma:

$$\epsilon_{t,s} = \frac{q(x^{(t)}) \cdot k(x^{(s)})}{\sqrt{d}}$$

El valor de d corresponde a la dimensión del modelo; es decir, la dimensión de los vectores de valor.

3. **Probabilidad Softmax:** Una vez que se han obtenido los valores del producto punto escalado, se obtienen las probabilidades con la función softmax, el cálculo es de la siguiente forma:

$$\alpha_{t,s} = \frac{e^{\mathcal{E}_{t,s}}}{\sum_u e^{\mathcal{E}_{t,u}}}$$

A estos valores se les conoce como pesos de atención. Los pesos de atención conforman una matriz de atención. Estos productos puntos serán las entradas a una función softmax.

4. **Representación:** Finalmente, se obtiene la representación de cada uno de las entradas a partir de los pesos de auto-atención y de los vectores de valor. Esta representación se estima de la siguiente forma:

$$h^{(t)} = \sum_s \alpha_{t,s} v(x^{(s)})$$

Es decir, la representación que se obtiene de la capa de auto-atención es una combinación lineal de los elementos de la entrada ponderados por los pesos de atención.

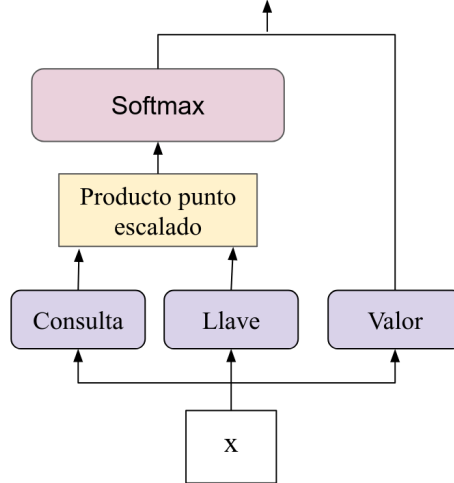


Figura 8.3: Esquema de los procesos realizados en una capa de auto-atención

En la Figura 8.3 se muestra la forma en que interactúan los diferentes procesos en la capa de auto-atención para obtener las representaciones de salida. En la capa de auto-atención, tenemos que derivar desde los valores de salida, que corresponden a la representación $h^{(t)}$ y actualizar los pesos de la capa, los cuáles corresponden a las capas lineales en cada uno de los valores de consulta, llave y valor. Estos son los únicos pesos

con los que la capa cuenta. Por tanto, debemos retro-propagar la derivada hasta estos pesos.

La capa de auto-atención trabaja con varios valores de salida, y por tanto, podemos pensar su derivada de forma secuencial. La función objetivo con la que trabajaremos, por tanto, considera una suma sobre los diferentes tiempos t , que se corresponde al número de valores de entrada. Esta función objetivo tendrá la siguiente forma:

$$R(\theta) = \sum_t \sum_y L_t(f_y(x^{(t)}), y)$$

Recuérdese que L_t es la función de pérdida, $f_y(x^{(t)})$ es la salida en el tiempo t en la neurona y . Como hemos construido la capa de atención, los pesos más próximos corresponden a los pesos en los vectores de valor. Por tanto, podemos comenzar a obtener los gradientes en sobre las proyecciones en valores.

Lema 3. Sean $x^{(1)}, x^{(2)}, \dots, x^{(T)}$ vectores de entrada de una capa de auto-atención con representaciones de salida $h^{(1)}, h^{(2)}, \dots, h^{(T)}$. Entonces, las derivadas parciales sobre los pesos de los valores está dada como:

$$\frac{\partial L_t(f_y(x^{(t)}), y)}{\partial W_{i,j}^V} = \frac{\partial L_t(f_y(x^{(t)}), y)}{\partial h^{(t)}} \sum_s \alpha_{t,i} x_j^{(s)}$$

Para todo $t \in \{1, \dots, T\}$ y donde $\frac{\partial L_t(f_y(x^{(t)}), y)}{\partial h^{(t)}}$ es el gradiente de las capas superiores.

Demostración. Consideremos el elemento t de las salidas, es decir $h^{(t)}$ y derivemos sobre los pesos de las proyecciones a los valores W^V , asumiendo que se trata de una capa lineal. Entonces tenemos:

$$\frac{\partial L_t(f_y(x^{(t)}), y)}{\partial W_{i,j}^V} = \frac{\partial L_t(f_y(x^{(t)}), y)}{\partial h^{(t)}} \frac{\partial h^{(t)}}{\partial W_{i,j}^V}$$

Como en casos previamente vistos, $\frac{\partial L_t(f_y(x^{(t)}), y)}{\partial h^{(t)}}$ denota la derivada que se retro-propaga desde las capas superiores, por lo que ahora no nos enfocaremos en ésta. Nos centramos en la derivada parcial en $\frac{\partial h^{(t)}}{\partial W_{i,j}^V}$, donde tenemos que:

$$\begin{aligned} \frac{\partial h^{(t)}}{\partial W_{i,j}^V} &= \frac{\partial h^{(t)}}{\partial v(x^{(s)})_i} \frac{\partial v(x^{(s)})_i}{\partial W_{i,j}^V} \\ &= \sum_s \frac{\partial \alpha_{t,s} v(x^{(s)})_i}{\partial v(x^{(s)})_i} \frac{\partial W_i^V x^{(s)}}{\partial W_{i,j}^V} \\ &= \sum_s \alpha_{t,i} x_j^{(s)} \end{aligned}$$

De ambos resultados se obtiene el lema. □

La derivada en este caso es sencilla ya que el valor se obtiene de forma lineal y no existe una recurrencia, sino que las operaciones se aplican sobre los datos de forma paralela. En el caso de las consultas y las llaves, las derivadas parciales pasan por la función softmax dentro de la suma ponderada, así como por el producto escalado (véase la Figura 8.3). Ambas derivadas son similares, pero muestran sus particularidades.

Lema 4. Sean $x^{(1)}, x^{(2)}, \dots, x^{(T)}$ vectores de entrada de una capa de auto-atención con representaciones de salida $h^{(1)}, h^{(2)}, \dots, h^{(T)}$. Entonces, las derivadas parciales sobre los pesos de las llaves está dada como:

$$\frac{\partial L_t(f_y(x^{(t)}), y)}{\partial W_{i,j}^K} = \frac{\partial L_t(f_y(x^{(t)}), y)}{\partial h^{(t)}} \frac{1}{\sqrt{d}} \sum_s v(x^{(s)})_i \alpha_{t,i} (\delta_{i,s} - \alpha_{t,s}) q(x^{(s)})_i x_j^{(s)}$$

Para todo $t \in \{1, \dots, T\}$ y donde $\frac{\partial L_t(f_y(x^{(t)}), y)}{\partial h^{(t)}}$ es el gradiente de las capas superiores.

Demostración. De igual forma que en el caso de las derivadas parciales en los valores, debemos sólo fijarnos en la derivada de la salida de la auto-atención $h^{(t)}$ sobre los pesos de las consultas W^K . Siendo $\epsilon_{t,s}$ el prudcto punto escalado entre el elemento t y elemento s de la entrada, tenemos que:

$$\begin{aligned} \frac{\partial h^{(t)}}{\partial W_{i,j}^K} &= \frac{\partial h^{(t)}}{\alpha_{t,s}} \frac{\partial \alpha_{t,s}}{\partial \epsilon_{t,s}} \frac{\partial \epsilon_{t,s}}{\partial W_{i,j}^K} \\ &= \sum_s \frac{\partial \alpha_{t,s} v(x^{(s)})}{\alpha_{t,i}} \frac{\partial \alpha_{t,i}}{\partial \epsilon_{t,s}} \frac{1}{\sqrt{d}} \frac{\partial q(x^{(t)}) k(x^{(s)})}{\partial k(x^{(s)})_i} \frac{\partial W_{i,j}^K x^{(s)}}{\partial W_{i,j}^K} \\ &= \frac{1}{\sqrt{d}} \sum_s v(x^{(s)})_i \alpha_{t,i} (\delta_{i,s} - \alpha_{t,s}) q(x^{(t)})_i x_j^{(s)} \end{aligned}$$

Recuérdese que $\alpha_{t,s}$ está determinado por la función softmax, por lo que aquí hemos utilizado su derivada. □

Finalmente, podemos obtener las derivadas parciales sobre los pesos de las consultas. Estas derivadas guardan semejanzas con las de las llaves, pero no son iguales, aunque podemos aprovechar la estructura de la capa para poder retro-propagar tanto en las consultas como en las llaves.

Lema 5. Sean $x^{(1)}, x^{(2)}, \dots, x^{(T)}$ vectores de entrada de una capa de auto-atención con representaciones de salida $h^{(1)}, h^{(2)}, \dots, h^{(T)}$. Entonces, las derivadas parciales sobre los pesos de las consultas está dada como:

$$\frac{\partial L_t(f_y(x^{(t)}), y)}{\partial W_{i,j}^Q} = \frac{\partial L_t(f_y(x^{(t)}), y)}{\partial h^{(t)}} \frac{1}{\sqrt{d}} \sum_s v(x^{(s)})_i \alpha_{t,i} (\delta_{i,s} - \alpha_{t,s}) k(x^{(s)})_i x_j^{(t)}$$

Para todo $t \in \{1, \dots, T\}$ y donde $\frac{\partial L_t(f_y(x^{(t)}), y)}{\partial h^{(t)}}$ es el gradiente de las capas superiores.

Su derivación es sencilla y se deja como ejercicio. Tanto en las derivadas parciales de las consultas, como el de las llaves podemos notar que hay factores que se reutilizan, en particular, en ambos casos aparece la derivada de la función softmax y el producto por los vectores de valor. Por tanto, ambos valores pueden guardarse en variables para que se utilicen en ambos casos. Esto viene de la estructura de la capa de auto-atención que se muestra en la Figura 8.3, donde tanto la consulta como la llave se mandan, después del producto escalado a una activación softmax y por un producto por los vectores de valor. Utilizar esta estructura para obtener las derivadas, como una gráfica computacional, facilita la implementación de este tipo de capas.

Si antes de la capa de auto-atención existieran capas previas, las derivadas en la capa de auto-atención pueden retro-propagar a una capa anterior tomando las derivadas anteriores (omitiendo los valores de $x_j^{(s)}$ y $x_j^{(t)}$). De esta forma, las capa de auto-atención pueden integrarse en arquitecturas más complejas. En particular, este tipo de capas conforman la parte central de las arquitecturas de transformadores, en la cuales en los pasos previos se suelen incorporar algunas capas. Antes de adentrarnos en estas arquitecturas revisamos otros tipos de atención que se utilizarán dentro de los transformadores.

8.3. Auto-atención enmascarada

La auto-atención tiene la característica que toma en cuenta, para la representación de un elemento de entrada, todos los elementos que aparecen en su contexto, incluyendo aquellos elementos que aparecen posteriormente a este elemento. Es decir, asume que la representación de un elemento en posición t requiere de la información de todos los otros elementos en la cadena de entrada. En la matriz de pesos de atención podemos ver que todas las entradas tienen un peso de atención asignado, lo que implica que existe una relación entre los elementos. Esto es, si tenemos una entrada $x^{(t)}$, en la capa de auto-atención se asumirá que tiene una relación con cada elemento $x^{(i)}$, con $i \in \{1, \dots, T\}$, incluyendo a sí mismo, a los elementos a la izquierda y los de la derecha.

Sin embargo, esto puede ser problemático en varios casos, particularmente será problemático al aplicarlo en tareas de lenguaje natural donde lo que se busca es predecir los elementos de una secuencia dado elementos previos. Imaginemos que queremos entrenar este predictor de palabras; los datos de entrenamiento son las secuencias de palabras y en cada posición t buscamos predecir el elemento en la posición $t + 1$. Utilizar un mecanismo de auto-atención incluirá la información del siguiente elemento, por lo que habría un sesgo en el entrenamiento que no podrá encontrarse durante la inferencia.

Los mecanismos de atención, en general, pueden utilizarse para predecir secuencias a partir de secuencias de entrada. Por tanto, es importante que en el tiempo de predicción de las secuencias no tengan acceso a información de los elementos siguientes. Para evitar este sesgo, se utiliza un enmascaramiento de los elementos subsecuentes. El enmascaramiento es un procedimiento que evitará que elementos de la secuencia que se trabaja sean vistos por la capa de auto-atención.

Definición 127 (Enmascaramiento). Sea $x^{(1)}, x^{(2)}, \dots, x^{(T)}$ una cadena de entrada en una capa de auto-atención. El enmascaramiento reemplaza uno o más de estos ele-

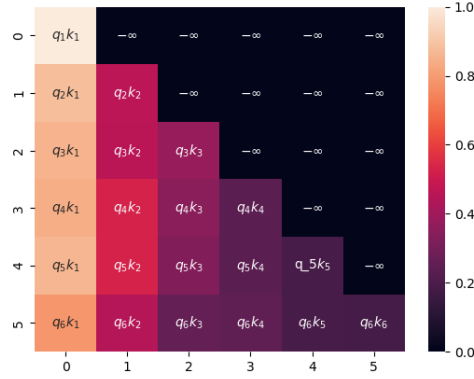


Figura 8.4: Matriz de atención para el mecanismo de auto-atención enmascarada

mentos $x^{(i)}$ por una etiqueta estándar, generalmente denominada *MASK*, para que el modelo no tenga información sobre este elemento.

El enmascaramiento tiene varias aplicaciones. Por ejemplo, en modelos de lenguaje como BERT (Devlin et al., 2019) se utiliza el enmascaramiento para ocultar ciertas palabras que después pueden ser predichas por el modelo. En particular, cuando el objetivo es predecir los elementos siguientes de una secuencia se habla de un modelo auto-regresivo, donde se enmascaran todos los elementos siguientes a una posición i .

Definición 128 (Enmascaramiento auto-regresivo). *Un proceso de enmascaramiento auto-regresivo es un proceso de enmascaramiento de una secuencias $x^{(1)}, x^{(2)}, \dots, x^{(T)}$, donde para cada posición i todos los elementos $x^{(j)}$, para $j > i$, son enmascarados.*

Prácticamente, el enmascaramiento se puede implementar dentro de la matriz de pesos del producto punto escalado, asignando a todos los valores subsecuentes de la posición actual el valor $-\infty$, de tal forma que al aplicar la función softmax para obtener la matriz de auto-atención, estos valores serán 0. En el enmascaramiento auto-regresivo, este enmascaramiento reemplazará los valores en toda la parte triangular superior de la matriz con los valores $-\infty$, como se observa en la Figura 8.4.

Como mencionamos, al aplicar el enmascaramiento a todos los elementos siguientes dada una posición i , la función softmax en la auto-atención arrojará valores 0, por lo que estos elementos no tendrán influencia en la representación obtenida en la capa. A partir de esto podremos obtener nuevas representaciones que dependan únicamente de los valores previos, lo cual será útil al predecir secuencias dentro de las arquitecturas de transformadores.

Definición 129 (Auto-atención con enmascaramiento auto-regresivo). *Sea $x^{(1)}, \dots, x^{(T)}$ una secuencia de elementos de entrada, una capa de auto-atención con enmascaramiento auto-regresivo es una capa cuyas representaciones $h^{(1)}, h^{(2)}, \dots, h^{(T)}$ son obte-*

nidas considerando sólo los elementos previos; esto es:

$$h^{(i)} = \sum_{j \leq i} \alpha_{i,j} \psi_v(x^{(j)})$$

Donde $\alpha_{i,j}$ son pesos de atención obtenidos con enmascaramiento auto-regresivo y ψ_v es la función de proyección sobre el espacio de valores.

En la auto-atención con enmascaramiento se considera en la representación de salida el elemento actual. De hecho, el primer elemento sólo estará representado por sí mismo (específicamente, por su proyección sobre el espacio de valores).

Ejemplo 27. Retomemos el ejemplo de la Sección 8.2 donde teníamos una matriz de valores del producto punto escalado dado en el Cuadro 8.3, en donde ya hemos reemplazado los valores de la parte triangular superior de la matriz por valores de $-\infty$. Hemos utilizado las mismas proyecciones que en el ejemplo citado. Al aplicar la

	$x^{(1)}$	$x^{(2)}$	$x^{(3)}$
$x^{(1)}$	1	$-\infty$	$-\infty$
$x^{(2)}$	3	8	$-\infty$
$x^{(3)}$	7	18	40

Cuadro 8.3: Matriz de productos punto escalados con enmascaramiento

función softmax obtendremos una matriz como la que se muestra en el Cuadro 8.4, donde los valores subsecuentes son ceros. De esta forma, las representaciones de cada

	$x^{(1)}$	$x^{(2)}$	$x^{(3)}$
$x^{(1)}$	1	0	0
$x^{(2)}$	0.006	0.994	0
$x^{(3)}$	0.005	0.005	0.99

Cuadro 8.4: Matriz de pesos de atención con enmascaramiento

elemento dependerán únicamente de los elementos anteriores o iguales a la posición actual. De hecho, vemos que el primer elemento se representará sólo a partir de sí mismo. Si asumimos que la proyección al espacio de valores es la identidad, obtenemos las siguientes representaciones de salida de la capa:

$$h = \begin{pmatrix} 0 & 1 \\ 0,99 & 1,99 \\ 2,98 & 3,99 \end{pmatrix}$$

En este caso, las representaciones son similares a las entradas, pues los pesos de atención son mayores con relación a cada uno de los datos consigo mismo.

8.4. Cabezas de atención y atención multi-cabeza

Dentro de las redes neuronales es común el uso de varias capas que se aplican de manera secuencial: una capa toma la salida de la capa anterior y envía una nueva representación a la capa siguiente. Este proceso tiene una analogía con el procesamiento de las capas recurrentes donde un estado anterior es procesado de manera previa para después obtener la representación del estado siguiente. Al igual que con las recurrentes, el procesamiento secuencial de las capas no es paralelizable, pues se requiere de todas las capas previas para computar la capa actual.

Para poder paralelizar un conjunto de representaciones distintas, Vaswani et al. (2017) proponen el uso de múltiples cabezas. El concepto de multi-cabeza busca tener varias posibles representaciones que capturen diferentes relaciones entre los elementos de entrada del modelo. Para ayudar al modelo a hacer mejores predicciones se puede aprender un conjunto de relaciones distinto y utilizar ese conjunto de relaciones para poder estimar diferentes representaciones de los datos de entrada. Así cada cabeza se enfocaría a una representación particular, permitiendo una mayor variación en éstas.

El primer concepto que debemos determinar es el de una cabeza de atención. Estas cabezas no sólo utilizan capas de atención o auto-atención, sino que incorporan otras capas para procesar los datos de entrada. En particular, una cabeza de atención suele incorporar una capa de normalización por lotes y una capa feedforward.

Definición 130 (Cabeza de atención). Sea $x^{(1)}, \dots, x^{(T)}$ una secuencia de entrada (generalmente por lotes). Una cabeza de atención aplica a la entrada los siguientes procesos:

1. Una capa de auto-atención, auto-atención con enmascaramiento o con atención estándar.
2. Una capa de suma y normalización.
3. Una capa de feedforward con ReLU.

Las capas de auto-atención y auto-atención enmascarada ya ha sido revisadas. Por su parte, la última capa de feedforward aplica una capa lineal con activación ReLU seguida de otra capa lineal. Esto es, se obtiene la capa:

$$ffw(h^{(i)}) = WReLU(W'h^{(i)} + b') + b$$

Donde $h^{(i)}$ es la salida de la capa de suma y normalización. Es esta última capa la que revisamos a continuación. Se trata de una capa que realiza una normalización por lotes (Ioffe and Szegedy, 2015).

Definición 131 (Normalización por lotes). Dado un conjunto de datos x , la normalización por lotes realiza una normalización de la distribución del lote en base a la siguiente formulación:

$$Norm(x) = a \odot \frac{x - \hat{\mu}}{\hat{\sigma} + \varepsilon} + \beta \quad (8.4)$$

Donde $a, \beta \in \mathbb{R}^d$ son parámetros y ε es un valor pequeño para evitar división entre 0.

Esta normalización se aplica generalmente en lotes (véase Sección 4.1.1). Esta normalización busca evitar que la distribución en las capas ocultas sea muy distinta entre el entrenamiento y la evaluación. Podemos observar los siguientes resultados.

Proposición 32. *Dado un conjunto de datos x normalizados por medio de la normalización por lotes, la distribución normalizada tiene media $\hat{\mu}$:*

$$\hat{\mu} = \mathbb{E}[\text{Norm}(x)] = \beta$$

Donde $\beta \in \mathbb{R}^d$ es el parámetro de la normalización.

Demostración. Por las propiedades de la esperanza, se observa con facilidad que se cumple:

$$\begin{aligned} \mathbb{E}[\text{Norm}(x)] &= \mathbb{E}\left[a \odot \frac{x - \hat{\mu}}{\hat{\sigma} + \varepsilon} + \beta\right] \\ &= \frac{a}{\hat{\sigma} + \varepsilon} \odot \mathbb{E}[x - \hat{\mu}] + \mathbb{E}[\beta] \\ &= \frac{a}{\hat{\sigma} + \varepsilon} \odot (\hat{\mu} - \hat{\mu}) + \beta \\ &= \beta \end{aligned}$$

□

Proposición 33. *Dado un conjunto de datos x normalizados por medio de la normalización por lotes, la distribución tiene una varianza $\hat{\sigma}^2$ dada como:*

$$\hat{\sigma}^2 = \mathbb{E}[(\text{Norm}(x) - \hat{\mu})^2] = a^2$$

Por tanto, la normalización por lotes lleva los datos hacia una distribución con parámetros que son aprendidos durante el proceso de entrenamiento. Esto permite tener menos variedad al interior de las capas y suele obtener mejores resultados. Finalmente, la cabeza de atención tiene una estructura como la que se muestra en la Figura 8.5.

Ya que tenemos varias cabezas de atención, el siguiente paso es, a partir de todas las cabezas, obtener una representación única que combine toda la información en las diferentes cabezas. Cabe señalar que cada cabeza se calcula de forma independiente a las otras cabezas, lo que permite su paralelización. Para obtener una sola representación se debe transformar cada una de las cabezas en un sólo conjunto de vectores que represente la secuencia de entrada.

Definición 132 (Atención multi cabeza). *Dada una secuencia de entrada $x^{(1)}, \dots, x^{(T)}$ la atención multi cabeza primero computa un conjunto de cabezas $\text{head}_1, \dots, \text{head}_N$ que se combinan para obtener una representación final h como:*

$$h = [\text{head}_1; \dots; \text{head}_N]W + b \quad (8.5)$$

Donde $[\text{head}_1; \dots; \text{head}_N]$ es la concatenación de las cabezas, $W \in \mathbb{R}^{Nd \times d}$ y $b \in \mathbb{R}^d$ son pesos a aprender.

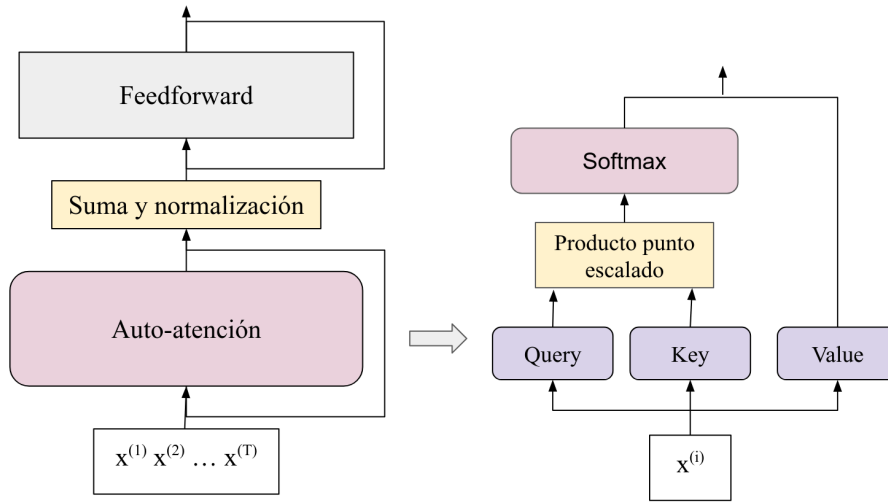


Figura 8.5: Diagrama de una cabeza de atención

Generalmente, se utiliza un bias igual a 0, por lo que sólo se aprende la matriz de pesos. Como se puede inferir por las dimensiones de W , esta matriz lleva la concatenación de todas las cabezas a un sólo conjunto de vectores de dimensión d . En la Figura 8.6 se muestra el diagrama de la atención multi cabeza. Como se puede ver, a la entrada se aplican múltiples proyecciones a espacios de consulta, llaves y valores. Cada una de estas proyecciones es independiente en cada cabeza. Posteriormente se hace un proceso de atención en cada una de las cabezas. Finalmente, las salidas de cada cabeza se concatenan y se aplica la transformación lineal para obtener una única representación.

La atención multi cabeza es un procedimiento esencial para la arquitectura de transformadores. Las múltiples cabezas sirven como diferentes capas que procesan a los datos de entrada de forma independiente y con el objetivo de aprender diferentes características de éstos. Al final, lo aprendido en cada cabeza se utiliza para obtener la representación final de la atención. A continuación revisamos la aplicación de todos los procesos que hemos revisado hasta ahora en la arquitectura de transformadores.

8.5. Transformadores

Los transformadores son quizá las arquitecturas de atención más reconocidas en la actualidad. Estas arquitecturas utilizan diferentes capas de atención, de forma distinta, como la auto-atención, la auto-atención enmascarada y la atención estándar. Estas arquitecturas fueron propuestas por Vaswani et al. (2017) para sustituir el uso de capas recurrentes, principalmente en tareas de procesamiento del lenguaje natural. La idea principal radica en obtener representaciones de los datos de entrada con base a su relación en los otros elementos de la misma entrada, pero de forma paralelizable; es decir, prescindiendo de un cómputo secuencial de la cadena de entrada.

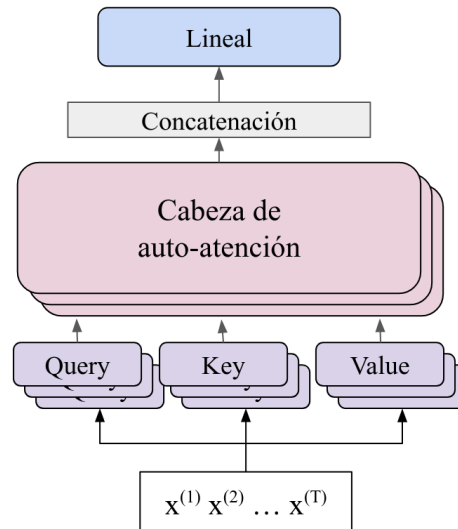


Figura 8.6: Diagrama de la atención multi cabeza

Por tanto, para codificar tanto las cadenas de entrada como las de salida se utilizan capas de atención que procesan los datos para obtener salidas que contengan información de las cadenas. De forma general, la arquitectura es de tipo encoder-decoder, pero tanto el encoder como el decoder utilizan capas de atención y no recurrencias. La arquitectura general de los transformadores se muestra en la Figura 8.7.

En la arquitectura resaltan dos secciones principales que se corresponden al encoder y al decoder. Tanto el encoder como el decoder cuentan con una entrada que corresponden a los datos de entrada y los datos de salida. Estos datos suelen ser cadenas. En el caso de la salida, durante el entrenamiento esta entrada corresponde a los datos de supervisión. Durante la inferencia se utilizan técnicas de enmascaramiento. Las entradas en ambos casos pasan por una capa de encajes (o *embeddings*) y de codificación posicional. Posteriormente se aplican las cabezas de atención. Como se puede observar, el decoder aplica la atención en dos partes, auto-atención para la codificación de las salidas y otra atención estándar para combinar los elementos de salidas con entradas. A continuación nos enfocamos a describir cada una de las partes de la arquitectura del transformador:

Encoder: El encoder codifica la cadena de entrada. Consta de N copias (lo que en la Figura 8.7 se describe como Nx) y cada una de estas copias consta de las siguientes capas:

Encajes: La representación en vectores distribuidos de los elementos de entrada.

Codificación posicional: Vectores, de la misma dimensión que los encajes, que codifican la posición de los elementos en la cadena.

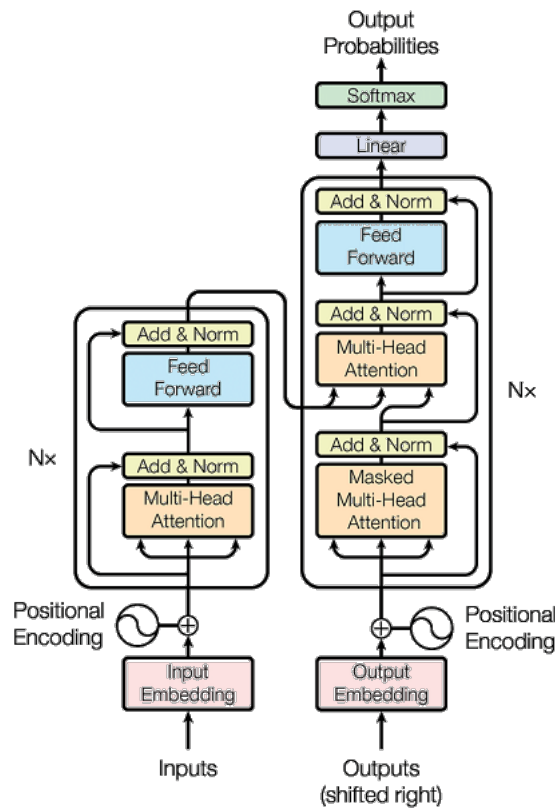


Figura 8.7: Arquitectura de un modelo de transformador propuesto por Vaswani et al. (2017)

Auto-atención: Capa de auto-atención multi cabeza, que utiliza diferentes cabezas (o *heads*), es decir, diferentes mecanismos de atención que se calculan paralelamente y se concatenan para tener una representación final.

Suma y normalización: Se trata de una conexión residual que agrega (suma) la información de entrada a la salida de la capa de atención. Además, realiza una normalización de los vectores.

Feedforward: Una vez que se obtiene la salida de la capa de atención y se agrega información de la entrada, se pasa por una capa feedforward, cuya salida, después se pasara por una capa de suma y normalización, serán los vectores de entrada del decoder. Se trata, generalmente, de una capa de la forma $W'ReLU(Wx + b) + b'$.

Decoder: Decodifica los elementos del encoder para obtener las probabilidades de salida. También se cuenta con N copias de este módulo. Se compone de:

Encajes: La representación en vectores distribuidos de los elementos de la ca-

dena de salida.

Codificación posicional: La codificación en vectores de la posición de los elementos de la salida.

Auto-atención enmascarada: Se trata de una capa de auto-atención que se aplica sobre la cadena de salida y también incorpora varias cabezas. Sin embargo, se distingue por enmascarar varias de los elementos de salidas, para así poder paralelizar los procesos de predicción del decoder.

Atención: El decoder incorpora una capa mas de atención que, como se ve en la Figura 8.7, es donde incorpora los elementos del decoder al encoder. Esta capa de atención relaciona la entrada con la salida; por tanto, se puede equiparar a la atención usada en las redes recurrentes.

Feedforward: Se incorpora, como en el decoder, una capa feedforward que se aplica sobre los vectores obtenidos en la capa previa de atención.

Suma y normalización: En cada capa del decoder, se agrega la operación de suma y normalización que integra los elementos de la capa previa a los de la capa actual, además de normalizar los vectores.

Salida: La salida del transformador serán las probabilidades que respondan a nuestra tarea; en este caso, la salida se compone de una capa común (lineal) con activación softmax. En este sentido, la salida del transformador será similar al de otras arquitecturas neuronales que hemos revisado.

La mayoría de las partes que componen al transformador ya han sido revisadas en las secciones anteriores; principalmente aquellas que tienen que ver con la atención. A continuación describimos cada una de las partes del transformador, enfocándonos en las secciones que no han sido atendidas.

8.5.1. Encoder

Dentro de la arquitectura de los transformadores, el encoder o decodificador toma como entrada un conjunto de vectores representando una secuencia; estos vectores comúnmente se presentan en una matriz. Se puede decir, entonces, que la entrada del encoder es una matriz cuyas columnas son los encajes de los elementos en las cadenas que, además, contienen información de su posición. Los encajes y la información de posición se suman: si $c^{(t)}$, $t = 1, \dots, T$, es el encaje del elemento t de la entrada, entonces, la representación que entrará al encoder será la suma de las matrices:

$$x = \begin{pmatrix} c^{(1)} \\ c^{(2)} \\ \vdots \\ c^{(T)} \end{pmatrix} + \begin{pmatrix} PE_1 \\ PE_2 \\ \vdots \\ PE_T \end{pmatrix}$$

El encoder consta de dos sub-capas: una capa de auto-atención y en segundo lugar una capa feedforward. Entre ambas sub-capas, además se agregan mecanismos de suma

de las capas previas (conexiones residuales) y normalización. A continuación explicamos los elementos de entrada y cada una de estas sub-capas en la arquitectura de los transformadores.

Encajes y codificación posicional

Los encajes son representaciones distribuidas que representan elementos categóricos de las entradas. Estos se han definido en la Sección 1.5.1 dentro de la Definición 32. En esta definición se señala que dado un vector de codificación binaria $u^{(t)} \in \{0, 1\}^k$, donde usualmente se tiene un 1 en la entrada que corresponde al índice asociado a la categoría que se representa en el vector mientras las otras entradas son ceros, la capa de encaje se define como:

$$c^{(t)} = Wu^{(t)}$$

En este caso, $c^{(t)}$ es un vector que corresponde al encaje de la categoría t dentro de los elementos de entrada y $W \in \mathbb{R}^{d \times k}$ es la matriz de pesos donde d es un hiperparámetro que corresponde a la dimensión del modelo. Se puede notar sin mucha dificultad que cuando el vector de entrada $u^{(t)}$ corresponde a un vector binario con un sólo 1 en la entrada t , el producto por W corresponde a tomar la t -ésima columna de esta matriz. Esta simplicidad permite que los datos categóricos se representen de manera eficiente en vectores continuos.

Ya mencionábamos que la auto-atención sufre la pérdida de información posicional que sí tenían las capas recurrentes. Para lidiar con esto, las arquitecturas de transformadores incorporan la llamada codificación posicional. Se trata de un vector que busca codificar la posición de cada elemento en la cadena de entrada para que se puedan procesar cadenas secuenciales de datos.

Definición 133 (Codificación posicional). *Dada una secuencia de elementos indexada por $t \in \{1, 2, \dots, T\}$ la codificación posicional es un vector $PE_t \in \mathbb{R}^d$ cuyas entradas se definen según su paridad. Si la entrada $2i$ del vector posicional es par, entonces se tiene que:*

$$PE_{2i}(t) = \sin\left(\frac{t}{\omega^{2i/d}}\right)$$

Por su parte, para las entradas impares del vector se tiene:

$$PE_{2i+1}(t) = \cos\left(\frac{t}{\omega^{2i/d}}\right)$$

Donde ω es un hiperparámetro, generalmente igual a 10000.

La idea que radica detrás de la codificación posicional es que cada vector PE_t sea un vector que represente una posición en base a la superposición de funciones seno y coseno. Como se puede ver en la Figura 8.8, donde se ven los valores posicionales en 512 dimensiones para 100 posiciones, en cada posición los valores negativos van abarcando más parte de las primeras entradas de los vectores, mientras que en los vectores de las primeras posiciones los valores positivos abarcan la mayoría de las entradas.

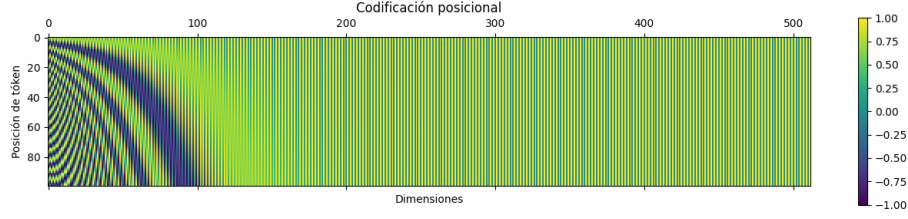


Figura 8.8: Visualización de la matriz de codificación posicional para un modelo de dimensión 512

Definición 134 (Representación de entrada). *Dada una secuencia de entrada de valores categóricos con índices $t \in \{1, 2, \dots, T\}$, las representaciones de entrada de un transformador están dadas como:*

$$x = \sqrt{d} \begin{pmatrix} c^{(1)} \\ c^{(2)} \\ \vdots \\ c^{(T)} \end{pmatrix} + \begin{pmatrix} PE_1 \\ PE_2 \\ \vdots \\ PE_T \end{pmatrix} \quad (8.6)$$

Donde $c^{(t)} \in \mathbb{R}^d$ son encajes que codifican a los elementos con índice t y $PE_t \in \mathbb{R}^d$ son vectores de codificación posicional; d es la dimensión del modelo.

Las representaciones dadas por los encajes se multiplican por la raíz cuadrada de la dimensión del modelo para escalar estos valores. Por tanto, se puede ver que cada vector de entrada dentro del transformador es la suma de su encaje más su codificación posicional:

$$x^{(t)} = \sqrt{d}c^{(t)} + PE_t$$

Estas representaciones serán la entrada de la primera capa de auto-atención en las copias del encoder. Remarcamos que los transformadores fueron en un principio arquitecturas prensadas para procesar cadenas textuales. De allí viene la relevancia de realizar una codificación posicional que conserve la secuencia de palabras dentro de un texto, así como la necesidad de los encajes para obtener representaciones a través de las palabras. Actualmente, las arquitecturas de transformadores se pueden aplicar a otro tipo de datos como imágenes (Han et al., 2020), superficies (Lu et al., 2022) u otras estructuras. La representación de entrada dependerá en gran parte de los tipos de datos que estemos procesando.

Ejemplo 28. *Supóngase que se tiene una secuencia $w_0w_2w_1$ donde los índices $i \in \{0, 1, 2\}$ corresponden a los subíndices de cada elemento. Entonces, la matriz de entrada de datos estaría dada por una matriz binaria en $\mathbb{R}^3$:*

$$u = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

Definamos entonces una matriz de pesos en la capa de encajes $W \in \mathbb{R}^{2 \times 3}$ tal que los encajes estén dados como:

$$c = uW^T = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 3 & 2 \\ 1 & 5 \\ 4 & 2 \end{pmatrix} = \begin{pmatrix} 3 & 2 \\ 4 & 2 \\ 1 & 5 \end{pmatrix}$$

En este caso, sólo se han cambiado las columnas de la matriz, pues el índice 2 aparece antes que el 1 en la secuencia de entrada. Ahora, podemos calcular los vectores de codificación posicional. Para simplificar usemos $\omega = 10$:

$$PE = \begin{pmatrix} \sin(0) & \cos(0) \\ \sin(1/10^{1/2}) & \cos(1/10^{1/2}) \\ \sin(2/10^{1/2}) & \cos(2/10^{1/2}) \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 0,84 & 0,54 \\ 0,9 & -0,41 \end{pmatrix}$$

En este caso la dimensión del modelo es $d = 2$, pues los encajes tienen dimensión 2, por lo que cada vector posicional, para poder sumarse, debe tener la misma posición. Por tanto, la representación final es de la forma:

$$x = \begin{pmatrix} 3 & 2 \\ 4 & 2 \\ 1 & 5 \end{pmatrix} + \begin{pmatrix} 0 & 1 \\ 0,84 & 0,54 \\ 0,9 & -0,41 \end{pmatrix} = \begin{pmatrix} 3 & 3 \\ 4,84 & 2,54 \\ 1,9 & 4,59 \end{pmatrix}$$

Cada uno de los renglones de esta matriz es la representación de un dato de entrada para el transformador en la posición correspondiente.

Salida del encoder

Una vez obtenidas las representaciones de entrada se aplica una capa de auto-atención multi cabeza (Secciones 8.2 y 8.4), en la cual se realiza posteriormente una normalización por lotes (Definición 131) y una suma residual. En este caso, la suma residual corresponde a sumar a la salida de la normalización por lotes la entrada de esa sub-capas. En particular, se tiene que esta suma y normalización se aplican como:

$$h' = \text{Norm}(\text{Att}(x)) + x$$

Donde Att aplica la auto-atención a la entrada x y Norm es la normalización por lotes (Definición 131). Esta conexión residual con los elementos de entrada busca recuperar información relevante de la entrada que pueda pasar a las siguientes capas. Posteriormente se aplica una capa simple de feedforward con activación ReLU como:

$$\begin{aligned} \text{ffw}(x) &= \text{ReLU}(W'x + b') + b \\ &= W \max\{0, W'x + b'\} + b \end{aligned}$$

A la salida de esta capa feedforward se vuelve aplicar una capa de suma y normalización, donde se retoman los valores de la sub-capas previa. Esto es, siendo h' las salidas de la auto-atención más la suma y normalización, se tiene:

$$h = \text{Norm}(\text{ffw}(h')) + h'$$

Por tanto, el resultado final de la capa de encoder serán vectores $h^{(1)}, h^{(2)}, \dots, h^{(t)}$ (renglones de la matriz h) que representan a la cadena de entrada. Estos vectores de codificación jugarán un papel similar a la codificación recurrente, pues serán pasados al decoder para obtener salidas en base a un mecanismo de atención entre el encoder y el decoder.

8.5.2. Decoder

El decoder se encarga de decodificar los vectores obtenidos en la salida del encoder para obtener una respuesta adecuada (una secuencia de salida) que responda al problema; es decir, una vez que se tiene una codificación de la cadena de entrada, el decoder buscará decodificarla para reconstruirla en una nueva cadena. Al igual que en el encoder, el decoder consta con N copias que trabajan paralelamente. Cada una de estas copias cuenta además con tres sub-capas: la capa de auto-atención enmascarada, la sub-capas de atención entre encoder y decoder, y finalmente una sub-capas feedforward.

Auto-atención enmascarada

La auto-atención enmascarada se utiliza en el decoder para producir secuencias de salida, en las que la información subsecuente al estado actual no debe ser vista o no se conoce. La auto-atención enmascarada se ha descrito en la Sección 8.3; este tipo de auto-atención busca ocultar relaciones entre los elementos de salida que pueden provocar sesgos en el aprendizaje. En particular, se utiliza una auto-atención en forma auto-regresiva, es decir, se enmascaran todos los elementos posteriores al estado actual en la salida.

La auto-atención enmascarada se aplica tanto en el entrenamiento como en la inferencia. Durante el entrenamiento, la secuencia de salida es conocida en tanto se cuenta con un conjunto de datos supervisado (o auto-supervisado). Por tanto, enmascarar las secuencias sólo evita que el modelo entrene con conocimiento de lo que busca predecir: los elementos que siguen al estado actual en la secuencia. Es común que se utilice un símbolo de inicio de cadena. Este símbolo corresponde a una etiqueta que no es parte de las categorías del problema y cuya función es indicar el inicio de una nueva secuencia. Así, si se tiene una secuencia de entrada $x^{(1)} \dots x^{(T)}$ se agregará un nuevo elemento al inicio $x^{(0)}$ que estará conformado por un valor que indique el inicio de la cadena.

Cuando se trata de datos en formato de texto, este primer elemento suele ser un símbolo que no corresponde a un elemento del lenguaje natural, tales como un símbolo [bos] (de *beginning of string*) o bien otro tipo de indicador que posteriormente pasa por las capas de encajes y codificación posicional. Este símbolo tiene especial interés en el tiempo de inferencia, pues siempre será el elemento con el que comenzará la predicción de una cadena, de tal forma que al aplicar la auto-atención enmascarada siempre el primer estado será este símbolo inicial. Esto permitirá realizar las predicciones de las secuencias de salida partiendo de un estado inicial dado.

Por tanto, en tiempo de inferencia, una vez que se ha aplicado el encoder, el decoder comenzará sólo con el símbolo inicial, aplicando la auto-atención enmascarada sólo a este símbolo para obtener una salida. En los siguientes estados se pueden utilizar las

predicciones previas para incorporarlas a la cadena predicha y obtener nuevas salidas (véase Sección 8.5.3).

La auto-atención enmascarada en el decoder sólo se encarga de codificar los elementos de salida para que posteriormente sean procesados por la sub-capa de atención estándar (encoder-decoder) y de esta forma obtener una salida que sea capaz de predecir los elementos secuenciales con base en poner atención a los elementos de entrada.

Atención encoder-decoder

La aplicación de el mecanismo de atención entre el encoder y el decoder puede considerarse como la parte central de la arquitectura de transformador, pues en esta parte el modelo se encarga de utilizar los elementos de salida, que han sido procesados previamente por la auto-atención enmascarada, para combinarlos con los elementos de la entrada y aplicar la idea esencial de la atención; esto es, que la salida actual debe poner atención a los elementos de entrada, generando la salida en base a una suma de las codificaciones de las entradas ponderadas por una peso de atención probabilístico.

La idea esencial de la atención entre encoder y el decoder es la misma que se aplica en las redes recurrentes (Sección 8.1). Sin embargo, la diferencia entre los transformadores y este tipo de redes recurrentes radica en la forma en que se codifican tanto las entradas como las salidas. Mientras que en las redes recurrentes las entradas se codifican por medio de capas recurrentes y las salidas depende de estas mismas capas en los estados previos, los transformadores codifican tanto las entradas como las salidas por medio de la auto-atención.

Por tanto, la atención en los transformadores no es radicalmente distinta a la de las redes recurrentes; se basa en la misma idea y es equivalente a la que revisamos en la Sección 8.1.

Definición 135 (Atención encoder-decoder). Sea $x^{(1)}, \dots, x^{(T)}$ una secuencia de entrada con codificación $h_{enc}^{(1)}, \dots, h_{enc}^{(T)}$ obtenida por un encoder en un transformador. La atención entre el encoder y el decoder obtiene representaciones de salida dadas como:

$$h_y^{(i)} = \sum_{j=1}^T \alpha_{i,j} \psi_v(h_{enc}^{(j)})$$

Donde ψ_v es una función de proyección al espacio de valores y $\alpha_{i,j}$ son los pesos de atención obtenidos como:

$$\alpha_{i,j} = \text{Softmax}\left(\frac{\psi_q(h_{dec}^{(i)}) \psi_k(h_{enc}^{(j)})}{\sqrt{d}}\right)$$

Tal que $h_{dec}^{(i)}$ es la codificación del estado actual de la salida realizado con auto-atención enmascarada, y ψ_q, ψ_k son funciones de proyección al espacio de consultas y de llaves, respectivamente.

Se pueden ver las similitudes con la Definición 121 donde para obtener la representación de la salida en el estado actual se realiza una suma ponderada de las representaciones del encoder. La diferencia radica en que estas representaciones son proyectadas

al espacio de valores antes de sumarse y en la forma en que se calculan los pesos de atención. Sin embargo, podemos observar que bajo ciertas condiciones, la atención en transformers se acerca a la atención en redes recurrentes.

Proposición 34. Si ψ_q y ψ_k son proyecciones lineales (productos por matrices), entonces los pesos de atención se pueden expresar como una forma bilineal; esto es:

$$\alpha_{i,j} = h_{dec}^{(i)} W h_{enc}^{(j)}$$

Para alguna matriz de pesos W .

En particular, esta forma de calcular la atención es la forma propuesta por Luong (2015) en redes recurrentes y que se muestra en la Ecuación 8.2 de la Sección 8.1. Podríamos pedir que la proyección ψ , sobre los valores esté dada por la identidad, de tal forma que la estimación de las representaciones por atención se parezca más a la usada en las redes recurrentes.

Corolario 2. El mecanismo de atención en redes recurrentes es un caso particular de la atención en transformadores.

Este último resultado queda claro al suponer las condiciones de proyecciones lineales en las consultas y llaves, y del uso de la identidad en los valores. Entonces, la atención entre el encoder y el decoder en los transformadores es más general, por lo que puede llevarnos a resultados con mejor desempeño. Asimismo, aclaramos que el corolario anterior sólo habla del mecanismo de atención entre encoder y decoder, pues otra diferencia esencial entre las redes recurrentes y los transformadores es la forma en que se representan los datos de entrada por el encoder y los estados de salida en el decoder.

Utilizando la notación de Vaswani et al. (2017), en el que se utilizan las matrices Q_{dec} para la matriz de consultas en el decoder, y K_{enc}, V_{enc} como las matrices de llaves y valores del encoder, respectivamente, tenemos que la sub-capa de atención en el transformador se puede escribir como:

$$Attention(Q_{dec}, K_{enc}, V_{enc}) = Softmax\left(\frac{Q_{dec}K_{enc}^T}{\sqrt{d_k}}\right)V_{enc} \quad (8.7)$$

Los pesos de atención también pueden representarse en una matriz. En este caso se trata de una matriz rectangular en tanto se comparan los elementos de entrada con los de salida. También en esta sub-capa de atención entre el encoder y el decoder suelen usarse múltiples cabezas (Sección 8.4) para poder obtener relaciones entre la entrada y la salida distintas, que se concatenan y se reducen a un sólo vector que alimentará a la siguiente sub-capa de feedforward, la cual es similar a la usada en el encoder, utilizando una activación ReLU intermedia. Como se puede observar en la Figura 8.7, después de cada sub-capas se aplica una suma y normalización.

La salida del decoder no es todavía la salida final del transformador, pues a lo que arroja el decoder todavía tiene que pasar por una capa lineal y una activación softmax para obtener los resultados. En la siguiente sección exploramos la fase final de transformador y abordamos brevemente su entrenamiento.

8.5.3. Salida e inferencia

La arquitectura de los transformadores está pensada para predecir secuencias de salida a partir de secuencias de entrada. Supongamos, entonces, que la salida del decoder en el transformador es $h = h^{(1)} \dots h^{(t)}$, la cual codifica unas salidas hasta el tiempo t . Asumimos que estas salidas responde a una secuencia de entrada que denotaremos como $x = x^{(1)} \dots x^{(T)}$. Entonces la salida del transformador será:

$$f(x) = \text{Softmax}(hW + b)$$

Donde $W \in \mathbb{R}^{d \times N}$ es la matriz de pesos de esta capa de salida, N es el número de neuronas de salida, y $b \in \mathbb{R}^N$ es el bias. Esta salida será una secuencia de vectores que contienen la probabilidad basada en la función softmax para cada tiempo t . Es decir, la salida se puede expresar como una secuencia $f(x)^{(1)} \dots f(x)^{(t)}$, donde cada $f(x)^{(i)}$ es un vector de probabilidades softmax. A partir de estas probabilidades, el objetivo es obtener una secuencia $\hat{y}^{(1)} \dots \hat{y}^{(t)}$ con los elementos que maximicen la probabilidad:

$$\hat{y}^{(1)} \dots \hat{y}^{(t)} = \arg \max_{i_1 \dots i_t} f_{i_1}^{(1)}(x) \dots f_{i_t}^{(t)}(x)$$

Es decir, la salida será la secuencia más probable dadas las probabilidades de salida. La estimación de una probabilidad máxima de secuencia se vuelve compleja con respecto a la longitud de la secuencia de salida, puesto que se requieren explorar todas las posibles combinaciones de valores de salida, lo que requiere de un cálculo de orden $O(t^N)$, con t la longitud de la secuencia de salida, y N los posibles valores de la salida. Para poder estimar la probabilidad máxima, por tanto, se suelen utilizar estrategias que permitan calcular probabilidades de manera más eficiente, aunque sin garantizar un máximo global. Una de estas primeras estrategias es la llamada ambiciosa.

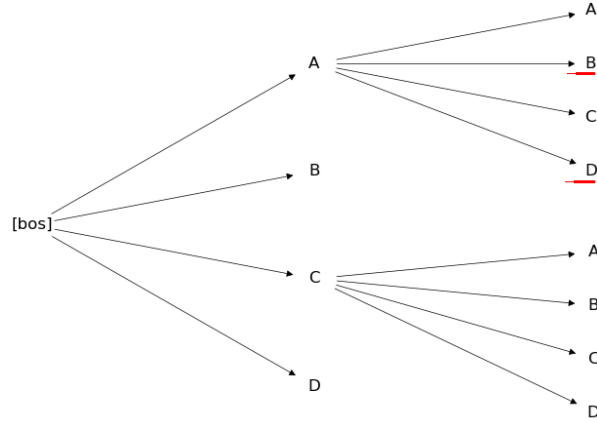
Definición 136 (Estimación ambiciosa). *Dada una secuencia $f(x) = f(x)^{(1)} \dots f(x)^{(t)}$ de vectores de probabilidad, la estimación ambiciosa obtiene cada una de las salidas $\hat{y}^{(i)}$, $i \in \{1, 2, \dots, t\}$, como:*

$$\hat{y}^{(i)} = \arg \max_j f(x)_j^{(i)}$$

La estimación ambiciosa maximiza cada una de las salidas sin considerar otras salidas; por tanto, no puede garantizar maximizar globalmente la probabilidad de la secuencia de salida, pues pueden existir posibles combinaciones que, si bien no presenten valores máximos al principio, terminen por maximizar la probabilidad sobre toda la cadena. Sin embargo, la complejidad del método ambicioso es de orden lineal, lo que lo hace mucho más eficiente que una búsqueda exhaustiva de la probabilidad máxima. Una estrategia intermedia es la búsqueda por haces (o *beam search*).

Definición 137 (Búsqueda por haces). *Dada una secuencia $f(x) = f(x)^{(1)} \dots f(x)^{(t)}$ de vectores de probabilidad, la búsqueda por haces obtiene salidas $\hat{y}^{(i)}$, $i \in \{1, 2, \dots, t\}$, reduciendo el número de elementos de la búsqueda a sólo k por cada tiempo t .*

Las búsqueda por haces evita hacer una búsqueda exhaustiva al ignorar todos los caminos posibles que estén por debajo de los k más probables. Esto también ayuda

Figura 8.9: Esquema de la búsqueda por haces con $k = 2$

a realizar una mayor exploración al contrario de cómo lo haría el método ambicioso. Puede verse que la estimación ambiciosa es un caso particular del método por haces cuando $k = 1$.

Ejemplo 29. Tomemos un ejemplo simple en que las posibles salidas en cada tiempo t son sólo 4: A, B, C y D, indexadas con los valores 1, 2, 3 y 4, respectivamente. Iniciamos la salida con un símbolo inicial [bos], entonces supongamos que en el siguiente tiempo, las probabilidades obtenidas son:

	A	B	C	D
$f(x)^{(1)}$	0.3	0.2	0.4	0.1

Tomemos $k = 2$, esto es que nos quedaremos únicamente con los dos valores con las probabilidades más altas. En este caso corresponden a los valores C y A. Por lo tanto, en el siguiente tiempo obtendremos las probabilidades condicionadas únicamente a A y C (lo denotamos agregando la condición en la función).

	A	B	C	D
$f(x A)^{(2)}$	0.1	0.4	0.1	0.4
$f(x C)^{(2)}$	0.3	0.2	0.2	0.3

En este caso, sólo hemos realizado las combinaciones posibles de los símbolos en el tiempo $t = 2$ con los símbolos de salida. En el siguiente tiempo, tendríamos que quedarnos con únicamente los dos caminos con mayor probabilidad, que en este ejemplo son AB y AD. En la Figura 8.9 se visualiza este ejemplo.

La búsqueda por haces obtiene k secuencias posibles al final de la inferencia, de tal forma que podemos elegir la secuencia que maximice la probabilidad final, u optar por otras secuencias. Cabe señalar que es común que en los modelos de transformadores se determine un tamaño máximo de la secuencia de salida para determinar hasta cuándo detener la inferencia. Otra forma de detener la inferencia es a partir de un símbolo

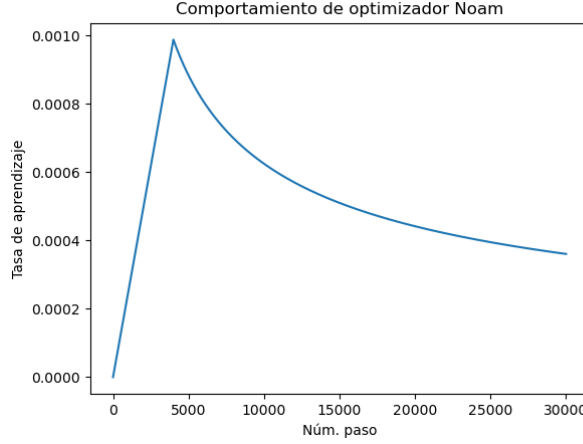


Figura 8.10: Valores de la tasa de aprendizaje en diferentes pasos para el optimizador Noam

especial que indique el final de la secuencia, de tal forma que al predecir este símbolo la secuencia se terminará.

8.5.4. Optimizador Noam

El último aspecto que explicamos sobre la arquitectura de los transformadores es el optimizador que es comúnmente utilizado para entrenar estos modelos. Si bien, la arquitectura puede ser entrenada bajo cualquier optimizador (véase Sección 4.2), Vaswani et al. (2017) notan que el modelo converge mejor cuando la tasa de aprendizaje se normaliza. Por tanto, proponen la implementación de un optimizador, llamado Noam (de *Normalized Adam*) debido a que es una versión normalizada de Adam (véase la Definición 77).

Definición 138 (Optimizador Noam). *El optimizador Noam es un optimizador basado en Adam donde la tasa de aprendizaje η se normaliza en cada paso como:*

$$\eta \leftarrow \frac{1}{\sqrt{d}} \min\left\{\frac{1}{\sqrt{t}}, \frac{t}{\sqrt{\omega^3}}\right\}$$

Donde t es el paso actual y ω es un hiperparámetro.

En el optimizador Noam es común que al hiperparámetro ω se le asigne un valor igual a 4 000. A este hiperparámetro se le conoce como *warmup*. Vaswani et al. (2017) muestran que este optimizador tiene mejores resultados que otros optimizadores bajo la arquitectura de los transformadores. Este optimizador sólo modifica la forma en que se obtiene la tasa de aprendizaje en cada paso de la optimización, pero la actualización de los pesos se hace con el método de Adam. En la Figura 8.10 se muestra el comportamiento de la tasa de aprendizaje en 3 000 pasos de actualización con el optimizador

Noam. El valor de la tasa de aprendizaje inicial se ha elegido igual a 0 y se utiliza $\omega = 4000$.

Ejercicios

1. Demuestra que, siendo x una matriz de datos de entrada, y $h^{(i)}$ el i -ésimo renglón de la matriz de representaciones de $h = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$, entonces:

$$h^{(i)} = \sum_{j=1}^T \text{Softmax}\left(\frac{\psi_q(x^{(i)})\psi_k(x^{(j)})}{\sqrt{d}}\right)\psi_v(x^{(j)})$$

Tal que ψ_q , ψ_k y ψ_v son las proyecciones al espacio de consultas, llaves y valores, respectivamente.

2. Demuestra que en una capa de auto-atención, las derivadas parciales sobre las consultas son de la forma:

$$\frac{\partial L_t(f_y(x^{(t)}), y)}{\partial W_{i,j}^Q} = \frac{\partial L_t(f_y(x^{(t)}), y)}{\partial h^{(t)}} \frac{1}{\sqrt{d}} \sum_s v(x^{(s)})_i \alpha_{i,i} (\delta_{i,s} - \alpha_{t,s}) k(x^{(s)})_i x_j^{(t)}$$

3. Demuestra que la normalización por lotes cumple que su varianza es de la forma:

$$\hat{\sigma}^2 = \mathbb{E}[(\text{Norm}(x) - \mu)^2] = a^2$$

4. Demuestra que si ψ_q y ψ_k son proyecciones lineales (productos por matrices), entonces los pesos de atención se pueden expresar como una forma bilineal; es decir que existe una matriz de pesos W tal que:

$$\alpha_{i,j} = h_{dec}^{(i)} W h_{enc}^{(j)}$$

5. Determina la complejidad de la búsqueda por haces.
6. Implementa una arquitectura de transformador para un problema de traducción automática, usando PyTorch. Puedes apoyarte en *The Annotated Transformer*¹ o en el material de apoyo².

¹<https://nlp.seas.harvard.edu/2018/04/03/attention.html>

²<https://github.com/VictorMijangosDeLaCruz/MecanismosAtencion>

Notación

$x + y$	Suma de elementos: puede referir a suma de escalares o a suma de vectores, tal que si $x, y \in \mathbb{R}^d$, entonces $(x + y)_i = x_i + y_i$.
$x \cdot y$	Producto punto: entre vectores $x, y \in \mathbb{R}^d$ tal que $x \cdot y = x_1 y_1 + x_2 y_2 + \dots + x_d y_d = \sum_{i=1}^d x_i y_i$.
Wx	Producto matriz-vector: Dado $W \in \mathbb{R}^{n \times d}$ y $x \in \mathbb{R}^d$, entonces $Wx \in \mathbb{R}^n$ tal que $(Wx)_i = W_{i \cdot} \cdot x$.
$W^T U$	Producto entre matrices: Dadas $W \in \mathbb{R}^{d \times n}$ y $U \in \mathbb{R}^{n \times m}$, entonces $W^T U \in \mathbb{R}^{d \times m}$ tal que $(W^T U)_{i,j} = W_{\cdot i} \cdot U_{j \cdot}$.
$x \otimes y$	Producto externo: Dado dos vectores $x \in \mathbb{R}^d$, $y \in \mathbb{R}^n$, se tiene que $x \otimes y \in \mathbb{R}^{d \times n}$ tal que $(x \otimes y)_{i,j} = x_i y_j$.
$x \odot y$	Producto punto a punto: Dados los vectores $x, y \in \mathbb{R}^d$, el resultado es un vector d -dimensional tal que $(x \odot y)_i = x_i y_i$.
$\delta_{i,j}$	Función delta de Kronecker: Definida como:

$$\delta_{i,j} = \begin{cases} 1 & \text{si } i = j \\ 0 & \text{si } i \neq j \end{cases}$$

$\sigma(x)$	Función sigmoide: Definida como $\sigma(x) = \frac{1}{1+e^{-x}} = \frac{e^x}{1+e^x}$
$\text{Softmax}(x)$	Función softmax: Una versión suave de arg máx, dado un vector $x \in \mathbb{R}^n$ genera un vector de probabilidad, tal que $\text{Softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$
$\arg \min_{x \in X} f(x)$	Argumento que minimiza: Determina el valor que toma el mínimo de una función $f : X \rightarrow Y$, es decir $x^* = \arg \min_{x \in X} f(x)$ es el valor tal que para todo otro $x \in X$ $f(x^*) \leq f(x)$.
$\arg \max_{x \in X} f(x)$	Argumento que maximiza: Determina el valor que toma el máximo de una función $f : X \rightarrow Y$, es decir $x^* = \arg \max_{x \in X} f(x)$ es el valor tal que para todo otro $x \in X$ $f(x) \leq f(x^*)$.
$\frac{\partial f(x)}{\partial x_i}$	Derivada parcial: Dada una función $f : \mathbb{R}^d \rightarrow \mathbb{R}$ y $x \in \mathbb{R}^d$, tenemos que $\frac{\partial f(x)}{\partial x_i} = \lim_{h \rightarrow 0} \frac{f(x + h e_i) - f(x)}{h}$, donde e_i es un vector base de $\mathbb{R}^d$.
$\nabla_x f(x)$	Gradiente: Dada una función $f : \mathbb{R}^d \rightarrow \mathbb{R}$, su gradiente es el vector de derivadas parciales $\nabla f(x) = \left(\frac{\partial f(x)}{\partial x_1} \quad \frac{\partial f(x)}{\partial x_2} \quad \dots \quad \frac{\partial f(x)}{\partial x_d} \right)^T$
$W \star x$	Correlación cruzada: Usualmente referida en la literatura de aprendizaje profundo como convolución , refiere a la función $(W \star x)_{i,j} = \sum_{l=0}^H \sum_{l'=0}^L W_{l,l'} x_{i+l,j+l'}$

Bibliografía

- Bahdanau, D. (2014). Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*.
- Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. (2018). Automatic differentiation in machine learning: a survey. *Journal of machine learning research*, 18(153):1–43.
- Beck, A. and Teboulle, M. (2009). A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM journal on imaging sciences*, 2(1):183–202.
- Breiman, L. (1996). Bagging predictors. *Machine learning*, 24:123–140.
- Brézis, H. (1984). *Análisis Funcional Teoría y Aplicaciones*. Alianza Editorial.
- Cho, K., van Merriënboer, B., Bahdanau, D., and Bengio, Y. (2014). On the properties of neural machine translation: Encoder-decoder approaches.
- Chung, J. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186.
- Duan, Y., Ji, G., Cai, Y., et al. (2023). Minimum width of leaky-relu neural networks for uniform universal approximation. In *International Conference on Machine Learning*, pages 19460–19470. PMLR.
- Duchi, J., Hazan, E., and Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research*, 12(7).
- Elman, J. L. (1990). Finding structure in time. *Cognitive science*, 14(2):179–211.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep learning*. The MIT Press.

- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. (2014). Generative adversarial nets. *Advances in neural information processing systems*, 27.
- Han, K., Wang, Y., Chen, H., Chen, X., Guo, J., Liu, Z., Tang, Y., Xiao, A., Xu, C., Xu, Y., et al. (2020). A survey on visual transformer. *arXiv preprint arXiv:2012.12556*.
- Ho, J., Jain, A., and Abbeel, P. (2020). Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851.
- Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8):1735–1780.
- Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural networks*, 4(2):251–257.
- Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366.
- Huang, C. (2020). Relu networks are universal approximators via piecewise linear or constant functions. *Neural Computation*, 32(11):2249–2278.
- Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *CoRR*, abs/1502.03167.
- Kingma, D. (2014). Adam: a method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Kingma, D. P. and Welling, M. (2013). Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.
- LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324.
- Leshno, M., Lin, V. Y., Pinkus, A., and Schocken, S. (1993). Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural networks*, 6(6):861–867.
- Lin, J., Song, C., He, K., Wang, L., and Hopcroft, J. E. (2019). Nesterov accelerated gradient and scale invariance for adversarial attacks. In *International Conference on Learning Representations*.
- Lu, D., Xie, Q., Wei, M., Gao, K., Xu, L., and Li, J. (2022). Transformers in 3d point clouds: A survey. *arXiv preprint arXiv:2205.07417*.
- Luong, M.-T. (2015). Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025*.
- McCulloch, W. S. and Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, 5:115–133.

- Mienye, I. D. and Sun, Y. (2022). A survey of ensemble learning: Concepts, algorithms, applications, and prospects. *IEEE Access*, 10:99129–99149.
- Minsky, M. and Papert, S. (1969). *Perceptron: An introduction to computational geometry*. The MIT Press.
- Mitchell, T. (1997). *Machine Learning*. McGraw Hill.
- Mukkamala, M. C. and Hein, M. (2017). Variants of rmsprop and adagrad with logarithmic regret bounds. In *International conference on machine learning*, pages 2545–2553. PMLR.
- Park, S., Yun, C., Lee, J., and Shin, J. (2020). Minimum width for universal approximation. *arXiv preprint arXiv:2006.08859*.
- Raskutti, G., Wainwright, M. J., and Yu, B. (2014). Early stopping and non-parametric regression: an optimal data-dependent stopping rule. *The Journal of Machine Learning Research*, 15(1):335–366.
- Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, 65(6):386.
- Ruder, S. (2016). An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*.
- Russell, S. and Norvig, S. (2021). *Artificial Intelligence: A Modern Approach*. Pearson Education Press.
- Schuster, M. and Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE transactions on Signal Processing*, 45(11):2673–2681.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958.
- Valiant, L. G. (1984). A theory of the learnable. *Communications of the ACM*, 27(11):1134–1142.
- Vapnik, V. (1998). *Statistical learning theory*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Warde-Farley, D., Goodfellow, I. J., Courville, A., and Bengio, Y. (2014). An empirical analysis of dropout in piecewise linear networks. *arXiv preprint arXiv:1312.6197*.
- Watanabe, S. (2009). *Algebraic geometry and statistical learning theory*. Cambridge university press.
- Werbos, P. J. (1990). Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560.

- Zeiler, M. D. (2012). Adadelata: an adaptive learning rate method. *arXiv preprint arXiv:1212.5701*.